

Contents

第九册简介：大模型数据工程与数据智能	11
本书定位	11
为什么需要这本书	11
适合读者	11
学完后应具备的能力	11
阅读建议	11
与其他书的关系	11
第九册：大模型数据工程与数据智能	12
本书定位	12
章节文件	12
第一章：数据总览	12
本章资料边界	12
本章目标	12
1. 来龙去脉：大模型为什么越来越重视数据	13
1.1 早期深度学习：数据是燃料	13
1.2 GPT-3 时代：规模和 web-scale 数据	13
1.3 Chinchilla 时代：模型不是越大越好，token 也要跟上	13
1.4 高质量数据时代：Textbooks Are All You Need 的启发	13
1.5 今天的数据观念	14
2. 小白例子：训练模型像培养学生	14
3. 大模型数据的五个核心维度	14
3.1 规模	14
3.2 质量	15
3.3 覆盖	15
3.4 配比	15
3.5 治理	16
3.6 数据工程公式速查	16
4. 大模型数据类型全景图	17
4.1 预训练数据	17
4.2 SFT 数据	17
4.3 偏好数据	17
4.4 安全数据	18
4.5 评估数据	18
4.6 RAG 数据	18
4.7 多模态数据	19
5. 大模型数据 Pipeline	19
5.1 数据来源	19
5.2 解析和规范化	19
5.3 过滤和质量评分	20
5.4 去重	20
5.5 配比和采样	20
5.6 版本管理	20
6. 数据问题如何变成模型问题	20
6.1 幻觉	20
6.2 过度拒答	20
6.3 代码能力弱	20
6.4 数学推理弱	21
6.5 评估分数虚高	21
7. 数据质量的层次	21
7.1 文本层质量	21
7.2 内容层质量	21
7.3 任务层质量	21

7.4 行为层质量	21
8. 数据配比为什么难	21
9. 数据治理为什么是核心能力	22
9.1 可追溯	22
9.2 可复现	22
9.3 可删除	22
9.4 可评估	22
9.5 可审计	22
10. 最小代码：数据集审计和上线门禁	22
11. 面向专家：数据不是静态资产，而是控制面	26
12. 真实项目中的数据团队分工	27
13. 面试官会怎么问	27
问题 1：为什么数据决定模型能力？	27
问题 2：Chinchilla 对数据工程有什么启发？	27
问题 3：高质量数据为什么重要？	27
问题 4：预训练数据、SFT 数据、偏好数据有什么区别？	27
问题 5：数据问题如何导致安全问题？	28
14. 标准回答模板	28
15. 常见误区	28
15.1 误区：数据越多越好	28
15.2 误区：清洗就是删垃圾文本	28
15.3 误区：SFT 数据越多越好	28
15.4 误区：评估数据也是数据，可以混着用	28
15.5 误区：数据工程只是训练前工作	28
16. 小练习	28
练习 1	28
练习 2	28
练习 3	29
练习 4	29
练习 5	29
17. 本章总结	29

第二章：Web-Scale 数据采集

29

本章目标	29
0. 本讲资料边界与第二轮精修口径	30
1. 来龙去脉：为什么 Web 成了大模型的数据源	30
1.1 早期 NLP 的数据规模很小	30
1.2 预训练改变了数据需求	30
1.3 GPT-3 之后：Web-scale 变成标配	30
1.4 The Pile 的启发	31
1.5 今天的观点	31
2. 小白例子：做一座城市图书馆	31
3. 常见数据来源	31
3.1 网页数据	31
3.2 书籍	32
3.3 论文和技术文档	32
3.4 代码数据	32
3.5 论坛和问答	33
3.6 对话数据	33
3.7 多语言数据	33
4. Web-scale 采集 Pipeline	33
4.1 Source Planning	34
4.2 Legal and Policy Review	34
4.3 Collection	34
4.4 Raw Storage	34
4.5 Parsing	34

4.6 Boilerplate Removal	35
4.7 Language and Domain Classification	35
4.8 Quality Filtering	35
4.9 Deduplication	35
4.10 Versioning and Audit	35
4.11 关键公式与采集审计指标	36
5. 数据采集中的合规问题	37
5.1 版权	37
5.2 隐私	37
5.3 Terms of Service	37
5.4 Robots 和访问控制	37
5.5 数据可追溯	37
6. 采集不是越多越好	38
6.1 噪声成本	38
6.2 记忆风险	38
6.3 偏见放大	38
6.4 评估污染	38
6.5 安全风险	38
7. 数据源元数据设计	38
8. Web 数据和专门数据的平衡	38
9. 多语言采集	39
9.1 低资源语言问题	39
9.2 语言识别	39
9.3 翻译数据	39
9.4 多语言配比	39
10. 代码数据采集的特殊问题	39
10.1 许可证	39
10.2 Fork 和重复	39
10.3 密钥和凭证	40
10.4 安全漏洞	40
10.5 Repo-level 上下文	40
11. 对话和用户数据采集	40
12. 真实项目架构	40
12.1 Source registry	40
12.2 Raw data lake	40
12.3 Dataset builder	41
12.4 Audit dashboard	41
12.5 最小可运行 Web 采集审计 demo	41
13. 面向专家：采集策略会改变模型行为	46
14. 面试官会怎么问	47
问题 1：如何构建 web-scale 预训练数据集?	47
问题 2：网页数据有哪些风险?	47
问题 3：为什么要保留数据元数据?	47
问题 4：如何处理版权和合规?	47
问题 5：The Pile 对数据工程有什么启发?	48
15. 标准回答模板	48
16. 常见误区	48
16.1 误区：能访问就能训练	48
16.2 误区：网页数据天然代表真实世界	48
16.3 误区：只要数据够多，质量无所谓	48
16.4 误区：去重是清洗阶段的小问题	48
16.5 误区：用户日志可以直接拿来训练	48
17. 小练习	48
练习 1	48
练习 2	48

练习 3	49
练习 4	49
练习 5	49
18. 本章总结	49
第三章：清洗过滤与质量评分	49
0. 本讲资料边界与第二轮精修口径	49
1. 先建立直觉：清洗过滤到底在过滤什么？	50
2. 来龙去脉：从 NLP 数据预处理到大模型数据治理	50
3. 为什么“更多数据”不等于“更好数据”？	50
4. 清洗过滤流水线总览	51
4.1 关键公式与审计指标	51
5. 基础有效性过滤：先把明显坏数据挡在外面	52
6. 乱码和编码问题	52
7. 模板页、导航栏和 boilerplate 去除	53
8. 广告、SEO 垃圾和低信息密度文本	53
9. 质量评分：从二元过滤到连续打分	54
10. 规则过滤 vs 模型过滤	54
11. 质量分类器怎么训练？	55
12. 困惑度过滤：用语言模型判断“像不像正常文本”	55
13. 毒性和有害内容过滤	56
14. PII 和隐私过滤	56
15. 低质量机器生成数据	56
16. 不同数据类型的过滤策略不同	57
16.1 网页正文	57
16.2 代码数据	57
16.3 数学数据	57
16.4 论文和技术文档	57
16.5 对话数据	57
16.6 多语言数据	58
17. 阈值怎么定？	58
18. 误删和漏删：过滤系统的两个错误	58
19. 过滤后如何评估数据质量？	59
19.1 最小可运行清洗过滤审计 demo	59
20. 数据清洗和训练目标的关系	62
21. 面向专家：清洗过滤改变模型的归纳偏置	63
22. 一个可落地的清洗过滤方案	63
23. 常见面试题	64
23.1 为什么大模型训练前要做数据清洗？	64
23.2 规则过滤和模型过滤怎么取舍？	64
23.3 如何设计质量分类器？	64
23.4 怎么处理 PII？	64
23.5 毒性内容是不是全部删除？	64
23.6 如何判断清洗策略是否有效？	64
24. 常见误区	65
25. 本章小结	65
第四章：去重与污染检测	65
0. 本讲资料边界与第二轮精修口径	66
1. 先建立直觉：为什么重复数据很危险？	66
2. 来龙去脉：从数据库去重到大模型去重	66
3. exact dedup：完全去重	67
4. 文档级、段落级和子串级去重	67
4.1 文档级去重	67
4.2 段落级去重	68
4.3 子串级去重	68

5. near dedup: 近重复去重	68
6. MinHash: 把集合相似度变成可扩展检索	68
6.1 关键公式与审计指标	69
7. LSH: 避免全量两两比较	70
8. SimHash: 适合快速近重复检测	70
9. embedding 去重: 语义相似, 但更危险	71
10. 去重不是越狠越好	71
11. 代码数据去重: fork、vendor 和生成文件	72
12. benchmark contamination: 评测污染是什么?	72
13. 污染检测的基本方法	73
14. 评测污染和训练污染的区别	73
15. canary: 用探针检测记忆风险	73
16. 去重与记忆风险	74
17. 去重策略会影响数据配比	74
18. 去重系统的工程架构	74
19. 阈值和审计	75
20. 面向专家: 去重会改变 scaling behavior	75
21. 一个可落地的去重与污染检测方案	75
21.1 最小可运行去重与污染检测审计 demo	76
22. 常见面试题	80
22.1 exact dedup 和 near dedup 的区别是什么?	80
22.2 为什么大模型训练必须去重?	80
22.3 MinHash 解决什么问题?	80
22.4 LSH 的作用是什么?	80
22.5 benchmark contamination 怎么检测?	80
22.6 去重是不是越彻底越好?	81
22.7 canary 是什么?	81
23. 常见误区	81
24. 本章小结	81

第五章: Data Mixture 与配比 81

0. 本讲资料边界与第二轮精修口径	82
1. 先建立直觉: 配比决定模型 “读过什么世界”	82
2. 来龙去脉: 从 “越多越好” 到 “配比优化”	82
3. Data mixture 到底在配什么?	83
4. 为什么配比会影响能力?	83
4.1 关键公式与审计指标	83
5. 自然分布 vs 人工配比	84
6. 通用文本的角色	85
7. 代码数据的配比	85
8. 数学和推理数据的配比	86
9. 多语言数据的配比	86
10. 专业领域数据的配比	86
11. 合成数据的配比	87
12. 数据配比和训练阶段	87
12.1 预训练	87
12.2 继续预训练	87
12.3 SFT	87
12.4 偏好训练	87
12.5 安全训练	88
13. 静态配比 vs 动态配比	88
14. Temperature sampling 和多语言平滑	88
15. 配比实验怎么做?	88
16. 评估矩阵: 配比优化看什么指标?	89
17. 数据上采样和过拟合	89
18. 配比和 tokenizer 的关系	89

19. 配比和去重清洗的关系	90
20. 面向专家: Data mixture 是隐式目标函数设计	90
21. 一个可落地的 data mixture 方案	90
21.1 最小可运行 data mixture 审计 demo	91
22. 常见面试题	93
22.1 什么是 data mixture?	93
22.2 为什么不能按数据自然规模混合?	93
22.3 代码数据比例提高一定好吗?	93
22.4 多语言配比怎么做?	93
22.5 合成数据应该占多少?	93
22.6 如何优化 data mixture?	93
22.7 配比和 Chinchilla scaling 有什么关系?	93
23. 常见误区	94
24. 本章小结	94

第六章: Code、Math 与 Domain Data 94

0. 本讲资料边界与第二轮精修口径	95
1. 先建立直觉: 为什么这三类数据值得单独讲?	95
2. 来龙去脉: 从通用语言建模到能力定向数据	95
3. 三类数据的共同特征	95
3.1 关键公式与审计指标	96
4. Code data: 代码数据为什么重要?	97
5. 代码数据从哪里来?	97
6. 代码数据清洗	97
7. 代码数据的结构化处理	98
8. 单测为什么重要?	98
9. 代码数据的风险	98
10. Math data: 数学数据为什么重要?	99
11. 数学数据从哪里来?	99
12. 数学数据清洗	99
13. 过程数据、答案数据和 verifier 数据	100
14. 数学数据的污染风险	100
15. Domain data: 专业领域数据为什么重要?	100
16. 专业领域数据来源	101
17. 专业领域数据清洗和治理	101
18. 医学、法律、金融数据的差异	101
18.1 医学数据	101
18.2 法律数据	101
18.3 金融数据	101
19. 专业数据与 RAG 的关系	102
20. 继续预训练 vs SFT vs RAG	102
21. 三类数据的配比策略	102
22. 面向专家: 专项数据改变模型的能力拓扑	102
23. 一个可落地的专项数据建设方案	103
23.1 最小可运行专项数据审计 demo	103
24. 常见面试题	106
24.1 代码数据为什么对大模型重要?	106
24.2 代码数据清洗重点是什么?	106
24.3 为什么单测对代码模型重要?	106
24.4 数学数据和普通文本有什么不同?	106
24.5 verifier 数据有什么价值?	106
24.6 专业领域数据应该怎么用?	107
24.7 为什么不能把医学法律金融数据直接大量混进预训练?	107
25. 常见误区	107
26. 本章小结	107

第七章: Synthetic Data 与 Distillation Data	107
0. 本讲资料边界与第二轮精修口径	108
1. 先建立直觉: 为什么要合成数据?	108
2. 来龙去脉: 从数据增强到大模型自举	108
3. Synthetic data 和 distillation data 的区别	109
3.1 关键公式与审计指标	109
4. 合成数据常见类型	110
5. 合成指令数据	110
6. Evol-Instruct: 让指令变复杂	111
7. 推理轨迹数据	111
8. Teacher-student 蒸馏	111
9. Distillation data 不等于复制能力	112
10. 合成数据生成 pipeline	112
11. 质量验证: 合成数据最关键的一步	113
12. 去重和多样性控制	113
13. 数据退化风险	113
14. 合成数据在不同训练阶段的用法	114
14.1 预训练	114
14.2 继续预训练	114
14.3 SFT	114
14.4 偏好训练	114
14.5 安全训练	114
15. 合成数据和真实用户数据	114
16. 合成数据配比	114
17. 面向专家: 合成数据是分布编辑	115
18. 一个可落地的合成与蒸馏数据方案	115
18.1 最小可运行合成与蒸馏数据审计 demo	115
19. 常见面试题	118
19.1 什么是 synthetic data?	118
19.2 什么是 distillation data?	118
19.3 Self-Instruct 的核心思想是什么?	118
19.4 合成数据有什么风险?	118
19.5 如何验证合成数据质量?	118
19.6 合成数据比例应该是多少?	118
19.7 蒸馏能不能让小模型超过 teacher?	119
20. 常见误区	119
21. 本章小结	119
第八章: Preference Data 与安全数据	119
0. 本讲资料边界与第二轮精修口径	120
1. 先建立直觉: 为什么只做 SFT 不够?	120
2. 来龙去脉: 从 RLHF 到 DPO	120
3. Preference data 到底是什么?	121
3.1 关键公式与审计指标	121
4. 偏好维度: helpful、honest、harmless	122
5. 偏好数据从哪里来?	122
6. chosen/rejected 构造方式	123
7. 偏好标注规范	123
8. 偏好数据的质量问题	123
9. 安全数据是什么?	124
10. 安全数据分层	124
11. 拒答数据	124
12. 边界允许数据	125
13. 红队数据	125
14. 误拒和漏拒	125
15. 偏好数据和安全数据的关系	126

16. 专业高风险领域安全数据	126
17. 多语言安全数据	126
18. LLM judge 在偏好和安全数据中的作用	127
19. 数据隐私与标注员安全	127
20. Preference data 的评估	127
21. 安全数据的评估	128
22. 面向专家: Preference data 是价值函数样本	128
23. 一个可落地的偏好与安全数据方案	128
23.1 最小可运行偏好与安全数据审计 demo	129
24. 常见面试题	131
24.1 什么是 preference data?	131
24.2 RLHF 和 DPO 对数据的要求有什么共同点?	131
24.3 偏好标注看哪些维度?	132
24.4 安全数据是不是只要拒答数据?	132
24.5 红队数据怎么用于训练?	132
24.6 如何平衡误拒和漏拒?	132
24.7 LLM judge 能否替代人工偏好标注?	132
25. 常见误区	132
26. 本章小结	132
第九章: 多模态、语音与视频数据	133
0. 本讲资料边界与第二轮精修口径	133
1. 先建立直觉: 多模态数据为什么难?	133
2. 来龙去脉: 从人工标注到 web-scale 弱监督	134
3. 多模态数据的基本类型	134
3.1 关键公式与审计指标	134
4. 图文对数据	135
5. 图文对质量过滤	136
6. Caption 数据	136
7. OCR 和文档图像数据	136
8. 图像数据去重和污染	137
9. 语音数据	137
10. 语音转录和时间对齐	138
11. 语音数据隐私和合规	138
12. 视频数据	138
13. 视频切片和时间对齐	139
14. 多模态指令数据	139
15. 多模态数据的安全问题	139
16. 版权和许可	140
17. 多模态质量评估	140
18. 面向专家: 多模态数据是对齐问题, 不是拼接问题	140
19. 一个可落地的多模态数据方案	141
19.1 最小可运行多模态数据审计 demo	141
20. 常见面试题	143
20.1 图文对数据最大的问题是什么?	143
20.2 caption 数据怎么评估质量?	144
20.3 OCR 数据为什么不能只存纯文本?	144
20.4 语音数据清洗重点是什么?	144
20.5 视频数据为什么比图像更难?	144
20.6 多模态数据有哪些隐私风险?	144
20.7 如何避免多模态模型学到幻觉?	144
21. 常见误区	144
22. 本章小结	145
第十章: Data Attribution 与 Valuation	145
0. 本讲资料边界与第二轮精修口径	145

1. 先建立直觉：为什么需要数据归因和估值？	145
2. 来龙去脉：从解释单点预测到评估数据资产	146
3. Data Attribution 和 Data Valuation 的区别	146
4. 为什么大模型数据价值很难精确计算？	146
4.1 关键公式与估值指标	147
5. 最朴素的方法：数据消融	148
6. 数据源级估值	148
7. 样本级估值	149
8. Influence functions: 从预测追溯训练点	149
9. Shapley value 和 Data Shapley	149
10. 近似方法：从精确估值到可用信号	150
11. 数据选择	150
12. 主动学习和标注优先级	150
13. 负价值数据	151
14. Attribution 在错误分析中的用途	151
15. 数据估值和 Data Mixture 的关系	151
16. 数据估值的工程指标	152
17. 大模型中的 practical attribution	152
18. 面向专家：数据价值是目标依赖的边际贡献	152
19. 一个可落地的数据归因和估值方案	153
19.1 最小可运行数据归因与估值 demo	153
20. 常见面试题	156
20.1 什么是 data attribution?	156
20.2 什么是 data valuation?	156
20.3 Influence functions 的核心思想是什么?	156
20.4 Data Shapley 的核心思想是什么?	156
20.5 大模型中如何实际评估数据源价值?	156
20.6 数据价值是不是一个固定分数?	156
20.7 如何发现负价值数据?	156
21. 常见误区	156
22. 本章小结	157

第十一章：Dataset Versioning 与 Governance

157

0. 本讲资料边界与第二轮精修口径	157
1. 先建立直觉：为什么数据需要版本?	157
2. 来龙去脉：从数据集文档到基础模型治理	158
3. Dataset versioning 到底版本化什么?	158
4. 数据 lineage: 数据从哪里来，到哪里去	158
5. 元数据 schema	159
5.1 关键公式与治理指标	159
6. 数据快照和不可变性	160
7. 数据 manifest	161
8. 权限控制	161
9. 删除请求和数据撤回	161
10. 数据审计	162
11. Datasheet: 给数据集写说明书	162
12. 数据治理和 model card 的关系	163
13. 数据版本和训练可复现	163
14. 数据 schema 演进	163
15. 数据治理中的角色分工	163
16. 数据治理指标	164
17. 面向专家：治理是模型行为控制面的底座	164
18. 一个可落地的数据版本与治理方案	164
18.1 最小可运行数据版本治理 demo	165
19. 常见面试题	167
19.1 为什么大模型数据需要 versioning?	167

19.2 什么是 data lineage?	167
19.3 manifest 有什么作用?	167
19.4 删除请求为什么难?	167
19.5 Datasheet for Datasets 解决什么问题?	167
19.6 数据治理和 model card 有什么关系?	168
19.7 如何设计数据权限?	168
20. 常见误区	168
21. 本章小结	168

第十二章：数据面试题

168

0. 本讲资料边界与第二轮精修口径	169
1. 面试总框架：大模型数据题怎么答?	169
1.1 关键公式与指标速查	169
2. 高频题一：如何构建大模型预训练数据集?	171
标准回答框架	171
面试加分点	171
3. 高频题二：如何做 web-scale 数据采集?	171
标准回答框架	171
常见追问	172
4. 高频题三：如何做数据清洗和质量评分?	172
标准回答框架	172
面试加分点	172
5. 高频题四：如何做去重?	172
标准回答框架	172
常见追问	173
6. 高频题五：如何检测 benchmark contamination?	173
标准回答框架	173
面试加分点	173
7. 高频题六：如何设计 data mixture?	173
标准回答框架	173
常见追问	173
8. 高频题七：如何建设代码数据?	173
标准回答框架	173
面试加分点	174
9. 高频题八：如何设计数学和 reasoning 数据?	174
标准回答框架	174
常见追问	174
10. 高频题九：如何建设 synthetic data?	174
标准回答框架	174
面试加分点	174
11. 高频题十：如何建设 preference data?	175
标准回答框架	175
常见追问	175
12. 高频题十一：如何建设安全数据?	175
标准回答框架	175
合规提醒	175
13. 高频题十二：如何建设多模态数据?	175
标准回答框架	175
常见追问	176
14. 高频题十三：如何评估数据价值?	176
标准回答框架	176
面试加分点	176
15. 高频题十四：如何做 dataset versioning 和 governance?	176
标准回答框架	176
常见追问	176
16. 系统设计题：给你 100TB 原始 web 数据，如何做成训练集?	176

推荐回答	176
深挖回答	177
17. 系统设计题：如何构建一个代码大模型的数据体系？	177
推荐回答	177
18. 系统设计题：如何构建一个医疗领域模型的数据体系？	177
推荐回答	177
19. 行为面试题：数据策略和模型效果冲突怎么办？	177
推荐回答	177
20. 专家追问：为什么数据质量比数据规模更重要？	178
21. 专家追问：为什么数据问题会表现成模型问题？	178
22. 专家追问：小模型数据实验能外推到大模型吗？	178
23. 专家追问：如何避免数据工程过度优化 benchmark？	178
24. 专家追问：数据治理会不会拖慢训练迭代？	178
25. 开放题：如果让你从零搭建数据团队，你怎么分工？	179
26. 面试中的常见错误表达	179
27. 最后一套万能回答模板	179
27.1 最小可运行数据面试复盘 demo	180
28. 本章小结	182

第九册简介：大模型数据工程与数据智能

本书定位

第九册专门讲大模型数据，目标是理解数据如何决定模型能力，以及如何构建高质量、可治理、可评估的数据体系。

为什么需要这本书

大模型能力不是只由架构决定，数据质量、数据配比、去重、污染控制、合成数据和偏好数据都会决定最终行为。许多训练和评估问题，本质上都是数据问题。

适合读者

1. 想准备数据工程、训练数据、后训练数据和评估数据相关岗位的读者。
2. 需要构建 SFT、DPO、RAG、多模态或安全数据集的人。
3. 想理解数据价值评估和数据治理的人。

学完后应具备的能力

1. 能设计 web-scale 数据采集、清洗、过滤和去重流程。
2. 能分析 benchmark contamination 和数据泄漏。
3. 能设计 data mixture、代码/数学/领域数据和多模态数据配比。
4. 能构造 preference data、安全数据和合成数据。
5. 能做 dataset versioning、governance 和 data valuation。

阅读建议

建议和第五册训练全流程、第七册评估、第八册安全一起学习。

与其他书的关系

第九册是训练、对齐、评估、RAG、多模态和安全专题的数据基础。

第九册：大模型数据工程与数据智能

本书定位

本书专门介绍大模型数据。目标是理解数据如何决定模型能力，以及如何构建高质量、可治理、可评估的数据体系。

章节文件

1. chapters/01-数据总览.md
2. chapters/02-web-scale 数据采集.md
3. chapters/03-清洗过滤与质量评分.md
4. chapters/04-去重与污染检测.md
5. chapters/05-data-mixture 与配比.md
6. chapters/06-code-math-domain-data.md
7. chapters/07-synthetic-data 与 distillation-data.md
8. chapters/08-preference-data 与安全数据.md
9. chapters/09-多模态语音视频数据.md
10. chapters/10-data-attribution 与 valuation.md
11. chapters/11-dataset-versioning 与 governance.md
12. chapters/12-数据面试题.md

第一章：数据总览

重点：数据是大模型能力来源，数据规模、质量、覆盖、配比和治理共同决定训练上限。

面试重点：不要把大模型训练理解成“有足够 GPU 就行”。模型能力、行为、安全、评估可信度和上线风险，都强烈依赖数据工程。

本章资料边界

本章第二轮精修前，按 WRITING_PLAN.md 联网核对了 GPT-3、Chinchilla、phi-1 / Textbooks Are All You Need、Dolma、RefinedWeb、C4 / T5、训练数据去重论文和 NIST AI RMF / Generative AI Profile 等资料边界。

本章定位为第九册的数据工程总览章。重点是把数据规模、质量、覆盖、配比、去重、污染控制、PII、许可、版本和治理串成一套面试可讲、工程可落地的框架。

本章不展开完整爬虫系统、数据湖架构、版权法细节、生产级 PII 检测模型、复杂 data attribution 算法或特定闭源模型的数据配方。后续章节会分别展开 web-scale 采集、质量评分、去重污染、data mixture、合成数据、偏好数据、多模态数据和数据治理。

本章目标

学完本章，你要能回答：

1. 为什么说数据决定大模型能力上限？
2. 数据规模、数据质量、数据覆盖、数据配比分别影响什么？
3. 从 GPT-3 到 Chinchilla，再到高质量小模型路线，数据观念发生了什么变化？
4. 预训练数据、SFT 数据、偏好数据、评估数据、RAG 数据有什么区别？
5. 大模型数据工程的完整 pipeline 是什么？
6. 数据问题如何表现为训练问题、评估问题、安全问题和产品问题？
7. 面试中如何系统回答“大模型数据工程怎么做”？

1. 来龙去脉：大模型为什么越来越重视数据

1.1 早期深度学习：数据是燃料

在传统深度学习时代，大家就知道数据重要。

ImageNet 让视觉模型发生质变。

大规模语料让词向量、语言模型和机器翻译显著进步。

但那个阶段，很多讨论仍集中在：

1. 模型结构。
2. 损失函数。
3. 优化器。
4. 正则化。
5. 训练技巧。

数据常被看成“给模型喂的材料”，但还没有形成今天这种系统工程视角。

1.2 GPT-3 时代：规模和 web-scale 数据

GPT-3 证明了大规模自回归语言模型在海量文本上预训练后，可以获得强 few-shot 能力。

这个阶段的重要启发是：

模型规模 + 数据规模 + 计算规模，可以带来涌现式能力提升。

因此，web-scale 数据采集、清洗和训练变成核心能力。

但 GPT-3 时代也暴露了问题：

1. 数据来源混杂。
2. 质量差异巨大。
3. 训练集中可能有偏见、有害内容和隐私信息。
4. benchmark contamination 难控制。
5. 数据配比不透明。
6. 低质量数据会消耗大量训练 compute。

1.3 Chinchilla 时代：模型不是越大越好，token 也要跟上

Chinchilla 论文系统研究了给定 compute budget 下模型参数量和训练 token 数的关系。

它的重要启发是：很多早期大模型相对“数据不足”，即模型很大，但训练 token 不够。

更 compute-optimal 的做法是让模型大小和训练 token 数一起增长。

这改变了很多团队的训练观念：

不是只堆参数，数据 token 数和质量同样关键。

1.4 高质量数据时代：Textbooks Are All You Need 的启发

phi-1 相关工作展示了另一条思路：在特定任务上，高质量、教材式、结构化、合成增强的数据，可能让较小模型获得很强能力。

这并不意味着“只要教材就够了”。

更准确的启发是：

数据质量、任务匹配和数据组织方式，可以显著影响模型效率和能力形成。

1.5 今天的数据观念

今天的大模型数据工程不再只是“爬更多网页”。

而是综合考虑：

1. 数据规模。
2. 数据质量。
3. 数据覆盖。
4. 数据配比。
5. 去重和污染控制。
6. 合成数据。
7. 偏好数据。
8. 安全数据。
9. 多模态数据。
10. 数据版本和治理。

这也是第九册要解决的问题。

2. 小白例子：训练模型像培养学生

把模型想象成一个学生。

训练数据就是他的教材、作业、考试和老师反馈。

如果教材很多但质量差，学生会学到错误知识。

如果教材只覆盖一个领域，学生会偏科。

如果作业里都是简单题，学生不会难题。

如果考试题提前泄露，分数不可信。

如果老师反馈有偏，学生会学会讨好老师而不是真懂。

对应到大模型：

1. 预训练数据像教材。
2. SFT 数据像示范答案。
3. 偏好数据像老师打分。
4. 评估数据像考试。
5. RAG 数据像开卷资料。
6. 安全数据像行为规范训练。

所以数据工程不是后勤工作，而是模型能力和行为塑造的核心。

3. 大模型数据的五个核心维度

3.1 规模

规模指 token 数、样本数、文档数和模态数据量。

数据规模影响：

1. 语言覆盖。
2. 世界知识覆盖。
3. 长尾概念学习。
4. 模型泛化。
5. Scaling law。

但规模不是越大越好。

如果数据质量差，大规模只会放大噪声。

3.2 质量

质量包括：

1. 正确性。
2. 可读性。
3. 信息密度。
4. 结构清晰度。
5. 领域可靠性。
6. 有害内容比例。
7. 重复程度。

高质量数据能提高训练效率。

低质量数据会导致：

1. 幻觉。
2. 格式混乱。
3. 有害行为。
4. 错误知识。
5. 训练 compute 浪费。

3.3 覆盖

覆盖指数据是否覆盖目标能力和目标用户场景。

例如：

1. 多语言。
2. 代码。
3. 数学。
4. 科学。
5. 法律。
6. 医疗。
7. 对话。
8. 工具调用。
9. 多模态。

覆盖不足会导致模型偏科。

3.4 配比

配比指不同数据类型在训练中的比例。

例如：

1. Web 文本多少。
2. 代码多少。
3. 数学多少。
4. 中文多少。
5. 高质量合成数据多少。
6. 安全数据多少。

配比影响模型能力结构。

代码比例高，模型代码能力可能更强，但可能影响自然语言分布。

安全数据比例过高，可能 over-refusal。

3.5 治理

治理指数据是否可追溯、可审计、可删除、可复现、可合规。

包括：

1. 数据来源。
2. 许可证。
3. PII 处理。
4. 去重策略。
5. 数据版本。
6. 评估集隔离。
7. 删除请求。
8. 安全审计。

没有治理，模型越大风险越大。

3.6 数据工程公式速查

数据工程面试中，不一定要推复杂统计公式，但要能把数据口径说成可度量、可验收的指标。

把一个数据样本抽象成：

$$d_i=(x_i,n_i,q_i,z_i,l_i,r_i)$$

其中 x_i 是文本、代码、图片、音频或多模态内容， n_i 是 token 数或样本权重， q_i 是质量分， z_i 是领域、语言、任务、来源等元数据， l_i 是许可或合规标签， r_i 是风险标签。

过滤后的 token 保留率：

$$R_{\{\mathrm{keep}\}}=\frac{\sum_i n_i g_i}{\sum_i n_i}$$

其中 g_i 是样本是否通过质量、安全、许可、PII、污染等门禁的 0/1 标记。保留率太低可能说明过滤过狠，保留率太高也可能说明质量门禁太松。

加权质量分：

$$Q_{\{\mathrm{data}\}}=\frac{\sum_i n_i q_i g_i}{\sum_i n_i g_i}$$

数据配比：

$$p_k=\frac{\sum_i n_i g_i \cdot \mathbb{I}[z_i=k]}{\sum_i n_i g_i}$$

其中 k 可以是 code、math、zh、safety、medical、RAG docs 等领域或任务切片。

覆盖率可以粗略写成：

$$C_{\{\mathrm{cov}\}}=\frac{\sum_k \mathbb{I}[T_k \geq \tau_k]}{K}$$

其中 T_k 是切片 k 的保留 token 数， τ_k 是该切片的最低覆盖阈值， K 是目标切片数量。

重复率：

$$R_{\{\mathrm{dup}\}}=1-\frac{N_{\{\mathrm{unique}\}}}{N_{\{\mathrm{raw}\}}}$$

评估污染风险：

$$R_{\{\mathrm{contam}\}}=\frac{1}{M} \sum_{j=1}^M \mathbb{I}[\text{left[}$$

$$\max_i \mathrm{sim}(e_j, d_i) \geq \tau_{\mathrm{sim}} \\ \text{\texttt{\textbackslash right}}$$

其中 e_j 是评估样本, d_i 是训练样本, sim 可以是 n-gram overlap、Jaccard、embedding similarity 或更强的改写检测模型。

数据上线门禁可以抽象成：

$$G_{\mathrm{data}} = \\ \mathbb{I}[Q_{\mathrm{data}} \geq \tau_q], \\ \mathbb{I}[R_{\mathrm{dup}} \leq \tau_{\mathrm{dup}}], \\ \mathbb{I}[R_{\mathrm{contam}} \leq \tau_c], \\ \mathbb{I}[R_{\mathrm{pii}} \leq \tau_p], \\ \mathbb{I}[R_{\mathrm{license}} \leq \tau_l], \\ \mathbb{I}[C_{\mathrm{cov}} \geq \tau_{\mathrm{cov}}]$$

这组公式的核心价值不是“算一个漂亮总分”，而是强迫你把数据工程从经验判断变成可审计口径：保留了多少、质量怎样、覆盖哪些能力、重复和污染有多高、PII 和许可风险是否过线。

4. 大模型数据类型全景图

4.1 预训练数据

目标：学习语言、知识、代码、世界模式和基础能力。

来源可能包括：

1. Web 文本。
2. 书籍。
3. 论文。
4. 代码。
5. 问答。
6. 百科。
7. 多语言语料。

关键问题：

1. 数据质量。
2. 去重。
3. PII。
4. 版权。
5. 毒性和偏见。
6. 评估污染。

4.2 SFT 数据

目标：教模型按照指令回答。

特点：

1. 数量比预训练小得多。
2. 质量要求高。
3. 格式和 chat template 很重要。
4. 覆盖任务类型和回答风格。

SFT 数据影响模型“像不像助手”。

4.3 偏好数据

目标：让模型学会人类偏好。

形式：

prompt + chosen answer + rejected answer

用于 RLHF、DPO、RLAIF 等。

关键问题：

1. 标注偏差。
2. 长度偏差。
3. Safety/helpfulness trade-off。
4. Reward hacking。
5. 多轮偏好。

4.4 安全数据

目标：让模型理解风险边界。

包括：

1. 应该拒绝的请求。
2. 正常安全教育请求。
3. 边界请求。
4. 多轮诱导。
5. Prompt injection。
6. 隐私保护。
7. 安全替代回答。

安全数据质量直接影响 over-refusal 和 harmful compliance。

4.5 评估数据

目标：衡量模型能力和风险。

关键要求：

1. 不能污染训练。
2. 分层清晰。
3. 有标准答案或 rubric。
4. 能做 error analysis。
5. 能做 regression。

评估数据不是训练数据。

混用会导致 benchmark contamination。

4.6 RAG 数据

目标：给模型提供外部知识。

关键问题：

1. 文档切分。
2. 元数据。
3. 权限控制。
4. 更新机制。
5. 检索质量。
6. 引用和 faithfulness。
7. Prompt injection。

4.7 多模态数据

包括：

1. 图文对。
2. 图像问答。
3. OCR 数据。
4. 视频字幕。
5. 音频转写。
6. 语音对话。
7. 图表和文档。

多模态数据难点是对齐、质量和标注成本。

5. 大模型数据 Pipeline

一个典型 pipeline：

```
data sourcing
-> crawling / collection
-> parsing
-> language identification
-> normalization
-> filtering
-> deduplication
-> PII and safety processing
-> quality scoring
-> domain classification
-> mixture design
-> tokenization
-> packing / sharding
-> training
-> evaluation and attribution
-> versioning and governance
```

5.1 数据来源

先决定从哪里来。

包括：

1. 公开网页。
2. 许可数据。
3. 代码仓库。
4. 企业文档。
5. 合成数据。
6. 人工标注数据。

5.2 解析和规范化

原始数据往往很脏。

需要处理：

1. HTML。
2. PDF。
3. Markdown。
4. 表格。
5. 代码。

6. 多语言编码。
7. boilerplate。

5.3 过滤和质量评分

过滤低质量、重复、垃圾、营销噪声、乱码、有害和隐私数据。

质量评分可以用规则、分类器、模型或混合方法。

5.4 去重

去重非常关键。

重复数据会导致：

1. 训练效率下降。
2. 记忆风险增加。
3. 评估污染。
4. 某些来源过度影响模型。

5.5 配比和采样

不同数据源要按目标能力配比。

这不是静态问题，而是需要实验迭代。

5.6 版本管理

数据必须可复现。

否则训练结果无法解释。

6. 数据问题如何变成模型问题

6.1 幻觉

可能来自：

1. 训练数据错误。
2. 知识冲突。
3. 高质量事实数据不足。
4. SFT 奖励完整回答而不是不确定性。

6.2 过度拒答

可能来自：

1. 安全数据过于粗糙。
2. 负样本缺少正常安全教育请求。
3. 偏好数据奖励保守拒答。

6.3 代码能力弱

可能来自：

1. 代码数据比例不足。
2. 低质量代码多。
3. 缺少测试和解释。
4. 缺少 repo-level 数据。

6.4 数学推理弱

可能来自：

1. 数学数据覆盖不足。
2. 步骤错误。
3. 合成数据质量差。
4. 缺少可验证答案。

6.5 评估分数虚高

可能来自：

1. Benchmark 泄漏。
2. 训练集包含测试题。
3. 反复用同一评估集调参。
4. 评估集被合成数据污染。

7. 数据质量的层次

7.1 文本层质量

1. 是否乱码。
2. 是否重复。
3. 是否语法清晰。
4. 是否包含 boilerplate。
5. 是否营销噪声或垃圾内容。

7.2 内容层质量

1. 是否事实正确。
2. 是否信息密度高。
3. 是否逻辑清楚。
4. 是否有专业来源。
5. 是否过时。

7.3 任务层质量

1. 是否适合目标任务。
2. 是否有明确输入输出。
3. 是否覆盖难度梯度。
4. 是否能验证。

7.4 行为层质量

1. 是否符合助手行为。
2. 是否诚实表达不确定性。
3. 是否安全。
4. 是否不过度拒答。
5. 是否风格稳定。

8. 数据配比为什么难

数据配比不是简单“多放好数据”。

原因：

1. 不同能力互相竞争。

2. 领域数据可能影响通用能力。
3. 安全数据过多可能 over-refusal。
4. 代码数据影响语言风格。
5. 合成数据可能模式单一。
6. 多语言数据需要平衡。

配比设计需要实验。

常见方法：

1. 小规模 proxy training。
2. Ablation。
3. 分层评估。
4. Data attribution。
5. Curriculum。
6. 动态采样。

9. 数据治理为什么是核心能力

没有治理，数据工程会不可控。

9.1 可追溯

知道数据从哪里来。

9.2 可复现

知道训练用了哪个版本。

9.3 可删除

能响应版权、隐私和合规要求。

9.4 可评估

知道哪些数据带来收益或风险。

9.5 可审计

能解释模型出了问题时数据链路在哪里。

10. 最小代码：数据集审计和上线门禁

下面这个 0 依赖 demo 用一个 toy 数据集模拟数据总览章的核心审计流程。输入是 10 条候选训练样本和 3 条评估题；输出包括 token 保留率、加权质量分、领域配比、覆盖率、重复率、污染风险、PII、许可缺口和上线门禁。

它演示的重点是：一个数据集即使平均质量分不错，也可能因为评估污染、PII 或许可问题不能直接进入训练。

```
import re
from collections import defaultdict
```

```
docs = [
    {
        "id": "web_intro",
        "domain": "web",
        "tokens": 1200,
        "quality": 0.82,
```

```

    "license": "ok",
    "split": "train",
    "text": "Transformer attention tutorial with examples and clear explanation.",
  },
  {
    "id": "web_intro_dup",
    "domain": "web",
    "tokens": 1180,
    "quality": 0.80,
    "license": "ok",
    "split": "train",
    "text": "Transformer attention tutorial with examples and clear explanation.",
  },
  {
    "id": "code_tests",
    "domain": "code",
    "tokens": 900,
    "quality": 0.91,
    "license": "permissive",
    "split": "train",
    "text": "Python function solve array problem with unit tests and edge cases.",
  },
  {
    "id": "math_steps",
    "domain": "math",
    "tokens": 700,
    "quality": 0.88,
    "license": "ok",
    "split": "train",
    "text": "Algebra word problems with step by step reasoning and verified answers.",
  },
  {
    "id": "safety_refusal",
    "domain": "safety",
    "tokens": 520,
    "quality": 0.74,
    "license": "ok",
    "split": "train",
    "text": "Safe assistant examples for privacy boundary and refusal with alternatives.",
  },
  {
    "id": "spam_page",
    "domain": "web",
    "tokens": 480,
    "quality": 0.32,
    "license": "ok",
    "split": "train",
    "text": "Keyword stuffed boilerplate navigation repeated repeated repeated.",
  },
  {
    "id": "pii_note",
    "domain": "web",
    "tokens": 360,
    "quality": 0.68,
    "license": "ok",

```

```

        "split": "train",
        "text": "Customer support transcript contact alice@example.com invoice 1380.",
    },
    {
        "id": "license_gap",
        "domain": "medical",
        "tokens": 610,
        "quality": 0.86,
        "license": "unknown",
        "split": "train",
        "text": "Clinical guideline summary with unclear redistribution permission.",
    },
    {
        "id": "eval_leak",
        "domain": "math",
        "tokens": 240,
        "quality": 0.90,
        "license": "ok",
        "split": "train",
        "text": "Benchmark question what is 17 plus 25 answer 42 with final answer.",
    },
    {
        "id": "zh_dialog",
        "domain": "zh",
        "tokens": 760,
        "quality": 0.77,
        "license": "ok",
        "split": "train",
        "text": "中文多轮对话示例 覆盖 工具 使用 和 回答 风格。",
    },
],

eval_prompts = [
    "what is 17 plus 25",
    "explain transformer attention heads",
    "privacy boundary refusal",
]

def normalize(text):
    return " ".join(re.findall(r"[a-z0-9\u4e00-\u9fff]+", text.lower()))

def words(text):
    return set(normalize(text).split())

def jaccard(a, b):
    wa, wb = words(a), words(b)
    return len(wa & wb) / max(1, len(wa | wb))

seen = set()
reports = []
for doc in docs:

```



```

norm = normalize(doc["text"])
exact_dup = norm in seen
seen.add(norm)

max_eval_sim = max(jaccard(doc["text"], prompt) for prompt in eval_prompts)
pii = bool(re.search(r"[\w.-]+@[ \w.-]+", doc["text"]))
license_ok = doc["license"] in {"ok", "permissive"}
low_quality = doc["quality"] < 0.60
contaminated = max_eval_sim >= 0.40
keep = not (exact_dup or pii or not license_ok or low_quality or contaminated)

reports.append(
    {
        **doc,
        "exact_dup": exact_dup,
        "pii": pii,
        "license_ok": license_ok,
        "low_quality": low_quality,
        "contaminated": contaminated,
        "keep": keep,
    }
)

raw_tokens = sum(doc["tokens"] for doc in reports)
kept_docs = [doc for doc in reports if doc["keep"]]
kept_tokens = sum(doc["tokens"] for doc in kept_docs)

raw_quality = sum(doc["quality"] * doc["tokens"] for doc in reports) / raw_tokens
kept_quality = sum(doc["quality"] * doc["tokens"] for doc in kept_docs) / kept_tokens

tokens_by_domain = defaultdict(int)
for doc in kept_docs:
    tokens_by_domain[doc["domain"]] += doc["tokens"]

mixture = {
    domain: round(tokens / kept_tokens, 3)
    for domain, tokens in sorted(tokens_by_domain.items())
}

required_domains = {"web", "code", "math", "safety", "zh"}
covered = sorted(required_domains & set(tokens_by_domain))
missing = sorted(required_domains - set(tokens_by_domain))

risk_rates = {
    "exact_dup": sum(doc["exact_dup"] for doc in reports) / len(reports),
    "contamination": sum(doc["contaminated"] for doc in reports) / len(reports),
    "pii": sum(doc["pii"] for doc in reports) / len(reports),
    "license_gap": sum(not doc["license_ok"] for doc in reports) / len(reports),
    "low_quality": sum(doc["low_quality"] for doc in reports) / len(reports),
}

failed_ids = {
    "duplicate": [doc["id"] for doc in reports if doc["exact_dup"]],
    "contamination": [doc["id"] for doc in reports if doc["contaminated"]],
    "pii": [doc["id"] for doc in reports if doc["pii"]],
}

```

```

    "license": [doc["id"] for doc in reports if not doc["license_ok"]],
    "low_quality": [doc["id"] for doc in reports if doc["low_quality"]],
}

gates = {
    "retention": kept_tokens / raw_tokens >= 0.55,
    "quality": kept_quality >= 0.78,
    "coverage": not missing,
    "dup": risk_rates["exact_dup"] <= 0.15,
    "contamination": risk_rates["contamination"] == 0.0,
    "pii": risk_rates["pii"] == 0.0,
    "license": risk_rates["license_gap"] == 0.0,
}

print(f"raw_tokens={raw_tokens} kept_tokens={kept_tokens} retention={kept_tokens / raw_tokens:.3f}")
print(f"quality raw={raw_quality:.3f} kept={kept_quality:.3f}")
print(f"mixture={mixture}")
print(f"coverage={{'covered':{covered}, 'missing':{missing}}}")
print(
    "risk_rates={"
    + ", ".join(f"'{name}':{rate:.3f}" for name, rate in risk_rates.items())
    + "}"
)
print(f"failed_ids={failed_ids}")
print(f"gates={gates}")
print(f"data_ready={all(gates.values())}")

```

典型输出：

```

raw_tokens=6950 kept_tokens=4080 retention=0.587
quality raw=0.787 kept=0.831
mixture={'code': 0.221, 'math': 0.172, 'safety': 0.127, 'web': 0.294, 'zh': 0.186}
coverage={'covered': ['code', 'math', 'safety', 'web', 'zh'], 'missing': []}
risk_rates={'exact_dup': 0.100, 'contamination': 0.100, 'pii': 0.100, 'license_gap': 0.100, 'low_quality': 0.100}
failed_ids={'duplicate': ['web_intro_dup'], 'contamination': ['eval_leak'], 'pii': ['pii_note'], 'license': ['license_g
gates={'retention': True, 'quality': True, 'coverage': True, 'dup': True, 'contamination': False, 'pii': False, 'license'
data_ready=False

```

这个例子里，保留样本的加权质量分从 0.787 提升到 0.831，说明质量过滤确实有收益；但 data_ready=False，因为评估污染、PII 和许可门禁没有通过。真实项目里也一样，不能只用“质量分变高了”证明数据可以训练，必须同时看污染隔离、隐私、许可、覆盖和版本记录。

11. 面向专家：数据不是静态资产，而是控制面

很多人把数据看成训练前准备好的静态资产。

更成熟的理解是：数据是控制模型能力和行为的控制面。

你可以通过数据控制：

1. 模型学什么语言。
2. 擅长什么任务。
3. 如何回答用户。
4. 是否诚实表达不确定。
5. 如何拒绝危险请求。
6. 是否引用证据。
7. 是否会调用工具。
8. 是否偏向某种风格。

因此，数据工程和 alignment、evaluation、safety、product 都是连接的。

12. 真实项目中的数据团队分工

一个大模型数据团队可能包括：

1. 数据采集工程。
2. 数据清洗和过滤。
3. 质量评分模型。
4. 数据混合和采样。
5. 合成数据生成。
6. 标注平台。
7. 偏好数据团队。
8. 安全数据团队。
9. 数据治理和合规。
10. 数据评估和 attribution。

数据团队不是简单 ETL 团队，而是模型能力团队的一部分。

13. 面试官会怎么问

问题 1：为什么数据决定模型能力？

回答要点：

1. 预训练数据决定基础语言 and 知识覆盖。
2. SFT 数据决定指令跟随和助手风格。
3. 偏好数据决定对齐行为。
4. 安全数据决定风险边界。
5. 评估数据决定模型选择是否可信。
6. RAG 数据决定外部知识质量。

标准回答：

大模型能力不是只由参数和算力决定。预训练数据决定模型能学到什么语言、知识和模式；数据质量决定训练效率和错误知识；数据

问题 2：Chinchilla 对数据工程有什么启发？

回答要点：

1. 给定 compute 下，参数量和 token 数要共同 scaling。
2. 早期很多模型 undertrained。
3. 数据 token 数不是附属项，而是 compute-optimal 训练的核心变量。
4. 需要高质量、大规模、可治理的数据管线。

问题 3：高质量数据为什么重要？

回答要点：

1. 提高训练效率。
2. 减少错误知识和噪声。
3. 对小模型尤其关键。
4. 影响模型风格、推理和安全行为。
5. 但高质量数据通常更贵、更难规模化。

问题 4：预训练数据、SFT 数据、偏好数据有什么区别？

回答要点：

1. 预训练数据学基础语言和知识。
2. SFT 数据教模型按指令回答。
3. 偏好数据教模型区分好坏回答。
4. 三者数据量、格式、质量要求和风险不同。

问题 5：数据问题如何导致安全问题？

回答要点：

1. 有害内容进入预训练。
2. 安全样本覆盖不足。
3. 拒答数据过粗导致 over-refusal。
4. 隐私数据导致泄露。
5. Prompt injection 样本不足导致 RAG/Agent 风险。
6. 偏好数据奖励看似安全但无用的回答。

14. 标准回答模板

面试中可以这样回答：

我会把大模型数据工程看成从数据来源到模型行为的全链路系统。预训练阶段需要 web-scale 数据采集、解析、语言识别、清洗、过滤。

数据的核心维度包括规模、质量、覆盖、配比和治理。规模决定上限，质量决定效率，覆盖决定能力边界，配比决定能力结构，治理决定安全。

15. 常见误区

15.1 误区：数据越多越好

纠正：低质量、重复和污染数据会浪费 compute，增加记忆和偏差风险。

15.2 误区：清洗就是删垃圾文本

纠正：清洗还包括去重、PII、版权、安全、质量评分、领域分类和评估污染控制。

15.3 误区：SFT 数据越多越好

纠正：SFT 更看质量、覆盖和格式一致性。低质量 SFT 会毁掉模型行为。

15.4 误区：评估数据也是数据，可以混着用

纠正：评估数据必须隔离，否则 benchmark contamination 会让分数失真。

15.5 误区：数据工程只是训练前工作

纠正：数据贯穿训练、后训练、评估、RAG、上线监控和治理全周期。

16. 小练习

练习 1

画出一个大模型数据 pipeline。

要求包含：采集、解析、过滤、去重、PII、质量评分、配比、tokenization、版本管理。

练习 2

解释 GPT-3、Chinchilla、phi-1 对数据观念的不同启发。

练习 3

设计一个 SFT 数据质量检查表。

要求覆盖：任务覆盖、格式、答案质量、安全边界、多轮和拒答。

练习 4

给出 5 个数据问题导致模型问题的例子。

练习 5

设计一个数据治理方案。

要求覆盖：数据来源、许可、版本、PII、评估隔离、删除请求和审计。

17. 本章总结

数据是大模型能力、行为和安全的核心来源。

大模型数据工程经历了从“更多 web-scale 数据”，到“compute-optimal token 数”，再到“高质量、任务匹配、可治理数据”的演化。

数据的五个核心维度是规模、质量、覆盖、配比和治理。

预训练数据、SFT 数据、偏好数据、安全数据、评估数据、RAG 数据和多模态数据分别塑造不同能力和风险。

大模型数据 pipeline 包括采集、解析、过滤、去重、PII、安全处理、质量评分、配比、tokenization、版本和治理。

很多训练问题、评估问题、安全问题和产品问题，本质上都是数据问题。

面试中要把数据工程讲成模型能力和行为控制系统，而不是简单 ETL。

第二章：Web-Scale 数据采集

重点：网页、书籍、论文、代码、论坛、对话、多语言数据来源和法律合规问题。

面试重点：Web-scale 数据采集不是“写爬虫把网页都扒下来”，而是一个涉及数据来源、解析、质量、版权、隐私、反滥用、可追溯和后续训练目标的系统工程。

合规边界：本章讨论合法合规的数据采集、公开数据集、数据治理和工程流程，不提供绕过访问控制、规避反爬、抓取未授权内容或违反网站条款的操作方法。

本章目标

学完本章，你要能回答：

1. Web-scale 数据采集为什么是大模型训练的基础？
2. 大模型常见数据来源有哪些？
3. 网页、书籍、论文、代码、论坛、对话、多语言数据各有什么特点？
4. Common Crawl、The Pile 这类公开语料给了我们什么启发？
5. Web 数据采集 pipeline 应该如何设计？
6. 如何处理版权、隐私、robots、Terms of Service、数据许可和合规？
7. 如何做数据来源记录和版本治理？
8. 面试中如何回答“如何从零构建 web-scale 训练数据集”？

0. 本讲资料边界与第二轮精修口径

按照 WRITING_PLAN.md 的要求, 本讲精修前核对了 Common Crawl 的 WARC / WAT / WET 数据形态、IETF RFC 9309 的 Robots Exclusion Protocol、T5 / C4、The Pile、RefinedWeb、FineWeb、Dolma 和 DataComp-LM / DCLM 等公开资料。

本讲聚焦大模型训练数据采集的工程闭环:

数据源规划 -> 合规审查 -> 公开或授权采集 -> 原始存储 -> 解析 -> 过滤 -> 去重 -> 配比 -> 版本审计

本讲不提供绕过登录、破解访问控制、规避反爬、批量抓取未授权内容或违反网站条款的做法。法律、版权和隐私问题在真实项目中需要法务、安全、隐私和业务团队共同确认; 面试中也不应把技术可访问性说成 “天然可用于训练”。

1. 来龙去脉: 为什么 Web 成了大模型的数据源

1.1 早期 NLP 的数据规模很小

早期 NLP 任务常用人工标注数据集。

例如:

1. 情感分类数据集。
2. 机器翻译平行语料。
3. 问答数据集。
4. 命名实体识别数据集。
5. 文本分类 benchmark。

这些数据集质量高, 但规模有限。

它们适合训练专项模型, 不适合训练通用语言模型。

1.2 预训练改变了数据需求

预训练语言模型不再只学一个任务。

它要从海量文本中学习:

1. 语言结构。
2. 世界知识。
3. 常识。
4. 代码模式。
5. 多语言表达。
6. 文档格式。
7. 推理模式。

这需要非常大的语料。

互联网自然成为主要来源。

1.3 GPT-3 之后: Web-scale 变成标配

GPT-3 展示了大规模语言模型在海量语料上训练后可以获得强 few-shot 能力。

这让业界意识到:

通用模型能力来自大规模、多样化、跨领域文本。

但互联网数据不是天然干净。

它包含:

1. 高质量文章。
2. 论坛讨论。

3. 广告。
4. SEO 垃圾。
5. 重复页面。
6. 错误事实。
7. 有害内容。
8. 个人信息。
9. 版权内容。

所以 Web-scale 采集之后，真正难的是筛选和治理。

1.4 The Pile 的启发

The Pile 是一个由多种高质量子集构成的大规模英文文本语料，强调多样性和高质量来源。

它的重要启发不是“照搬某个数据集”，而是：

1. 数据来源需要分类型组织。
2. 不同子集代表不同能力和领域。
3. 数据集构建过程需要文档化。
4. 数据本身也需要分析潜在风险。
5. 单一 web crawl 不足以代表全部高质量知识。

1.5 今天的观点

今天更成熟的观点是：

Web-scale 数据采集只是起点，真正的壁垒是合法来源、质量筛选、覆盖设计、合规治理、版本复现和持续评估。

2. 小白例子：做一座城市图书馆

构建大模型数据集像建一座城市图书馆。

你不能只把所有纸都搬进来。

你需要决定：

1. 哪些书可以收。
2. 哪些书版权允许。
3. 哪些书质量高。
4. 哪些书重复。
5. 哪些书有隐私信息。
6. 哪些书适合儿童区。
7. 哪些书需要标注来源。
8. 哪些书过时。
9. 哪些书属于专业馆藏。

Web-scale 数据也是这样。

采集只是“搬书”。

数据工程是“建馆、分类、审查、维护和更新”。

3. 常见数据来源

3.1 网页数据

网页是最大的数据来源之一。

优点：

1. 规模大。

2. 领域广。
3. 更新快。
4. 多语言丰富。
5. 包含真实用户表达。

缺点：

1. 噪声大。
2. 重复多。
3. 广告和导航多。
4. SEO 内容多。
5. 版权和许可复杂。
6. PII 风险高。

3.2 书籍

书籍通常质量高、结构完整。

适合学习：

1. 长文结构。
2. 系统知识。
3. 叙事能力。
4. 专业表达。

风险：

1. 版权复杂。
2. 获取渠道限制。
3. 数据格式多样。
4. OCR 错误。

3.3 论文和技术文档

优点：

1. 专业性强。
2. 信息密度高。
3. 适合科学和技术能力。
4. 结构规范。

风险：

1. 公式、图表、引用解析困难。
2. PDF 解析噪声。
3. 许可证差异。
4. 学术文本和普通用户语言风格不同。

3.4 代码数据

代码数据是模型编程能力的基础。

来源包括：

1. 开源仓库。
2. 文档示例。
3. 教程。
4. 问答网站。
5. 单元测试。

关键问题：

1. 许可证。
2. 重复和 fork。
3. 生成代码质量。
4. 安全漏洞。
5. 密钥泄露。
6. 多文件上下文。

3.5 论坛和问答

优点：

1. 问题真实。
2. 对话自然。
3. 包含 debug 和经验。
4. 覆盖长尾问题。

风险：

1. 答案质量不稳定。
2. 可能过时。
3. 可能包含攻击性语言。
4. 隐私风险。
5. 平台条款复杂。

3.6 对话数据

对话数据对助手能力很重要。

来源可能包括：

1. 人工标注对话。
2. 用户日志。
3. 合成对话。
4. 客服记录。
5. 多轮任务数据。

风险：

1. 隐私敏感。
2. 用户同意问题。
3. 标注成本高。
4. 场景偏差。
5. 安全边界复杂。

3.7 多语言数据

多语言数据决定模型国际化能力。

难点：

1. 高质量低资源语言数据少。
2. 语言识别难。
3. 不同语言数据质量差异大。
4. 翻译数据可能带来翻译腔。
5. 文化和地区偏差。

4. Web-scale 采集 Pipeline

一个合规 pipeline 可以分成十步。

4.1 Source Planning

先定义目标。

问题：

1. 训练什么模型？
2. 目标语言是什么？
3. 目标领域是什么？
4. 是否需要代码、数学、科学、多模态？
5. 是否允许商业使用？
6. 是否需要可公开发布？

没有目标，采集会变成无差别堆数据。

4.2 Legal and Policy Review

采集前先审查：

1. 数据许可。
2. robots 协议。
3. Terms of Service。
4. 版权风险。
5. 隐私风险。
6. 数据使用目的。
7. 地区合规要求。

4.3 Collection

合法合规地获取数据。

方式包括：

1. 使用公开许可数据集。
2. 使用数据提供方 API。
3. 使用授权数据。
4. 使用组织内部有权使用的数据。
5. 使用公开 crawl dump。

本书不讨论绕过限制或规避反爬的方法。

4.4 Raw Storage

原始数据要保存元信息。

包括：

1. URL 或来源。
2. 抓取时间。
3. 许可信息。
4. 内容哈希。
5. MIME type。
6. 语言。
7. 数据版本。

4.5 Parsing

把 HTML、PDF、代码仓库、Markdown、论坛页面解析成结构化文本。

要保留：

1. 正文。
2. 标题。
3. 层级结构。
4. 代码块。
5. 表格。
6. 元数据。

4.6 Boilerplate Removal

去掉：

1. 导航栏。
2. 页脚。
3. Cookie banner。
4. 广告。
5. 推荐链接。
6. 社交按钮。
7. 重复模板。

4.7 Language and Domain Classification

识别：

1. 语言。
2. 领域。
3. 文档类型。
4. 代码语言。
5. 安全风险类别。

这些标签后续用于配比和过滤。

4.8 Quality Filtering

过滤：

1. 乱码。
2. 低信息密度。
3. 重复。
4. 垃圾内容。
5. 自动生成 SEO 内容。
6. 恶意或有害内容。
7. 隐私信息。

4.9 Deduplication

去重包括：

1. Exact dedup。
2. Near dedup。
3. Document-level dedup。
4. Paragraph-level dedup。
5. Code clone dedup。

4.10 Versioning and Audit

每次处理都要记录版本。

包括：

1. 原始数据版本。
2. 过滤规则版本。
3. 去重版本。
4. 质量模型版本。
5. 最终训练集版本。
6. 样本级 provenance。

4.11 关键公式与采集审计指标

把 web-scale 采集看成一个数据流，而不是一个爬虫脚本。

设候选数据源集合为：

$$\mathcal{S} = \{s_1, s_2, \dots, s_M\}$$

每个数据源可以记录成：

$$s_k = (a_k, l_k, r_k, p_k, w_k)$$

其中 a_k 是访问方式， l_k 是 license / ToS 状态， r_k 是 robots 或访问策略， p_k 是隐私风险， w_k 是目标配比权重。

对第 i 个原始样本，采集准入门禁可以抽象为：

$$A_i = g_{\{\mathrm{license}\}}(i) \wedge g_{\{\mathrm{tos}\}}(i) \wedge g_{\{\mathrm{robots}\}}(i) \wedge g_{\{\mathrm{privacy}\}}(i) \wedge g_{\{\mathrm{access}\}}$$

这里每个 g 都是 0/1 检查项。任意一项为 0，样本就不应进入训练数据候选集。它不是法律结论，而是工程系统中必须显式记录的审计状态。

设原始样本集合为 D_{raw} ，解析后的集合为 D_{parse} ，通过质量、隐私、安全、污染和去重后的集合为 D_{keep} 。按 token 数计算的保留率为：

$$R_{\{\mathrm{keep}\}} = \frac{\sum_{x_i \in D_{\{\mathrm{keep}\}}} T_i}{\sum_{x_i \in D_{\{\mathrm{raw}\}}} T_i}$$

其中 T_i 是样本 x_i 的 token 数。这个指标帮助你判断过滤是否过松或过严。

对某个语言、领域或来源分组 c ，最终配比为：

$$m_c = \frac{\sum_{x_i \in D_{\{\mathrm{keep}\}}, c_i=c} T_i}{\sum_{x_i \in D_{\{\mathrm{keep}\}}} T_i}$$

如果目标覆盖集合是 C_{target} ，可以定义覆盖率：

$$C_{\{\mathrm{cover}\}} = \frac{1}{|C_{\{\mathrm{target}\}}|} \sum_{c \in C_{\{\mathrm{target}\}}} I[T_c \geq \tau_c]$$

其中 T_c 是分组 c 的保留 token 数， τ_c 是最低覆盖阈值。这个公式适合说明低资源语言、代码、数学、科学和安全数据不能只靠自然网页比例决定。

去重率可以写成：

$$R_{\{\mathrm{dup}\}} = \frac{|D_{\{\mathrm{parse}\}}| - |D_{\{\mathrm{dedup}\}}|}{|D_{\{\mathrm{parse}\}}|}$$

PII 或密钥风险率可以写成：

$$R_{\{\mathrm{risk}\}} = \frac{1}{|D_{\{\mathrm{parse}\}}|} \sum_i I_i$$

其中 $I_i=1$ 表示样本命中隐私、密钥、评估污染或高风险安全规则。

样本级 provenance 至少应包含：

$$p_i = (\mathrm{source}_i, \mathrm{crawl}_i, \mathrm{urlhash}_i, \mathrm{hash}_i, \mathrm{license}_i, t_i, v_i)$$

也就是来源、crawl 批次、URL hash、内容 hash、许可状态、采集时间和数据版本。没有 provenance，后续删除请求、污染排查、风险回溯和模型版本追责都会很困难。

一个最小 web 数据采集门禁可以写成：

```
G_{\mathrm{web}}=
g_{\mathrm{source}}\,
g_{\mathrm{policy}}\,
g_{\mathrm{parse}}\,
g_{\mathrm{quality}}\,
g_{\mathrm{pii}}\,
g_{\mathrm{contam}}\,
g_{\mathrm{dedup}}\,
g_{\mathrm{version}}
```

这些检查项任意一个缺失, 都说明项目还只是“拿到一些文本”, 不能算完成了可审计的训练数据采集 pipeline。

5. 数据采集中的合规问题

5.1 版权

训练数据可能涉及版权。

需要关注:

1. 数据是否有明确许可。
2. 许可是否允许训练。
3. 是否允许商业使用。
4. 是否允许再分发。
5. 是否需要署名。
6. 是否支持删除请求。

不同地区法律和判例可能不同。面试中不要给法律结论, 而要说明需要法务和合规审查。

5.2 隐私

数据中可能包含 PII。

需要处理:

1. 邮箱。
2. 电话。
3. 地址。
4. 证件号。
5. 医疗信息。
6. 私密聊天。
7. 密钥和 token。

5.3 Terms of Service

网站条款可能限制抓取和训练使用。

工程团队不能只从技术可行性判断。

5.4 Robots 和访问控制

Robots 和访问控制体现网站对爬虫行为的限制。

合规数据采集要尊重这些边界。

5.5 数据可追溯

如果未来需要删除某类数据, 必须知道它进入了哪个数据版本和哪个模型版本。

6. 采集不是越多越好

6.1 噪声成本

低质量数据会消耗训练 compute。

6.2 记忆风险

重复数据和稀有敏感数据增加 memorization 风险。

6.3 偏见放大

互联网分布不等于真实世界分布。

某些群体、语言和观点可能被过度或不足表示。

6.4 评估污染

网页中可能包含 benchmark 题目和答案。

如果进入训练集，会让评估失真。

6.5 安全风险

未过滤的网络数据可能包含有害内容、恶意代码、危险指导和注入文本。

7. 数据源元数据设计

每个样本最好带元数据。

常见字段：

1. source_id。
2. source_type。
3. license。
4. url_hash。
5. crawl_time。
6. language。
7. domain。
8. quality_score。
9. safety_score。
10. pii_flag。
11. dedup_cluster_id。
12. dataset_version。

元数据的价值：

1. 配比。
2. 过滤。
3. 回溯。
4. 删除。
5. 审计。
6. 数据 attribution。

8. Web 数据和专门数据的平衡

Web 数据覆盖广，但质量参差不齐。

专门数据质量高，但覆盖有限。

常见组合：

1. Web crawl 提供广覆盖。
2. 书籍和论文提供长结构和专业知识。
3. 代码数据提供编程能力。
4. 数学数据提供推理能力。
5. 对话数据提供助手风格。
6. 安全数据提供边界。
7. 合成数据补足稀缺任务。

关键不是二选一，而是配比和治理。

9. 多语言采集

9.1 低资源语言问题

低资源语言数据少，且质量更不稳定。

如果只按互联网自然比例采样，模型会偏向高资源语言。

9.2 语言识别

语言识别在短文本、混合语言、代码混杂和低资源语言上容易出错。

9.3 翻译数据

机器翻译可以扩展数据，但会带来：

1. 翻译腔。
2. 文化信息丢失。
3. 错误传播。
4. 风格单一。

9.4 多语言配比

需要考虑：

1. 用户分布。
2. 目标市场。
3. 语言资源稀缺度。
4. 任务重要性。
5. 模型容量。

10. 代码数据采集的特殊问题

10.1 许可证

开源不等于无限制使用。

不同 license 对训练、分发和商业使用的解释需要法务判断。

10.2 Fork 和重复

代码仓库大量 fork 和复制。

不去重会让热门代码过度影响模型。

10.3 密钥和凭证

代码中可能包含泄露密钥、token、密码和内部 URL。

必须扫描和过滤。

10.4 安全漏洞

训练不安全代码可能让模型生成漏洞模式。

需要结合静态分析、安全标签和高质量代码筛选。

10.5 Repo-level 上下文

函数片段不等于真实工程代码。

如果要训练 coding agent，需要保留目录结构、依赖、测试和 commit 信息。

11. 对话和用户数据采集

用户数据最敏感。

必须考虑：

1. 用户同意。
2. 用途说明。
3. 脱敏。
4. 数据保留期限。
5. 删除请求。
6. 人工审核权限。
7. 是否允许用于训练。

真实产品中，用户日志可以帮助发现问题，但不能默认无限制进入训练。

12. 真实项目架构

一个采集系统可以分成：

1. Source registry。
2. Legal review workflow。
3. Collector。
4. Raw data lake。
5. Parser。
6. Metadata extractor。
7. Quality filter。
8. PII and safety scanner。
9. Dedup service。
10. Dataset builder。
11. Version registry。
12. Audit dashboard。

12.1 Source registry

记录每个数据源的合法性、用途、联系人、许可和风险等级。

12.2 Raw data lake

保存原始数据和 hash，方便复现和审计。

12.3 Dataset builder

根据训练目标和配比规则构建最终数据集。

12.4 Audit dashboard

展示：

1. 数据来源占比。
2. 语言占比。
3. 领域占比。
4. PII 命中率。
5. 去重率。
6. 质量分布。
7. 许可证分布。

12.5 最小可运行 Web 采集审计 demo

下面这个 demo 不联网、不读取真实网页，也不需要第三方库。它用内存里的 toy HTML 模拟一个合规采集 pipeline，覆盖 source policy、HTML 解析、质量过滤、PII / 密钥扫描、评估污染扫描、exact dedup、元数据和配比统计。

它演示的不是 “怎么爬网站”，而是 “采集系统上线前应该检查什么”。

```
import hashlib
import re
from collections import Counter, defaultdict
```

```
sources = {
    "tech_blog": {"license": "cc-by", "tos_train": True, "robots_allowed": True, "access": "public"},
    "oss_docs": {"license": "apache-2.0", "tos_train": True, "robots_allowed": True, "access": "public"},
    "science_preprint": {"license": "cc-by", "tos_train": True, "robots_allowed": True, "access": "public"},
    "zh_news": {"license": "authorized", "tos_train": True, "robots_allowed": True, "access": "public"},
    "private_forum": {"license": "unknown", "tos_train": False, "robots_allowed": False, "access": "login_required"},
}
```

```
documents = [
    {
        "id": "blog_attention",
        "source_id": "tech_blog",
        "url": "https://example.org/blog/attention",
        "crawl_time": "2026-06-01T00:00:00Z",
        "mime": "text/html",
        "language": "en",
        "domain": "web_ml",
        "html": """
<html><body><nav>Home Archive</nav><article>
Transformer attention uses queries keys and values for language model training.
This article explains data quality, deduplication, provenance, and evaluation contamination.
</article><footer>Contact</footer></body></html>
""",
    },
    {
        "id": "oss_vector_db",
        "source_id": "oss_docs",
        "url": "https://docs.example.org/vector-db",
    }
]
```

```

    "crawl_time": "2026-06-01T00:05:00Z",
    "mime": "text/html",
    "language": "en",
    "domain": "code_docs",
    "html": ""
<main>Open source vector database documentation explains indexes, tests,
examples, licenses, failure modes, and reproducible deployment commands.</main>
    "",
},
{
    "id": "blog_attention_dup",
    "source_id": "tech_blog",
    "url": "https://mirror.example.org/blog/attention-copy",
    "crawl_time": "2026-06-01T00:10:00Z",
    "mime": "text/html",
    "language": "en",
    "domain": "web_ml",
    "html": ""
<html><body><nav>Home Archive</nav><article>
Transformer attention uses queries keys and values for language model training.
This article explains data quality, deduplication, provenance, and evaluation contamination.
</article><footer>Contact</footer></body></html>
    "",
},
{
    "id": "forum_private",
    "source_id": "private_forum",
    "url": "https://forum.example.org/private/thread/7",
    "crawl_time": "2026-06-01T00:15:00Z",
    "mime": "text/html",
    "language": "en",
    "domain": "forum",
    "html": "<article>Private member discussion with personal project details.</article>",
},
{
    "id": "spam_seo",
    "source_id": "tech_blog",
    "url": "https://example.org/seo/spam",
    "crawl_time": "2026-06-01T00:20:00Z",
    "mime": "text/html",
    "language": "en",
    "domain": "spam",
    "html": "<body>Buy now!!! promo promo promo $$$ click click click</body>",
},
{
    "id": "pii_secret",
    "source_id": "tech_blog",
    "url": "https://example.org/leak",
    "crawl_time": "2026-06-01T00:25:00Z",
    "mime": "text/html",
    "language": "en",
    "domain": "security",
    "html": "<article>Contact jane@example.com and use api key sk-live-abcdef for tests.</article>",
},
{

```

```

        "id": "eval_leak",
        "source_id": "tech_blog",
        "url": "https://example.org/benchmark/answer",
        "crawl_time": "2026-06-01T00:30:00Z",
        "mime": "text/html",
        "language": "en",
        "domain": "eval",
        "html": "<article>Benchmark answer key: GSM8K solution and hidden test answer.</article>",
    },
    {
        "id": "paper_scaling",
        "source_id": "science_preprint",
        "url": "https://papers.example.org/scaling",
        "crawl_time": "2026-06-01T00:35:00Z",
        "mime": "text/html",
        "language": "en",
        "domain": "science",
        "html": ""
        <article>Scaling law experiments compare model size, data tokens, compute budget,
        validation loss, ablation controls, and reproducible training recipes.</article>
        "",
    },
    {
        "id": "zh_data_quality",
        "source_id": "zh_news",
        "url": "https://news.example.cn/data-quality",
        "crawl_time": "2026-06-01T00:40:00Z",
        "mime": "text/html",
        "language": "zh",
        "domain": "zh_web",
        "html": ""
        <article> 高质量中文语料需要覆盖教育、科技、政策、生活服务和真实问答，
        同时记录来源、许可、时间、语言、质量分、隐私风险和去重状态。</article>
        "",
    },
]

```

```

REQUIRED_META = ["id", "source_id", "url", "crawl_time", "mime", "language", "domain"]
PII_OR_SECRET = [re.compile(r"[\w.-]+@[ \w-]+\.[\w.-]+"), re.compile(r"sk-[A-Za-z0-9-]{8,}")]
CONTAMINATION = ["benchmark answer", "gsm8k solution", "hidden test answer"]

```

```

def strip_html(html):
    html = re.sub(r"<script.*?</script>|<style.*?</style>", " ", html, flags=re.I | re.S)
    text = re.sub(r"<[^>]+>", " ", html)
    return re.sub(r"\s+", " ", text).strip()

```

```

def tokenize(text):
    return re.findall(r"[A-Za-z0-9_-]+|[\u4e00-\u9fff]", text.lower())

```

```

def content_hash(text):
    normalized = " ".join(tokenize(text))
    return hashlib.shal(normalized.encode("utf-8")).hexdigest()[ :12]

```

```

def quality_score(text):
    toks = tokenize(text)
    if not toks:
        return 0.0
    unique_ratio = len(set(toks)) / len(toks)
    alpha_ratio = sum(ch.isalpha() for ch in text) / max(len(text), 1)
    length_score = min(len(toks) / 24, 1.0)
    return round(0.45 * length_score + 0.35 * unique_ratio + 0.20 * min(alpha_ratio / 0.65, 1.0), 3)

def policy_allowed(doc):
    source = sources[doc["source_id"]]
    return (
        source["license"] not in {"unknown", "restricted"}
        and source["tos_train"]
        and source["robots_allowed"]
        and source["access"] == "public"
    )

def has_pii_or_secret(text):
    return any(pattern.search(text) for pattern in PII_OR_SECRET)

def has_eval_contamination(text):
    lower = text.lower()
    return any(term in lower for term in CONTAMINATION)

def sum_tokens(items, field):
    sums = defaultdict(int)
    for item in items:
        sums[item[field]] += item["tokens"]
    return sums

def audit_web_collection(items):
    seen_hashes = set()
    kept = []
    rejected = {}
    stage_counts = Counter(raw=len(items))

    for doc in items:
        missing = [field for field in REQUIRED_META if not doc.get(field)]
        if missing:
            rejected[doc["id"]] = "missing_metadata"
            continue

        text = strip_html(doc["html"])
        doc_hash = content_hash(text)
        score = quality_score(text)

        if not policy_allowed(doc):

```

```

        rejected[doc["id"]] = "policy_block"
        continue
    stage_counts["policy_pass"] += 1

    if has_pii_or_secret(text):
        rejected[doc["id"]] = "pii_or_secret"
        continue
    stage_counts["pii_pass"] += 1

    if has_eval_contamination(text):
        rejected[doc["id"]] = "eval_contamination"
        continue
    stage_counts["contamination_pass"] += 1

    if score < 0.62:
        rejected[doc["id"]] = "low_quality"
        continue
    stage_counts["quality_pass"] += 1

    if doc_hash in seen_hashes:
        rejected[doc["id"]] = "exact_duplicate"
        continue
    seen_hashes.add(doc_hash)
    kept.append(**doc, "text": text, "hash": doc_hash, "tokens": len(tokenize(text)), "quality": score)
    stage_counts["dedup_pass"] += 1

raw_tokens = sum(len(tokenize(strip_html(doc["html"]))) for doc in items)
kept_tokens = sum(doc["tokens"] for doc in kept)
language_mix = {lang: round(count / kept_tokens, 3) for lang, count in sorted(sum_tokens(kept, "language").items())}
domain_mix = {dom: round(count / kept_tokens, 3) for dom, count in sorted(sum_tokens(kept, "domain").items())}
gates = {
    "policy": rejected.get("forum_private") == "policy_block",
    "pii": rejected.get("pii_secret") == "pii_or_secret",
    "contamination": rejected.get("eval_leak") == "eval_contamination",
    "quality": rejected.get("spam_seo") == "low_quality",
    "dedup": rejected.get("blog_attention_dup") == "exact_duplicate",
    "provenance": all(doc.get("hash") and doc.get("url") and doc.get("crawl_time") for doc in kept),
}

return {
    "kept_ids": [doc["id"] for doc in kept],
    "rejected": dict(sorted(rejected.items())),
    "stage_counts": dict(stage_counts),
    "retention": round(kept_tokens / max(raw_tokens, 1), 3),
    "language_mix": language_mix,
    "domain_mix": domain_mix,
    "avg_quality": round(sum(doc["quality"] for doc in kept) / max(len(kept), 1), 3),
    "gates": gates,
    "gate_pass": all(gates.values()),
}

report = audit_web_collection(documents)
print("kept_ids=", report["kept_ids"])
print("rejected=", report["rejected"])

```

```

print("stage_counts=", report["stage_counts"])
print("retention=", report["retention"])
print("language_mix=", report["language_mix"])
print("domain_mix=", report["domain_mix"])
print("avg_quality=", report["avg_quality"])
print("gates=", report["gates"])
print("gate_pass=", report["gate_pass"])

assert report["kept_ids"] == ["blog_attention", "oss_vector_db", "paper_scaling", "zh_data_quality"]
assert report["rejected"] == {
    "blog_attention_dup": "exact_duplicate",
    "eval_leak": "eval_contamination",
    "forum_private": "policy_block",
    "pii_secret": "pii_or_secret",
    "spam_seo": "low_quality",
}
assert report["stage_counts"] == {
    "raw": 9,
    "policy_pass": 8,
    "pii_pass": 7,
    "contamination_pass": 6,
    "quality_pass": 5,
    "dedup_pass": 4,
}
assert report["retention"] == 0.639
assert report["language_mix"] == {"en": 0.537, "zh": 0.463}
assert report["domain_mix"] == {"code_docs": 0.148, "science": 0.167, "web_ml": 0.222, "zh_web": 0.463}
assert report["gate_pass"] is True

```

预期输出类似：

```

kept_ids= ['blog_attention', 'oss_vector_db', 'paper_scaling', 'zh_data_quality']
rejected= {'blog_attention_dup': 'exact_duplicate', 'eval_leak': 'eval_contamination', 'forum_private': 'policy_block',
stage_counts= {'raw': 9, 'policy_pass': 8, 'pii_pass': 7, 'contamination_pass': 6, 'quality_pass': 5, 'dedup_pass': 4}
retention= 0.639
language_mix= {'en': 0.537, 'zh': 0.463}
domain_mix= {'code_docs': 0.148, 'science': 0.167, 'web_ml': 0.222, 'zh_web': 0.463}
avg_quality= 0.922
gates= {'policy': True, 'pii': True, 'contamination': True, 'quality': True, 'dedup': True, 'provenance': True}
gate_pass= True

```

这个 demo 的关键不是分数阈值本身，而是工程习惯：每个样本都要有来源、许可、时间、内容 hash、拒绝原因和最终版本。真正的 web-scale pipeline 只是把这里的 toy 规则替换成更强的解析器、质量模型、PII 检测器、污染检测器、near-dedup 和数据版本系统。

13. 面向专家：采集策略会改变模型行为

采集策略不是中性的。

它会影响模型：

1. 语言风格。
2. 世界知识。
3. 价值观分布。
4. 安全边界。
5. 代码习惯。
6. 多语言能力。
7. 长文本能力。

例如：

1. 论坛数据多，模型更口语化。
2. 论文数据多，模型更学术化。
3. 代码数据多，模型更擅长编程。
4. 低质量网页多，模型更容易胡说。

因此，数据采集和模型行为是强耦合的。

14. 面试官会怎么问

问题 1：如何构建 web-scale 预训练数据集？

回答要点：

1. 先定义模型目标和数据需求。
2. 建立合法数据源 registry。
3. 采集公开许可、授权或合规来源。
4. 做解析、语言识别、boilerplate removal。
5. 做质量过滤、PII、安全扫描、去重。
6. 做领域分类和数据配比。
7. 做版本管理和审计。

标准回答：

我会先定义模型目标，例如通用、多语言、代码还是领域模型。然后建立 source registry，记录每个数据源的许可、用途、风险和隐私。

问题 2：网页数据有哪些风险？

回答要点：

1. 噪声。
2. 重复。
3. 广告和 SEO。
4. 版权。
5. PII。
6. 有害内容。
7. 评估污染。
8. 分布偏差。

问题 3：为什么要保留数据元数据？

回答要点：

1. 配比。
2. 过滤。
3. 版本复现。
4. 删除请求。
5. 合规审计。
6. 数据 attribution。
7. 问题回溯。

问题 4：如何处理版权和合规？

回答要点：

1. 数据源许可审查。
2. Terms of Service。
3. robots 和访问控制。
4. 商业使用限制。

5. 数据删除机制。
6. 法务和合规参与。
7. 样本级 provenance。

问题 5: The Pile 对数据工程有什么启发?

回答要点:

1. 多来源高质量数据比单一 web crawl 更有价值。
2. 数据子集要按来源和领域组织。
3. 数据集需要文档化和风险分析。
4. 数据多样性影响跨领域泛化。

15. 标准回答模板

面试中可以这样回答:

Web-scale 数据采集不是简单写爬虫, 而是一个从 source planning 到 governance 的完整 pipeline。首先要明确模型目标和数据需求。

我会特别关注四类风险: 版权合规、隐私泄露、低质量噪声和评估污染。因为这些问题最后都会变成模型幻觉、安全问题、评估失真。

16. 常见误区

16.1 误区: 能访问就能训练

纠正: 技术可访问不等于法律、许可和政策允许。

16.2 误区: 网页数据天然代表真实世界

纠正: 互联网分布有强偏差。

16.3 误区: 只要数据够多, 质量无所谓

纠正: 低质量数据会浪费 compute, 并引入错误和风险。

16.4 误区: 去重是清洗阶段的小问题

纠正: 去重影响训练效率、记忆风险和评估污染。

16.5 误区: 用户日志可以直接拿来训练

纠正: 需要同意、脱敏、保留策略、删除机制和访问控制。

17. 小练习

练习 1

设计一个 web-scale 数据采集系统架构。

要求包含: source registry、legal review、collector、raw data lake、parser、quality filter、PII scanner、dedup、dataset builder、version registry。

练习 2

列出网页、书籍、论文、代码、论坛、对话数据各自的优缺点。

练习 3

为代码数据采集设计合规和安全 checklist。

练习 4

解释为什么 robots、ToS、license 和 provenance 都和大模型训练相关。

练习 5

设计一个数据源元数据 schema。

要求至少包含 10 个字段。

18. 本章总结

Web-scale 数据采集是大模型训练的基础，但采集本身只是起点。

大模型数据来源包括网页、书籍、论文、代码、论坛、对话、多语言和多模态数据。

不同来源有不同质量、覆盖、许可、隐私和安全风险。

一个成熟 pipeline 应覆盖 source planning、legal review、collection、raw storage、parsing、boilerplate removal、language/domain classification、quality filtering、deduplication、versioning 和 audit。

采集策略会直接影响模型行为、能力、偏见、安全和合规风险。

面试中要把 Web-scale 数据采集讲成合法、可追溯、可治理、可评估的数据系统，而不是简单爬虫工程。

第三章：清洗过滤与质量评分

如果说上一章的 Web-scale 数据采集解决的是“从哪里拿到足够多的数据”，那么本章要解决的是“哪些数据值得进入训练集”。这一步看似像脏活累活，实际上决定了大模型预训练的上限、稳定性、安全性和产品体验。

很多初学者会以为：只要模型足够大、数据足够多，模型自然能从噪声中学到规律。这个想法只对了一半。大模型确实有很强的统计归纳能力，但它不会自动知道哪些网页是正文、哪些是广告、哪些是模板、哪些是乱码、哪些是低质量 SEO 内容、哪些是隐私信息、哪些是测试集污染、哪些是重复复制的水文。模型会把训练数据中的一切都当作“世界的一部分”去拟合。

所以，清洗过滤不是训练前的边角流程，而是大模型工程的核心环节之一。

本章重点讨论：乱码、模板页、广告、低质量文本、毒性、PII、质量分类器和规则过滤。

合规边界：本章讨论数据质量治理、安全过滤、隐私保护和工程评估，不提供规避网站限制、绕过访问控制、恢复或利用个人敏感信息的方法。

0. 本讲资料边界与第二轮精修口径

按照 `WRITING_PLAN.md` 的要求，本讲精修前核对了 T5 / C4、Gopher / MassiveText、RefinedWeb、FineWeb、Dolma、DataComp-LM / DCLM、Microsoft Presidio 和 OWASP secrets 相关资料。

本讲聚焦训练数据清洗和质量评分的可迁移方法：

解析质量 -> 基础有效性 -> 正文与模板 -> 质量特征 -> 模型评分 -> 隐私安全 -> 污染去重 -> 抽样审计 -> 小模型验证

本讲不把某篇论文或某个数据集的过滤规则写成通用最优规则。真实项目中，过滤阈值必须按语言、领域、来源、数据类型和训练目标校准；PII、版权、安全和用户数据处理需要合规、安全和法务参与。

1. 先建立直觉：清洗过滤到底在过滤什么？

把训练语料想象成给模型读的“图书馆”。采集阶段像把全世界的书、网页、笔记、论坛帖子、代码仓库搬进仓库。清洗过滤阶段则像图书管理员、编辑、法律顾问和安全审核员一起做筛选。

他们要问的问题不是一个，而是一组：

1. 这段文本是不是正常可读？
2. 这段文本是不是正文，还是网页导航、版权声明、广告、按钮、评论区垃圾文本？
3. 这段文本有没有重复？
4. 这段文本有没有极低信息密度？
5. 这段文本有没有明显机器生成垃圾、采集错误或格式损坏？
6. 这段文本属于什么语言、领域、风格？
7. 这段文本是否包含个人敏感信息？
8. 这段文本是否包含过高风险的有害内容？
9. 这段文本是否可能污染评测集或 benchmark？
10. 这段文本是否适合当前模型的目标能力？

注意，清洗过滤不是简单地“删脏数据”。更准确地说，它是在做训练分布塑形。

如果你过滤掉大量口语和论坛数据，模型可能更像一本书，但聊天感下降。如果你保留太多网页评论，模型可能更接地气，但事实性、格式稳定性和安全性可能下降。如果你过度过滤少数语言，模型会在这些语言上能力薄弱。如果你只保留“教科书质量”数据，小模型可能受益很大，但大模型的世界知识覆盖可能不足。

这就是数据清洗的难点：它不是一个单目标优化问题，而是质量、覆盖率、多样性、安全性、版权合规、成本和下游目标之间的权衡。

2. 来龙去脉：从 NLP 数据预处理到大模型数据治理

早期 NLP 任务的数据处理比较直接。比如情感分类、机器翻译、命名实体识别，数据集通常由研究者或标注团队提前整理好。常见预处理包括分词、去特殊字符、统一大小写、去停用词、长度截断。这些操作服务于一个固定任务，规模通常从几千到几百万样本。

预训练语言模型出现后，数据处理的目标发生了变化。BERT、GPT 这类模型不再只面向一个任务，而是要从大规模无标注文本中学习通用语言能力。于是，过滤不再只是为了让某个分类器输入更干净，而是为了构造一个能支撑通用能力的语料宇宙。

到了 GPT-3、Gopher、Chinchilla 之后，数据规模进一步变大。数据质量开始和 scaling law 直接相关：同样的算力和参数量，如果数据噪声更低、重复更少、领域配比更合理，模型可以获得更好的 loss 和下游能力。反过来，低质量数据会浪费 token budget，让模型在重复模板、垃圾页面和无意义文本上消耗训练算力。

再后来，phi-1 等工作强调“textbook quality”数据的重要性。它提醒大家：对于某些目标，尤其是代码、数学、推理和教学型能力，高质量、小规模、结构清晰的数据可能比巨量杂乱 web 数据更有效。这不意味着大规模数据不重要，而是说明数据质量会改变效率曲线。

从历史上看，数据清洗经历了三个阶段：

1. 规则预处理阶段：主要靠长度、字符、语言、重复、黑名单等启发式规则。
2. 统计过滤阶段：加入语言模型困惑度、n-gram 分布、词频、压缩率等统计特征。
3. 模型化治理阶段：使用质量分类器、毒性分类器、PII 检测器、领域分类器、embedding 聚类 and 人工审计闭环。

今天的大模型数据工程通常不是单一过滤器，而是一套多级流水线。

3. 为什么“更多数据”不等于“更好数据”？

大模型训练有一个重要事实：每个 token 都要消耗算力。一个 1T token 的训练集，不只是硬盘上更大而已，它意味着训练过程中模型会在这些 token 上反复做前向、反向和参数更新。

如果其中大量 token 是重复页脚、模板导航、乱码、SEO 堆词、低质量问答、采集错误，那么模型就会把昂贵算力花在学习这些噪声上。

更糟的是，低质量数据不是“无影响”。它可能带来具体坏处：

1. 降低训练效率：loss 下降慢，有效信息密度低。
2. 诱导坏格式：模型学会输出网页残留、版权页脚、无意义列表。
3. 增加幻觉：低可信网页和拼接文本会污染事实知识。
4. 增加重复记忆：大量重复文本让模型更容易记忆原文。
5. 放大偏见和毒性：低治理语料可能包含攻击性、歧视性或高风险内容。
6. 影响下游风格：模型输出可能变得啰嗦、营销化、标题党或论坛化。
7. 污染评测：如果 benchmark 或答案泄漏进训练集，评估结果会虚高。

因此，一个成熟团队不会只汇报“我们有多少 TB 数据”，而会汇报：有效 token 数、来源构成、重复率、语言分布、质量分布、安全过滤策略、PII 处理策略、版本和审计信息。

4. 清洗过滤流水线总览

一个典型的大模型数据清洗流水线可以拆成下面几层：

1. 解析层：从 HTML、PDF、代码仓库、文档格式中抽取文本和结构。
2. 基础有效性过滤：去空文本、乱码、异常字符、极短或极长样本。
3. 正文抽取和模板去除：去导航栏、菜单、页脚、版权声明、广告块。
4. 语言识别和领域识别：判断语言、领域、文体和来源类型。
5. 规则质量过滤：基于长度、字符比例、重复行、标点比例、词汇分布等规则。
6. 模型质量评分：用分类器或小模型判断文本是否高质量。
7. 安全和隐私过滤：处理毒性、敏感内容、PII、政策风险内容。
8. 去重和污染检测：文档级、段落级、近重复、benchmark contamination。
9. 抽样审计：人工检查过滤前后样本，估计误删和漏删。
10. 版本化产出：保存过滤规则、阈值、模型版本、数据 lineage。

这条流水线不是一次性脚本，而是反复迭代的系统。每次训练结果、人工审计、下游评测和安全评估都会反过来修改过滤策略。

4.1 关键公式与审计指标

设原始样本集合为：

$$D_{\{\mathrm{raw}\}} = \{x_1, x_2, \dots, x_N\}$$

每条样本 x_i 经过解析后，可以抽取一组质量特征：

$$f_i = (l_i, u_i, a_i, r_i, s_i, b_i, p_i)$$

其中 l_i 是长度特征， u_i 是 unique token ratio， a_i 是字母或中日韩字符比例， r_i 是重复行或重复 n-gram 特征， s_i 是符号或异常字符比例， b_i 是 boilerplate / link / SEO 特征， p_i 是 PII、secret、安全或污染风险特征。

一个简单质量分可以写成：

$$q_i = \sum_k w_k f_{ik} - \lambda_r r_i - \lambda_s s_i - \lambda_b b_i - \lambda_p p_i$$

这里 w_k 是正向质量特征权重， λ 是风险或噪声惩罚。真实系统不一定用线性分数，也可以用分类器、ranker 或 LLM-as-judge 辅助打分，但面试时用这个式子能讲清“多信号融合”的本质。

过滤门禁可以写成：

$$G_i = I(q_i \geq \tau_{\{c_i\}}) \wedge I_{\{\mathrm{valid}\}}(i) \wedge I_{\{\mathrm{privacy}\}}(i) \wedge I_{\{\mathrm{safety}\}}(i) \wedge I_{\{\mathrm{contam}\}}(i)$$

其中 c_i 是样本所属语言、领域或来源桶， $\tau_{\{c_i\}}$ 是桶级阈值。 I_{valid} 、 I_{privacy} 、 I_{safety} 、 I_{contam} 、 I_{dedup} 分别表示基础有效性、隐私、风险、污染和去重检查是否通过。

按 token 数计算的整体保留率为：

$$R_{\{\mathrm{keep}\}} = \frac{\sum_i G_i T_i}{\sum_i T_i}$$

对某个语言、领域或来源桶 c 的保留率为：

$$R_{\{\mathrm{keep}\}}(c) = \frac{\sum_{i:c_i=c} G_i T_i}{\sum_{i:c_i=c} T_i}$$

风险命中率可以写成：

$$R_{\{\mathrm{risk}\}} = \frac{1}{N} \sum_i I(p_i > 0)$$

如果有人工审计集 A ，其中 $h_i=1$ 表示人工认为样本应保留， $h_i=0$ 表示应过滤，则误删率和漏删率分别是：

$$R_{\{\mathrm{fp}\}} = \frac{\sum_{i \in A} I(h_i=1, G_i=0)}{\sum_{i \in A} I(h_i=1)}$$

$$R_{\{\mathrm{fn}\}} = \frac{\sum_{i \in A} I(h_i=0, G_i=1)}{\sum_{i \in A} I(h_i=0)}$$

这里的 fp 指过滤系统的 false positive，也就是好数据被误删； fn 指坏数据被漏掉。

困惑度过滤可以作为一个特征，而不是最终裁判：

$$\mathrm{PPL}_i = \exp\left(\frac{1}{T_i} \sum_{t=1}^{T_i} -\log p_{\{\mathrm{ref}\}}(x_t \mid x_{1:t-1})\right)$$

更稳的做法不是全局统一阈值，而是在语言、领域、来源桶内看极端异常值：

$$z_i = \frac{\log \mathrm{PPL}_i - \mu_{\{c_i\}}}{\sigma_{\{c_i\}}}$$

只有当 z_i 明显偏离同桶分布时，才把困惑度作为强过滤信号。这样可以减少对代码、数学、专业文本和低资源语言的误伤。

5. 基础有效性过滤：先把明显坏数据挡在外面

基础过滤的目标很简单：排除“明显不能训练”的样本。

常见规则包括：

1. 文本长度过短：比如只有几个词、一个标题、一个按钮文本。
2. 文本长度异常过长：可能是拼接错误、日志倾倒、重复模板堆叠。
3. 非文本字符比例过高：大量乱码、控制字符、二进制残留。
4. 数字或符号比例异常：例如满屏电话号码、价格、ID、乱码编码。
5. 重复字符过多：例如“哈哈哈哈哈”或同一符号重复数千次。
6. 行重复率过高：网页菜单、商品列表、采集错误经常出现这种模式。
7. Unicode 异常：混入不可见字符、方向控制符、错误编码。
8. 语言不匹配：目标语料是中文，样本却是其他语言或混合乱码。

这些规则看起来朴素，但非常有效。因为大规模 web 数据里，最先要处理的不是微妙质量问题，而是大量显而易见的脏数据。

一个常见误区是：规则越多越好。实际上，规则过滤要非常谨慎。比如“数字比例高”可能过滤掉财报、科学论文、数学题、代码日志；“符号比例高”可能过滤掉代码、公式、配置文件；“重复行高”可能过滤掉诗歌、歌词或结构化文档。

所以规则不能脱离数据源和目标任务。对通用自然语言、代码、数学、法律、医学、金融，规则阈值应该不同。

6. 乱码和编码问题

乱码是 web-scale 数据中最常见的问题之一。它来自编码识别错误、网页解析错误、PDF 抽取错误、压缩解压错误、复制粘贴损坏和混合语言环境。

乱码对模型的危害主要有三类：

1. 浪费 token：Tokenizer 会把异常字符拆成大量罕见 token。

2. 干扰分布：模型学习到无意义字符组合。
3. 影响稳定性：异常长序列或奇怪字符可能放大训练和推理中的边缘问题。

常见检测方法包括：

1. 统计不可打印字符比例。
2. 统计 Unicode replacement character 的比例。
3. 检测异常 byte pattern。
4. 用语言识别模型判断文本是否像自然语言。
5. 统计 tokenizer 后的 token/character 比例。

其中 token/character 比例很实用。如果一段中文正常文本 1000 个字符会变成相对合理的 token 数，但一段乱码可能被切成大量碎片 token。异常高的比例说明这段文本对训练很不友好。

但也要注意：代码、数学公式、多语言文本和表格可能天然 token 密度较高，不能简单一刀切。

7. 模板页、导航栏和 boilerplate 去除

网页不是为大模型训练设计的。一个网页里真正有价值的正文，可能只占 HTML 的一小部分。其他内容包括导航栏、推荐链接、版权信息、cookie 提示、页脚、侧边栏、评论框、广告位、分享按钮等。

如果不去除 boilerplate，模型会大量学习这些模板：

1. “点击这里查看更多”
2. “版权所有，未经许可不得转载”
3. “上一篇 / 下一篇”
4. “分享到微信微博”
5. “热门推荐”
6. “接受 Cookie”

这些文本本身不一定有害，但信息密度很低，而且重复度极高。大规模保留会稀释正文质量。

常见方法包括：

1. DOM 结构分析：识别正文节点、导航节点、页脚节点。
2. 文本密度方法：正文区域通常文本密度高、链接密度低。
3. 链接比例过滤：链接文本占比过高的段落更像菜单或推荐栏。
4. 重复模板检测：同一站点大量页面中重复出现的段落可能是模板。
5. 规则和模型结合：先用规则粗筛，再用模型判断正文质量。

面试中经常会问：“为什么不能直接从 HTML 提取所有文本？”标准回答是：因为 HTML 中包含大量非正文信息，直接提取会引入模板噪声、广告噪声和重复文本，降低有效 token 密度，并改变模型输出风格。

8. 广告、SEO 垃圾和低信息密度文本

广告和 SEO 垃圾是 web 数据里非常难处理的一类问题。它们往往是正常自然语言，不一定乱码，也不一定短，但训练价值很低。

低质量 SEO 文本常见特征包括：

1. 关键词堆砌。
2. 标题党。
3. 大量同义改写。
4. 内容空泛，信息密度低。
5. 多段结构高度相似。
6. 夹杂商品、服务、联系方式或推广话术。
7. 句子语义连贯性弱，但表面语法正常。

这类文本很难用单条规则识别。因为它们不是“坏字符”，而是“低价值语言”。

常见处理方式有三类：

1. 规则特征：关键词密度、链接密度、重复 n-gram、广告词表、联系方式模式。
2. 统计特征：困惑度、压缩率、句子相似度、段落重复率。
3. 质量分类器：训练一个模型判断文本是否像高质量文章、教材、问答或文档。

但广告词表不能无限扩大。因为很多领域文本天然包含价格、地址、电话、产品名、品牌名，比如电商评论、商业案例、法律合同和新闻报道。过度过滤会损失真实世界知识。

更稳妥的策略是多信号融合：单一广告词不直接删除，而是与链接比例、重复程度、文本质量分、来源信誉、人工审计结果一起判断。

9. 质量评分：从二元过滤到连续打分

早期过滤常常是二元决策：保留或删除。但在大模型训练中，更常见的是给每个样本一个质量分。

质量分可以用于多个目的：

1. 删除最低质量样本。
2. 按质量分采样，高质量样本更高概率进入训练。
3. 对不同来源设置不同阈值。
4. 做 data mixture 配比时作为权重。
5. 支持后续审计和版本回溯。

一个简单质量评分系统可能包含：

1. 可读性分：语言是否自然、句子是否完整。
2. 信息密度分：是否包含实质内容。
3. 结构分：段落、标题、列表是否清晰。
4. 来源分：来源是否可靠、领域是否重要。
5. 重复惩罚：是否与已有样本高度重复。
6. 安全惩罚：是否包含高风险或不适合训练的内容。
7. 隐私惩罚：是否疑似包含个人敏感信息。

质量评分不是绝对真理。它反映的是团队对“什么是好训练数据”的定义。

例如，百科、教材、论文和技术文档通常在事实性和结构上得分高；论坛、聊天和社交媒体在结构上可能得分低，但对对话、口语和真实用户表达很重要。因此质量分不能替代数据配比策略。

10. 规则过滤 vs 模型过滤

数据过滤常见两大路线：规则过滤和模型过滤。

规则过滤的优点：

1. 可解释。
2. 便宜。
3. 容易并行。
4. 适合处理明显错误。
5. 便于审计和复现。

规则过滤的缺点：

1. 覆盖有限。
2. 容易误伤边缘数据。
3. 很难识别语义低质。
4. 阈值依赖经验。

模型过滤的优点：

1. 能捕捉语义质量。
2. 能识别复杂模式。
3. 可以适配不同领域。
4. 可以输出连续分数。

模型过滤的缺点：

1. 成本更高。
2. 需要标注或弱监督数据。
3. 可能继承标注偏见。
4. 可解释性较差。
5. 分布外数据容易误判。

成熟系统通常不是二选一，而是组合使用：先用规则过滤掉显然坏的数据，再用模型评分处理更微妙的质量问题，最后用人工抽样审计校准阈值。

11. 质量分类器怎么训练？

质量分类器的目标是判断一段文本是否适合进入训练集，或给出质量分。常见训练方式包括：

1. 人工标注：让标注员按质量标准打分。
2. 弱监督：把高质量来源当正例，把低质量来源当负例。
3. 对比学习：让模型区分教材式文本和垃圾网页。
4. LLM-as-judge：用更强模型辅助生成质量标签，再人工抽检。
5. 下游反馈：根据训练后模型表现反推数据质量策略。

弱监督很常见。例如，可以把高质量教材、百科、论文摘要、技术文档作为正例，把采集错误页、低质 SEO 页、模板页、垃圾评论作为负例。分类器学到的不是“真理”，而是这些来源之间的统计差异。

这会带来一个风险：分类器可能把“文风像教材”误认为“质量高”，从而偏好正式、标准、主流语言，压低口语、方言、少数语言、社区讨论和新兴表达。

因此，质量分类器需要配套审计：

1. 分语言审计。
2. 分领域审计。
3. 分来源审计。
4. 分长度审计。
5. 分人群和风格审计。

否则，一个看似提升平均质量的分类器，可能会系统性删除某些重要数据。

12. 困惑度过滤：用语言模型判断“像不像正常文本”

困惑度过滤是一种经典方法。直觉是：如果一个参考语言模型觉得某段文本非常“不像正常语言”，它可能是乱码、拼接错误、机器垃圾或异常文本。

困惑度低，说明文本在参考模型看来更可预测；困惑度高，说明文本更异常。

但困惑度过滤有明显局限：

1. 高质量专业文本可能困惑度高，因为参考模型不熟悉该领域。
2. 低质量套话可能困惑度低，因为语言模式很常见。
3. 少数语言可能被误判为异常。
4. 代码和数学公式不适合用普通自然语言困惑度直接过滤。

所以困惑度更适合作为一个特征，而不是最终裁判。

一个更稳健的用法是：对每个语言、领域、来源分别统计困惑度分布，只过滤极端异常样本，而不是用统一阈值处理所有数据。

13. 毒性和有害内容过滤

大模型训练语料中不可避免会出现攻击性语言、仇恨表达、骚扰、暴力、成人内容、自伤相关内容、危险行为描述等。是否过滤、过滤多少、如何保留上下文，是非常复杂的问题。

如果完全不过滤，模型可能更容易复现有害表达。如果过度过滤，模型可能不了解这些内容，导致安全分类、风险识别和拒答能力变弱。很多安全训练需要模型见过风险语义，但以受控方式学习如何识别、拒绝和安全回应。

因此，安全过滤通常不是简单删除所有敏感内容，而是分层处理：

1. 明显非法或高风险内容：严格移除或隔离。
2. 攻击性和仇恨内容：根据用途过滤、降权或放入安全专用数据池。
3. 新闻、研究、教育语境中的风险讨论：保留时需要上下文判断。
4. 安全问答和拒答示例：用于后续 SFT/RLHF/安全训练，而不是混入普通预训练池。

这里的关键是上下文。比如一篇讨论网络安全治理的政策文章和一段鼓励攻击行为的文本，表面关键词可能相似，但训练用途完全不同。

所以安全过滤需要模型分类、规则、人工审核和数据分层共同完成。

14. PII 和隐私过滤

PII 指 personally identifiable information，即可以识别个人身份的信息。常见类型包括姓名、地址、电话号码、邮箱、身份证件号、账号、地理位置、医疗记录、财务信息等。

大模型训练中处理 PII 的原因有两个：

1. 合规和伦理：训练数据不应不当包含个人敏感信息。
2. 记忆风险：模型可能记住罕见、重复或结构化的敏感文本。

PII 过滤常见方法包括：

1. 正则和规则：识别邮箱、电话、证件号、银行卡号等结构化模式。
2. NER 模型：识别人名、地名、机构名等实体。
3. 上下文分类器：判断实体是否构成隐私风险。
4. 哈希或脱敏：对某些字段做不可逆处理或替换。
5. 来源级策略：对高风险来源直接排除或隔离。

PII 检测的难点在于误报和漏报。

误报例子：小说里的人名、公开名人信息、公司地址、论文作者邮箱、代码示例中的 fake email。

漏报例子：非标准格式的联系方式、图片 OCR 后的个人信息、论坛中的口语化地址、多个字段组合后可识别个人。

工程上通常不会只依赖一个 PII detector，而是用多层策略，并保留审计记录。对高风险数据源，宁可更保守；对公开、专业、低风险来源，可以结合上下文处理。

15. 低质量机器生成数据

随着生成式 AI 普及，web 上的机器生成文本越来越多。训练新模型时，如果大量采集到旧模型生成的低质量内容，可能出现“模型吃模型输出”的问题。

机器生成数据不一定坏。高质量 synthetic data 可以用于数学、代码、指令跟随和安全训练。但未经筛选的大量机器生成垃圾会带来风险：

1. 信息多样性下降。
2. 事实错误被放大。
3. 语言风格同质化。
4. 模型过度学习模板化表达。
5. 长尾真实表达被稀释。

检测机器生成文本很难，尤其当生成模型越来越强时。更实际的策略不是追求完美检测，而是从来源、时间、结构和质量信号综合控制。

例如：

1. 对已知低质内容农场降权。
2. 监控重复模板和语义相似段落。
3. 对疑似批量生成内容抽样审计。
4. 保留高质量合成数据，但明确标注来源和生成方式。
5. 不把 synthetic data 和自然语料混在一起不可追踪地使用。

关键不是“机器生成一定不能用”，而是“不能无标识、无评估、无配比地用”。

16. 不同类型数据的过滤策略不同

同一套规则不能处理所有数据。

16.1 网页正文

重点是正文抽取、模板去除、广告过滤、语言识别、质量评分和去重。

16.2 代码数据

代码数据不能用普通文本规则直接过滤。符号多、缩进多、重复结构多都是正常现象。代码过滤更关注：

1. 文件是否可解析。
2. 许可证是否可用。
3. 是否是自动生成文件。
4. 是否包含 secrets 或凭据。
5. 是否重复 fork 或 vendor 代码。
6. 是否包含测试答案或 benchmark 泄漏。

16.3 数学数据

数学文本可能包含大量公式、符号、编号和短句。普通困惑度或符号比例过滤容易误伤。更合适的方法是领域分类、结构识别和人工抽样。

16.4 论文和技术文档

这类数据质量通常较高，但 PDF 抽取容易产生页眉页脚、断行、参考文献噪声、公式损坏和表格错乱。

16.5 对话数据

对话数据要关注角色标注、轮次完整性、隐私、攻击性内容和 consent。不能把真实用户对话简单当普通 web 文本处理。

16.6 多语言数据

多语言过滤要防止主流语言规则误伤低资源语言。语言识别模型在短文本、混合文本和低资源语言上容易不准，需要抽样审计。

17. 阈值怎么定？

过滤系统里最常见的问题是：阈值怎么定？比如质量分低于多少删除？重复率高于多少删除？毒性分高于多少隔离？

成熟做法不是拍脑袋，而是建立验证闭环。

常见步骤：

1. 从原始数据中抽样。
2. 对不同阈值过滤后的样本做人工审计。
3. 估计误删率和漏删率。
4. 训练小规模模型或做数据消融实验。
5. 观察验证 loss、下游 benchmark、安全评估和生成样例。
6. 按语言、领域和来源分别调整阈值。
7. 固化版本并记录规则。

阈值选择要服务于目标。训练通用 base model、代码模型、数学模型、对话模型、安全模型，阈值可能完全不同。

比如，代码模型不能因为符号多就过滤；安全模型不能完全删除所有风险语义；对话模型需要保留真实口语；教材型小模型可能更偏好高质量结构化文本。

18. 误删和漏删：过滤系统的两个错误

过滤系统有两个基本错误：

1. False Positive：把好数据删掉。
2. False Negative：把坏数据留下。

很多团队只关注漏删，因为坏数据留下来容易被看到。但误删同样危险，因为它会改变训练分布。

误删的后果包括：

1. 某些语言能力下降。
2. 某些领域知识缺失。
3. 口语表达能力下降。
4. 少数群体或低资源文化表达被削弱。
5. 模型变得过于书面、僵硬或单一。

漏删的后果包括：

1. 训练效率下降。
2. 输出风格变差。
3. 安全风险上升。
4. 隐私记忆风险上升。
5. benchmark 污染。

一个好答案不能只说“加强过滤”，而要说清楚如何平衡误删和漏删，如何用审计和实验评估。

19. 过滤后如何评估数据质量？

数据过滤是否有效，不能只看删掉了多少。删掉 50% 数据不代表质量提升，可能只是删掉了某些语言或领域。

常见评估指标包括：

1. 保留率：每个来源、语言、领域保留了多少。
2. 重复率：文档级、段落级、近重复率是否下降。
3. 质量分分布：过滤前后质量评分是否改善。
4. PII 命中率：敏感信息风险是否下降。
5. 毒性分布：高风险内容比例是否下降。
6. 语言分布：是否意外损失低资源语言。
7. 领域分布：是否过度偏向百科、新闻或技术文档。
8. 人工审计通过率：抽样人工检查是否更好。
9. 小模型验证：用过滤前后数据训练小模型比较 loss 和下游表现。
10. 生成样例：观察模型是否输出模板、广告、乱码、隐私信息或不良风格。

其中，小模型验证很重要。因为有些过滤规则看起来合理，但对最终模型效果未必好。先用较小训练预算做 ablation，可以避免把错误策略放大到昂贵的大训练中。

19.1 最小可运行清洗过滤审计 demo

下面这个 demo 不依赖外部库，也不读写文件。输入是一组 toy 样本，每条样本有 id、language、domain 和 text；输出包括保留样本、拒绝原因、保留率、语言 / 领域配比、质量分预览和门禁检查。

它演示的是清洗过滤的最小机制，不是生产级 PII detector、安全分类器或质量模型。

```
import hashlib
import re
from collections import Counter, defaultdict
```

```
samples = [
    {"id": "good_web_ml", "language": "en", "domain": "web_ml", "text": "Transformer data cleaning improves training"},
    {"id": "good_zh_policy", "language": "zh", "domain": "zh_web", "text": "高质量中文语料需要保留真实问答、政策解读"},
    {"id": "good_code_doc", "language": "en", "domain": "code", "text": "Function parse_config(path) reads YAML, validates"},
    {"id": "good_math_note", "language": "en", "domain": "math", "text": "For a random variable X, the expectation E[X^2] ="},
    {"id": "dup_web_ml", "language": "en", "domain": "web_ml", "text": "Transformer data cleaning improves training efficiency"},
    {"id": "boilerplate_menu", "language": "en", "domain": "web", "text": "Home About Contact Subscribe Privacy Home"},
    {"id": "seo_low_info", "language": "en", "domain": "seo", "text": "Best cheap model training service buy now discount"},
    {"id": "garbled_text", "language": "unknown", "domain": "web", "text": "xxxxxx %%% \u0000 \u0001 click"},
    {"id": "pii_email_phone", "language": "en", "domain": "forum", "text": "Please contact alice@example.com or call 123-456-7890"},
    {"id": "secret_key", "language": "en", "domain": "code", "text": "The test config accidentally contains api_key = 'sk-12345678901234567890123456789012'"},
    {"id": "safety_risk", "language": "en", "domain": "comment", "text": "This comment contains hate speech, harassment, and threats."},
    {"id": "eval_leak", "language": "en", "domain": "eval", "text": "Benchmark answer key: hidden test answer for a model."}
]
```

```
PII_PATTERNS = [re.compile(r"[\w.+~]+@[ \w-]+\.[\w.-]+"), re.compile(r"1[3-9]\d{9}")]
SECRET_PATTERNS = [re.compile(r"sk-[A-Za-z0-9-]{10,}"), re.compile(r"AKIA[0-9A-Z]{12,}")]
SAFETY_TERMS = {"hate", "harassment", "demeaning", "insults"}
CONTAM_TERMS = {"benchmark answer", "hidden test answer", "answer key"}
PROMO_TERMS = {"buy", "discount", "promo", "click", "cheap"}
```

```
def tokens(text):
    return re.findall(r"[A-Za-z0-9_]+|[\u4e00-\u9fff]", text.lower())
```

```

def content_hash(text):
    normalized = " ".join(tokens(text))
    return hashlib.sha1(normalized.encode("utf-8")).hexdigest()[:12]

def line_repeat_ratio(text):
    parts = [p.strip().lower() for p in re.split(r"[\n.!?:!? ]", text) if p.strip()]
    if not parts:
        return 1.0
    counts = Counter(parts)
    return sum(count - 1 for count in counts.values() if count > 1) / len(parts)

def symbol_ratio(text):
    if not text:
        return 1.0
    ok = lambda ch: ch.isspace() or ch.isalnum() or "\u4e00" <= ch <= "\u9fff"
    return sum(not ok(ch) for ch in text) / len(text)

def quality_features(sample):
    text = sample["text"]
    tok = tokens(text)
    return {
        "tokens": len(tok),
        "length_score": min(len(tok) / 24, 1.0),
        "unique_ratio": len(set(tok)) / max(len(tok), 1),
        "alpha_or_cjk": sum(ch.isalpha() or "\u4e00" <= ch <= "\u9fff" for ch in text) / max(len(text), 1),
        "repeat": line_repeat_ratio(text),
        "symbol_ratio": symbol_ratio(text),
        "replacement_ratio": text.count("ⓧ") / max(len(text), 1),
        "promo_ratio": sum(t in PROMO_TERMS for t in tok) / max(len(tok), 1),
    }

def quality_score(sample):
    f = quality_features(sample)
    symbol_penalty = 0.05 if sample["domain"] in {"code", "math"} else 0.20
    score = (
        0.30 * f["length_score"]
        + 0.25 * f["unique_ratio"]
        + 0.25 * min(f["alpha_or_cjk"] / 0.65, 1.0)
        - 0.30 * f["repeat"]
        - symbol_penalty * f["symbol_ratio"]
        - 0.50 * f["replacement_ratio"]
        - 0.50 * f["promo_ratio"]
    )
    return round(max(0.0, min(1.0, score)), 3)

def risk_flags(sample):
    text = sample["text"]
    lower = text.lower()
    tok = set(tokens(text))
    return {

```

```

    "pii": any(p.search(text) for p in PII_PATTERNS),
    "secret": any(p.search(text) for p in SECRET_PATTERNS),
    "safety": bool(tok & SAFETY_TERMS),
    "contamination": any(term in lower for term in CONTAM_TERMS),
}

```

```

def audit(items, threshold=0.62):
    seen = {}
    kept, rejected, rows = [], {}, []
    for item in items:
        f = quality_features(item)
        score = quality_score(item)
        flags = risk_flags(item)
        reason = None
        if f["tokens"] < 8 or f["replacement_ratio"] > 0.05:
            reason = "invalid_or_garbled"
        elif flags["pii"] or flags["secret"]:
            reason = "privacy_or_secret"
        elif flags["contamination"]:
            reason = "eval_contamination"
        elif flags["safety"]:
            reason = "safety_isolate"
        elif score < threshold:
            reason = "low_quality"
        else:
            key = content_hash(item["text"])
            reason = "exact_duplicate" if key in seen else None
            seen[key] = item["id"]
        if reason:
            rejected[item["id"]] = reason
        else:
            kept.append(item)
        rows.append({"id": item["id"], "score": score, "reason": reason or "kept"})

    kept_tokens = sum(len(tokens(item["text"])) for item in kept)
    raw_tokens = sum(len(tokens(item["text"])) for item in items)
    domain_tokens, language_tokens = defaultdict(int), defaultdict(int)
    for item in kept:
        token_count = len(tokens(item["text"]))
        domain_tokens[item["domain"]] += token_count
        language_tokens[item["language"]] += token_count
    reason_counts = Counter(rejected.values())
    gates = {
        "quality": reason_counts["low_quality"] >= 2,
        "privacy": reason_counts["privacy_or_secret"] == 2,
        "safety": reason_counts["safety_isolate"] == 1,
        "contamination": reason_counts["eval_contamination"] == 1,
        "dedup": reason_counts["exact_duplicate"] == 1,
        "coverage": len(domain_tokens) >= 4 and len(language_tokens) >= 2,
    }
    return {
        "kept_ids": [item["id"] for item in kept],
        "rejected": dict(sorted(rejected.items())),
        "reason_counts": dict(sorted(reason_counts.items())),
    }

```

```

    "retention": round(kept_tokens / raw_tokens, 3),
    "domain_mix": {k: round(v / kept_tokens, 3) for k, v in sorted(domain_tokens.items())},
    "language_mix": {k: round(v / kept_tokens, 3) for k, v in sorted(language_tokens.items())},
    "score_preview": {row["id"]: row["score"] for row in rows[:6]},
    "gates": gates,
    "gate_pass": all(gates.values()),
}

```

```

report = audit(samples)
print("kept_ids=", report["kept_ids"])
print("rejected=", report["rejected"])
print("reason_counts=", report["reason_counts"])
print("retention=", report["retention"])
print("domain_mix=", report["domain_mix"])
print("language_mix=", report["language_mix"])
print("score_preview=", report["score_preview"])
print("gates=", report["gates"])
print("gate_pass=", report["gate_pass"])

assert report["kept_ids"] == ["good_web_ml", "good_zh_policy", "good_code_doc", "good_math_note"]
assert report["reason_counts"] == {
    "eval_contamination": 1,
    "exact_duplicate": 1,
    "invalid_or_garbled": 1,
    "low_quality": 2,
    "privacy_or_secret": 2,
    "safety_isolate": 1,
}
assert report["retention"] == 0.496
assert report["domain_mix"] == {"code": 0.175, "math": 0.211, "web_ml": 0.193, "zh_web": 0.421}
assert report["language_mix"] == {"en": 0.579, "zh": 0.421}
assert report["gate_pass"] is True

```

运行后会看到类似输出：

```

kept_ids= ['good_web_ml', 'good_zh_policy', 'good_code_doc', 'good_math_note']
rejected= {'boilerplate_menu': 'low_quality', 'dup_web_ml': 'exact_duplicate', 'eval_leak': 'eval_contamination', 'garb
reason_counts= {'eval_contamination': 1, 'exact_duplicate': 1, 'invalid_or_garbled': 1, 'low_quality': 2, 'privacy_or_se
retention= 0.496
domain_mix= {'code': 0.175, 'math': 0.211, 'web_ml': 0.193, 'zh_web': 0.421}
language_mix= {'en': 0.579, 'zh': 0.421}
score_preview= {'good_web_ml': 0.77, 'good_zh_policy': 0.735, 'good_code_doc': 0.748, 'good_math_note': 0.763, 'dup_web_
gates= {'quality': True, 'privacy': True, 'safety': True, 'contamination': True, 'dedup': True, 'coverage': True}
gate_pass= True

```

这个 demo 刻意保留代码和数学样本，避免普通文本规则因为符号比例误伤它们；同时把 PII / secret、评估污染、安全风险和 exact duplicate 分成不同拒绝原因，便于后续审计。

20. 数据清洗和训练目标的关系

清洗过滤必须服从模型目标。

如果目标是通用 base model，需要覆盖广泛世界知识，不能只保留高雅书面文本。如果目标是代码模型，要保留大量高质量代码、文档、issue、问答和测试，但严格处理 license、secrets 和重复。如果目标是数学模型，要

保留题目、解答、教材、证明、公式和合成推理数据。如果目标是企业助手，要重点治理隐私、安全、事实性和指令风格。

这说明“数据质量”不是抽象概念，而是相对于目标能力定义的。

一段论坛讨论对百科问答可能质量低，但对训练用户意图理解很有价值。一段严谨论文对聊天产品可能不够自然，但对专业问答很重要。一段安全风险说明对普通预训练可能需要隔离，但对安全分类器训练很关键。

所以面试中不要说“低质量数据全部删除”。更专业的说法是：分层、打标、降权、隔离和按用途进入不同数据池。

21. 面向专家：清洗过滤改变模型的归纳偏置

从专家视角看，数据清洗不是中性的。它会改变模型的归纳偏置。

模型最终学到的是训练分布的压缩表示。你过滤掉什么，模型就少学什么；你保留什么，模型就多拟合什么；你给什么更高采样权重，模型就把什么视作更重要的模式。

例如：

1. 过度偏好教科书数据，模型可能更会解释概念，但对真实用户混乱输入鲁棒性下降。
2. 过度过滤社交媒体，模型可能更安全、更正式，但对流行表达、讽刺、错别字和多轮口语理解变弱。
3. 过度清理代码重复，可能减少记忆风险，但也可能删除常见 API 用法模式。
4. 过度过滤毒性语料，可能减少有害输出，但也可能削弱模型识别和处理有害内容的能力。
5. 过度依赖英文质量分类器，可能系统性压低非英语和低资源语言。

因此，过滤系统应该和 data mixture、训练目标、评估体系一起设计，而不是由数据团队孤立完成。

22. 一个可落地的清洗过滤方案

如果面试官问：“给你 100TB 原始 web 数据，你怎么清洗成预训练语料？”可以按下面框架回答。

第一步，建立数据 schema。为每条样本记录来源 URL、时间、语言、解析方式、域名、长度、hash、许可证信息、过滤状态和质量分。

第二步，做解析和正文抽取。对 HTML、PDF、文档、代码分别用不同 parser，保留结构信息，不只保留纯文本。

第三步，基础过滤。删除空文本、乱码、异常字符、极端长度、明显模板和解析失败样本。

第四步，语言和领域分类。识别语言、领域和内容类型，为后续分桶阈值做准备。

第五步，规则质量过滤。按语言和领域设置长度、符号比例、重复行、链接比例、低信息密度等规则。

第六步，模型质量评分。训练或使用质量分类器，输出连续质量分，而不是简单删除。

第七步，安全和隐私处理。使用 PII 检测、毒性分类、安全分类和来源级策略，做删除、脱敏、隔离或降权。

第八步，去重和污染检测。做文档级去重、近重复检测，并与 benchmark、评测集和高风险泄漏集比对。

第九步，人工抽样审计。分来源、语言、质量分桶抽样，估计误删和漏删。

第十步，小规模训练验证。用不同过滤策略训练小模型，对比验证 loss、下游任务、安全评估和生成质量。

第十一步，版本化发布。记录所有规则、阈值、分类器版本、数据 hash、过滤统计和审计报告。

这套回答的重点是：不是写一个脚本清洗，而是构建可复现、可审计、可迭代的数据治理系统。

23. 常见面试题

23.1 为什么大模型训练前要做数据清洗?

回答要点:

1. 提升有效 token 密度。
2. 减少乱码、模板、广告、重复和低质内容。
3. 降低隐私和安全风险。
4. 减少模型记忆和 benchmark 污染。
5. 改善训练效率、下游能力和生成风格。

一句话回答: 大模型会拟合训练分布, 脏数据不是无害噪声, 而会消耗算力、改变模型行为并带来安全和隐私风险。

23.2 规则过滤和模型过滤怎么取舍?

回答要点:

1. 规则过滤便宜、可解释、适合明显坏样本。
2. 模型过滤能识别语义质量, 但成本高、可能有偏差。
3. 实际系统通常先规则粗筛, 再模型评分, 最后人工审计校准。
4. 不同语言、领域、数据类型需要不同阈值。

23.3 如何设计质量分类器?

回答要点:

1. 定义质量标准: 可读性、信息密度、结构、来源、安全和隐私。
2. 构造训练数据: 人工标注、弱监督、正负来源对比、LLM 辅助标注。
3. 输出连续分数, 而不是只做二分类。
4. 分语言、领域和来源审计, 避免系统性偏差。
5. 用小模型训练或下游评估验证分类器效果。

23.4 怎么处理 PII?

回答要点:

1. 用规则识别结构化 PII, 如邮箱、电话、证件号。
2. 用 NER 和上下文模型识别非结构化个人信息。
3. 对高风险样本删除、脱敏或隔离。
4. 记录处理策略和数据 lineage。
5. 通过抽样审计估计漏检和误检。

23.5 毒性内容是不是全部删除?

回答要点:

1. 不能简单全部删除, 因为模型需要识别风险语义。
2. 明显高风险内容应删除或隔离。
3. 新闻、研究、教育和安全语境要做上下文判断。
4. 安全专用数据可进入 SFT 或安全训练池, 而不是普通混入预训练池。
5. 需要结合分类器、规则和人工审核。

23.6 如何判断清洗策略是否有效?

回答要点:

1. 看保留率、重复率、质量分、PII 命中率、安全风险分布。
2. 分语言、领域、来源做统计, 防止分布坍塌。

3. 人工抽样审计误删和漏删。
 4. 做小模型 ablation，比对 loss、benchmark、安全评估和生成样例。
 5. 记录版本，保证可复现。
-

24. 常见误区

误区一：数据越干净越好。

更准确地说，数据要适合目标。过度清洗会删除真实世界多样性。

误区二：规则过滤足够了。

规则能处理明显坏数据，但难以识别语义低质、SEO 垃圾和复杂安全语境。

误区三：质量分类器分数高就一定好。

分类器反映训练标签和来源偏好，可能压制少数语言、口语和非主流风格。

误区四：毒性内容全部删除最安全。

模型需要学会识别和安全处理风险内容，关键是分层、隔离和受控使用。

误区五：PII 用正则就能解决。

正则只能处理结构化模式，非结构化和组合型隐私需要上下文判断和审计。

误区六：清洗完成后就不用管了。

数据清洗是迭代系统，训练结果、评估反馈和安全事件都会反向修正过滤策略。

25. 本章小结

清洗过滤与质量评分是大模型数据工程的核心。它决定哪些信息进入训练分布，哪些风险被移除或隔离，哪些能力被强化，哪些能力被削弱。

本章要记住几句话：

1. 大模型不会自动忽略脏数据，它会拟合训练分布。
2. 清洗过滤不是删脏数据这么简单，而是在塑造模型能力和行为。
3. 规则过滤、模型评分、安全过滤、PII 处理和人工审计需要组合使用。
4. 质量不是绝对概念，而是相对于模型目标定义的。
5. 过滤策略必须评估误删和漏删，不能只看删除比例。
6. 成熟数据工程必须可复现、可审计、可版本化。

如果面试中被问到数据清洗，最好的回答不是列几个规则，而是讲清楚完整闭环：数据 schema、解析、基础过滤、质量评分、安全隐私、去重污染、人工审计、小模型验证和版本治理。

第四章：去重与污染检测

上一章讨论了如何从原始数据中清理乱码、模板、低质量文本和隐私风险。本章继续往下走，讨论大模型数据工程中另一个非常关键的问题：去重与污染检测。

很多初学者会把去重理解成“删掉完全一样的文本”。这只是最简单的一层。真实的大模型训练集中，重复和污染有很多形态：同一篇文章被多个网站转载，同一个网页被不同时间采集多次，同一个代码仓库有大量 fork，同一段 benchmark 答案出现在训练数据中，同一段隐私文本被复制粘贴到多个页面，同一个模板句子重复上万次。

这些问题不是小瑕疵。重复数据会浪费训练 token、增加记忆风险、扭曲数据分布；污染数据会让评测结果虚高，让团队误判模型真实能力。

本章重点：exact dedup、near dedup、MinHash、SimHash、benchmark contamination、canary。

合规边界：本章讨论去重、污染检测、评估可信度和隐私风险治理，不提供提取、复原或利用训练数据中敏感内容的方法。

0. 本讲资料边界与第二轮精修口径

按照 WRITING_PLAN.md 的要求，本讲精修前核对了 Deduplicating Training Data Makes Language Models Better、GPT-3 论文中的 benchmark contamination 分析、RefinedWeb、FineWeb、Dolma、DataComp-LM / DCLM、Google Research deduplicate-text-datasets、MinHash / LSH、SimHash 和 canary / 记忆风险相关公开资料。

本讲聚焦训练数据内部去重、训练集与评估集重叠检测、benchmark contamination 风险审计和数据版本治理：

规范化 -> exact hash -> 段落 / 子串重复 -> MinHash / SimHash 候选召回 -> 聚类保留 -> train-eval overlap -> canary / 记忆风险审计 -> 版本报告

本讲不提供训练数据抽取、隐私文本复原或绕过评估隔离的做法。canary 只讨论受控合成探针，用于记忆风险评估，不应使用真实用户隐私或真实凭据。

1. 先建立直觉：为什么重复数据很危险？

想象你准备考试，有一本 1000 页的复习资料。表面上看内容很多，但里面有 300 页是同一篇文章的不同排版，有 100 页是网页页脚，有 50 页是同一道题和答案反复出现。你会误以为自己读了很多知识，其实有效信息远少于纸面页数。

大模型训练也是这样。训练集的大小通常以 token 数衡量，但 token 数不等于有效信息量。如果 1T token 中有大量重复，那么模型不是在学习更丰富的世界，而是在反复强化少数文本。

重复数据主要带来五类问题：

1. 浪费算力：模型多次看到相同内容，训练预算被低效消耗。
2. 降低泛化：模型过度拟合高频重复文本，而不是学习广泛规律。
3. 增加记忆：重复出现的长文本更容易被模型原样记住。
4. 扭曲分布：某些来源、网站、模板或观点被过度放大。
5. 污染评测：训练集中含有测试题或答案，会导致 benchmark 分数虚高。

Lee 等人在 Deduplicating Training Data Makes Language Models Better 中指出，现有语言建模数据集包含大量近重复样本和长重复子串，去重可以减少模型输出训练数据原文的频率，并减少 train-test overlap 对评估的影响。这是数据去重从“工程清理”变成“大模型训练质量关键环节”的典型证据。

2. 来龙去脉：从数据库去重到大模型去重

传统数据工程里的去重，常常是数据库问题：同一个用户、订单、日志或文档出现多次，通过唯一 ID、时间戳、hash 或主键合并即可。

但大模型语料不同。它通常没有可靠主键，也没有统一 schema。两个文档可能：

1. URL 不同，但正文完全相同。
2. 正文基本相同，只是标题、广告、页脚不同。
3. 同一篇文章被改写、转载、翻译或摘录。
4. 同一份代码出现在原仓库、fork、vendor 目录和镜像中。
5. 同一份 benchmark 题目被博客、教程、题解网站收录。

因此，大模型去重从简单 ID matching 变成了大规模文本相似性问题。

它需要解决三个层次：

1. 完全重复：文本一模一样。

2. 近似重复：文本大部分相同，但有少量修改。
3. 语义污染：文本不一定字面相同，但包含评测题、答案或等价解法。

前两类主要靠 hash、n-gram、MinHash、SimHash、embedding 和分布式索引处理。第三类更难，通常需要 benchmark 原始数据、变体生成、相似检索、人工审计和评估解释共同完成。

3. exact dedup：完全去重

exact dedup 是最简单也最便宜的去重方式：对规范化后的文本计算 hash，相同 hash 只保留一份。

这里的关键词是“规范化后”。如果直接对原始文本 hash，很多本质相同的文档会因为空格、换行、HTML 实体、编码差异、大小写差异而被视为不同。

常见规范化包括：

1. 去除首尾空白。
2. 统一换行符。
3. 合并连续空格。
4. 统一 Unicode 表示。
5. 去掉明显模板字段。
6. 可选地统一大小写。
7. 可选地去除标点或 HTML 残留。

然后计算 hash，比如 SHA 系列、xxHash、MurmurHash 等。密码学安全不是这里的主要目标，吞吐、碰撞概率和工程生态更重要。

exact dedup 的优点：

1. 简单。
2. 快。
3. 可解释。
4. 容易分布式实现。
5. 非常适合文档级完全重复。

缺点也明显：

1. 不能发现轻微改写。
2. 不能发现转载时加了广告和页脚的版本。
3. 不能发现段落级重复。
4. 对 OCR、PDF 抽取和格式差异敏感。

所以 exact dedup 通常是第一层，而不是全部。

4. 文档级、段落级和子串级去重

去重的粒度很重要。

4.1 文档级去重

文档级去重把一整篇文章、一页网页或一个文件作为单位。它适合去除完整重复页面、镜像站、重复下载和数据版本合并中的重复。

优点是简单，缺点是粗糙。两篇文章只要有一个页脚不同，就可能无法 exact match。

4.2 段落级去重

段落级去重把文档拆成段落，识别重复段落。它适合处理网页模板、转载文章中的固定段落、免责声明和重复介绍。

但段落级去重会带来上下文破坏。删除一个段落后，文档可能变得不连贯。所以很多系统不直接删除段落，而是用段落重复率作为文档质量特征。

4.3 子串级去重

子串级去重关注长重复片段。例如某个 50 词句子在数据集中重复几次，即使所在文档不同，也会造成记忆风险。

子串级去重更细，但计算成本更高。通常需要 n-gram 索引、后缀数组、rolling hash 或专门的大规模文本去重工具。

5. near dedup: 近重复去重

近重复是大模型语料中更常见的问题。

例如：

1. 同一篇文章被多个网站转载，开头结尾略有不同。
2. 新闻稿被不同媒体改标题。
3. 教程被复制后插入广告。
4. 代码仓库 fork 后只改了 README。
5. PDF 抽取版本和 HTML 版本内容基本一致。

如果只做 exact dedup，这些样本都会留下来。

near dedup 的目标是识别“相似到不值得重复训练”的样本。关键问题是：如何定义相似？

常见相似度包括：

1. 字符级相似度。
2. token 级相似度。
3. n-gram Jaccard 相似度。
4. 编辑距离。
5. SimHash 汉明距离。
6. embedding cosine 相似度。

在 web-scale 规模下，不能两两比较所有文档。假设有 10 亿篇文档，两两比较是不可接受的。因此需要近似检索和局部敏感哈希。

6. MinHash: 把集合相似度变成可扩展检索

MinHash 常用于估计两个集合的 Jaccard 相似度。对于文本，可以把一篇文档表示成 n-gram 集合。例如按词切成 5-gram 或按字符切成 shingles。

直觉上，如果两篇文档共享很多 n-gram，它们就很可能是近重复。

MinHash 的核心思想是：不用保存完整 n-gram 集合，而是用多个 hash 函数对集合做摘要。两个集合越相似，它们的 MinHash 签名越可能相同。

在工程上常见流程是：

1. 文本规范化。
2. 分词或字符切片，生成 shingles。
3. 计算 MinHash signature。

4. 使用 LSH 把相似 signature 放入同一候选桶。
5. 只在候选桶内做精确相似度计算。
6. 对超过阈值的文档聚类或删除。

MinHash 的优点:

1. 适合大规模近重复检测。
2. 对小改动较鲁棒。
3. 相比全量 n-gram 比较节省空间。
4. 可与 LSH 结合降低候选对数量。

缺点:

1. 参数敏感, 如 shingle 大小、signature 长度、LSH band 设置。
2. 对短文本不稳定。
3. 对语义改写无能为力。
4. 多语言和代码场景需要调整切片策略。

面试时, 不需要推导 MinHash 数学细节, 但要讲清它解决的是 “大规模近重复检索不能两两比较” 的问题。

6.1 关键公式与审计指标

设清洗后的训练候选集为:

$$D_{\{\mathrm{train}\}} = \{d_1, d_2, \dots, d_N\}$$

规范化函数为 $\mathrm{norm}(d_i)$, 文档级 exact hash 为:

$$h_i = H(\mathrm{norm}(d_i))$$

某个 hash 桶 k 对应的完全重复集合为:

$$C_k = \{i: h_i = k\}$$

按文档数估计的 exact duplicate rate 可以写成:

$$R_{\{\mathrm{exact}\}} = \frac{\sum_k \max(|C_k| - 1, 0)}{N}$$

把文档 d_i 切成 shingle 集合 S_i , 两个文档的 Jaccard 相似度为:

$$J_{\{ij\}} = \frac{|S_i \cap S_j|}{|S_i \cup S_j|}$$

MinHash 的核心性质是: 随机排列 π 下两个集合最小 hash 相同的概率等于 Jaccard 相似度:

$$P(\min \pi(S_i) = \min \pi(S_j)) = J_{\{ij\}}$$

使用 K 个 hash 函数后, 可以用签名相同的比例估计相似度:

$$\hat{J}_{\{ij\}} = \frac{1}{K} \sum_{k=1}^K I(m_{ik} = m_{jk})$$

如果 LSH 使用 b 个 band、每个 band 包含 r 行, 那么真实相似度为 s 的样本对被召回为候选的概率近似为:

$$P_{\{\mathrm{cand}\}}(s) = 1 - (1 - s^r)^b$$

SimHash 可以看成把特征加权投票成二进制指纹。设第 b 位的加权和为:

$$v_{\{ib\}} = \sum_{g \in G_i} w_g I(\phi_b(g)=1) - \sum_{g \in G_i} w_g I(\phi_b(g)=0)$$

则 SimHash 第 b 位为:

$$z_{\{ib\}} = I(v_{\{ib\}} \geq 0)$$

两个 SimHash 指纹的汉明距离为:

$$H_{\{ij\}} = \sum_b I(z_{\{ib\}} \neq z_{\{jb\}})$$

near dedup 可以写成一个候选门禁:

$$M_{\{ij\}} = I(J_{\{ij\}} \geq \tau_J) \vee I(H_{\{ij\}} \leq \tau_H)$$

其中 τ_J 是 Jaccard 阈值, τ_H 是汉明距离阈值。实际系统通常先用 LSH 或索引召回候选, 再计算更精确的相似度。

对评测集 $E=\{e_1, \dots, e_M\}$, 训练污染风险可以写成最大相似度:

$$c_j = \max_i \mathrm{sim}(e_j, d_i)$$

超过阈值的污染率为:

$$R_{\{\mathrm{contam}\}}(\tau) = \frac{1}{M} \sum_{j=1}^M I(c_j \geq \tau)$$

去重和污染隔离后的 token 保留率为:

$$R_{\{\mathrm{keep}\}} = \frac{\sum_i G_i T_i}{\sum_i T_i}$$

这里 G_i 表示样本 d_i 通过 exact dedup、near dedup、污染检测、canary / privacy 检查和保留策略后的最终门禁。

如果有人工审计候选对集合 A , 其中 $y_{ij}=1$ 表示人工认为两篇文档应合并, 则近重复检测的误删率和漏检率可以写成:

$$R_{\{\mathrm{merge_fp}\}} = \frac{\sum_{(i,j) \in A} I(y_{ij}=0, M_{ij}=1)}{\sum_{(i,j) \in A} I(y_{ij}=0)}$$

$$R_{\{\mathrm{merge_fn}\}} = \frac{\sum_{(i,j) \in A} I(y_{ij}=1, M_{ij}=0)}{\sum_{(i,j) \in A} I(y_{ij}=1)}$$

面试里要强调: 这些公式不是让你背参数, 而是让你能解释 “exact 重复、near 重复、污染相似度、保留率、误删和漏检” 各自衡量什么。

7. LSH: 避免全量两两比较

LSH 是 Locality-Sensitive Hashing, 局部敏感哈希。普通 hash 希望相似输入也完全打散; LSH 则相反, 它希望相似输入更容易落到同一个桶里。

在去重里, LSH 的作用是候选召回。

没有 LSH 时, 10 亿篇文档两两比较不可行。有 LSH 后, 系统只比较落入相同或相关桶的文档, 候选对数量大幅下降。

但 LSH 不是最终判定器。它只是帮你找到 “可能相似” 的文档。最终仍然要计算 Jaccard、编辑距离、SimHash 距离或其他相似度, 并根据阈值决定是否去重。

LSH 的核心 trade-off 是召回率和计算量:

1. 桶太宽: 候选太多, 计算量大。
2. 桶太窄: 漏掉相似文档。
3. 阈值太高: 只能去掉几乎完全重复。
4. 阈值太低: 可能误删同主题但不同内容的文档。

因此, 大规模去重要通过抽样审计和小模型实验调参, 而不是只看理论阈值。

8. SimHash: 适合快速近重复检测

SimHash 也是一种常见近重复方法。它把文档映射成固定长度指纹, 相似文档的指纹汉明距离较小。

直觉上, SimHash 先把文本特征映射到向量空间, 再压缩成一个二进制签名。两个文档越相似, 签名越接近。

SimHash 常用于网页去重和重复页面检测, 因为它速度快、存储小、易于索引。

相比 MinHash, SimHash 更像向量特征的随机投影摘要; MinHash 更直接对应集合 Jaccard 相似度。工程上两者都可用, 选择取决于文本类型、相似定义和系统生态。

SimHash 的优点:

1. 快。
2. 指纹短。
3. 适合海量网页。
4. 对小修改有一定鲁棒性。

缺点：

1. 参数和特征选择会影响效果。
 2. 对短文本不稳定。
 3. 对深度语义改写能力有限。
 4. 汉明距离阈值需要审计校准。
-

9. embedding 去重：语义相似，但更危险

近几年很多团队会考虑用 embedding 做语义去重。做法是把文档、段落或问题编码成向量，用近似最近邻检索找相似样本。

embedding 去重能发现字面不同但语义相近的内容。例如同一道数学题的不同表述、同一个 FAQ 的改写、同一段代码功能的解释。

但它也更危险，因为“语义相似”不等于“应该删除”。

例如：

1. 两篇讲 Transformer 的文章可能主题相似，但细节不同，都有价值。
2. 两道数学题方法相似，但数字和推理不同，都能训练泛化。
3. 两段法律条文解释相似，但适用场景不同。
4. 多个用户问法语义相似，但对指令泛化很有帮助。

因此，embedding 更适合用于污染检测、聚类分析、采样平衡和人工审计辅助，而不是直接大规模删除所有近邻。

如果用于删除，阈值必须很高，并且要分领域评估误删。

10. 去重不是越狠越好

去重的目标不是让训练集没有任何相似样本，而是避免不合理重复。

有些重复是有害的，比如网页模板、转载垃圾、重复 benchmark 答案、重复隐私片段。有些重复是自然的，比如常见概念的不同讲解、代码 API 的常见用法、数学定理的标准表述。

完全去掉所有相似内容会带来问题：

1. 常识知识频率被压低。
2. 常见表达模式学习不足。
3. 代码模型对常用库和 API 不熟。
4. 数学模型对标准题型 exposure 不够。
5. 对话模型对常见用户意图鲁棒性下降。

所以更专业的策略是：

1. 对明显重复删除。
2. 对近重复聚类后保留代表样本。
3. 对高质量来源重复可降权，而不是全删。
4. 对低质量来源重复严格过滤。
5. 对 benchmark 和隐私相关重复更严格。

去重本质上是在控制数据分布的频率，而不是追求集合意义上的唯一性。

11. 代码数据去重：fork、vendor 和生成文件

代码数据的重复问题非常严重。

常见来源包括：

1. fork 仓库。
2. vendor 目录。
3. 自动生成代码。
4. minified 文件。
5. 复制粘贴的模板工程。
6. 依赖包镜像。
7. 教程代码和题解代码。

如果不去重，代码模型可能大量记忆常见仓库、题解和库源码。训练时看起来 loss 很低，但真实泛化和安全性未必好。

代码去重可以按多个粒度做：

1. 文件级 hash。
2. 函数级 hash。
3. AST 结构相似。
4. token n-gram near dedup。
5. 仓库级 fork 识别。
6. license 和来源级过滤。

代码去重的难点是：代码天然有大量模板和重复模式。比如 import、main 函数、配置文件、常见算法实现、测试框架样板。如果阈值过严，会删除有用模式；如果阈值过松，又会保留大量复制代码。

工程上通常要区分：

1. 自动生成文件：尽量移除。
2. 第三方 vendor：视 license 和目标处理。
3. fork 重复：保留主源或高质量版本。
4. 常见 API 用法：不宜完全去除。
5. 题库答案：对评估污染要特别严格。

12. benchmark contamination：评测污染是什么？

评测污染是指训练数据中包含了评测集的问题、答案、解析或高度相似变体，导致模型评测分数不能反映真实泛化能力。

这是大模型评估中非常严重的问题。因为很多 benchmark 是公开的，而 web-scale 训练数据会抓取论文、博客、GitHub、题解网站、课程材料和论坛讨论。公开 benchmark 一旦发布，就可能被复制到各种网页中。

污染有多种形式：

1. 题目原文在训练集中。
2. 题目和答案都在训练集中。
3. 答案解析在训练集中。
4. 同一道题的改写版本在训练集中。
5. benchmark 构造代码或数据文件在训练集中。
6. 评测 leaderboard 讨论和题解在训练集中。

污染的后果是：模型看起来“会推理”，实际上可能是在记忆题目或答案模式。

BIG-bench、MMLU、HumanEval、GSM8K 等公开评测都可能面临不同程度的污染风险。越流行、越公开、越被讨论的 benchmark，污染风险越高。

13. 污染检测的基本方法

污染检测通常需把评测集作为 query，在训练语料中检索相似内容。

常见方法包括：

1. exact match：题目原文 hash 匹配。
2. n-gram overlap：查找高重叠文本。
3. MinHash/LSH：大规模近重复检索。
4. embedding 检索：发现语义相似或改写题目。
5. answer leakage 检测：检查答案、解析、选项是否一起出现。
6. code benchmark 检测：查找函数签名、测试样例、题目描述和参考解法。
7. 时间切分：只使用 benchmark 发布前数据，降低污染风险。
8. 人工审计：对高相似候选做人工判断。

污染检测不能只看题目。很多时候答案和解析才是关键。如果训练集中只有题目没有答案，污染程度较轻；如果题目、选项、答案和解析一起出现，评测可信度就严重下降。

污染检测也不能只靠 exact match。许多题目会被改写、翻译、格式化或嵌入博客文章中，需要 near dedup 和语义检索辅助。

14. 评测污染和训练污染的区别

面试里容易混淆两个概念：训练数据重复和评测污染。

训练数据重复主要影响训练效率、记忆和分布。它发生在训练集内部。

评测污染主要影响评估可信度。它发生在训练集和测试集之间。

两者相关但不相同。一个训练集内部可以高度去重，但仍然包含 benchmark。反过来，一个训练集可能有很多重复，但不一定污染某个特定评测。

所以工程上要分别做：

1. intra-train dedup：训练集内部去重。
 2. train-eval overlap detection：训练集和评测集重叠检测。
 3. benchmark-specific quarantine：对评测相关数据隔离。
 4. eval report annotation：评测报告中说明污染检测方法和风险。
-

15. canary：用探针检测记忆风险

canary 是一种用于检测模型记忆和数据泄漏风险的探针思想。直觉是：在训练数据中插入或标记一些特殊、罕见、可识别的字符串，然后观察模型是否容易输出这些字符串。

在研究和安全评估中，canary 可以帮助估计模型对罕见序列的记忆倾向。它不是为了泄露数据，而是为了评估风险和验证防护策略。

使用 canary 时要注意：

1. 只能使用受控、非真实敏感信息的合成字符串。
2. 不能把真实用户隐私当 canary。
3. 需要控制出现次数、上下文和训练阶段。
4. 评估结果要用于降低记忆风险，而不是复原训练内容。

canary 的价值在于建立因果对照：如果一个罕见字符串出现次数越多，模型越容易记住，那么重复数据和敏感数据的风险就更清楚。

16. 去重与记忆风险

大模型的记忆风险和数据重复高度相关。罕见、长、重复出现的文本更容易被记住，尤其是包含结构化信息、固定格式或低熵内容时。

去重可以降低记忆风险，原因很直接：同一段文本出现次数减少，模型在训练中反复强化它的机会也减少。

但去重不是解决记忆的全部。记忆风险还与以下因素有关：

1. 模型规模。
2. 训练轮数。
3. 数据采样策略。
4. 文本罕见程度。
5. 文本结构化程度。
6. 是否包含 PII 或 secrets。
7. 后训练是否强化了复述行为。

因此，去重应和 PII 过滤、安全训练、输出过滤、隐私评估和日志治理一起使用。

17. 去重策略会影响数据配比

去重不是孤立步骤，它会改变 data mixture。

假设某个来源重复率很高，去重后它的 token 占比大幅下降；另一个来源本来重复率低，去重后占比相对上升。这会改变模型学习到的领域分布。

例如：

1. 新闻转载很多，去重后新闻占比下降。
2. 代码 fork 很多，去重后代码占比下降。
3. 百科文本重复较少，去重后占比相对上升。
4. 论坛数据短文本多，文档级去重影响较小。
5. 数学题库重复模板多，去重策略会影响题型频率。

所以去重后必须重新统计语言、领域、来源和质量分布。否则团队可能以为 data mixture 没变，但实际训练分布已经明显改变。

18. 去重系统的工程架构

一个可落地的 web-scale 去重系统通常包括：

1. 文本规范化模块。
2. 文档 hash 和 exact dedup。
3. 段落 hash 和重复段落统计。
4. shingling 和 MinHash signature 生成。
5. LSH 候选召回。
6. 候选对精确相似度计算。
7. 连通分量或聚类。
8. 保留策略选择。
9. benchmark contamination 检索。
10. 统计报告和人工审计。
11. 数据版本和 lineage 记录。

保留策略非常重要。发现一组近重复文档后，保留哪一份？常见原则包括：

1. 保留质量分最高的版本。
2. 保留来源最可信的版本。
3. 保留解析质量最好的版本。

4. 保留时间最早或最权威的版本。
5. 对不同语言或地区版本分开处理。

如果随便保留第一份，可能保留了转载垃圾，删掉了原始高质量来源。

19. 阈值和审计

去重阈值不能拍脑袋。

如果 Jaccard 阈值过高，只能去掉几乎完全相同的文档，重复残留多。如果阈值过低，会误删同主题但有不同内容的文章。

实际做法通常是：

1. 从候选对中按相似度分桶抽样。
2. 人工判断每个桶中多少是真重复、多少是误判。
3. 按语言、领域、来源分别统计。
4. 对不同数据类型设置不同阈值。
5. 用过滤前后数据训练小模型做 ablation。
6. 观察验证 loss、下游 benchmark、记忆指标和生成质量。

特别要注意短文本。短文本的 n-gram overlap 很容易虚高。比如两个短问题都问“什么是 Transformer”，它们相似度很高，但可能都值得保留。对短文本可以用更严格的规则、聚类后采样或按意图保留多样表达。

20. 面向专家：去重会改变 scaling behavior

从专家视角看，去重不只是减少数据量。它会改变训练的有效样本分布，从而影响 scaling behavior。

如果重复很多，训练 loss 可能下降更快，因为模型反复看到熟悉文本。但这不代表泛化更好。去重后，训练集更难，短期 loss 可能变化，但下游泛化、记忆风险和评测可信度可能更好。

这也是为什么只看 pretraining loss 不够。一个模型可能在重复数据上 loss 很漂亮，但在真实任务中不稳。另一个模型训练 loss 稍高，但因为数据多样性更好，泛化更强。

去重还会影响 token budget 使用方式。对于固定训练 token 数，如果去重后有效多样性提高，模型在同样 token budget 下能覆盖更多模式。但如果去重过度，常见模式 exposure 不足，也会损害能力。

所以专家级回答要强调：去重要和 data mixture、训练 token budget、sampling strategy、memorization evaluation 和 downstream evaluation 一起考虑。

21. 一个可落地的去重与污染检测方案

如果面试官问：“你有 100TB 清洗后的预训练语料，如何做去重和污染检测？”可以按下面思路回答。

第一步，文本规范化。统一编码、空白、换行、HTML 残留和 Unicode 表示，保留原始版本和规范化版本。

第二步，做 exact dedup。对文档级规范化文本计算 hash，相同 hash 合并，记录重复次数和来源。

第三步，做段落级统计。计算段落 hash，识别高频模板段落、页脚、免责声明和重复短语。

第四步，做 near dedup。对文档生成 shingles，计算 MinHash signature，用 LSH 召回候选，再计算精确相似度。

第五步，聚类和保留。把近重复文档组成簇，按质量分、来源可信度、解析质量和时间选择代表样本。

第六步，代码专项去重。对代码做文件级、函数级、仓库级和 fork 级去重，处理自动生成文件、vendor 和题库答案。

第七步, benchmark contamination 检测。把评测集题目、选项、答案、解析、代码签名和测试样例作为 query, 做 exact、n-gram、MinHash 和 embedding 检索。

第八步, 隔离风险数据。对高相似 benchmark 候选, 不一定永久删除, 但要从训练池或评测报告中隔离和标注。

第九步, 审计和评估。抽样检查误删和漏删, 统计去重前后语言、领域、来源、质量分和 token 分布, 训练小模型做 ablation。

第十步, 版本化。记录 hash、规则、阈值、MinHash 参数、LSH 参数、评测集版本、污染检测结果和数据 lineage。

这套回答体现了系统性: 先解决训练集内部重复, 再解决训练集与评测集重叠, 最后用审计和版本保证可信。

21.1 最小可运行去重与污染检测审计 demo

下面这个 demo 不依赖外部库, 也不读写文件。输入是一组 toy 训练候选文档和 toy 评测样本; 输出包括 exact duplicate group、near duplicate pair、SimHash 近邻、聚类、保留样本、benchmark contamination 命中、canary 命中、token 保留率和门禁检查。

它演示的是审计机制, 不是生产级 web-scale 去重系统。真实系统需要分布式 shingling、MinHash/LSH 索引、embedding 或 BM25 检索、人工审计、权限隔离和数据版本系统。

```
import hashlib
import re
from collections import defaultdict
from itertools import combinations
```

```
train_docs = [
    {"id": "attention_blog", "domain": "web_ml", "quality": 0.92, "text": "Scaled dot product attention compares quer"},
    {"id": "attention_blog_dup", "domain": "mirror", "quality": 0.61, "text": "Scaled dot product attention compares"},
    {"id": "attention_blog_near", "domain": "web_ml", "quality": 0.85, "text": "Scaled dot product attention compares"},
    {"id": "python_code", "domain": "code", "quality": 0.88, "text": "def load_config(path): read yaml settings, vali"},
    {"id": "python_code_fork", "domain": "code", "quality": 0.70, "text": "def load_config(file_path): read yaml sett"},
    {"id": "math_note", "domain": "math", "quality": 0.91, "text": "Jaccard similarity measures overlap between two s"},
    {"id": "benchmark_leak", "domain": "eval", "quality": 0.40, "text": "Benchmark problem: Alice has 3 apples and bu"},
    {"id": "canary_note", "domain": "privacy", "quality": 0.30, "text": "Synthetic canary QXZ-9142-LMN appears in a c"},
    {"id": "zh_data_note", "domain": "zh_web", "quality": 0.86, "text": "去重系统需要记录来源、规范化文本、哈希、注"}
]
```

```
eval_items = [
    {"id": "gsm_toy", "text": "Alice has 3 apples and buys 4 more apples. What is the correct answer?", "answer": "7"},
    {"id": "attention_eval", "text": "Explain scaled dot product attention and why softmax is applied to key scores."}
]
```

```
CANARIES = {"QXZ-9142-LMN"}
```

```
def tokens(text):
    return re.findall(r"[A-Za-z0-9_+|[\u4e00-\u9fff]", text.lower())
```

```
def normalize(text):
    return " ".join(tokens(text))
```

```
def stable_int(value):
    return int(hashlib.sha1(value.encode("utf-8")).hexdigest()[:16], 16)
```

```

def exact_hash(text):
    return hashlib.sha1(normalize(text).encode("utf-8")).hexdigest()[12:]

def shingles(text, n=3):
    toks = tokens(text)
    if len(toks) < n:
        return set(toks)
    return {" ".join(toks[i:i + n]) for i in range(len(toks) - n + 1)}

def jaccard(a, b):
    if not a and not b:
        return 1.0
    return len(a & b) / len(a | b)

def minhash_signature(shingle_set, size=64):
    if not shingle_set:
        return [0] * size
    return [
        min(stable_int(f"{seed}:{shingle}") for shingle in shingle_set)
        for seed in range(size)
    ]

def minhash_similarity(sig_a, sig_b):
    return sum(x == y for x, y in zip(sig_a, sig_b)) / len(sig_a)

def simhash(text, bits=32):
    weights = [0] * bits
    for tok in tokens(text):
        h = stable_int(tok)
        for bit in range(bits):
            weights[bit] += 1 if (h >> bit) & 1 else -1
    value = 0
    for bit, weight in enumerate(weights):
        if weight >= 0:
            value |= 1 << bit
    return value

def hamming(a, b):
    return (a ^ b).bit_count()

id_to_doc = {doc["id"]: doc for doc in train_docs}
shingle_map = {doc["id"]: shingles(doc["text"]) for doc in train_docs}
signatures = {doc_id: minhash_signature(s) for doc_id, s in shingle_map.items()}
simhashes = {doc["id"]: simhash(doc["text"]) for doc in train_docs}

hash_groups = defaultdict(list)
for doc in train_docs:

```

```

hash_groups[exact_hash(doc["text"])] .append(doc["id"])
exact_groups = [ids for ids in hash_groups.values() if len(ids) > 1]

near_pairs, simhash_pairs = [], []
for a, b in combinations([doc["id"] for doc in train_docs], 2):
    jac = jaccard(shingle_map[a], shingle_map[b])
    est = minhash_similarity(signatures[a], signatures[b])
    dist = hamming(simhashes[a], simhashes[b])
    is_exact_pair = any(set([a, b]).issubset(group) for group in exact_groups)
    if not is_exact_pair and (jac >= 0.42 or dist <= 6):
        near_pairs.append((a, b, round(jac, 3), round(est, 3), dist))
    if dist <= 7 and a != b:
        simhash_pairs.append((a, b, dist))

parent = {doc["id"]: doc["id"] for doc in train_docs}

def find(x):
    while parent[x] != x:
        parent[x] = parent[parent[x]]
        x = parent[x]
    return x

def union(a, b):
    ra, rb = find(a), find(b)
    if ra != rb:
        parent[rb] = ra

for group in exact_groups:
    for other in group[1:]:
        union(group[0], other)
for a, b, *_ in near_pairs:
    union(a, b)

clusters_by_root = defaultdict(list)
for doc in train_docs:
    clusters_by_root[find(doc["id"])].append(doc["id"])
clusters = [sorted(ids) for ids in clusters_by_root.values() if len(ids) > 1]

def choose_representative(ids):
    return max(ids, key=lambda doc_id: (
        id_to_doc[doc_id]["quality"],
        len(tokens(id_to_doc[doc_id]["text"])),
    ))

removed_by_dedup = set()
for cluster in clusters:
    rep = choose_representative(cluster)
    removed_by_dedup.update(set(cluster) - {rep})
dedup_kept = sorted(set(id_to_doc) - removed_by_dedup)

```

```

contamination_hits = {}
for item in eval_items:
    query_set = shingles(item["text"])
    best_doc, best_score = None, 0.0
    for doc in train_docs:
        score = jaccard(query_set, shingle_map[doc["id"]])
        answer_leak = item["answer"].lower() in tokens(doc["text"])
        if answer_leak and item["id"] == "gsm_toy":
            score += 0.20
        if score > best_score:
            best_doc, best_score = doc["id"], score
    if best_score >= 0.30:
        contamination_hits[item["id"]] = (best_doc, round(best_score, 3))

canary_hits = sorted(
    doc["id"] for doc in train_docs
    if any(canary in doc["text"] for canary in CANARIES)
)
quarantine_ids = {doc_id for doc_id, _ in contamination_hits.values()} | set(canary_hits)
final_kept = sorted(set(dedup_kept) - quarantine_ids)

raw_tokens = sum(len(tokens(doc["text"])) for doc in train_docs)
dedup_tokens = sum(len(tokens(id_to_doc[doc_id]["text"])) for doc_id in dedup_kept)
final_tokens = sum(len(tokens(id_to_doc[doc_id]["text"])) for doc_id in final_kept)
metrics = {
    "exact_groups": len(exact_groups),
    "near_pairs": len(near_pairs),
    "dedup_retention": round(dedup_tokens / raw_tokens, 3),
    "final_retention": round(final_tokens / raw_tokens, 3),
    "quarantine_count": len(quarantine_ids),
}
gates = {
    "exact_detected": len(exact_groups) == 1,
    "near_detected": len(near_pairs) >= 3,
    "contamination_detected": contamination_hits.get("gsm_toy", (None,))[0] == "benchmark_leak",
    "canary_detected": canary_hits == ["canary_note"],
    "representatives_kept": {"attention_blog", "python_code"}.issubset(final_kept),
}

print("exact_groups=", exact_groups)
print("near_pairs=", near_pairs)
print("simhash_close_pairs=", simhash_pairs[:5])
print("clusters=", sorted(clusters))
print("dedup_kept=", dedup_kept)
print("contamination_hits=", contamination_hits)
print("canary_hits=", canary_hits)
print("final_kept=", final_kept)
print("metrics=", metrics)
print("gates=", gates)
print("gate_pass=", all(gates.values()))

assert exact_groups == [["attention_blog", "attention_blog_dup"]]
assert len(near_pairs) >= 3
assert contamination_hits["gsm_toy"][0] == "benchmark_leak"
assert canary_hits == ["canary_note"]

```

```

assert final_kept == ["attention_blog", "math_note", "python_code", "zh_data_note"]
assert metrics == {
    "exact_groups": 1,
    "near_pairs": 3,
    "dedup_retention": 0.665,
    "final_retention": 0.461,
    "quarantine_count": 2,
}
assert all(gates.values())

```

运行后会看到类似输出：

```

exact_groups= [['attention_blog', 'attention_blog_dup']]
near_pairs= [('attention_blog', 'attention_blog_near', 0.576, 0.594, 4), ('attention_blog_dup', 'attention_blog_near', 0.576, 0.594, 4)]
simhash_close_pairs= [('attention_blog', 'attention_blog_dup', 0), ('attention_blog', 'attention_blog_near', 4), ('attention_blog_dup', 'attention_blog_near', 4)]
clusters= [['attention_blog', 'attention_blog_dup', 'attention_blog_near'], ['python_code', 'python_code_fork']]
dedup_kept= ['attention_blog', 'benchmark_leak', 'canary_note', 'math_note', 'python_code', 'zh_data_note']
contamination_hits= {'gsm_toy': ('benchmark_leak', 0.476)}
canary_hits= ['canary_note']
final_kept= ['attention_blog', 'math_note', 'python_code', 'zh_data_note']
metrics= {'exact_groups': 1, 'near_pairs': 3, 'dedup_retention': 0.665, 'final_retention': 0.461, 'quarantine_count': 2}
gates= {'exact_detected': True, 'near_detected': True, 'contamination_detected': True, 'canary_detected': True, 'representation_detected': True}
gate_pass= True

```

这个 demo 刻意把训练集内部去重和 train-eval contamination 分开：attention_blog、attention_blog_dup、attention_blog_near 属于训练集内部重复簇；benchmark_leak 是评测污染，应隔离；canary_note 是受控记忆风险探针，也不进入普通训练池。

22. 常见面试题

22.1 exact dedup 和 near dedup 的区别是什么？

exact dedup 识别完全相同的样本，通常通过规范化文本 hash 实现。near dedup 识别高度相似但不完全相同的样本，比如转载、改标题、加广告、轻微改写，通常用 n-gram、MinHash、SimHash、LSH 或 embedding 检索。

22.2 为什么大模型训练必须去重？

因为重复数据会浪费训练 token、降低有效多样性、增加记忆风险、扭曲来源分布，并可能造成 train-test overlap。去重能提升训练效率、降低复述训练数据的风险，并提高评估可信度。

22.3 MinHash 解决什么问题？

MinHash 用紧凑签名估计集合相似度，常用于大规模文本近重复检测。它把文档表示成 n-gram 集合，通过 signature 和 LSH 召回相似候选，避免全量两两比较。

22.4 LSH 的作用是什么？

LSH 的作用是候选召回。它让相似文档更可能落入同一桶，从而只比较少量候选对，而不是对所有文档两两比较。LSH 不是最终判定器，最终还要计算相似度并设置阈值。

22.5 benchmark contamination 怎么检测？

把评测集题目、答案、选项、解析、代码签名和测试样例作为 query，在训练语料中做 exact match、n-gram overlap、MinHash/LSH、embedding 检索和人工审计。对高相似结果隔离或在评测报告中标注风险。

22.6 去重是不是越彻底越好？

不是。过度去重会删除常见表达、常用代码模式、标准数学题型和真实用户意图多样性。目标是控制不合理重复，而不是消灭所有相似内容。需要按来源、领域、语言和任务目标设置策略。

22.7 canary 是什么？

canary 是受控插入或标记的特殊合成字符串，用来评估模型记忆和泄漏风险。它应使用非真实敏感信息，并服务于风险评估和防护验证，而不是复原训练数据。

23. 常见误区

误区一：去重就是删除完全相同的文件。

真实系统还需要近重复、段落重复、子串重复、代码 fork 和 benchmark contamination 检测。

误区二：去重后 token 数下降就是坏事。

如果删掉的是低价值重复，虽然 token 数减少，但有效信息密度提高。

误区三：embedding 相似就应该删除。

语义相似不等于重复。很多同主题文本都有不同细节，直接删除会损害覆盖率。

误区四：评测污染只需要查题目原文。

答案、解析、选项、测试样例和题解代码同样可能污染评估。

误区五：去重不会影响 data mixture。

去重会改变来源、语言、领域和质量分布，必须重新统计和调配。

误区六：训练集内部去重做好了，评测就一定可信。

不一定。还必须检测 train-eval overlap 和 benchmark-specific contamination。

24. 本章小结

去重与污染检测是大模型数据工程中连接训练质量、隐私风险和评估可信度的关键环节。

本章要记住几句话：

1. 重复数据会浪费算力、增加记忆、扭曲分布。
2. exact dedup 只能解决完全重复，near dedup 才能处理转载、改写和模板差异。
3. MinHash、SimHash 和 LSH 是 web-scale 去重的重要工具，但阈值必须审计。
4. benchmark contamination 会让评测分数虚高，必须单独检测。
5. canary 是评估记忆风险的受控探针，不应涉及真实敏感信息。
6. 去重不是越狠越好，它会改变 data mixture 和模型归纳偏置。
7. 成熟系统要记录规则、参数、版本、污染检测结果和审计报告。

如果面试中被问到去重，最好的回答不是只说“用 hash 去重”，而是讲完整链路：规范化、exact dedup、段落重复、near dedup、MinHash/LSH、代码专项、benchmark contamination、审计、ablation 和版本治理。

第五章：Data Mixture 与配比

前几章讨论了数据从哪里来、如何清洗、如何去重、如何避免污染。本章进入一个更隐蔽但更关键的问题：数据配比，也就是 data mixture。

所谓 data mixture，是指训练集中不同数据来源、语言、领域、模态和任务类型之间的比例安排。它回答的问题不是“有没有这些数据”，而是“这些数据各占多少、以什么顺序出现、用什么采样权重参与训练”。

很多人第一次接触大模型训练时，会以为数据配比只是工程配置：网页多少、书籍多少、代码多少、中文多少、英文多少，随便设一个比例就可以。实际上，data mixture 是训练目标的一部分。模型最终学到什么能力、偏向什么语言、擅长什么领域、输出什么风格，都会受到配比影响。

本章重点：通用文本、代码、数学、多语言、专业领域数据的混合策略和实验方法。

0. 本讲资料边界与第二轮精修口径

按照 WRITING_PLAN.md 的要求，本讲精修前核对了 Chinchilla、T5 / mT5、Gopher / MassiveText、RefinedWeb、FineWeb、Dolma、DataComp-LM / DCLM、Llama 系列公开报告和 phi-1 / textbook-quality data 相关资料。

本讲聚焦 data mixture 作为训练目标设计的一部分：数据池标签、token 配比、温度采样、质量加权、目标能力权重、风险约束、effective epoch、配比 ablation 和版本审计。

数据池 -> 清洗去重后 token 数 -> 质量 / 风险 / 能力标签 -> 采样权重 -> 训练预算分配 -> 评估矩阵 -> ablation -> 版本化

本讲不把任何公开模型的数据比例写成通用最佳答案。真实项目的 mixture 要由模型目标、token budget、数据质量、tokenizer、许可证、安全约束和评估结果共同决定。

1. 先建立直觉：配比决定模型“读过什么世界”

可以把预训练想象成给一个学生安排阅读书单。你不能只问“读了多少页”，还要问读了什么。

如果 90% 是网络论坛，学生会很熟悉口语和情绪表达，但可能缺少严谨知识。如果 90% 是论文，学生会更正式、更专业，但可能不懂普通用户怎么提问。如果 50% 是代码，学生会很会写程序，但普通对话和文学表达可能被压缩。如果英语占绝对多数，其他语言能力会被牺牲。

大模型也是一样。模型不是抽象地学习“智能”，它是在训练分布中学习统计规律。data mixture 决定了训练分布的形状。

因此，配比不是训练前随便拼数据，而是能力设计。

一个通用助手模型、代码模型、数学模型、医疗模型、金融模型、多语言模型、儿童教育模型，它们的理想 data mixture 都不一样。

2. 来龙去脉：从“越多越好”到“配比优化”

早期语言模型规模较小，数据主要来自新闻、百科、书籍或任务数据集。那时数据配比的重要性已经存在，但没有今天这么突出，因为模型容量和训练数据规模有限。

GPT-3 之后，web-scale 数据成为大模型训练主流。论文强调大规模无监督预训练带来 few-shot 能力，但也暴露出大型网页语料的质量、污染、偏见和来源分布问题。模型能力不只由参数量决定，也由训练语料决定。

Chinchilla 进一步提醒大家：在固定 compute budget 下，参数量和训练 token 数需要共同扩展。它把行业注意力从“模型越大越好”拉回到“同样算力下，模型大小和数据 token 数如何平衡”。但 Chinchilla 讨论的是 token 数和模型规模的宏观关系，真正落地时还要问：这些 token 来自哪里？什么质量？什么语言？什么领域？

phi-1 则从另一个角度强调高质量数据的重要性。它使用 textbook-quality web 数据和合成教材/练习数据，在较小模型规模下取得较强代码能力。这说明在某些目标上，数据质量和配比可以显著改变训练效率。

所以大模型数据工程的发展路线大致是：

1. 先追求规模：尽可能收集更多网页、书籍和代码。
2. 再追求质量：清洗、过滤、去重、降低污染。

3. 再追求配比：让不同数据类型按目标能力合理组合。
4. 最后追求闭环：用训练实验和评估反馈反向优化 data mixture。

3. Data mixture 到底在配什么？

data mixture 不只是“网页占多少”。它至少包含以下维度：

1. 来源维度：Common Crawl、书籍、论文、百科、代码仓库、论坛、问答、文档、课程、合成数据。
2. 语言维度：英语、中文、日语、法语、西班牙语、低资源语言、混合语言。
3. 领域维度：通用知识、科学、工程、法律、医学、金融、教育、娱乐、生活服务。
4. 能力维度：事实知识、推理、代码、数学、写作、翻译、对话、工具使用、安全拒答。
5. 质量维度：高质量教材、普通网页、低质量网页、人工标注、合成数据。
6. 风格维度：正式、口语、学术、新闻、社交媒体、教程、文档、对话。
7. 阶段维度：预训练、继续预训练、SFT、偏好训练、安全训练、领域适配。

同一份数据在不同维度上可能有不同标签。例如一份 Python 教程同时属于代码、英文、教育、技术文档、高质量结构化文本。配比系统最好支持多标签，而不是把数据硬塞进一个单一类别。

4. 为什么配比会影响能力？

模型训练本质上是在最小化训练分布上的预测误差。某类数据出现得越多，模型越有机会优化这类模式。

如果代码 token 占比提高，模型会更多看到缩进、函数、变量、API、注释、测试和错误信息，因此代码生成能力通常会增强。但如果代码占比过高，模型在普通自然语言上的训练机会相对减少，可能影响聊天、写作和开放问答。

如果数学数据占比提高，模型更常见到题目、证明、公式、推理步骤和答案格式，数学能力可能增强。但数学数据质量不高时，模型也可能学到错误推理模板。

如果多语言数据占比提高，非英语能力会提升，但在固定 token budget 下，英语和高资源领域的比例会下降。

这就是配比的基本 trade-off：训练 token 是有限预算。提高一种数据的采样权重，通常意味着降低另一种数据的有效权重。

4.1 关键公式与审计指标

设清洗、去重和污染隔离后的数据池集合为：

$$\mathcal{P} = \{P_1, P_2, \dots, P_K\}$$

每个数据池 P_k 有 token 数 n_k 、质量分 q_k 、风险分 r_k 和能力标签向量：

$$a_k = (a_{k, \text{general}}, a_{k, \text{code}}, a_{k, \text{math}}, a_{k, \text{multi}}, a_{k, \text{safety}})$$

最简单的自然配比是按清洗后 token 数采样：

$$p_k^{\text{nat}} = \frac{n_k}{\sum_j n_j}$$

多语言和多领域常用平滑采样。用 $0 \leq \alpha \leq 1$ 表示平滑强度，可以写成：

$$p_k(\alpha) = \frac{n_k^\alpha}{\sum_j n_j^\alpha}$$

其中 $\alpha=1$ 接近自然规模采样， $\alpha=0$ 接近均匀采样。实际工程里也可以把 α 写成温度的倒数，面试时重点讲清“平滑大池、抬高小池”的作用。

如果还要考虑质量、目标能力和风险，可以定义目标能力权重 v_m ，数据池效用为：

$$u_k = \sum_m v_m a_{k,m}$$

一个可解释的采样分数可以写成：

$$s_k = n_k^\alpha \exp(\beta q_k + \gamma u_k - \lambda r_k)$$

最终采样配比为：

$$p_k = \frac{s_k}{\sum_j s_j}$$

如果总训练预算为 B 个 token，则第 k 个数据池计划采样 token 数为：

$$b_k = B p_k$$

effective epoch 用于衡量小数据池被重复使用的程度：

$$e_k = \frac{b_k}{n_k}$$

如果 e_k 过高，说明该数据池被重复上采样，可能带来记忆、过拟合或污染风险。

对于某个能力维度 m ，mixture 的能力覆盖估计可以写成：

$$A_m(p) = \sum_k p_k a_{km}$$

配比方案的多目标效用可以粗略写成：

$$U(p) = \sum_m v_m A_m(p) - \lambda_r \sum_k p_{kr} - \lambda_e \sum_k \max(0, e_k - e_{\max})$$

这里第一项鼓励目标能力覆盖，第二项惩罚风险，第三项惩罚过度重复小数据池。

配比相对 baseline 的漂移可以用 KL divergence 监控：

$$D_{\{\mathrm{KL}\}}(p \parallel p^0) = \sum_k p_k \log \frac{p_k}{p_k^0}$$

它不是越小越好，而是帮助审计“这次 mixture 改动到底有多大”。

上线训练前可以把门禁写成：

$$G_{\{\mathrm{mix}\}} = I(\min_m A_m(p) \geq a_{\min}) I(\max_k e_k \leq e_{\max}) I(\sum_k p_{kr} \leq r_{\max}) I(D_{\{\mathrm{KL}\}}(p \parallel p^0) \leq \delta)$$

面试里要强调：这些公式只是把配比思路显式化。真实训练仍要靠 ablation、小模型预实验、多维评估和大规模复验来校准。

5. 自然分布 vs 人工配比

一种简单做法是按数据自然规模混合：哪个来源 token 多，就占更大比例。比如网页最多，网页就占绝大多数；英语最多，英语就占绝大多数。

这种做法有好处：

1. 简单。
2. 接近真实 web 分布。
3. 不需要太多人工假设。
4. 覆盖范围广。

但缺点很明显：

1. web 分布不等于理想训练分布。
2. 高资源语言会压制低资源语言。
3. 娱乐、营销、模板和低质内容可能过多。
4. 数学、代码、专业知识等高价值数据可能被稀释。
5. 模型能力会被“互联网上什么最多”支配。

因此，成熟系统通常会人工调配。比如上采样代码、数学、高质量书籍、技术文档和低资源语言；下采样低质量网页、重复来源和模板化内容。

人工配比的风险是引入主观偏置。团队认为重要的数据，不一定对最终模型最优。所以配比必须通过实验验证，而不是只靠直觉。

6. 通用文本的角色

通用文本通常包括网页正文、百科、书籍、新闻、论坛、问答、博客和文档。它是 base model 的主体，因为它提供世界知识、语言建模能力、常识、事实、表达风格和多样话题。

通用文本的优点：

1. 覆盖广。
2. 规模大。
3. 风格多样。
4. 能支撑开放域问答和通用语言能力。

缺点：

1. 质量不均。
2. 噪声多。
3. 重复和污染风险高。
4. 事实可信度不稳定。
5. 可能包含偏见、毒性和隐私信息。

配比时要避免两个极端：通用文本太少，模型缺少世界知识；通用文本太多，高价值专业能力被稀释。

7. 代码数据的配比

代码数据对现代大模型非常重要。即使模型不是专门代码模型，适量代码数据也可能提升结构化推理、符号处理、工具使用和长程依赖建模能力。

代码数据包括：

1. 源代码文件。
2. README 和技术文档。
3. issue 和 pull request。
4. 单元测试。
5. API 文档。
6. 编程问答。
7. 教程和代码解释。

提高代码比例通常会增强：

1. 代码补全。
2. bug 修复。
3. API 使用。
4. 结构化输出。
5. 算法题能力。

但代码占比过高可能带来：

1. 普通自然语言能力下降。
2. 输出风格更像代码或模板。
3. 许可证和版权风险上升。
4. secrets、凭据和私有片段风险增加。
5. benchmark contamination 风险增加，尤其是编程题库。

代码数据配比要结合模型定位。如果是通用助手，可以保留适中比例；如果是代码模型，需要显著提高代码及相关解释数据；如果是行业模型，代码可能只是辅助能力。

8. 数学和推理数据的配比

数学数据通常比普通 web 文本更稀缺，但对推理能力很重要。它包括教材、题库、解答、证明、竞赛题、合成推理题、符号推导和数学论坛。

数学数据的特殊性在于：质量比数量更重要。错误解答、跳步解答、格式混乱和答案泄漏会严重误导模型。

提高数学数据比例可能带来：

1. 算术能力提升。
2. 多步推理能力提升。
3. 公式理解能力提升。
4. 解题格式更稳定。
5. 对 Chain-of-Thought 类数据更敏感。

但风险包括：

1. 题库污染。
2. 模板化推理。
3. 错误步骤被学习。
4. 对自然语言表达挤占。
5. 合成数据同质化。

数学配比要特别重视评估闭环。不能只看数学 benchmark 提升，还要看通用能力是否下降、污染是否可控、推理是否真实泛化。

9. 多语言数据的配比

多语言配比是大模型训练中最难的问题之一。互联网上英语数据规模和质量通常远超许多语言。如果按自然分布训练，模型会强烈偏向英语。

常见策略包括：

1. 对低资源语言上采样。
2. 对高资源语言下采样。
3. 按语言质量分设置不同阈值。
4. 使用平滑采样，而不是完全按 token 数采样。
5. 对关键目标市场语言设置最低比例。
6. 使用翻译数据或平行语料增强跨语言能力。

多语言配比的 trade-off 很明显：低资源语言比例提高，会改善这些语言能力，但会占用固定 token budget。对于通用英文能力、代码能力和专业领域能力，可能产生机会成本。

还要注意，低资源语言的数据质量过滤不能直接套用英语规则。否则会因为语言识别误差、字符分布差异、短文本特征不同而误删大量有效数据。

10. 专业领域数据的配比

专业领域数据包括医学、法律、金融、材料、生物、芯片、机器人、教育等。它的价值在于提升专业问答、术语理解、领域推理和行业落地能力。

但专业数据配比有几个难点：

1. 高质量数据稀缺。
2. 授权和合规要求更高。
3. 专家审核成本高。
4. 错误知识风险更大。
5. 与通用能力存在 trade-off。

专业领域数据通常有三种用法：

1. 预训练阶段少量混入，提供基本术语和背景。
2. 继续预训练阶段提高比例，做领域适配。
3. SFT/RAG/工具阶段解决高准确率和实时知识问题。

不要把所有专业能力都压到预训练配比上。对于医学、法律、金融这类高风险领域，单靠预训练记忆不是可靠方案，通常需要检索、工具、引用、审核和安全策略配合。

11. 合成数据的配比

合成数据可以来自规则生成、程序生成、模拟器、强模型生成、专家模板或数据增强。它在代码、数学、指令跟随、安全训练和低资源语言中很常见。

合成数据的优点：

1. 可控。
2. 可定向补能力短板。
3. 格式统一。
4. 能生成稀缺任务。
5. 可用于构造难例和安全样本。

缺点也明显：

1. 同质化。
2. 事实错误。
3. 风格模板化。
4. 可能继承教师模型偏差。
5. 过量使用会让模型学习生成器的分布，而不是现实分布。

phi-1 的经验说明，高质量合成教材和练习对代码能力很有帮助。但这并不意味着合成数据越多越好。关键在于质量、覆盖、多样性、验证和配比。

一个稳妥原则是：合成数据要有明确目的、明确来源、明确标签、明确评估，不要不可追踪地混入自然语料。

12. 数据配比和训练阶段

不同训练阶段的 data mixture 应该不同。

12.1 预训练

预训练目标是建立广泛语言能力、世界知识和基础模式。配比要重视覆盖面、多样性和有效 token 数。

12.2 继续预训练

继续预训练常用于领域适配、语言适配或能力增强。此时可以提高目标领域数据比例，但要防止 catastrophic forgetting，即模型在新领域变强的同时通用能力下降。

12.3 SFT

SFT 的数据更偏指令、问答、对话、工具调用和格式规范。这里的配比决定助手风格、任务覆盖和指令遵循能力。

12.4 偏好训练

偏好训练数据配比影响模型对回答风格、安全边界、简洁性、帮助性和拒答策略的偏好。

12.5 安全训练

安全数据不能简单按自然分布采样。需要覆盖风险类别、拒答边界、合规场景、误拒问题和多语言安全表达。

13. 静态配比 vs 动态配比

静态配比是在整个训练过程中使用固定采样比例。例如网页 60%、代码 15%、数学 5%、书籍 10%、多语言 10%。

优点是简单、可复现、易分析。缺点是不够灵活。

动态配比会随训练阶段调整。例如：

1. 前期更多通用文本，建立基础语言能力。
2. 中期增加代码、数学和高质量知识数据。
3. 后期增加高质量、低噪声和目标能力数据。
4. 对 loss 下降慢或评估薄弱的数据域增加采样。

动态配比更像 curriculum learning，但也更难评估。因为最终效果来自配比、顺序、学习率、数据质量和训练阶段共同作用。

面试中可以说：静态配比适合第一版稳定训练，动态配比适合有充分实验体系的成熟团队。

14. Temperature sampling 和多语言平滑

多语言训练常用一种思想：不要完全按原始 token 数采样，而是对语言分布做平滑。

如果按原始规模采样，英语等高资源语言会占据绝大多数。若完全均匀采样，低资源语言可能被过度重复，导致过拟合和质量下降。因此需要介于两者之间的策略。

直观做法是：对每种语言的 token 占比做平滑，降低大语言的优势，提高小语言的采样概率。

温度越高，分布越接近均匀；温度越低，越接近原始分布。不同团队记号可能不同，面试中不必纠结公式，关键是讲清目的：在高资源语言和低资源语言之间做折中。

这种思想也可以用于领域配比：对小而高价值的数据源上采样，对大而低价值的数据源下采样。

15. 配比实验怎么做？

data mixture 不能只靠直觉，必须通过实验闭环。

典型流程：

1. 定义目标能力：通用问答、代码、数学、多语言、专业领域、安全等。
2. 建立候选数据池：每个池有来源、质量分、语言、领域、去重状态和许可证信息。
3. 设计 baseline mixture：基于经验和目标设初始比例。
4. 做小规模训练：使用较小模型和较少 token 快速验证趋势。
5. 做 ablation：单独提高或降低某类数据比例。
6. 建立评估矩阵：不要只看一个 benchmark。
7. 分析 trade-off：某项能力提升是否损害其他能力。
8. 固化版本：记录 mixture 配置、采样权重、数据版本和训练结果。
9. 放大验证：在更大模型或更多 token 上验证是否保持趋势。

配比实验最难的是外推。小模型上的最佳配比不一定适用于大模型；短训练上的最佳配比不一定适用于长训练。因此需要多尺度实验，而不是一次 ablation 就决定最终配比。

16. 评估矩阵：配比优化看什么指标？

一个配比方案不能只看平均分。它应该看多维指标：

1. 通用语言 loss。
2. 各来源 validation loss。
3. 下游 benchmark。
4. 代码能力。
5. 数学能力。
6. 多语言能力。
7. 专业领域能力。
8. 安全评估。
9. 幻觉和事实性。
10. 输出风格。
11. 记忆和污染风险。
12. 人工评测和产品样例。

例如，提高代码比例后，HumanEval 类指标可能提升，但多语言聊天能力下降。提高数学合成数据后，GSM 类指标可能提升，但模型回答变得模板化。提高论坛数据后，聊天自然度提升，但安全风险上升。

因此配比优化本质是多目标优化，而不是单指标爬山。

17. 数据上采样和过拟合

当某类数据很重要但规模小，团队常会对它上采样。比如低资源语言、数学题、代码解释、专业文档。

上采样能提高模型 exposure，但也带来过拟合和记忆风险。

如果一个小数据池被重复训练太多次，模型可能记住具体样本，而不是学到泛化规律。尤其是题库、隐私文本、专业文档和代码数据，更要谨慎。

缓解方法包括：

1. 增加数据多样性。
 2. 做去重和污染检测。
 3. 控制 epoch 数。
 4. 使用数据增强或合成变体，但要验证质量。
 5. 在评估中加入记忆检测。
 6. 对小数据池使用更高质量阈值。
-

18. 配比和 tokenizer 的关系

不同语言和数据类型在 tokenizer 下的 token 化效率不同。同样 1000 个字符，英文、中文、代码、数学公式和低资源语言可能对应不同 token 数。

如果配比按 token 计算，tokenizer 会影响实际信息量。如果某种语言 token 化效率低，它在相同 token budget 下能表达的信息更少，训练成本更高。

所以多语言配比不能只看原始字符数或文档数，也要看 token 分布、压缩效率和模型实际 loss。

代码和数学也类似。符号、缩进、特殊字符会影响 token 数。配比实验需要基于最终 tokenizer 统计，而不是原始文本大小。

19. 配比和去重清洗的关系

配比发生在清洗和去重之后，但也会反过来影响清洗策略。

比如某个领域数据很稀缺，如果用普通规则过滤，可能被误删很多；为了保留这个领域，团队可能降低某些过滤阈值，同时加强人工审计。又比如某个来源规模巨大且重复率高，去重后占比下降，需要重新计算采样权重。

因此，data mixture 不是单纯地在最后拼接文件，而是和数据清洗、质量评分、去重、污染检测共同设计。

成熟的数据表通常会保存：source、language、domain、quality score、dedup cluster、safety label、license、token count、sampling weight。训练时 sampler 根据这些字段动态采样。

20. 面向专家：Data mixture 是隐式目标函数设计

从专家视角看，data mixture 实际上是在设计隐式目标函数。

预训练通常形式上只有一个 next-token prediction loss，但这个 loss 是在 mixture 分布上求期望。改变 mixture，就改变了优化目标。

如果把代码采样权重加倍，优化器会更频繁地沿着代码样本梯度更新。如果把数学数据放到训练后期，模型后期参数会更受数学分布影响。如果把低质量网页下采样，模型对这些风格的拟合会减少。

因此，配比不是数据团队的外围工作，而是训练目标、模型能力和产品定位之间的接口。

专家级 trade-off 包括：

1. 多样性 vs 高质量。
 2. 通用能力 vs 专项能力。
 3. 高资源语言 vs 低资源语言。
 4. 自然数据 vs 合成数据。
 5. 预训练吸收 vs 后训练适配。
 6. 短期 benchmark 提升 vs 长期泛化。
 7. 上采样能力短板 vs 记忆和污染风险。
-

21. 一个可落地的 data mixture 方案

如果面试官问：“你要训练一个通用中文/英文大模型，如何设计数据配比？”可以按以下框架回答。

第一步，定义模型目标。明确是通用助手、代码助手、数学模型、多语言模型还是行业模型，以及核心评估指标。

第二步，建立数据池。把数据按来源、语言、领域、质量、安全、许可证和去重状态打标签。

第三步，设计初始配比。通用文本作为主体，适量加入高质量书籍、百科、论文、技术文档、代码、数学、多语言和合成数据。

第四步，设置采样权重。不要只按原始 token 数采样，对高质量、小规模、目标能力相关数据上采样，对低质量、大规模、重复来源下采样。

第五步，做小规模实验。训练多个小模型，对比不同代码比例、数学比例、多语言比例和高质量数据比例。

第六步，建立评估矩阵。覆盖通用问答、事实性、代码、数学、多语言、安全、长文本和人工样例。

第七步，分析 trade-off。看专项能力提升是否损害通用能力，是否增加幻觉、污染、记忆或安全风险。

第八步，放大验证。在更大模型和更多 token 上复验趋势，避免小规模结论误导。

第九步，版本化记录。保存 mixture 配置、数据版本、采样权重、过滤规则、训练曲线和评估结果。

第十步，迭代优化。根据训练结果、产品反馈和安全评估继续调整。

这套回答比直接报一个比例更好，因为真实项目中没有放之四海皆准的比例，只有目标驱动的实验闭环。

21.1 最小可运行 data mixture 审计 demo

下面这个 demo 不依赖外部库，也不读写文件。输入是一组 toy 数据池，每个池有清洗后 token 数、质量分、风险分和能力标签；输出包括自然配比、目标采样配比、计划采样 token、effective epoch、上采样倍数、能力覆盖、平均质量、风险和门禁结果。

它演示的是配比审计机制，不是生产级 sampler。真实训练还需要和 tokenizer 统计、数据版本、分布式采样器、训练曲线、验证 loss、下游评估和安全审计联动。

```
import math
```

```
pools = [
    {"name": "general_web", "tokens": 500_000, "quality": 0.72, "risk": 0.06, "cap": {"general": 0.85, "code": 0.10, "math": 0.20, "multilingual": 0.15, "safety": 0.10}},
    {"name": "books_reference", "tokens": 120_000, "quality": 0.88, "risk": 0.02, "cap": {"general": 0.75, "code": 0.95, "math": 0.25, "multilingual": 0.10, "safety": 0.05}},
    {"name": "code_docs", "tokens": 80_000, "quality": 0.90, "risk": 0.04, "cap": {"general": 0.25, "code": 0.95, "math": 0.25, "multilingual": 0.10, "safety": 0.05}},
    {"name": "math_reasoning", "tokens": 30_000, "quality": 0.93, "risk": 0.03, "cap": {"general": 0.20, "code": 0.25, "math": 0.95, "multilingual": 0.10, "safety": 0.05}},
    {"name": "zh_multilingual", "tokens": 70_000, "quality": 0.82, "risk": 0.04, "cap": {"general": 0.45, "code": 0.05, "math": 0.20, "multilingual": 0.95, "safety": 0.10}},
    {"name": "domain_science", "tokens": 40_000, "quality": 0.87, "risk": 0.05, "cap": {"general": 0.55, "code": 0.10, "math": 0.20, "multilingual": 0.10, "safety": 0.95}},
    {"name": "synthetic_reasoning", "tokens": 20_000, "quality": 0.86, "risk": 0.08, "cap": {"general": 0.35, "code": 0.10, "math": 0.20, "multilingual": 0.10, "safety": 0.10}},
    {"name": "safety_data", "tokens": 10_000, "quality": 0.92, "risk": 0.02, "cap": {"general": 0.15, "code": 0.05, "math": 0.20, "multilingual": 0.10, "safety": 0.95}},
]
```

```
target = {"general": 0.35, "code": 0.20, "math": 0.20, "multilingual": 0.15, "safety": 0.10}
```

```
BUDGET = 200_000
```

```
SMOOTHING_POWER = 0.65
```

```
QUALITY_BETA = 1.2
```

```
TARGET_GAMMA = 0.9
```

```
RISK_LAMBDA = 1.8
```

```
MAX_EPOCH = 2.5
```

```
def normalize(weights):
    total = sum(weights.values())
    return {k: v / total for k, v in weights.items()}
```

```
def utility(pool):
    return sum(target[k] * pool["cap"][k] for k in target)
```

```
raw_mix = normalize({p["name"]: p["tokens"] for p in pools})
raw_ability = {
    cap: round(sum(raw_mix[p["name"]] * p["cap"][cap] for p in pools), 3)
    for cap in target
}
```

```
scores = {}
for p in pools:
    scores[p["name"]] = (
        (p["tokens"] ** SMOOTHING_POWER)
        * math.exp(
            QUALITY_BETA * (p["quality"] - 0.75)
            + TARGET_GAMMA * utility(p)
            - RISK_LAMBDA * p["risk"]
        )
    )
```

```

mixture = normalize(scores)

plan = {}
for p in pools:
    name = p["name"]
    planned = round(BUDGET * mixture[name])
    plan[name] = {
        "mix": round(mixture[name], 3),
        "raw_mix": round(raw_mix[name], 3),
        "planned_tokens": planned,
        "effective_epoch": round(planned / p["tokens"], 3),
        "utility": round(utility(p), 3),
    }

ability = {
    cap: round(sum(mixture[p["name"]] * p["cap"][cap] for p in pools), 3)
    for cap in target
}
risk = round(sum(mixture[p["name"]] * p["risk"] for p in pools), 3)
avg_quality = round(sum(mixture[p["name"]] * p["quality"] for p in pools), 3)
up_sampling = {
    name: round(plan[name]["mix"] / max(plan[name]["raw_mix"], 1e-9), 2)
    for name in sorted(plan)
}
watchlist = sorted(
    name for name, row in plan.items()
    if row["effective_epoch"] > MAX_EPOCH
)
gates = {
    "code_floor": ability["code"] >= 0.21,
    "math_floor": ability["math"] >= 0.28,
    "multilingual_floor": ability["multilingual"] >= 0.25,
    "safety_floor": ability["safety"] >= 0.21,
    "risk_ok": risk <= 0.055,
    "epoch_ok": not watchlist,
}

print("raw_mix=", {k: round(raw_mix[k], 3) for k in sorted(raw_mix)})
print("target_mix=", {k: plan[k]["mix"] for k in sorted(plan)})
print("planned_tokens=", {k: plan[k]["planned_tokens"] for k in sorted(plan)})
print("effective_epoch=", {k: plan[k]["effective_epoch"] for k in sorted(plan)})
print("upsampling=", up_sampling)
print("raw_ability=", raw_ability)
print("target_ability=", ability)
print("avg_quality=", avg_quality, "risk=", risk)
print("watchlist=", watchlist)
print("gates=", gates)
print("gate_pass=", all(gates.values()))

assert round(sum(mixture.values()), 6) == 1.0
assert watchlist == []
assert ability == {"general": 0.591, "code": 0.218, "math": 0.285, "multilingual": 0.256, "safety": 0.212}
assert avg_quality == 0.823
assert risk == 0.045
assert all(gates.values())

```

运行后会看到类似输出：

```
raw_mix= {'books_reference': 0.138, 'code_docs': 0.092, 'domain_science': 0.046, 'general_web': 0.575, 'math_reasoning': 0.046}
target_mix= {'books_reference': 0.174, 'code_docs': 0.133, 'domain_science': 0.078, 'general_web': 0.351, 'math_reasoning': 0.046}
planned_tokens= {'books_reference': 34896, 'code_docs': 26671, 'domain_science': 15534, 'general_web': 70245, 'math_reasoning': 15534}
effective_epoch= {'books_reference': 0.291, 'code_docs': 0.333, 'domain_science': 0.388, 'general_web': 0.14, 'math_reasoning': 0.14}
upsampling= {'books_reference': 1.26, 'code_docs': 1.45, 'domain_science': 1.7, 'general_web': 0.61, 'math_reasoning': 2.0}
raw_ability= {'general': 0.693, 'code': 0.176, 'math': 0.206, 'multilingual': 0.261, 'safety': 0.218}
target_ability= {'general': 0.591, 'code': 0.218, 'math': 0.285, 'multilingual': 0.256, 'safety': 0.212}
avg_quality= 0.823 risk= 0.045
watchlist= []
gates= {'code_floor': True, 'math_floor': True, 'multilingual_floor': True, 'safety_floor': True, 'risk_ok': True, 'epoch_ok': True}
gate_pass= True
```

这个 demo 刻意把 `general_web` 从自然分布的 0.575 下调到 0.351，同时上采样代码、数学、安全和合成推理数据。结果是通用能力覆盖下降，但代码和数学覆盖上升；这正是 data mixture 的 trade-off，不能只看单项指标。

22. 常见面试题

22.1 什么是 data mixture?

data mixture 是训练集中不同来源、语言、领域、能力类型和质量层级数据的混合比例与采样策略。它决定模型在哪些数据分布上优化 next-token prediction，因此会影响模型能力、风格和安全表现。

22.2 为什么不能按数据自然规模混合?

因为互联网自然分布不等于理想训练分布。按自然规模混合会让高资源语言、大规模网页和低质来源占据过高比例，稀释代码、数学、专业领域和低资源语言等高价值数据。

22.3 代码数据比例提高一定好吗?

不一定。提高代码比例通常提升代码和结构化推理能力，但会占用 token budget，可能影响自然语言、多语言和对谈能力，还会增加 license、secrets、重复和题库污染风险。

22.4 多语言配比怎么做?

不能完全按原始 token 数采样，也不能简单均匀采样。通常对低资源语言上采样、对高资源语言适当下采样，用平滑采样在覆盖和质量之间折中，并分语言评估 loss、下游任务和人工样例。

22.5 合成数据应该占多少?

没有固定答案。合成数据要看目标任务、质量、覆盖、多样性和教师模型偏差。通常应明确标注来源，单独评估，逐步增加比例，通过 ablation 判断收益和副作用，而不是无追踪地混入训练集。

22.6 如何优化 data mixture?

先定义目标能力和评估矩阵，再构建带标签的数据池，设计 baseline mixture，做小规模 ablation，观察各领域 loss、benchmark、安全和人工评测，分析 trade-off，最后在更大规模上验证并版本化记录。

22.7 配比和 Chinchilla scaling 有什么关系?

Chinchilla 说明在固定 compute budget 下，模型参数和训练 token 数都很重要。但 token 数只是总量问题，data mixture 解决的是这些 token 来自哪里、质量如何、各能力如何分布。两者是互补关系。

23. 常见误区

误区一：data mixture 只是数据拼接。

它实际上决定训练分布，是隐式目标函数设计。

误区二：比例有标准答案。

不同模型目标、token budget、数据质量和评估体系下，最佳比例不同。

误区三：高质量数据越多越好。

高质量很重要，但过度偏向单一高质量风格会损失真实世界多样性。

误区四：专项数据越多专项能力越强。

超过一定比例后可能收益递减，并带来过拟合、记忆、污染和通用能力下降。

误区五：小模型 ablation 结论可以直接放大。

小模型趋势有参考价值，但大模型、长训练和不同 tokenizer 下可能变化。

误区六：SFT 可以弥补所有预训练配比问题。

SFT 能调整行为和格式，但很难补齐预训练阶段缺失的大规模知识和语言能力。

24. 本章小结

Data mixture 与配比是大模型数据工程中最接近“模型能力设计”的环节。它不是把数据随便混在一起，而是在有限 token budget 下决定模型读什么、读多少、什么时候读、以多大权重读。

本章要记住几句话：

1. data mixture 决定训练分布，也就改变隐式优化目标。
2. 通用文本提供覆盖，代码和数学提供结构化能力，多语言和专业数据提供目标能力扩展。
3. 配比不是自然规模决定的，而是目标、质量、风险和实验共同决定的。
4. 合成数据有价值，但必须可追踪、可评估、可控比例使用。
5. 配比优化是多目标问题，要看通用、代码、数学、多语言、安全、事实性和风格的 trade-off。
6. 最成熟的做法不是报固定比例，而是建立数据池、采样权重、评估矩阵、ablation 和版本化闭环。

如果面试中被问到 data mixture，最好的回答是：先定义模型目标，再建立带标签数据池，设计 baseline mixture，通过小规模实验和多维评估找 trade-off，最后在大规模训练中验证并版本化记录。

第六章：Code、Math 与 Domain Data

前一章讨论了 data mixture 与配比。本章进一步拆开三类在大模型训练中特别重要的数据：代码数据、数学数据和领域数据。

这三类数据有一个共同点：它们的规模通常不如通用网页文本大，但对模型能力的边际价值很高。代码数据能显著增强程序合成、结构化推理和工具使用能力；数学数据能增强多步推理、符号操作和验证意识；专业领域数据能让模型理解医学、法律、金融、科研、工程等高价值场景。

但它们也有共同风险：采集难、清洗难、授权难、污染风险高、质量判断成本高、过度上采样容易记忆和过拟合。

本章重点：代码仓库数据、单测、数学推理数据、医学法律金融等领域数据。

合规边界：本章讨论数据治理、质量控制、授权合规、安全评估和污染检测，不提供利用代码漏洞、泄露凭据、绕过系统或获取敏感领域数据的操作方法。

0. 本讲资料边界与第二轮精修口径

按照 WRITING_PLAN.md 的要求，本讲精修前核对了 Codex / HumanEval、GSM8K / verifier、MATH、The Stack / BigCode / StarCoder、GitHub secret scanning、Med-PaLM、PubMedQA、FinGPT 和 LegalBench 等公开资料。

本讲聚焦 code / math / domain data 的防御性数据治理：专项数据池、许可证和来源审查、secret / PII 过滤、题库和 benchmark 污染隔离、功能测试和 verifier、领域权威性 / 时效性 / 引用审计，以及这些数据如何进入 mixture、继续预训练、SFT 和 RAG。

专项数据池 -> 授权 / 来源 -> 结构解析 -> 质量验证 -> 风险扫描 -> 污染隔离 -> 采样配比 -> 评估闭环 -> 版本审计

本讲不提供漏洞利用、凭据提取、真实隐私复原、医疗法律金融决策建议或绕过数据权限的操作方法。高风险领域数据只讨论训练数据治理和模型边界，不把模型输出替代专家判断。

1. 先建立直觉：为什么这三类数据值得单独讲？

通用 web 数据像大海，覆盖广但质量不均。代码、数学和专业领域数据更像高浓度营养液。它们规模未必最大，但会强烈影响模型的某些核心能力。

代码数据让模型学习形式语言、抽象接口、变量绑定、长程依赖、测试驱动和机器可执行逻辑。数学数据让模型学习约束、步骤、推理链、答案校验和符号结构。专业领域数据让模型学习术语、概念体系、规范表达、领域证据和行业场景。

如果训练一个面向算法岗、工程岗或行业落地的大模型，这三类数据不能只是“顺便混一点”。它们必须被单独建池、单独清洗、单独评估、单独做污染检测。

2. 来龙去脉：从通用语言建模到能力定向数据

早期语言模型主要依赖新闻、百科、书籍和网页文本。模型重点学习自然语言流畅性和通用知识。随着 GPT-3 展示 few-shot 能力，行业意识到大规模预训练可以带来跨任务迁移。

但很快大家发现：通用语言能力不等于所有能力。模型能写文章，不代表会写正确代码；能解释概念，不代表能做多步数学；能给出医疗术语，不代表能在高风险专业场景中可靠。

Codex 的出现强化了代码数据的重要性。Evaluating Large Language Models Trained on Code 介绍了在公开 GitHub 代码上微调的 Codex，并用 HumanEval 评估从 docstring 合成程序的功能正确性。结果显示，代码数据和代码专项评估可以显著改变模型能力。

GSM8K 则提醒大家数学推理需要高质量问题和验证机制。Training Verifiers to Solve Math Word Problems 提出 GSM8K，并展示训练 verifier 对候选解排序可以改善数学 word problem 表现。这说明数学能力不只是背题，而涉及生成、验证和选择。

专业领域模型的发展也说明：通用语料覆盖不到的行业知识、术语和规范，需要专门的数据治理。医学、法律、金融等领域尤其不能只靠 web 噪声训练，因为错误回答可能带来现实风险。

3. 三类数据的共同特征

代码、数学和专业领域数据虽然内容不同，但工程问题相似。

共同特征包括：

1. 高价值：对特定能力提升明显。
2. 高稀缺：高质量数据远少于普通网页。
3. 高结构：代码有语法树，数学有步骤和公式，领域文本有术语和规范。

4. 高污染风险：很多 benchmark、题库、参考答案和标准文档公开可见。
5. 高合规要求：代码有 license，领域数据有隐私、版权和行业监管。
6. 高评估成本：需要功能测试、答案验证、专家审核或领域 benchmark。
7. 高过拟合风险：小数据池被上采样后容易记忆。

因此，这三类数据不能按普通网页处理。

3.1 关键公式与审计指标

设专项数据池由代码、数学和领域数据组成：

$$D_{\{\mathrm{spec}\}} = D_{\{\mathrm{code}\}} \cup D_{\{\mathrm{math}\}} \cup D_{\{\mathrm{domain}\}}$$

每个样本可以表示为：

$$x_i = (m_i, T_i, q_i, r_i, z_i)$$

其中 m_i 是数据类型，取值为 code、math 或 domain； T_i 是 token 数； q_i 是质量分； r_i 是风险分； z_i 是来源、许可证、时间戳、去重簇、污染标记等元数据。

代码数据常见的功能测试通过率为：

$$R_{\{\mathrm{test}\}, i} = \frac{n_{\{\mathrm{pass}\}, i}}{\max(n_{\{\mathrm{test}\}, i}, 1)}$$

一个可解释的代码样本质量分可以写成：

$$q_i^{\{\mathrm{code}\}} = w_{pI} \{\mathrm{parse}\}, i + w_{tR} \{\mathrm{test}\}, i + w_{dd} \lambda_g \lambda_u$$

其中 I_{parse} 表示语法或解析检查是否通过， d_i 是文档 / 注释 / 测试配套分， g_i 是生成文件、vendor、bundle 等低价值标记， u_i 是重复或 fork 风险。

数学样本可以同时看答案验证和过程质量：

$$q_i^{\{\mathrm{math}\}} = w_{aI} \{\mathrm{ans}\}, i + w_{sh} \lambda_c$$

其中 I_{ans} 表示答案可验证， s_i 是过程完整性分， h_i 是难度或覆盖价值， c_i 是 benchmark / 题库污染风险。

领域样本更强调权威性、时效性、引用和隐私风险：

$$q_i^{\{\mathrm{domain}\}} = w_{AA} + w_{R\rho} + w_{CI} \{\mathrm{cite}\}, i - \lambda_{PP} - \lambda_{SS} - \lambda_{VV}$$

其中 A_i 是来源权威性， ρ_i 是时效性， I_{cite} 表示是否保留出处或引用， P_i 是 PII 风险， S_i 是过期或 stale 风险， V_i 是高风险建议或未经验证观点风险。

专项数据的最终门禁可以写成：

$$G_i = I(q_i^{m_i} \geq \tau_{m_i}) I_{\{\mathrm{license}\}, i} I_{\{\mathrm{secret}\}, i} I_{\{\mathrm{contam}\}, i} I_{\{\mathrm{privacy}\}, i}$$

这里不同类型的数据使用不同阈值 τ_m 。代码要重视 license、secret、测试和 fork；数学要重视答案、过程和题库污染；领域数据要重视来源、时间、PII、引用和专家审计。

按类型计算 token 保留率：

$$R_{\{\mathrm{keep}\}}(m) = \frac{\sum_{i:m_i=m} G_i T_i}{\sum_{i:m_i=m} T_i}$$

按类型计算风险命中率：

$$R_{\{\mathrm{risk}\}}(m) = \frac{\sum_{i:m_i=m} I(r_i > 0)}{\sum_{i:m_i=m} 1}$$

如果专项数据计划采样 token 数为 b_m ，清洗后可用 token 数为 N_m ，则 effective epoch 为：

$$e_m = \frac{b_m}{N_m}$$

对于代码、数学、专业文档和合成推理数据， e_m 过高通常意味着更高的记忆和污染风险，需要降权、扩充数据、增强去重或重新设计采样策略。

专项数据上线前可以设置门禁：

$$G_{\{\mathrm{spec}\}} = I(R_{\{\mathrm{keep}\}}(m) \geq r_{\min}) I(R_{\{\mathrm{risk}\}}(m) \leq r_{\max}) I(e_m \leq e_{\max})$$

面试里要强调：代码、数学和领域数据不是“多多益善”，而是要用类型专属质量指标、风险指标和评估矩阵一起控制。

4. Code data：代码数据为什么重要？

代码是一种非常特殊的语言。它既像自然语言，又不是自然语言。它有严格语法、可执行语义、依赖关系、模块结构、类型约束和测试反馈。

代码数据可以训练模型学习：

1. 语法结构。
2. 变量绑定。
3. 函数抽象。
4. API 调用。
5. 模块组织。
6. 错误处理。
7. 测试用例。
8. 注释和实现的对应关系。
9. 需求到代码的映射。
10. 代码到解释的映射。

即使目标不是代码助手，代码数据也可能帮助模型形成更强的结构化输出能力。例如 JSON、SQL、工具调用、配置文件、算法步骤和形式化推理，都和代码训练有关系。

5. 代码数据从哪里来？

常见代码数据来源包括：

1. 开源代码仓库。
2. README、文档和教程。
3. issue、pull request 和 commit message。
4. 单元测试和集成测试。
5. API reference。
6. 编程问答社区。
7. 竞赛题、题解和教学材料。
8. notebook。
9. 配置文件和脚本。

不同来源价值不同。

源代码提供实现模式，但缺少自然语言意图。README 和文档提供意图、用法和解释。测试提供可验证行为。issue 和 PR 提供 bug、需求、修复过程和真实工程语境。问答数据提供问题到解决方案的映射。

如果只训练纯代码，模型可能会写语法正确但不理解用户需求的代码。如果加入文档、测试和问答，模型更容易学习“需求 -> 实现 -> 验证”的完整链路。

6. 代码数据清洗

代码数据清洗不能套普通文本规则。代码天然符号多、缩进多、短行多、重复模式多。普通规则可能把高质量代码当成异常文本删掉。

代码清洗常见步骤：

1. 文件类型识别：识别语言、扩展名和真实内容。
2. 编码和解析检查：排除损坏文件、二进制文件、无法解码文件。

3. 自动生成文件过滤：如压缩 JS、protobuf 生成文件、锁文件、构建产物。
4. vendor 和依赖目录处理：避免重复训练第三方库镜像。
5. fork 和镜像去重：降低重复和记忆风险。
6. license 识别：判断是否允许训练使用。
7. secrets 检测：识别并移除凭据、密钥、token、证书和私有端点。
8. benchmark 污染检测：排除或隔离评测题、参考解和测试样例。
9. 质量评分：根据语法可解析性、star、维护度、测试覆盖、文档质量等信号评分。

这里的 secrets 检测是防御性数据治理，目标是避免敏感信息进入训练集，而不是发现、利用或复原敏感信息。

7. 代码数据的结构化处理

代码不只是文本。更深入的处理可以利用结构信息。

常见结构化粒度包括：

1. 仓库级：项目、依赖、license、README、测试目录。
2. 文件级：语言、路径、模块、导入依赖。
3. 函数级：签名、docstring、实现、调用关系。
4. 类级：属性、方法、继承关系。
5. AST 级：语法结构、控制流、表达式树。
6. 测试级：输入、断言、预期行为。

这些结构可以用于构造训练样本。例如：

1. docstring 到函数实现。
2. 函数实现到解释。
3. 单元测试到函数实现。
4. bug 描述到 patch。
5. API 文档到调用示例。
6. 错误日志到修复建议。

这类样本比单纯拼接代码更接近真实代码助手任务。

8. 单测为什么重要？

单元测试是代码数据里非常有价值的一类。它把“代码应该做什么”以可执行约束表达出来。

单测的价值包括：

1. 提供输入输出约束。
2. 让模型学习边界条件。
3. 支持功能正确性评估。
4. 连接需求描述和实现。
5. 可用于生成候选代码后的自动验证。

HumanEval 这类评估强调 functional correctness，即代码是否通过测试，而不是文本相似度。这对代码模型非常关键。因为两个实现可以完全不同，但功能都正确；反过来，文本相似不代表代码可运行。

训练数据中如果有高质量测试和实现，模型更容易学习“写能跑的代码”，而不只是“写像代码的文本”。

9. 代码数据的风险

代码数据风险很集中：

1. license 风险：不同开源许可证对使用和再分发限制不同。

2. secrets 风险：仓库中可能误提交密钥、token、证书和密码。
3. 安全风险：代码可能包含漏洞、不安全 API、过时依赖。
4. 污染风险：编程评测题、参考答案、测试用例可能进入训练集。
5. 重复风险：fork、vendor、镜像和模板工程大量重复。
6. 质量风险：toy project、未维护代码、错误示例和过时写法很多。

对代码数据的正确态度不是“全收”，而是“来源治理 + license 策略 + secrets 过滤 + 去重 + benchmark 隔离 + 功能评估”。

10. Math data：数学数据为什么重要？

数学数据训练的不只是计算能力，而是约束下的推理能力。

数学任务要求模型：

1. 理解题意。
2. 抽象变量。
3. 建立关系。
4. 分解步骤。
5. 执行计算。
6. 检查答案。
7. 用清晰语言解释过程。

这些能力和很多高级任务相关：规划、科学推理、代码调试、财务分析、逻辑问答和工具调用。

但数学数据的难点是质量。一个错误解答比没有解答更糟，因为模型会学习错误推理模式。一个跳步答案会让模型学到表面格式，却没有学会中间推理。

11. 数学数据从哪里来？

数学数据常见来源包括：

1. 教材和讲义。
2. 题库和练习册。
3. 竞赛题和解答。
4. 数学论坛。
5. 课程材料。
6. 论文和证明。
7. 程序生成题。
8. 强模型生成的解题数据。
9. 人工标注的 step-by-step 解答。
10. verifier 或 judge 生成的正确性标签。

不同来源服务不同目标。

教材适合概念学习；题库适合练习模式；竞赛题适合难题推理；程序生成题适合可控覆盖；人工 step-by-step 解答适合训练过程表达；verifier 数据适合学习判断候选解正确性。

12. 数学数据清洗

数学数据清洗比普通文本难，因为公式、符号、编号和短句都很常见，不能用“符号比例高”这种普通规则粗暴过滤。

重点包括：

1. 公式解析: LaTeX、MathML、图片 OCR 公式可能损坏。
2. 题解对齐: 题目、步骤、答案必须对应。
3. 答案校验: 尽量用规则、计算器、CAS 或人工抽检验证。
4. 重复题检测: 同一题不同版本、翻译版、改数字版。
5. 难度分级: 小学、初中、高中、竞赛、本科、研究生。
6. 领域分类: 代数、几何、概率、微积分、离散数学等。
7. 污染检测: 与 GSM8K、MATH、竞赛评测集等进行 overlap 检测。
8. 过程质量评分: 步骤是否完整、是否跳步、是否存在错误推理。

数学数据最怕“答案对但过程错”或“过程看似合理但答案错”。因此需要结果验证和过程验证共同使用。

13. 过程数据、答案数据和 verifier 数据

数学训练数据可以分三类:

1. final-answer 数据: 只有题目和最终答案。
2. rationale 数据: 包含解题步骤和解释。
3. verifier 数据: 包含候选解及其正确性判断。

final-answer 数据便宜, 但教不会模型完整过程。rationale 数据更有价值, 但质量要求高。verifier 数据让模型学习判断一个推理链是否可靠, 适合和多候选采样结合。

GSM8K/verifier 的思路说明: 生成多个候选解, 再用 verifier 选择更可信答案, 是数学推理中的重要范式。对数据工程来说, 这意味着不仅要收集“正确解”, 还要构造“正确/错误候选 + 判断标签”的数据。

14. 数学数据的污染风险

数学 benchmark 很容易污染。原因是题目、答案、解析经常被发布到博客、课程、题解网站、GitHub 和论坛。

污染形式包括:

1. 题目原文出现。
2. 答案出现。
3. 完整解析出现。
4. 改写或翻译版本出现。
5. 同源题库中的相似题出现。
6. 合成数据生成时意外复现 benchmark 结构。

数学污染比普通问答更难, 因为“改数字版”和“同题型变体”很常见。不是所有相似题都必须删除, 但评测集原题、答案和解析必须严格隔离。

15. Domain data: 专业领域数据为什么重要?

专业领域数据解决的是模型的“行业知识”和“专业表达”问题。

例如医学模型要理解症状、疾病、检查、药物、指南和风险提示; 法律模型要理解法条、判例、合同、程序和司法解释; 金融模型要理解财报、风险、市场、监管和产品结构。

通用 web 数据里也有这些内容, 但通常不够系统、可信和及时。专业领域需要更可靠的数据来源、更严格的质量控制和更清晰的边界。

专业数据的目标不是让模型“取代专家”, 而是让模型具备领域语言理解、资料检索辅助、初步分析和规范表达能力。

16. 专业领域数据来源

常见专业数据包括：

1. 教材和专业书。
2. 行业标准和规范。
3. 法律法规和公开判例。
4. 医学指南和药品说明。
5. 金融公告、年报和监管文件。
6. 学术论文和综述。
7. 专利和技术白皮书。
8. 企业知识库和内部文档。
9. 专家标注问答。
10. 经授权的真实业务数据。

不同数据源的合规要求差异很大。公开可访问不等于可以随意训练；企业内部数据也必须处理授权、隐私、保密和访问控制。

17. 专业领域数据清洗和治理

专业数据治理要比普通网页更严格。

关键步骤包括：

1. 来源分级：官方、权威机构、教材、论文、论坛、用户内容分开。
2. 时间戳管理：医学、法律、金融知识会过期。
3. 版本管理：法规、指南、标准经常更新。
4. PII 和敏感信息处理：尤其是医疗、金融和企业数据。
5. 专家抽检：普通标注员可能无法判断专业正确性。
6. 引用和出处保留：便于后续 RAG、审计和更新。
7. 术语标准化：同义词、缩写、别名、跨语言术语。
8. 风险标签：高风险建议、诊断、投资、法律结论等要单独标注。

专业数据不能只追求数量。低质量专业数据会让模型学到危险的自信表达。

18. 医学、法律、金融数据的差异

18.1 医学数据

医学数据强调安全、证据等级、时效性和患者隐私。指南、药品说明、医学教材和综述比论坛问答更可靠。真实病历必须严格脱敏、授权和审计。

医学模型不能只学“给建议”，还要学会表达不确定性、建议就医、识别高风险症状和避免越权诊断。

18.2 法律数据

法律数据强调地域、时效、层级和适用条件。不同法域规则不同，同一法规也会修订。判例、法条、司法解释、合同模板和法律问答要分开处理。

法律模型要避免把一般信息说成确定法律意见，尤其要保留出处、适用范围和不确定性。

18.3 金融数据

金融数据强调时效、市场环境、监管边界和风险披露。财报、公告、研报、宏观数据和产品材料各自有不同用途。

金融模型要避免未经依据的投资建议，训练数据中也要区分事实信息、观点、预测和营销材料。

19. 专业数据与 RAG 的关系

很多专业能力不适合完全依赖预训练记忆。

原因包括：

1. 知识更新快。
2. 需要引用出处。
3. 错误成本高。
4. 领域边界复杂。
5. 企业私有数据不能全部进入 base model。

因此，专业数据常见用法是：

1. 预训练或继续预训练学习领域语言和概念。
2. SFT 学习专业问答格式和安全边界。
3. RAG 提供最新、可引用、可控的知识。
4. 工具调用完成计算、检索、校验和流程操作。

专业数据工程要同时服务训练和检索。保留文档结构、标题、章节、出处、日期和权限信息非常重要。

20. 继续预训练 vs SFT vs RAG

面对专业领域需求，常见问题是：该继续预训练、SFT，还是做 RAG？

继续预训练适合让模型熟悉领域语言、术语、文体和基础知识。它改变模型参数，但成本高，也可能造成通用能力遗忘。

SFT 适合让模型学会任务格式、问答风格、拒答边界和流程规范。它不能凭空补足大规模知识。

RAG 适合提供动态知识、私有知识和需要引用的内容。它不一定增强模型内在推理能力，但能提高可控性和可更新性。

实际项目通常三者结合，而不是选一个。

21. 三类数据的配比策略

代码、数学和专业领域数据都不能简单越多越好。

代码数据提高太多，可能损害自然语言和多语言能力。数学数据提高太多，可能造成模板化推理或 benchmark 过拟合。专业数据提高太多，可能让模型风格过窄，甚至在非专业场景中也过度严肃。

更稳妥的策略：

1. base 预训练中保留适量专项数据。
 2. 对目标能力做小规模 ablation。
 3. 用继续预训练强化目标领域。
 4. 用 SFT 学习任务格式。
 5. 用 RAG 和工具处理高准确、强时效和私有知识。
 6. 用评估矩阵监控副作用。
-

22. 面向专家：专项数据改变模型的能力拓扑

从专家视角看，代码、数学和领域数据不是简单增加几个能力点，而是改变模型内部能力之间的连接方式。

代码数据把自然语言意图连接到可执行结构；数学数据把语言理解连接到符号约束和验证；专业数据把通用语言连接到领域本体和规范表达。

这些数据可能产生迁移效应。例如代码训练可能帮助工具调用和结构化输出，数学训练可能帮助规划和验证，领域训练可能帮助术语消歧和长文档理解。

但迁移不是免费的。专项数据也会带来风格偏移、过拟合、污染、记忆和安全边界问题。专家级数据工程的关键不是“加更多专项数据”，而是“用数据、评估和训练阶段控制迁移方向”。

23. 一个可落地的专项数据建设方案

如果面试官问：“如何为大模型建设代码、数学和专业领域数据？”可以按下面回答。

第一步，明确目标能力。代码是补全、修复、测试生成还是工程问答？数学是小学 word problem、竞赛还是证明？专业领域是知识问答、文档检索还是流程辅助？

第二步，分池建设数据。代码、数学、医学、法律、金融等各自建立独立数据池，记录来源、授权、质量、语言、时间、版本、去重和污染状态。

第三步，专项清洗。代码处理 license、secrets、fork、vendor、生成文件和语法解析；数学处理公式、题解对齐、答案校验和评测污染；领域数据处理权威性、时效性、PII、出处和专家审计。

第四步，构造结构化样本。代码构造 docstring-code、test-code、bug-fix；数学构造 problem-solution、step-by-step、verifier；领域构造 question-answer、document-grounded answer、citation-aware answer。

第五步，设计配比和训练阶段。base 阶段适量混入，继续预训练强化领域，SFT 学习任务格式，RAG/工具解决实时和高风险知识。

第六步，做污染和记忆检测。对 HumanEval、GSM8K、MATH、领域 benchmark、题库、标准答案和私有数据做 overlap 检测。

第七步，建立评估矩阵。代码看功能正确性、编译运行、测试通过率；数学看答案正确率、过程质量、鲁棒变体；领域看事实性、引用、时效、安全边界和专家评分。

第八步，版本化和审计。记录数据版本、过滤规则、采样权重、授权信息、评估结果和专家抽检报告。

23.1 最小可运行专项数据审计 demo

下面这个 demo 不依赖外部库，也不读写文件。输入是一组 toy code / math / domain 样本；输出包括保留样本、拒绝原因、分类型 token 保留率、最终 mixture、质量分预览和门禁检查。

它演示的是专项数据治理机制，不是生产级 license scanner、secret scanner、医学 / 法律 / 金融审核器或数学 verifier。真实系统需要接入许可证审查、secret 扫描器、测试执行器、CAS / verifier、专家审计、权限系统和数据版本管理。

```
from collections import Counter, defaultdict
```

```
samples = [
    {"id": "code_api_doc", "kind": "code", "tokens": 1400, "license_ok": True, "syntax_ok": True, "tests_pass": 8, "t
    {"id": "code_secret_config", "kind": "code", "tokens": 700, "license_ok": True, "syntax_ok": True, "tests_pass":
    {"id": "code_eval_solution", "kind": "code", "tokens": 900, "license_ok": True, "syntax_ok": True, "tests_pass":
    {"id": "code_vendor_bundle", "kind": "code", "tokens": 2200, "license_ok": False, "syntax_ok": True, "tests_pass":
    {"id": "math_word_verified", "kind": "math", "tokens": 650, "answer_ok": True, "process_score": 0.92, "difficulty":
    {"id": "math_wrong_steps", "kind": "math", "tokens": 500, "answer_ok": False, "process_score": 0.35, "difficulty":
    {"id": "math_benchmark_leak", "kind": "math", "tokens": 620, "answer_ok": True, "process_score": 0.88, "difficulty":
    {"id": "math_proof_note", "kind": "math", "tokens": 820, "answer_ok": True, "process_score": 0.84, "difficulty":
    {"id": "domain_med_guideline", "kind": "domain", "tokens": 1500, "authority": 0.95, "recency": 0.90, "citation":
    {"id": "domain_forum_advice", "kind": "domain", "tokens": 750, "authority": 0.25, "recency": 0.60, "citation": Fa
    {"id": "domain_fin_report", "kind": "domain", "tokens": 1200, "authority": 0.90, "recency": 0.85, "citation": Tru
    {"id": "domain_private_case", "kind": "domain", "tokens": 900, "authority": 0.80, "recency": 0.75, "citation": Tr
]
```

```
THRESHOLDS = {"code": 0.72, "math": 0.70, "domain": 0.74}
```

```
def safe_div(a, b):  
    return a / b if b else 0.0
```

```
def score(item):  
    if item["kind"] == "code":  
        test_rate = safe_div(item["tests_pass"], item["tests_total"])  
        penalty = 0.25 * item["generated"] + 0.20 * item["duplicate"]  
        return round(  
            0.35 * item["syntax_ok"]  
            + 0.35 * test_rate  
            + 0.20 * item["doc_score"]  
            - penalty,  
            3,  
        )  
    if item["kind"] == "math":  
        return round(  
            0.45 * item["answer_ok"]  
            + 0.40 * item["process_score"]  
            + 0.15 * item["difficulty"],  
            3,  
        )  
    cite = 1.0 if item["citation"] else 0.0  
    return round(  
        0.42 * item["authority"]  
        + 0.25 * item["recency"]  
        + 0.20 * cite  
        - 0.35 * item["risk"]  
        - 0.20 * item["stale"],  
        3,  
    )
```

```
def reject_reason(item, q):  
    if item["kind"] == "code":  
        if not item["license_ok"] or item["secret"]:  
            return "license_or_secret"  
        if item["contam"]:  
            return "eval_contamination"  
        if item["generated"] or item["duplicate"]:  
            return "generated_or_duplicate"  
    elif item["kind"] == "math":  
        if item["contam"]:  
            return "eval_contamination"  
        if item["duplicate"]:  
            return "duplicate_math"  
        if not item["answer_ok"]:  
            return "unverified_answer"  
    else:  
        if item["pii"]:  
            return "privacy_or_sensitive"
```



```

        if item["stale"]:
            return "stale_domain"
    if q < THRESHOLDS[item["kind"]]:
        return "low_quality"
    return None

kept, rejected, rows = [], {}, []
for item in samples:
    q = score(item)
    reason = reject_reason(item, q)
    rows.append({"id": item["id"], "kind": item["kind"], "score": q, "reason": reason or "kept"})
    if reason:
        rejected[item["id"]] = reason
    else:
        kept.append(item)

raw_tokens = sum(item["tokens"] for item in samples)
kept_tokens = sum(item["tokens"] for item in kept)
kind_tokens, raw_kind_tokens = defaultdict(int), defaultdict(int)
for item in samples:
    raw_kind_tokens[item["kind"]] += item["tokens"]
for item in kept:
    kind_tokens[item["kind"]] += item["tokens"]

reason_counts = dict(sorted(Counter(rejected.values()).items()))
gates = {
    "code_has_tests": any(item["kind"] == "code" and item["tests_total"] > 0 for item in kept),
    "math_verified": all(item.get("answer_ok", True) for item in kept if item["kind"] == "math"),
    "domain_no_pii": all(not item.get("pii", False) for item in kept if item["kind"] == "domain"),
    "contamination_blocked": reason_counts.get("eval_contamination", 0) == 2,
    "secret_blocked": reason_counts.get("license_or_secret", 0) == 2,
    "coverage": set(kind_tokens) == {"code", "domain", "math"},
}

report = {
    "kept_ids": [item["id"] for item in kept],
    "rejected": dict(sorted(rejected.items())),
    "reason_counts": reason_counts,
    "retention": round(kept_tokens / raw_tokens, 3),
    "kind_retention": {k: round(kind_tokens[k] / raw_kind_tokens[k], 3) for k in sorted(raw_kind_tokens)},
    "mixture": {k: round(kind_tokens[k] / kept_tokens, 3) for k in sorted(kind_tokens)},
    "score_preview": {row["id"]: row["score"] for row in rows},
    "gates": gates,
    "gate_pass": all(gates.values()),
}

for key, value in report.items():
    print(f"{key}=", value)

assert report["kept_ids"] == [
    "code_api_doc",
    "math_word_verified",
    "math_proof_note",
    "domain_med_guideline",

```

```

    "domain_fin_report",
]
assert report["reason_counts"] == {
    "eval_contamination": 2,
    "license_or_secret": 2,
    "low_quality": 1,
    "privacy_or_sensitive": 1,
    "unverified_answer": 1,
}
assert report["retention"] == 0.459
assert report["kind_retention"] == {"code": 0.269, "domain": 0.621, "math": 0.568}
assert report["mixture"] == {"code": 0.251, "domain": 0.485, "math": 0.264}
assert report["gate_pass"] is True

```

运行后会看到类似输出：

```

kept_ids= ['code_api_doc', 'math_word_verified', 'math_proof_note', 'domain_med_guideline', 'domain_fin_report']
rejected= {'code_eval_solution': 'eval_contamination', 'code_secret_config': 'license_or_secret', 'code_vendor_bundle': 'code_vendor_bundle'}
reason_counts= {'eval_contamination': 2, 'license_or_secret': 2, 'low_quality': 1, 'privacy_or_sensitive': 1, 'unverified_answer': 1}
retention= 0.459
kind_retention= {'code': 0.269, 'domain': 0.621, 'math': 0.568}
mixture= {'code': 0.251, 'domain': 0.485, 'math': 0.264}
score_preview= {'code_api_doc': 0.87, 'code_secret_config': 0.8, 'code_eval_solution': 0.84, 'code_vendor_bundle': 0.06, 'math_word_verified': 0.901, 'math_wrong_steps': 0.2, 'math_benchmark_leak': 0.892, 'math_proof_note': 0.899, 'domain_med_guideline': 0.899, 'domain_fin_report': 0.899}
gates= {'code_has_tests': True, 'math_verified': True, 'domain_no_pii': True, 'contamination_blocked': True, 'secret_blocked': True}
gate_pass= True

```

这个 demo 的重点是把三类专项数据的质量逻辑分开：代码看 license、secret、污染和测试；数学看答案验证、过程质量和题库污染；领域数据看权威性、时效、引用和 PII。

24. 常见面试题

24.1 代码数据为什么对大模型重要？

代码数据提供形式语言、可执行逻辑、API 使用、结构化输出和测试反馈。它不仅提升代码生成，也可能增强工具调用、JSON/SQL 输出和结构化推理能力。

24.2 代码数据清洗重点是什么？

重点是语言识别、语法规则、license、secrets 过滤、fork 和 vendor 去重、自动生成文件过滤、benchmark 污染检测、测试和文档对齐，以及质量评分。

24.3 为什么单测对代码模型重要？

单测提供可执行约束，可以判断功能正确性。代码生成不能只看文本相似度，应该看是否通过测试。单测还能帮助模型学习边界条件和需求到实现的映射。

24.4 数学数据和普通文本有什么不同？

数学数据更强调步骤、符号、答案校验和过程正确性。普通文本清洗规则可能误伤公式和短句，数学数据还要防止错误解答、跳步、题库污染和 benchmark 泄漏。

24.5 verifier 数据有什么价值？

verifier 数据让模型学习判断候选解是否正确。对于数学题，可以生成多个候选答案，再用 verifier 选择更可靠的解，比只训练单一答案更能利用计算和数据。

24.6 专业领域数据应该怎么用？

预训练或继续预训练用于学习术语和领域语言，SFT 用于学习专业问答格式和边界，RAG 用于最新、私有和可引用知识。高风险领域还需要专家审核和安全策略。

24.7 为什么不能把医学法律金融数据直接大量混进预训练？

因为这些数据合规要求高、时效性强、错误成本大，且可能包含隐私和专业误导。大量混入还可能带来风格偏移和过拟合。应分层使用、保留出处、做专家审计，并结合 RAG。

25. 常见误区

误区一：代码数据就是 .py、.java 文件。

更完整的代码数据包括文档、测试、issue、PR、README、API reference 和问答。

误区二：能编译的代码就是高质量代码。

能编译只是基础，代码还可能过时、不安全、无 license、无测试或来自重复 fork。

误区三：数学数据只要题目和答案。

高质量数学训练更需要步骤、验证、难度、领域、答案校验和错误候选。

误区四：专业数据越多越专业。

低质量专业数据会制造危险自信，且过度配比会损害通用能力。

误区五：领域模型只靠继续预训练。

继续预训练、SFT、RAG、工具和安全策略各自解决不同问题。

误区六：公开数据一定可训练。

公开访问不等于授权训练，尤其是代码 license、领域文档版权和个人敏感信息。

26. 本章小结

Code、Math 与 Domain Data 是大模型能力定向增强的核心数据。它们不是普通 web 文本的附属品，而是需要独立治理的数据资产。

本章要记住几句话：

1. 代码数据训练可执行逻辑、结构化输出和工程能力。
2. 数学数据训练多步推理、符号约束和验证意识。
3. 专业领域数据训练术语、规范表达和行业语境。
4. 三类数据都需要专项清洗、授权治理、去重和污染检测。
5. 单测、verifier、专家审计和 RAG 是这些数据走向可靠应用的重要桥梁。
6. 专项数据不是越多越好，而要按目标、阶段、配比和评估闭环使用。

如果面试中被问到专项数据建设，最好的回答是：先明确目标能力，再分池治理代码、数学和领域数据，分别处理 license/secrets、题解验证/污染、专业合规/时效，用结构化样本、配比实验、评估矩阵和版本审计形成闭环。

第七章：Synthetic Data 与 Distillation Data

前面几章讨论的大多是自然产生的数据：网页、代码仓库、数学题、专业文档、论坛、书籍和论文。本章讨论另一类越来越重要的数据：合成数据和蒸馏数据。

合成数据是由规则、程序、模拟器或模型生成的数据。蒸馏数据通常是强模型作为 teacher，为弱模型或小模型生成回答、解释、偏好或推理轨迹，让 student 模型学习 teacher 的行为。

这类数据在今天的大模型训练中非常常见。指令微调、数学推理、代码训练、安全训练、多语言增强、领域适配、工具调用训练，都大量使用合成数据或 teacher-generated data。

但合成数据不是免费午餐。它可以快速扩充能力，也可能制造同质化、错误放大、评估污染、teacher 偏差继承和数据退化。

本章重点：合成指令数据、推理轨迹、self-instruct、teacher model distillation、数据退化风险。

合规边界：本章讨论合成数据的训练、评估、治理和风险控制，不提供绕过模型服务条款、复制专有模型能力、生成有害内容或规避安全策略的方法。

0. 本讲资料边界与第二轮精修口径

按照 WRITING_PLAN.md 的要求，本讲精修前核对了 Self-Instruct、WizardLM / Evol-Instruct、phi-1 / Textbooks Are All You Need、Orca、经典 knowledge distillation、Distilling Step-by-Step 和 model collapse / generated data recursion 等公开论文资料。

本讲聚焦 synthetic data 与 distillation data 的防御性数据工程闭环：目标能力定义、seed 设计、teacher 授权、prompt / sampling 记录、生成后过滤、正确性验证、去重和污染隔离、合成比例控制、自然数据锚点、训练 ablation 与版本审计。

目标能力 -> seed 数据 -> teacher / 规则生成 -> 验证 -> 去重 -> 安全与污染扫描 -> 配比 -> 小规模训练实验 -> 版本审计

本讲不讨论绕过模型服务条款、无授权复制专有模型输出、生成有害样本、规避安全策略或用合成数据替代真实合规审计。合成数据只能作为可验证、可追踪、可配比的训练信号。

1. 先建立直觉：为什么要合成数据？

自然数据有两个问题：不够可控，也不一定覆盖你想要的能力。

例如，你想训练模型学会复杂指令跟随。互联网上当然有问答和教程，但不一定有大量“清晰指令 -> 高质量回答”的样本。你想训练数学分步推理，网页上有题目和答案，但步骤可能缺失、错误或格式混乱。你想训练安全拒答，自然语料里可能有风险内容，但不一定有安全、合规、边界清晰的回答。

合成数据的价值在于：它可以按目标能力主动构造训练样本。

可以把合成数据想成“定制练习册”。自然数据像从世界各地收来的书，有价值但混杂；合成数据像老师根据学生短板编的练习，目标明确、格式统一、覆盖可控。

但练习册质量取决于出题老师。如果老师水平高、覆盖广、验证严格，练习册很有用。如果老师重复、偏科、答案错误，学生会被带偏。

2. 来龙去脉：从数据增强到大模型自举

合成数据不是大模型时代才有。传统机器学习里早就有数据增强：图像旋转裁剪、语音加噪、机器翻译回译、规则生成样本等。这些方法的目标是增加数据量、提升鲁棒性、覆盖边界情况。

大模型时代，合成数据的形态发生了变化。模型本身可以生成高质量文本、代码、题目、解释和对话，于是数据生成不再只是简单扰动，而是能力自举。

Self-Instruct 是一个代表性工作。它用模型生成 instructions、inputs 和 outputs，过滤无效或相似样本，再用这些合成指令数据微调模型，从而提升 instruction-following 能力。它说明：即使缺少大量人工指令数据，也可以通过模型生成和过滤构造可用训练数据。

WizardLM/Evol-Instruct 进一步强调复杂指令生成。它从初始指令出发，通过逐步改写成更复杂的指令，用于增强模型处理复杂任务的能力。

phi-1 则展示了合成教材和练习在代码模型上的价值。它使用 textbook-quality web 数据和 GPT-3.5 生成的合成教材/练习，让小规模代码模型获得较强能力。这说明合成数据不仅能补数量，也能塑造能力结构。

3. Synthetic data 和 distillation data 的区别

这两个概念有重叠，但关注点不同。

Synthetic data 强调数据是人工或模型生成的，不是自然采集来的。生成方式可以是规则、程序、模拟器或 LLM。

Distillation data 强调 teacher-student 关系。强模型生成回答、解释、偏好或分数，弱模型通过这些数据学习 teacher 的行为。

例子：

1. 用规则生成 10 万道算术题：synthetic data，但不一定是 distillation。
2. 用 GPT-4 给指令生成高质量答案：既是 synthetic data，也是 distillation data。
3. 用模拟器生成机器人状态轨迹：synthetic data。
4. 用强代码模型为问题生成参考实现和解释：distillation data。
5. 用 teacher 模型给多个回答排序：偏好蒸馏数据。

面试中可以这样说：合成数据关注“数据来源是否生成”，蒸馏数据关注“是否把 teacher 能力迁移给 student”。

3.1 关键公式与审计指标

训练数据可以分成自然数据、合成数据和蒸馏数据：

$$D = D_{\{\mathrm{nat}\}} \cup D_{\{\mathrm{syn}\}} \cup D_{\{\mathrm{distill}\}}$$

一条合成或蒸馏样本可以表示为：

$$s_i = (x_i, y_i, a_i, t_i, p_i, q_i, r_i, z_i)$$

其中 x_i 是指令或上下文， y_i 是生成答案或轨迹， a_i 是目标能力标签， t_i 是 teacher / generator 标识， p_i 是 prompt 和采样参数版本， q_i 是质量分， r_i 是风险分， z_i 是授权、验证、去重、污染和版本元数据。

如果是普通 SFT 合成样本，目标仍是条件语言建模：

$$L_{\{\mathrm{sft}\}} = -\frac{1}{N_{\{\mathrm{tok}\}}} \sum_i w_i \sum_j \log p_{\{\theta\}}(y_{\{i,j\}} \mid x_i, y_{\{i,<j\}})$$

其中 w_i 是样本权重， N_{tok} 是有效训练 token 数。 w_i 不应该只由 teacher 分数决定，还要叠加验证、风险、重复和多样性审计。

如果 teacher 提供 soft distribution，蒸馏损失常写成 KL 形式：

$$L_{\{\mathrm{kd}\}} = \tau^2 \sum_i \mathrm{KL}(p_T^{\{\tau\}}(\cdot \mid c_i) \parallel p_S^{\{\tau\}}(\cdot \mid c_i))$$

其中 p_T 是 teacher 分布， p_S 是 student 分布， τ 是 distillation temperature， c_i 是上下文。如果只有 teacher 的 hard answer，则退化为对 teacher output 做 supervised learning。

合成样本质量分可以写成可审计的加权形式：

$$q_i = w_v V_i + w_e E_i + w_d D_i + w_s S_i - \lambda_h H_i - \lambda_u U_i - \lambda_c C_i$$

其中 V_i 是正确性验证， E_i 是证据或测试支持， D_i 是多样性贡献， S_i 是安全边界通过情况， H_i 是 hallucination 风险， U_i 是近重复或模板化风险， C_i 是评测污染风险。

合成数据样本门禁可以写成：

$$G_i = I(q_i \geq \tau_q) I_{\{\mathrm{auth}\}, i} I_{\{\mathrm{verify}\}, i} I_{\{\mathrm{dedup}\}, i} I_{\{\mathrm{safe}\}, i} I_{\{\mathrm{contam}\}}$$

这里 I_{auth} 表示 teacher 输出或生成规则的使用授权成立； I_{verify} 表示答案、代码、引用、安全边界或工具参数通过验证； I_{contam} 表示没有命中评测污染。

合成和蒸馏数据在训练集中的 token 占比为：

$$R_{\{\mathrm{syn}\}} = \frac{\sum_{i: o_i \in \{\mathrm{syn}, \mathrm{distill}\}} G_{iT_i}}{\sum_i G_{iT_i}}$$

其中 o_i 是样本来源类型， T_i 是 token 数。 R_{syn} 过高时，要警惕同质化、teacher 偏差和 model collapse 风险。

能力覆盖可以按标签集合计算：

$$C_{\{\mathrm{cover}\}} = \frac{|A_{\{\mathrm{target}\}} \cap A_{\{\mathrm{kept}\}}|}{|A_{\{\mathrm{target}\}}|}$$

同质化或近重复率可以写成：

$$R_{\{\mathrm{dup}\}} = \frac{\sum_i I(\max_{j < i} \mathrm{sim}(s_i, s_j) > \tau_{\{\mathrm{sim}\}})}{n}$$

最终训练前门禁可以写成：

$$G_{\{\mathrm{syn}\}} = I(R_{\{\mathrm{syn}\}} \leq r_{\{\max\}}) I(C_{\{\mathrm{cover}\}} \geq c_{\{\min\}}) I(R_{\{\mathrm{dup}\}} \leq d_{\{\max\}}) I(R_{\{\mathrm{contam}\}} \leq \tau_{\{\mathrm{contam}\}})$$

面试里要强调：合成数据不是“便宜 token”，而是对训练分布的主动编辑。必须同时记录 teacher、prompt、sampling、验证、过滤、配比和训练效果。

4. 合成数据常见类型

大模型训练中的合成数据主要有以下类型：

1. 合成指令数据：instruction、input、output 三元组。
2. 合成对话数据：多轮用户和助手对话。
3. 合成数学数据：题目、解题步骤、答案、验证标签。
4. 合成代码数据：需求、代码、测试、解释、bug fix。
5. 合成领域数据：医学、法律、金融等场景问答和文档问答。
6. 合成安全数据：风险识别、拒答、边界解释、合规替代建议。
7. 合成工具调用数据：用户意图、工具选择、参数填充、结果总结。
8. 合成多语言数据：翻译、跨语言问答、低资源语言增强。
9. 合成偏好数据：chosen/rejected 回答对。
10. 合成评估数据：用于内部评测和回归测试的任务集。

不同类型的数据目标不同，生成和验证方法也不同。

5. 合成指令数据

指令数据是 instruction tuning 的核心。它让模型从“续写文本”转向“理解用户要求并完成任务”。

一条典型指令样本包括：

1. instruction：用户要求。
2. input：可选上下文。
3. output：理想回答。

合成指令数据的优势：

1. 可以快速扩大任务覆盖。
2. 可以补充长尾任务。
3. 可以控制格式和风格。
4. 可以构造不同难度。
5. 可以多语言扩展。

风险：

1. 指令类型重复。
2. 回答风格同质化。
3. teacher 幻觉被 student 学到。
4. 难度分布不真实。
5. 过度训练后模型变得模板化。

Self-Instruct 的关键启发不是“让模型随便生成数据”，而是“生成后必须过滤无效、重复和相似样本，并用评估验证收益”。

6. Evol-Instruct：让指令变复杂

普通合成指令容易停留在简单任务，例如总结、翻译、改写、列要点。模型如果只训练这些数据，面对复杂约束、多步骤任务、组合任务时容易失败。

Evol-Instruct 的思想是从简单指令出发，通过改写逐步提高复杂度，例如增加限制条件、增加推理步骤、增加输入复杂度、要求多角度分析或引入更具体场景。

它解决的问题是指令复杂度不足。

但复杂不等于高质量。复杂指令也可能不合理、不可回答、约束冲突或答案质量差。因此需要过滤和审计：

1. 指令是否清楚。
 2. 约束是否一致。
 3. 答案是否满足约束。
 4. 难度是否真实有用。
 5. 是否覆盖多种能力，而不是只会堆限制。
-

7. 推理轨迹数据

推理轨迹数据包含中间思路、步骤、证明、计算过程或解释。它在数学、代码、科学问答、规划和复杂决策中很重要。

推理轨迹的价值：

1. 给模型提供分步解决问题的模式。
2. 提升复杂任务可解释性。
3. 让模型学会检查中间结果。
4. 支持 verifier 或 judge 训练。
5. 可用于生成过程监督数据。

但推理轨迹也有明显风险：

1. 错误步骤会被学习。
2. 模型可能学会“看起来合理”的伪推理。
3. 轨迹可能过长，挤占有效上下文。
4. 不同任务不一定需要显式长推理。
5. 如果来自 teacher，student 可能模仿 teacher 的偏差和风格。

所以推理轨迹数据一定要重视正确性验证。数学题可以校验最终答案，代码题可以运行测试，领域题需要引用依据或专家抽检。

8. Teacher-student 蒸馏

蒸馏的基本思想是：用强模型教弱模型。

在大模型数据工程中，teacher 可以提供：

1. 高质量回答。
2. 分步解释。
3. 多候选答案。
4. 答案评分。
5. chosen/rejected 偏好对。
6. 错误分析。
7. 任务分解。
8. 工具调用轨迹。

student 通过监督微调或偏好训练学习这些数据。

蒸馏的好处：

1. 降低人工标注成本。
2. 把强模型行为迁移到小模型。
3. 构造更稳定的回答格式。
4. 补足长尾任务。
5. 支持私有模型能力定制。

蒸馏的风险：

1. teacher 的错误被继承。
2. teacher 的风格被过度模仿。
3. student 难以超过 teacher 的知识边界。
4. 可能违反 teacher 服务或数据使用条款。
5. 多代蒸馏可能导致数据分布退化。

工程上必须确认 teacher 输出的使用授权和合规边界。

9. Distillation data 不等于复制能力

面试中需要注意一个边界：蒸馏不是简单“复制某个闭源模型”。合规项目要关注授权、数据使用条款、输出归属和安全限制。

从技术角度看，蒸馏也不是直接复制参数。student 学到的是训练数据中的行为模式，受 student 容量、训练策略、数据覆盖、优化目标和评估体系限制。

合理的蒸馏场景包括：

1. 用自有 teacher 为自有 student 生成训练数据。
2. 使用授权允许的数据和模型输出。
3. 用专家审核后的 teacher 数据增强内部模型。
4. 用 teacher 辅助标注、评分和质量过滤。

不合理的做法包括无授权大规模复制专有模型输出、绕过限制获取数据或生成违反安全边界的内容。本书不讨论这类做法。

10. 合成数据生成 pipeline

一个成熟的合成数据 pipeline 通常包括：

1. 目标定义：要补什么能力。
2. 种子数据：初始任务、领域文档、题型、用户场景。
3. 生成策略：prompt、模板、规则、模型、采样参数。
4. 多样性控制：任务类型、难度、语言、领域、格式。
5. 初步过滤：去重、长度、格式、无效样本、明显错误。
6. 质量评分：LLM judge、规则校验、模型评分、人工抽样。
7. 正确性验证：测试、计算、引用、事实核验、专家审计。

8. 安全过滤：风险分类、PII、政策边界、拒答质量。
9. 配比控制：不要让合成数据淹没自然数据。
10. 训练实验：小规模 ablation 验证收益和副作用。
11. 版本化：记录 teacher、prompt、参数、过滤器和数据版本。

这条链路的核心不是“生成很多”，而是“生成、过滤、验证、评估、迭代”。

11. 质量验证：合成数据最关键的一步

合成数据的问题不是不能生成，而是太容易生成。真正难的是验证。

不同任务的验证方法不同：

1. 数学题：校验答案、步骤、单位、边界条件。
2. 代码题：运行单测、静态检查、编译、lint。
3. 事实问答：检索证据、引用来源、人工抽检。
4. 翻译：双向一致性、人工评估、术语一致性。
5. 安全回答：边界是否正确，是否误拒或漏拒。
6. 工具调用：参数是否正确，工具结果是否被正确使用。
7. 领域问答：专家审计和权威资料对照。

如果验证缺失，合成数据很容易把 hallucination 包装成训练信号。

12. 去重和多样性控制

合成数据经常高度重复。模型会生成相似指令、相似开头、相似答案结构和相似解释模板。

常见重复包括：

1. 指令重复。
2. 任务类型重复。
3. 回答模板重复。
4. 推理步骤重复。
5. 领域场景重复。
6. 安全拒答话术重复。

控制方法包括：

1. embedding 聚类去重。
2. n-gram overlap 过滤。
3. 按任务类型分桶采样。
4. 控制难度分布。
5. 多 teacher 或多 prompt 生成。
6. 人工审计高频模式。
7. 对同质模板降权。

多样性不是为了好看，而是为了避免 student 学到单一风格。

13. 数据退化风险

数据退化指模型生成的数据逐渐替代真实数据后，训练分布变窄、错误累积、长尾信息消失、语言风格同质化。

如果一代模型用自然数据训练，第二代大量用第一代生成数据，第三代再用第二代生成数据，可能出现信息损失。真实世界的复杂性被逐步压缩成模型喜欢生成的模式。

表现包括：

1. 语言更模板化。
2. 观点更平均。
3. 长尾知识减少。
4. 错误事实被重复强化。
5. 难题覆盖下降。
6. 安全边界变得机械。
7. 多语言和小众表达被削弱。

防止退化的关键是保留高质量自然数据、人工数据、真实用户分布、权威来源和严格验证。合成数据应是补充和定向增强，而不是无节制替代真实数据。

14. 合成数据在不同训练阶段的用法

14.1 预训练

预训练阶段可以少量使用高质量合成教材、代码练习、数学题和领域文本，但要谨慎控制比例，避免合成分布主导 base model。

14.2 继续预训练

继续预训练适合用合成数据补目标领域或能力短板，例如代码教材、数学练习、多语言问答。

14.3 SFT

SFT 是合成指令数据最常用的阶段。instruction-response、multi-turn dialogue、tool use、format following 都可以合成。

14.4 偏好训练

可以用 teacher 或 judge 生成 chosen/rejected 数据，但偏好质量必须审计，否则会让模型学到表面偏好。

14.5 安全训练

安全合成数据可以覆盖风险类别、拒答边界、误拒案例和安全替代建议。此类数据必须保持防御和合规导向。

15. 合成数据和真实用户数据

真实用户数据反映真实需求、真实表达和真实错误，但涉及隐私、授权和安全治理。合成数据可控、便宜、可扩展，但可能不真实、不多样。

两者不是替代关系。

真实用户数据适合发现真实分布和产品问题；合成数据适合针对性补齐覆盖不足、构造安全边界、生成难例和统一格式。

成熟做法是：用真实数据发现任务分布，用合成数据扩展和补齐，再用人工和在线反馈验证。

16. 合成数据配比

合成数据占比没有标准答案。不同阶段、任务和质量下差异很大。

判断合成数据比例时要看：

1. 目标能力是否稀缺。

2. 合成数据质量是否可验证。
3. 是否覆盖真实用户分布。
4. 是否和自然数据重复。
5. 是否导致风格同质化。
6. 是否提升目标指标但损害通用能力。
7. 是否增加幻觉、安全或记忆风险。

稳妥策略是从小比例开始，通过 ablation 逐步提高，观察收益曲线和副作用，而不是一次性大规模混入。

17. 面向专家：合成数据是分布编辑

从专家视角看，合成数据不是单纯增加样本，而是在编辑训练分布。

自然数据来自世界分布，合成数据来自生成器分布。生成器分布由 teacher 能力、prompt、采样参数、过滤器、任务模板和审计标准共同决定。

因此，合成数据引入的是一种新的偏置。如果这种偏置与目标能力一致，模型会受益；如果偏置过强，模型会被塑造成“像生成器一样说话”。

专家级问题包括：

1. 生成器分布和真实用户分布差多少？
 2. 合成数据是否覆盖目标任务的长尾？
 3. teacher 错误如何被发现和阻断？
 4. student 是否只是模仿格式，而没有获得能力？
 5. 合成数据提升 benchmark 是否来自污染或模板匹配？
 6. 多代合成是否导致数据退化？
-

18. 一个可落地的合成与蒸馏数据方案

如果面试官问：“如何建设 synthetic data 和 distillation data？”可以按下面回答。

第一步，定义目标能力。明确要补指令跟随、数学、代码、工具调用、多语言、安全还是领域问答。

第二步，准备种子数据。来自真实任务、公开高质量数据、专家模板、领域文档或人工设计任务。

第三步，选择生成方式。规则生成适合可验证任务，LLM 生成适合开放任务，teacher-student 适合蒸馏行为和格式。

第四步，生成多样样本。控制任务类型、难度、语言、领域、输出格式和边界场景。

第五步，过滤和去重。去掉无效、重复、低质量、过短、过长、格式错、相似度过高的样本。

第六步，验证正确性。数学用答案校验，代码用测试，事实用检索和引用，领域用专家审计，安全用策略检查。

第七步，质量评分和分桶。按任务、难度、风险、语言、领域、teacher、prompt 版本打标签。

第八步，做小规模训练实验。比较不同合成比例、不同 teacher、不同过滤阈值的收益和副作用。

第九步，和自然数据混合。控制合成数据比例，避免同质化和分布退化。

第十步，版本化和合规审计。记录 teacher、prompt、采样参数、过滤规则、授权信息、质量评估和训练效果。

18.1 最小可运行合成与蒸馏数据审计 demo

下面这个 demo 不依赖外部库，也不读写文件。输入是一组 toy natural / synthetic / distillation 样本；输出包括保留样本、拒绝原因、来源配比、任务配比、teacher 配比、多样性覆盖和门禁结果。

它演示的是合成数据治理机制，不是生产级合规审查、LLM judge、数学 verifier、代码沙箱、安全分类器或版权系统。真实系统要接入授权审查、teacher 版本管理、prompt registry、测试执行、检索证据、人工抽样、污染检测和训练 ablation。

```
from collections import Counter, defaultdict
```

```
samples = [
    {"id": "seed_user_math", "origin": "natural", "task": "math", "tokens": 420, "quality": 0.86, "validated": True,
    {"id": "seed_user_domain", "origin": "natural", "task": "domain", "tokens": 1200, "quality": 0.84, "validated": True,
    {"id": "syn_math_verified", "origin": "synthetic", "task": "math", "tokens": 620, "quality": 0.88, "validated": True,
    {"id": "syn_math_wrong", "origin": "synthetic", "task": "math", "tokens": 560, "quality": 0.83, "validated": False,
    {"id": "distill_tool_trace", "origin": "distill", "task": "tool", "tokens": 780, "quality": 0.91, "validated": True,
    {"id": "distill_unauthorized", "origin": "distill", "task": "general", "tokens": 700, "quality": 0.90, "validated": True,
    {"id": "syn_code_tests", "origin": "synthetic", "task": "code", "tokens": 900, "quality": 0.87, "validated": True,
    {"id": "syn_code_duplicate", "origin": "synthetic", "task": "code", "tokens": 880, "quality": 0.86, "validated": True,
    {"id": "syn_benchmark_leak", "origin": "synthetic", "task": "eval_like", "tokens": 500, "quality": 0.84, "validated": True,
    {"id": "syn_safety_refusal", "origin": "synthetic", "task": "safety", "tokens": 640, "quality": 0.82, "validated": True,
    {"id": "syn_unsafe_answer", "origin": "synthetic", "task": "safety", "tokens": 610, "quality": 0.74, "validated": True,
    {"id": "real_user_private", "origin": "natural", "task": "domain", "tokens": 540, "quality": 0.78, "validated": True,
]
```

```
TARGET_TAGS = {"math", "reasoning", "tool", "multi_step", "code", "tests", "safety", "boundary", "domain", "seed"}
MIN_QUALITY = 0.80
MAX_SYN_RATIO = 0.72
```

```
def reject_reason(item):
    if not item["authorized"]:
        return "unauthorized_teacher"
    if item["pii"]:
        return "privacy_or_pii"
    if item["contam"]:
        return "eval_contamination"
    if item["duplicate"]:
        return "near_duplicate"
    if not item["safety_ok"]:
        return "unsafe_or_policy_fail"
    if not item["validated"]:
        return "unverified_output"
    if item["quality"] < MIN_QUALITY:
        return "low_quality"
    return None
```

```
kept, rejected = [], {}
for item in samples:
    reason = reject_reason(item)
    if reason:
        rejected[item["id"]] = reason
    else:
        kept.append(item)
```

```
raw_tokens = sum(item["tokens"] for item in samples)
kept_tokens = sum(item["tokens"] for item in kept)
origin_tokens = defaultdict(int)
```

```

task_tokens = defaultdict(int)
teacher_tokens = defaultdict(int)
covered_tags = set()

for item in kept:
    origin_tokens[item["origin"]] += item["tokens"]
    task_tokens[item["task"]] += item["tokens"]
    teacher_tokens[item["teacher"] or "human_or_seed"] += item["tokens"]
    covered_tags.update(item["diversity"])

synthetic_like_tokens = origin_tokens["synthetic"] + origin_tokens["distill"]
report = {
    "kept_ids": [item["id"] for item in kept],
    "rejected": dict(sorted(rejected.items())),
    "reason_counts": dict(sorted(Counter(rejected.values()).items())),
    "retention": round(kept_tokens / raw_tokens, 3),
    "origin_mix": {k: round(origin_tokens[k] / kept_tokens, 3) for k in sorted(origin_tokens)},
    "task_mix": {k: round(task_tokens[k] / kept_tokens, 3) for k in sorted(task_tokens)},
    "teacher_mix": {k: round(teacher_tokens[k] / kept_tokens, 3) for k in sorted(teacher_tokens)},
    "synthetic_like_ratio": round(synthetic_like_tokens / kept_tokens, 3),
    "diversity_coverage": round(len(covered_tags & TARGET_TAGS) / len(TARGET_TAGS), 3),
}

gates = {
    "authorization": "unauthorized_teacher" in report["reason_counts"],
    "validation": "unverified_output" in report["reason_counts"],
    "contamination": "eval_contamination" in report["reason_counts"],
    "safety": "unsafe_or_policy_fail" in report["reason_counts"],
    "privacy": "privacy_or_pii" in report["reason_counts"],
    "diversity": report["diversity_coverage"] >= 0.85,
    "synthetic_ratio": report["synthetic_like_ratio"] <= MAX_SYN_RATIO,
    "natural_anchor": origin_tokens["natural"] > 0,
}
report["gates"] = gates
report["gate_pass"] = all(gates.values())

for key, value in report.items():
    print(f"{key}=", value)

assert report["kept_ids"] == [
    "seed_user_math",
    "seed_user_domain",
    "syn_math_verified",
    "distill_tool_trace",
    "syn_code_tests",
    "syn_safety_refusal",
]
assert report["reason_counts"] == {
    "eval_contamination": 1,
    "near_duplicate": 1,
    "privacy_or_pii": 1,
    "unauthorized_teacher": 1,
    "unsafe_or_policy_fail": 1,
    "unverified_output": 1,
}

```

```
assert report["retention"] == 0.546
assert report["origin_mix"] == {"distill": 0.171, "natural": 0.355, "synthetic": 0.474}
assert report["synthetic_like_ratio"] == 0.645
assert report["diversity_coverage"] == 1.0
assert report["gate_pass"] is True
```

运行后会看到类似输出：

```
kept_ids= ['seed_user_math', 'seed_user_domain', 'syn_math_verified', 'distill_tool_trace', 'syn_code_tests', 'syn_safe
rejected= {'distill_unauthorized': 'unauthorized_teacher', 'real_user_private': 'privacy_or_pii', 'syn_benchmark_leak'
reason_counts= {'eval_contamination': 1, 'near_duplicate': 1, 'privacy_or_pii': 1, 'unauthorized_teacher': 1, 'unsafe_or
retention= 0.546
origin_mix= {'distill': 0.171, 'natural': 0.355, 'synthetic': 0.474}
task_mix= {'code': 0.197, 'domain': 0.263, 'math': 0.228, 'safety': 0.14, 'tool': 0.171}
teacher_mix= {'human_or_seed': 0.355, 'owned_teacher': 0.368, 'policy_template': 0.14, 'rule_solver': 0.136}
synthetic_like_ratio= 0.645
diversity_coverage= 1.0
gates= {'authorization': True, 'validation': True, 'contamination': True, 'safety': True, 'privacy': True, 'diversity': T
gate_pass= True
```

这个 demo 的重点是：合成数据进入训练前必须被当成可审计数据产品，而不是 teacher 随手生成的文本。它要能证明授权成立、错误被验证拦截、污染被隔离、PII 被过滤、近重复被降掉，并且合成 / 蒸馏 token 没有压过自然数据锚点。

19. 常见面试题

19.1 什么是 synthetic data?

synthetic data 是由规则、程序、模拟器或模型生成的数据，不是直接从自然环境采集的数据。在大模型中常用于指令数据、数学题、代码任务、安全样本、工具调用和领域问答。

19.2 什么是 distillation data?

distillation data 是 teacher model 生成或标注的数据，用于训练 student model 学习 teacher 的回答、推理、偏好、评分或格式。它关注能力迁移，而不仅是数据是否人工生成。

19.3 Self-Instruct 的核心思想是什么?

用模型自举生成 instruction、input 和 output 样本，过滤无效或相似样本，再用于 instruction tuning。核心不是无脑生成，而是生成后过滤和评估。

19.4 合成数据有什么风险?

主要风险包括错误放大、teacher 偏差继承、同质化、数据退化、评估污染、过拟合、合规问题和模型学会模板而不是真能力。

19.5 如何验证合成数据质量?

按任务选择验证方法：数学校验答案，代码运行测试，事实问答检索证据，领域问答专家审计，安全数据检查边界，工具调用验证参数和结果使用。还要做去重、人工抽样和训练 ablation。

19.6 合成数据比例应该是多少?

没有固定比例。应从目标能力、数据质量、验证可靠性、自然数据覆盖、训练阶段和副作用出发，通过小规模 ablation 逐步确定。

19.7 蒸馏能不能让小模型超过 teacher?

通常 student 受 teacher 数据、模型容量和训练目标限制，很难在 teacher 全能力范围内全面超过 teacher。但在特定任务、特定分布、经过高质量筛选和额外数据增强后，student 可以在局部指标上接近甚至超过 teacher。

20. 常见误区

误区一：合成数据越多越好。

合成数据容易生成，但不一定高质量。过多会导致同质化和数据退化。

误区二：teacher 强，生成数据就一定好。

teacher 也会 hallucinate，也有偏差。生成数据必须验证。

误区三：蒸馏就是复制模型。

蒸馏是通过数据学习 teacher 行为，不等于复制参数或完整能力，还必须遵守授权和使用边界。

误区四：合成数据可以替代真实数据。

合成数据适合补齐和增强，真实数据仍然提供真实分布、长尾表达和实际用户需求。

误区五：只要 benchmark 提升就说明合成数据有效。

还要检查污染、泛化、风格、事实性、安全和人工样例。

误区六：复杂指令一定更好。

复杂但不合理、不可验证或答案低质的指令会伤害训练。

21. 本章小结

Synthetic Data 与 Distillation Data 是大模型数据工程从“收集世界”走向“主动构造训练分布”的关键工具。

本章要记住几句话：

1. 合成数据用于补齐自然数据覆盖不到或难以标注的能力。
2. 蒸馏数据用于把 teacher 的回答、推理、偏好和格式迁移给 student。
3. Self-Instruct 和 Evol-Instruct 的核心启发是生成后必须过滤、去重和评估。
4. phi-1 说明高质量合成教材和练习可以显著提升特定能力。
5. 合成数据最大的风险是错误放大、同质化、teacher 偏差和数据退化。
6. 合成数据不是越多越好，而要有目标、验证、配比、审计和版本化。

如果面试中被问到合成数据，最好的回答不是“用强模型生成一批数据”，而是讲完整闭环：目标定义、种子设计、生成、多样性控制、过滤、正确性验证、安全审计、配比实验、自然数据混合和版本治理。

第八章：Preference Data 与安全数据

前一章讨论了 synthetic data 和 distillation data。本章继续往后，讨论后训练阶段最关键的两类数据：preference data 和安全数据。

预训练让模型学会语言、知识和模式，SFT 让模型学会按指令回答。但一个模型是否真正“好用”，还取决于它能不能在多个可接受回答中选择更符合人类偏好的那个，能不能在风险请求前保持边界，能不能在安全与帮助之间做平衡。

这就需要偏好数据和安全数据。

偏好数据回答的是：两个或多个回答中，哪个更好，为什么更好。安全数据回答的是：什么请求可以帮助，什么请求应该拒绝，什么请求应该安全改写，什么请求需要给出风险提示或建议寻求专业帮助。

本章重点：偏好标注、chosen/rejected、拒答数据、安全分类、红队数据。

合规边界：本章讨论偏好数据、安全数据、红队评估和防御性数据治理，不提供绕过安全策略、构造攻击提示、实施有害行为或规避模型防护的方法。

0. 本讲资料边界与第二轮精修口径

按照 WRITING_PLAN.md 的要求，本讲精修前核对了 InstructGPT / RLHF、Learning from Human Preferences、Learning to Summarize from Human Feedback、DPO、Helpful and Harmless RLHF、Constitutional AI 和 Anthropic red teaming 等公开论文资料。

本讲聚焦 preference data 与 safety data 的防御性数据治理：prompt 池设计、chosen / rejected 构造、标注一致性、reward / DPO 数据质量、安全分层、误拒 / 漏拒、红队回归、隐私脱敏、标注员保护、偏好偏置审计和数据版本化。

行为准则 -> prompt 池 -> 候选回答 -> 偏好标注 -> 安全分层 -> 审计过滤 -> 偏好训练 -> 安全评估 -> 版本治理

本讲不提供绕过安全策略、构造攻击提示、实施有害行为、规避模型防护或传播红队细节的方法。红队数据只作为防御性评估、训练和回归测试信号使用。

1. 先建立直觉：为什么只做 SFT 不够？

SFT 数据通常是 prompt 到 ideal answer 的映射。它告诉模型：“遇到这个问题，可以这样答。”

但真实对话中，一个问题往往有很多可行回答。比如用户问“帮我解释 Transformer”，回答可以短、长、面向小白、面向专家、带公式、带代码、带类比。它们都可能正确，但质量不同。

SFT 很难显式告诉模型这些细粒度偏好：

1. 哪个回答更有帮助。
2. 哪个回答更诚实。
3. 哪个回答更安全。
4. 哪个回答更符合用户意图。
5. 哪个回答更少幻觉。
6. 哪个回答更简洁或更完整。
7. 哪个回答拒答得更合适。

偏好数据就是为了解决这个问题。它不是只给标准答案，而是给比较信号。

2. 来龙去脉：从 RLHF 到 DPO

InstructGPT 是现代 RLHF 路线的重要代表。它先收集示范数据做监督微调，再收集人类对模型输出的排序数据训练 reward model，最后用强化学习优化模型，使模型更符合人类偏好。论文显示，经过人类反馈训练的小模型在人工偏好评估中可以超过更大的原始 GPT-3，并改善有用性、真实性和毒性问题。

RLHF 的关键数据不是普通问答，而是人类比较：同一个 prompt 下，多个回答哪个更好。这个排序信号用于训练 reward model。

后来，DPO 简化了偏好优化流程。DPO 不再显式训练 reward model 再做复杂强化学习，而是直接用 chosen/rejected 偏好对优化语言模型。它说明偏好数据本身可以直接驱动模型行为调整。

从数据工程角度看，RLHF、DPO、IPO、KTO 等算法差异很重要，但底层都依赖高质量偏好数据。偏好数据质量差，算法再漂亮也会学偏。

3. Preference data 到底是什么？

最常见的偏好数据形态是三元组：

1. prompt：用户输入。
2. chosen：更好的回答。
3. rejected：更差的回答。

也可以是排序列表：

1. prompt。
2. answer A、B、C、D。
3. 人类或模型给出的排序。

还可以带解释：

1. chosen 为什么好。
2. rejected 为什么差。
3. 差异属于事实性、帮助性、安全性、格式、语气还是完整性。

偏好数据不是简单二分类。一个回答可能事实正确但啰嗦，另一个简洁但遗漏重要边界。标注者必须知道当前任务更重视什么。

3.1 关键公式与审计指标

一条偏好样本可以表示为：

$$d_i=(x_i,y_i^+,y_i^-,a_i,r_i,z_i)$$

其中 x_i 是 prompt, y_i^+ 是 chosen response, y_i^- 是 rejected response, a_i 是偏好维度标签, r_i 是风险类别, z_i 是标注者、rubric、语言、来源、隐私、污染和版本元数据。

Reward Model 常用成对排序损失：

$$L_{\{\mathrm{rm}\}}=-\frac{1}{n}\sum_i \log \sigma(R_{\{\phi\}}(x_i,y_i^+)-R_{\{\phi\}}(x_i,y_i^-))$$

这里 R_{ϕ} 给 prompt-response 打标量分数。它学的是偏好排序近似，不是绝对真值。

DPO 可以直接利用 chosen / rejected 偏好对：

$$\Delta_i=\log \pi_{\{\theta\}}(y_i^+|\mid x_i)-\log \pi_{\{\mathrm{ref}\}}(y_i^+|\mid x_i)-\log \pi_{\{\theta\}}(y_i^-|\mid x_i)+\log \pi_{\{\mathrm{ref}\}}(y_i^-|\mid x_i)$$

$$L_{\{\mathrm{dpo}\}}=-\frac{1}{n}\sum_i \log \sigma(\beta\Delta_i)$$

其中 π_{ref} 是 reference model, β 控制偏好优化强度。数据工程上，DPO 对数据质量非常敏感：chosen / rejected 太弱、标注噪声大或存在长度偏置，都会让模型学偏。

偏好 margin 可以写成：

$$m_i=s_i^+-s_i^-$$

其中 s_i^+ 和 s_i^- 是按 rubric 聚合后的 chosen / rejected 分数。过小的 m_i 说明偏好信号弱，过大的 m_i 可能说明 rejected 太差，训练信号不够细。

标注一致率可以写成：

$$A=\frac{1}{n}\sum_i I(l_{\{i,1\}}=l_{\{i,2\}})$$

真实项目里通常会用更多标注者、Kappa、分桶一致率或专家复核。面试中至少要说清楚：偏好数据必须审计一致性，而不是默认人类标注全对。

安全数据样本可以表示为：

$$u_i=(x_i,c_i,a_i,y_i,z_i)$$

其中 c_i 是风险类别, a_i 是期望动作, 例如 answer、refuse、safe_alt、clarify; y_i 是目标安全响应; z_i 记录语言、地区、政策版本、标注者、隐私和红队来源。

漏拒率和误拒率可以写成:

$$R_{\{\mathrm{leak}\}} = \frac{\sum_i I(a_i = \mathrm{refuse}) I(\hat{a}_i \neq \mathrm{refuse})}{\sum_i I(a_i = \mathrm{refuse})}$$

$$R_{\{\mathrm{over}\}} = \frac{\sum_i I(a_i = \mathrm{answer}) I(\hat{a}_i = \mathrm{refuse})}{\sum_i I(a_i = \mathrm{answer})}$$

这里 R_{leak} 关注高风险请求被错误回答, R_{over} 关注安全请求被过度拒绝。安全数据只优化其中一个指标会造成模型行为失衡。

长度偏置可以用 chosen 是否显著更长来审计:

$$B_{\{\mathrm{len}\}} = \frac{\sum_i I(|y_i^+| > \gamma |y_i^-|)}{n}$$

如果 B_{len} 很高, 要检查标注者是否把 “更长” 误当成 “更好”。

偏好与安全数据上线前可以设置门禁:

$$G_{\{\mathrm{pref}\}} = I(A_{\{\min\}}(\bar{m}_{\{\min\}})) I(R_{\{\mathrm{leak}\}} \leq \rho_{\{\mathrm{leak}\}}) I(R_{\{\mathrm{over}\}})$$

其中 C_{risk} 是安全风险类别覆盖率。面试里要强调: preference data 是模型行为价值函数的样本, 安全数据是边界行为样本, 两者都必须版本化和审计。

4. 偏好维度: helpful、honest、harmless

偏好标注常见维度可以概括为 helpful、honest、harmless。

Helpful 表示回答是否真正帮助用户解决问题。它关注相关性、完整性、可执行性、结构清晰度和是否理解用户意图。

Honest 表示回答是否真实、校准、不编造。它关注事实准确性、不确定性表达、引用边界和是否承认不知道。

Harmless 表示回答是否安全、合规、避免助长伤害。它关注危险内容、隐私、歧视、欺骗、越权建议和拒答边界。

实际标注中, 这三者经常冲突。

例如, 用户要求高风险操作细节。最 helpful 的表面回答可能是给出步骤, 但 harmless 要求拒绝危险细节, 并提供安全替代建议。用户问医学问题时, 回答要 helpful, 但 honest 要表达不确定性, harmless 要避免替代医生诊断。

高质量偏好数据必须把这些冲突写进标注规范。

5. 偏好数据从哪里来?

常见来源包括:

1. 人类标注者比较模型回答。
2. 专家标注专业领域回答。
3. 用户反馈, 如点赞、点踩、重试、投诉。
4. 模型生成多个候选后由人类排序。
5. teacher model 或 LLM judge 辅助排序。
6. 红队评估中的安全偏好。
7. A/B 测试和线上质量反馈。
8. 人工构造的边界案例。

这些来源各有问题。

人类标注质量高但成本高。用户反馈真实但噪声大。LLM judge 便宜但有偏差。专家标注可靠但扩展慢。线上反馈有产品偏置, 不能直接当作普适偏好。

成熟系统会混合多来源，并用审计和校准控制质量。

6. chosen/rejected 构造方式

构造偏好对有几种常见方法。

第一种，同一模型多采样。对同一个 prompt 采样多个回答，让标注者选择最好和最差。这能覆盖模型真实错误模式。

第二种，不同模型对比。让 base、SFT、RLHF、不同尺寸模型、不同版本模型回答同一 prompt，再做排序。这有助于学习版本间质量差异。

第三种，人工写 chosen，模型生成 rejected。适合明确高质量标准，但成本较高。

第四种，安全场景模板构造。对风险请求生成安全回答和不安全回答，用于训练拒答边界。这里必须只保留防御性、安全导向的标注内容，不扩散可操作风险细节。

第五种，自动批改或工具验证。代码、数学、事实检索任务可以用测试、计算或证据辅助判断 chosen/rejected。

关键是 rejected 不能太弱。如果 rejected 明显胡说，模型只学到粗糙偏好；如果 chosen 和 rejected 差异细微，训练信号更有价值。

7. 偏好标注规范

偏好数据质量高度依赖标注规范。没有清晰规范，不同标注者会按个人口味选择，数据噪声很大。

标注规范应该说明：

1. 优先级：安全、事实、帮助、格式、语气如何排序。
2. 拒答边界：哪些请求拒绝，哪些可以安全回答。
3. 不确定性：何时应该承认不知道。
4. 专业建议：医学、法律、金融如何表达边界。
5. 引用要求：需要依据时如何处理。
6. 多语言和文化差异：不同语言下如何保持一致标准。
7. 长度偏好：简洁和完整如何权衡。
8. 标注示例：提供正反例和边界例。

标注规范不是一次写完。训练中发现模型偏差后，要回到规范修订，再补数据。

8. 偏好数据的质量问题

偏好数据常见问题包括：

1. 标注者不一致。
2. 偏好过度主观。
3. 偏好维度混杂。
4. rejected 太弱。
5. prompt 分布不真实。
6. 长回答偏置：标注者更容易觉得长回答更好。
7. 安全过度偏置：模型学会过度拒答。
8. 迎合偏置：模型学会讨好用户而不是保持事实。
9. LLM judge 偏置：自动评审偏好某种风格。
10. 数据泄漏：评测集或线上隐私内容混入训练。

因此，偏好数据要做标注一致性评估、分桶审计、模型回归测试和线上观察。

9. 安全数据是什么？

安全数据是用于训练和评估模型安全行为的数据。它不只是“危险问题 -> 拒绝回答”。更完整地说，它包括：

1. 风险分类数据。
2. 安全拒答数据。
3. 安全替代建议数据。
4. 边界允许数据。
5. 误拒修复数据。
6. 红队评估数据。
7. 多语言安全数据。
8. 专业高风险领域数据。
9. 隐私和 PII 处理数据。
10. 政策解释和合规回答数据。

安全数据的目标不是让模型什么都拒绝，而是让模型学会区分：该帮助时帮助，该拒绝时拒绝，该提示风险时提示风险。

10. 安全数据分层

安全数据最好按风险分层，而不是只分“安全/不安全”。

可以分为：

1. 明确安全请求：正常学习、科普、创作、技术帮助。
2. 敏感但可安全回答：历史、新闻、政策、风险识别、教育性解释。
3. 高风险请求：可能造成现实伤害、违法、欺骗、隐私侵犯或危险操作。
4. 边界模糊请求：用户意图不清，需要澄清或给安全版本。
5. 紧急风险请求：自伤、医疗急症、暴力威胁等需要支持性安全回应。
6. 专业高风险请求：医学、法律、金融等需要免责声明、建议咨询专业人士和避免确定性结论。

分层的好处是避免模型“一刀切”。它能在风险内容中区分教育、预防、治理和操作性伤害。

11. 拒答数据

拒答数据教模型如何拒绝不该回答的请求。

一个好的拒答不是冷冰冰地说“不行”，而是：

1. 简短说明不能提供什么。
2. 不重复危险细节。
3. 给出安全替代方向。
4. 语气尊重，不训斥用户。
5. 对紧急风险提供求助建议。
6. 对可教育内容提供高层次安全解释。

拒答数据常见错误包括：

1. 过度解释风险细节，反而泄露有害信息。
2. 拒答太泛，用户无法获得安全帮助。
3. 语气生硬，产品体验差。
4. 把合法教育、研究、防御请求也拒绝。
5. 多语言拒答不一致。

因此，拒答数据要同时训练“拒绝什么”和“如何安全地帮助”。

12. 边界允许数据

安全训练不能只有拒答数据，还必须有边界允许数据。

例如，用户问安全治理、历史事件、疾病科普、法律常识、网络安全防御、化学安全规范等内容，模型应该在安全边界内提供帮助。如果训练集中只有风险关键词对应拒答，模型会学会看到关键词就拒绝。

边界允许数据的作用是减少误拒。

它教模型：

1. 教育性解释可以回答。
2. 防御性安全建议可以回答。
3. 高层次风险识别可以回答。
4. 合法合规的专业常识可以回答。
5. 不提供操作性伤害细节也能有帮助。

这是安全数据中最容易被忽略的一类。

13. 红队数据

红队数据来自对模型的系统性安全测试。目标是发现模型在边界场景下可能产生的有害输出，然后用这些发现改进模型。

Anthropic 的 red teaming 工作系统讨论了语言模型红队方法、规模行为和经验教训，并释放了红队攻击数据用于分析有害输出模式。这里的重点不是传播攻击方法，而是用系统化评估发现和降低伤害。

红队数据可以包括：

1. 风险类别。
2. 模型失败模式。
3. 安全响应示例。
4. 拒答边界。
5. 多轮对话风险。
6. 多语言风险。
7. 模型版本和策略变化后的回归样例。

在训练中使用红队数据时，要注意只保留必要的风险标签、上下文和安全响应，不传播可操作风险细节。

14. 误拒和漏拒

安全系统有两个基本错误：

1. 误拒：该回答的安全请求被拒绝。
2. 漏拒：该拒绝的高风险请求被回答。

只降低漏拒会导致模型保守、无用、动不动拒绝。只降低误拒会导致安全风险上升。

安全数据建设要同时收集两类样本：

1. 漏拒修复数据：高风险请求对应安全拒答。
2. 误拒修复数据：安全请求对应正常帮助。

高质量安全模型不是拒答率最高的模型，而是在风险边界上最稳定的模型。

15. 偏好数据和安全数据的关系

安全数据可以进入偏好训练。

例如同一个风险 prompt 下，可以构造：

1. chosen：拒绝危险细节，并提供安全替代建议。
2. rejected：提供危险细节，或拒答过度，或语气恶劣，或没有帮助。

这样模型不仅学会“拒绝”，还学会“什么样的拒绝更好”。

对边界允许场景也可以构造偏好对：

1. chosen：安全、高层次、教育性回答。
2. rejected：过度拒绝或不必要恐吓。

因此，偏好数据是安全行为精调的重要载体。

16. 专业高风险领域安全数据

医学、法律、金融等领域需要专门安全数据。

医学数据要教模型：

1. 区分科普和诊断。
2. 遇到急症建议及时求助。
3. 不给确定性治疗方案。
4. 表达不确定性和个体差异。

法律数据要教模型：

1. 区分法律信息和法律意见。
2. 提醒法域和时效差异。
3. 不替代律师判断。
4. 保留证据和程序边界。

金融数据要教模型：

1. 区分事实、观点和投资建议。
2. 提醒风险。
3. 避免承诺收益。
4. 对监管边界保持谨慎。

这些领域的偏好数据最好由专家参与标注和审计。

17. 多语言安全数据

安全能力不能只在英文上好。许多模型在英语下安全，在其他语言、混合语言或口语表达中边界变弱。

多语言安全数据要覆盖：

1. 主要目标语言。
2. 低资源语言。
3. 混合语言。
4. 方言和口语表达。
5. 跨文化敏感内容。
6. 多语言拒答风格。

直接翻译英文安全数据是不够的。不同语言的表达、法律环境、文化语境和用户习惯不同，需要本地化审计。

18. LLM judge 在偏好和安全数据中的作用

LLM judge 可以辅助偏好标注和安全审核。它能降低成本，快速筛选明显低质量样本，生成初始解释，帮助标注者聚焦难例。

但 LLM judge 不能无审计替代人类。

风险包括：

1. 偏好固定风格。
2. 对长回答有偏好。
3. 对事实错误不敏感。
4. 对安全边界理解不稳定。
5. 多语言能力不均。
6. 可能偏袒同源模型输出。

合理用法是：LLM judge 做初筛和辅助，关键样本、边界样本、高风险样本由人类或专家审计。

19. 数据隐私与标注员安全

偏好数据和安全数据常来自用户对话、红队样本和高风险场景，因此必须重视隐私和标注员安全。

隐私治理包括：

1. 用户授权和数据使用边界。
2. PII 删除或脱敏。
3. 访问控制。
4. 数据保留期限。
5. 审计日志。
6. 敏感样本隔离。

标注员安全包括：

1. 对高风险内容分级。
2. 提供心理支持和工作负载控制。
3. 允许跳过严重不适样本。
4. 最小化暴露不必要细节。
5. 用工具预过滤极端内容。

数据质量不能以标注员伤害为代价。

20. Preference data 的评估

评估偏好数据可以看：

1. 标注一致性。
2. 标注者间 agreement。
3. chosen/rejected 难度分布。
4. prompt 覆盖度。
5. 语言和领域覆盖。
6. 安全类别覆盖。
7. 标注理由质量。
8. 模型训练后的 win rate。
9. 是否引入长度偏置、拒答偏置或迎合偏置。
10. 人工回归测试。

注意，偏好训练后模型 win rate 提升不一定代表整体变好。还要看事实性、安全、通用能力、专业能力和产品体验。

21. 安全数据的评估

安全数据评估应同时看漏拒和误拒。

常见指标包括：

1. 高风险请求拒答率。
2. 安全请求正常回答率。
3. 边界场景正确处理率。
4. 多语言安全一致性。
5. 专业高风险场景合规率。
6. 拒答质量评分。
7. 安全替代建议质量。
8. 红队回归通过率。
9. 用户体验影响。
10. 线上安全事件率。

好的安全评估不是只看“拒绝了多少”，而是看“拒绝是否该拒绝，帮助是否安全地帮助”。

22. 面向专家：Preference data 是价值函数样本

从专家视角看，preference data 是对人类价值函数的稀疏采样。它不是客观真理，而是标注规范、标注者群体、任务分布、产品定位和社会规范共同作用的结果。

这带来几个专家级问题：

1. 偏好是否代表目标用户？
2. 标注者是否理解任务和领域？
3. 安全优先级是否压倒了帮助性？
4. 模型是否学会迎合而不是真实？
5. 线上用户偏好和离线标注偏好是否一致？
6. 不同文化和语言下偏好是否冲突？
7. 偏好数据是否会随时间过期？

因此，偏好数据不是一次性资产，而是持续更新的模型行为治理系统。

23. 一个可落地的偏好与安全数据方案

如果面试官问：“如何建设 preference data 和安全数据？”可以按下面回答。

第一步，定义行为准则。明确 helpful、honest、harmless 的优先级，以及不同风险类别的处理方式。

第二步，构建 prompt 池。覆盖真实用户请求、长尾任务、专业领域、多语言、边界案例和红队发现。

第三步，生成候选回答。使用不同模型版本、多采样、人工回答或 teacher 生成候选。

第四步，偏好标注。让标注者选择 chosen/rejected，并标注原因，如事实性、安全性、帮助性、格式或语气。

第五步，安全分层。把样本按明确安全、敏感可答、高风险、边界模糊、紧急风险、专业高风险分桶。

第六步，构造拒答和边界允许数据。既训练该拒绝的样本，也训练不该误拒的样本。

第七步，红队闭环。用红队评估发现失败模式，转化为安全训练和回归测试数据。

第八步，质量审计。检查标注一致性、长度偏置、拒答偏置、LLM judge 偏差和多语言一致性。

第九步，训练和评估。用于 RM、RLHF、DPO 或其他偏好优化，再通过 win rate、安全评估、误拒/漏拒和人工样例验证。

第十步，治理和版本化。记录标注规范、数据来源、标注者协议、隐私处理、策略版本、训练版本和评估结果。

23.1 最小可运行偏好与安全数据审计 demo

下面这个 demo 不依赖外部库，也不读写文件。输入是一组 toy preference / safety 样本；输出包括保留样本、拒绝原因、风险配比、语言配比、平均偏好 margin、误拒 / 漏拒修复覆盖和门禁结果。

它演示的是数据治理机制，不是生产级 reward model、DPO trainer、安全分类器、红队平台或隐私系统。真实项目要接入标注平台、专家复核、policy registry、PII 脱敏、红队回归集、人工一致性统计、训练 ablation 和线上安全监控。

```
from collections import Counter, defaultdict
```

```
samples = [
    {"id": "pref_summary_helpful", "kind": "preference", "risk": "normal", "lang": "en", "tokens": 760, "chosen": {"helpful": 0.9, "honest": 0.8, "harmless": 0.7}},
    {"id": "pref_factual_citation", "kind": "preference", "risk": "normal", "lang": "zh", "tokens": 820, "chosen": {"helpful": 0.8, "honest": 0.7, "harmless": 0.6}},
    {"id": "safe_high_risk_refusal", "kind": "safety", "risk": "high_risk", "lang": "en", "tokens": 690, "expected": "refusal", "action": "refusal"},
    {"id": "safe_boundary_allowed", "kind": "safety", "risk": "boundary_allowed", "lang": "en", "tokens": 610, "expected": "boundary_allowed", "action": "boundary_allowed"},
    {"id": "safe_professional_boundary", "kind": "safety", "risk": "professional", "lang": "zh", "tokens": 740, "expected": "professional", "action": "professional"},
    {"id": "safe_privacy_refusal", "kind": "safety", "risk": "privacy", "lang": "en", "tokens": 580, "expected": "refusal", "action": "refusal"},
    {"id": "pref_low_agreement", "kind": "preference", "risk": "normal", "lang": "en", "tokens": 650, "chosen": {"helpful": 0.6, "honest": 0.5, "harmless": 0.4}},
    {"id": "pref_length_bias", "kind": "preference", "risk": "normal", "lang": "en", "tokens": 920, "chosen": {"helpful": 0.7, "honest": 0.6, "harmless": 0.5}},
    {"id": "safe_wrong_action", "kind": "safety", "risk": "boundary_allowed", "lang": "zh", "tokens": 560, "expected": "boundary_allowed", "action": "wrong_action"},
    {"id": "pref_private_log", "kind": "preference", "risk": "normal", "lang": "en", "tokens": 500, "chosen": {"helpful": 0.5, "honest": 0.4, "harmless": 0.3}},
    {"id": "pref_eval_leak", "kind": "preference", "risk": "normal", "lang": "en", "tokens": 540, "chosen": {"helpful": 0.4, "honest": 0.3, "harmless": 0.2}}
]
```

```
WEIGHTS = {"helpful": 0.38, "honest": 0.32, "harmless": 0.30}
```

```
MIN_MARGIN = 0.08
```

```
MIN_AGREEMENT = 0.67
```

```
TARGET_RISKS = {"normal", "high_risk", "boundary_allowed", "professional", "privacy"}
```

```
def quality(answer):
    return sum(WEIGHTS[k] * answer[k] for k in WEIGHTS)
```

```
def reject_reason(item):
    chosen_q = quality(item["chosen"])
    rejected_q = quality(item["rejected"])
    margin = chosen_q - rejected_q
    if not item["rubric"]:
        return "missing_rubric"
    if item["pii"]:
        return "privacy_or_pii"
    if item["contam"]:
        return "eval_contamination"
    if item["agreement"] < MIN_AGREEMENT:
        return "low_labeler_agreement"
    if not item["chosen"]["safe"]:
        return "unsafe_chosen"
    if item["kind"] == "safety" and item["chosen"]["action"] != item["expected"]:
        return "wrong_safety_action"
    if margin < MIN_MARGIN:
        if item["chosen"]["length"] > 2.5 * item["rejected"]["length"]:
            return "length_bias_risk"
```

```

        return "weak_preference_margin"
    return None

kept, rejected, rows = [], {}, []
for item in samples:
    c_q = round(quality(item["chosen"]), 3)
    r_q = round(quality(item["rejected"]), 3)
    reason = reject_reason(item)
    rows.append({"id": item["id"], "risk": item["risk"], "chosen_q": c_q, "rejected_q": r_q, "margin": round(c_q - r_q, 3)})
    if reason:
        rejected[item["id"]] = reason
    else:
        kept.append(item)

risk_tokens, lang_tokens = defaultdict(int), defaultdict(int)
kind_counts = Counter()
for item in kept:
    risk_tokens[item["risk"]] += item["tokens"]
    lang_tokens[item["lang"]] += item["tokens"]
    kind_counts[item["kind"]] += 1

kept_tokens = sum(item["tokens"] for item in kept)
raw_tokens = sum(item["tokens"] for item in samples)
safety_kept = [item for item in kept if item["kind"] == "safety"]
leak_repairs = [item["id"] for item in safety_kept if item["expected"] == "refuse"]
over_refusal_repairs = [item["id"] for item in safety_kept if item["expected"] == "answer"]
coverage = len(set(risk_tokens) & TARGET_RISKS) / len(TARGET_RISKS)

report = {
    "kept_ids": [item["id"] for item in kept],
    "rejected": dict(sorted(rejected.items())),
    "reason_counts": dict(sorted(Counter(rejected.values()).items())),
    "retention": round(kept_tokens / raw_tokens, 3),
    "kind_counts": dict(sorted(kind_counts.items())),
    "risk_mix": {k: round(risk_tokens[k] / kept_tokens, 3) for k in sorted(risk_tokens)},
    "lang_mix": {k: round(lang_tokens[k] / kept_tokens, 3) for k in sorted(lang_tokens)},
    "avg_margin": round(sum(row["margin"] for row in rows if row["reason"] == "kept") / len(kept), 3),
    "coverage": round(coverage, 3),
    "leak_repairs": leak_repairs,
    "over_refusal_repairs": over_refusal_repairs,
    "score_preview": {row["id"]: (row["chosen_q"], row["rejected_q"], row["margin"]) for row in rows},
}

gates = {
    "agreement_filter": "low_labeler_agreement" in report["reason_counts"],
    "privacy_filter": "privacy_or_pii" in report["reason_counts"],
    "contamination_filter": "eval_contamination" in report["reason_counts"],
    "length_bias_filter": "length_bias_risk" in report["reason_counts"],
    "wrong_action_filter": "wrong_safety_action" in report["reason_counts"],
    "risk_coverage": report["coverage"] >= 1.0,
    "safety_balance": bool(leak_repairs) and bool(over_refusal_repairs),
    "preference_and_safety": kind_counts["preference"] >= 2 and kind_counts["safety"] >= 3,
}
report["gates"] = gates

```

```

report["gate_pass"] = all(gates.values())

for key, value in report.items():
    print(f"{key}=", value)

assert report["kept_ids"] == [
    "pref_summary_helpful",
    "pref_factual_citation",
    "safe_high_risk_refusal",
    "safe_boundary_allowed",
    "safe_professional_boundary",
    "safe_privacy_refusal",
]
assert report["reason_counts"] == {
    "eval_contamination": 1,
    "length_bias_risk": 1,
    "low_labeler_agreement": 1,
    "privacy_or_pii": 1,
    "wrong_safety_action": 1,
}
assert report["retention"] == 0.57
assert report["kind_counts"] == {"preference": 2, "safety": 4}
assert report["risk_mix"] == {"boundary_allowed": 0.145, "high_risk": 0.164, "normal": 0.376, "privacy": 0.138, "professional": 0.176}
assert report["coverage"] == 1.0
assert report["gate_pass"] is True

```

运行后会看到类似输出：

```

kept_ids= ['pref_summary_helpful', 'pref_factual_citation', 'safe_high_risk_refusal', 'safe_boundary_allowed', 'safe_professional_boundary', 'safe_privacy_refusal']
rejected= {'pref_eval_leak': 'eval_contamination', 'pref_length_bias': 'length_bias_risk', 'pref_low_agreement': 'low_labeler_agreement', 'pref_privacy_or_pii': 'privacy_or_pii', 'pref_wrong_safety_action': 'wrong_safety_action'}
reason_counts= {'eval_contamination': 1, 'length_bias_risk': 1, 'low_labeler_agreement': 1, 'privacy_or_pii': 1, 'wrong_safety_action': 1}
retention= 0.57
kind_counts= {'preference': 2, 'safety': 4}
risk_mix= {'boundary_allowed': 0.145, 'high_risk': 0.164, 'normal': 0.376, 'privacy': 0.138, 'professional': 0.176}
lang_mix= {'en': 0.629, 'zh': 0.371}
avg_margin= 0.241
coverage= 1.0
leak_repairs= ['safe_high_risk_refusal', 'safe_privacy_refusal']
over_refusal_repairs= ['safe_boundary_allowed']
gates= {'agreement_filter': True, 'privacy_filter': True, 'contamination_filter': True, 'length_bias_filter': True, 'professional_filter': True, 'wrong_safety_action_filter': True}
gate_pass= True

```

这个 demo 的重点是把偏好数据和安全数据放在同一个治理闭环里：偏好样本要检查标注一致性、margin 和长度偏置；安全样本要同时覆盖漏拒修复和误拒修复；所有数据都要过隐私、污染、rubric 和版本门禁。

24. 常见面试题

24.1 什么是 preference data?

preference data 是对同一 prompt 下多个回答的比较数据，常见形式是 prompt、chosen、rejected。它用于训练 reward model、DPO 或其他偏好优化方法，让模型更符合人类偏好。

24.2 RLHF 和 DPO 对数据的要求有什么共同点?

它们都依赖高质量偏好数据。RLHF 通常用排序数据训练 reward model，再用强化学习优化模型；DPO 直接用 chosen/rejected 偏好对优化模型。数据质量差，两者都会学偏。

24.3 偏好标注看哪些维度？

常见维度包括 helpful、honest、harmless，也包括事实性、完整性、简洁性、格式、语气、引用、专业边界和安全性。

24.4 安全数据是不是只要拒答数据？

不是。安全数据还包括风险分类、边界允许、误拒修复、安全替代建议、多语言安全、专业高风险场景和红队回归数据。只有拒答数据会导致模型过度拒绝。

24.5 红队数据怎么用于训练？

红队数据用于发现模型失败模式，并转化为安全训练样本和回归测试集。使用时应保留风险标签和安全响应，避免传播可操作风险细节。

24.6 如何平衡误拒和漏拒？

同时建设漏拒修复数据和误拒修复数据；评估时同时看高风险拒答率和安全请求正常回答率；对边界样本进行人工审计和多语言评估。

24.7 LLM judge 能否替代人工偏好标注？

不能完全替代。LLM judge 可用于初筛、辅助解释和规模化标注，但边界样本、高风险样本、专业领域和关键评测仍需要人类或专家审计。

25. 常见误区

误区一：偏好数据只是选更长的回答。

更长不一定更好。偏好要综合事实性、帮助性、安全性、简洁性和用户意图。

误区二：chosen/rejected 随便构造就能训 DPO。

如果 rejected 太弱或标注噪声大，模型学不到细粒度偏好，甚至会学到错误偏好。

误区三：安全数据越多，模型越安全。

如果数据只有拒答，会造成过度拒绝。安全需要边界允许、误拒修复和真实场景覆盖。

误区四：红队数据就是攻击样本库。

红队数据的目标是发现和降低伤害，应转化为防御性训练、评估和治理数据，而不是传播攻击方法。

误区五：偏好训练可以解决事实性问题。

偏好训练能改善行为，但不能完全替代知识质量、检索、引用和事实核验。

误区六：线上用户反馈都是真实偏好。

线上反馈有噪声、选择偏差和产品上下文，必须清洗、分桶和审计。

26. 本章小结

Preference Data 与安全数据是大模型从“会回答”走向“回答得更符合人类期望、更安全、更可控”的关键数据。

本章要记住几句话：

1. 偏好数据提供比较信号，常见形式是 prompt、chosen、rejected。

2. RLHF 和 DPO 算法不同，但都依赖高质量偏好数据。
3. 安全数据不只是拒答，还包括边界允许、误拒修复、红队回归和专业高风险场景。
4. 好的安全模型不是拒答最多，而是边界最稳定。
5. 红队数据应服务于防御、评估和治理，不应传播可操作风险细节。
6. 偏好和安全数据需要标注规范、隐私治理、标注员保护、审计和版本化。

如果面试中被问到偏好数据和安全数据，最好的回答是：先定义行为准则，再构建覆盖真实用户、边界场景、多语言和红队发现的 prompt 池，生成候选回答，标注 chosen/rejected 和安全类别，同时建设拒答与边界允许数据，用 RLHF/DPO 等方法训练，并通过误拒、漏拒、win rate、人工审计和红队回归形成闭环。

第九章：多模态、语音与视频数据

前面几章主要讨论文本、代码、数学、偏好和安全数据。本章进入多模态数据工程：图文、语音和视频数据。

多模态模型看起来是在模型结构上“加图像、加音频、加视频”，但真正决定上限的往往是数据：图像和文本是否真的对应，语音和转录是否准确，视频片段和描述是否时间对齐，OCR 是否保留，字幕是否错位，版权和隐私是否可控。

多模态数据比纯文本更难。文本只需要处理 token 序列，而图像、音频、视频还涉及采样率、分辨率、帧率、时间戳、模态对齐、视觉质量、声音质量、场景变化、人物隐私、版权来源和标注一致性。

本章重点：图文对、caption、OCR、视频片段、音频转录、时间对齐、版权和隐私。

合规边界：本章讨论合法合规的多模态数据治理、质量控制、隐私保护和评估，不提供抓取未经授权媒体、规避访问控制、识别或追踪个人身份的操作方法。

0. 本讲资料边界与第二轮精修口径

按照 WRITING_PLAN.md 的要求，本讲精修前核对了 CLIP、LAION-5B、Whisper、WebVid / Frozen in Time、VQA、TextVQA 和 DocVQA 等公开论文资料。

本讲聚焦图文、OCR、语音、视频和多模态指令数据的防御性数据治理：合法来源、许可证、图文 / 音文 / 视频字幕对齐、OCR 空间结构、ASR 时间戳、视频切片、隐私脱敏、评测污染、质量评分、配比和版本审计。

媒体来源 -> 模态解析 -> 质量检测 -> 跨模态对齐 -> 隐私 / 版权 -> 污染隔离 -> 训练样本 -> 评估闭环 -> 版本治理

本讲不提供未经授权媒体抓取、规避平台访问控制、身份识别、声纹追踪、车牌 / 人脸追踪或敏感个人信息复原方法。多模态数据只讨论训练数据治理和安全边界。

1. 先建立直觉：多模态数据为什么难？

纯文本训练样本通常是一个文本片段。多模态训练样本则可能是：

1. 一张图片和一句 caption。
2. 一张图片和 OCR 文本。
3. 一段语音和对应转录。
4. 一段视频和字幕。
5. 一段视频、音频、字幕、镜头切片和事件描述。
6. 一个网页中的图片、周围文本、标题、alt text 和链接上下文。

这些模态之间不一定对齐。网页上的图片可能只是装饰图；caption 可能是广告语；视频字幕可能延迟几秒；音频转录可能漏词；OCR 可能把表格读错；图片里的人脸、车牌、医疗影像可能涉及隐私。

所以多模态数据工程的核心不是“把图片和文字拼起来”，而是判断不同模态之间是否真的互相解释。

2. 来龙去脉：从人工标注到 web-scale 弱监督

早期视觉模型主要依赖人工标注数据集，例如图像分类、检测、分割数据集。每张图片有固定类别或框标注。这类数据质量高，但规模有限，概念空间也受类别表限制。

CLIP 代表了一条重要路线：从互联网上的大规模图文对中学习视觉表示。它不是让模型只预测固定类别，而是通过自然语言监督学习图像和文本的对应关系。CLIP 论文使用 4 亿图文对，展示了自然语言监督对 zero-shot transfer 的价值。

语音领域也经历了类似变化。Whisper 使用大规模、多语言、多任务弱监督音频转录数据训练，强调规模和多样性对鲁棒语音识别的价值。

视频领域更复杂，因为视频包含图像帧、动作、声音、字幕、时间结构和事件演化。大规模视频数据通常来自视频平台、课程、电影片段、会议、直播、教学和监控类场景，但合规和隐私要求更高。

这条历史线索说明：多模态能力的突破很大程度来自 web-scale 弱监督数据，但弱监督数据必须经过严格清洗和对齐。

3. 多模态数据的基本类型

常见多模态数据包括：

1. 图文对：image-caption、image-alt text、image-surrounding text。
2. 图像问答：image-question-answer。
3. OCR 数据：图像中的文字和结构。
4. 文档图像：PDF 页面、表格、票据、截图、网页截图。
5. 语音转录：audio-transcript。
6. 语音对话：speaker turns、timestamps、transcript。
7. TTS 数据：text-audio-speaker。
8. 视频字幕：video segment-subtitle。
9. 视频问答：video-question-answer。
10. 视频描述：clip-caption、event annotation、action label。
11. 多模态指令数据：image/video/audio + user instruction + answer。

不同数据服务不同能力。图文对适合视觉语言对齐；OCR 数据适合读图和文档理解；语音转录适合 ASR；视频字幕适合视频理解；多模态指令数据适合 VLM assistant。

3.1 关键公式与审计指标

多模态数据池可以写成：

$$D_{\{\mathrm{mm}\}} = D_{\{\mathrm{it}\}} \cup D_{\{\mathrm{ocr}\}} \cup D_{\{\mathrm{aud}\}} \cup D_{\{\mathrm{vid}\}} \cup D_{\{\mathrm{inst}\}}$$

其中 D_{it} 是 image-text 数据， D_{ocr} 是 OCR / 文档图像数据， D_{aud} 是 audio-transcript 数据， D_{vid} 是 video-caption / video-QA 数据， D_{inst} 是多模态指令数据。

一条样本可以表示为：

$$m_i = (u_i, t_i, q_i, a_i, r_i, z_i)$$

其中 u_i 是媒体对象或媒体片段元数据， t_i 是文本、caption、transcript、subtitle 或 answer， q_i 是媒体质量分， a_i 是跨模态对齐分， r_i 是隐私、版权、安全和污染风险， z_i 是来源、license、时间戳、hash、OCR / ASR 版本和标注版本。

图文对齐可以用归一化 embedding 的余弦相似度近似：

$$A_{\{\mathrm{it}\}, i} = \frac{f_I(I_i)^{\mathrm{top}} f_T(T_i)}{\|f_I(I_i)\|_2 \|f_T(T_i)\|_2}$$

这个分数只能作为过滤信号，不能替代人工抽检。图表、文档、新闻、商品图和截图都可能出现语义相关但描述不充分的问题。

OCR / 文档样本需要保留空间结构：

$$O_i = \{(s_{ij}, b_{ij}, p_{ij})\}_{j=1}^{n_i}$$

其中 s_{ij} 是识别文本, $b_{ij}=(x_{ij}, y_{ij}, h_{ij}, \omega_{ij})$ 是位置框, ω_{ij} 表示框宽度, p_{ij} 是 OCR 置信度。只保存纯文本会丢失阅读顺序、表格结构和版面关系。

语音转录质量常用词错误率:

$$WER = \frac{S+D+I}{N}$$

其中 S 是替换数, D 是删除数, I 是插入数, N 是参考 transcript 的词数。中文或字符级任务也可以看 CER。

音文时间对齐误差可以写成:

$$E_{\{\mathrm{aud}, i\}} = \frac{1}{K_i} \sum_{k=1}^{K_i} |\hat{t}_{ik} - t_{ik}|$$

如果字幕或 transcript 和音频错位, 模型会学到错误对齐。

视频切片后帧数可以粗略写成:

$$F_i = \lfloor \mathrm{fps}_i \cdot \Delta t_i \rfloor$$

其中 Δt_i 是片段时长。训练时还要记录关键帧、字幕时间、ASR 时间、OCR 时间和事件标签。

视频字幕或事件对齐误差可以写成:

$$E_{\{\mathrm{vid}, i\}} = |\hat{t}_{\{\mathrm{caption}, i\}} - t_{\{\mathrm{event}, i\}}|$$

视频数据比图像多一个时间维度, E_{vid} 过大时, 即使字幕文本质量高, 也不应直接进入训练。

隐私风险率可以按样本数或 token / 时长统计:

$$R_{\{\mathrm{privacy}\}} = \frac{\sum_i I(P_i=1)}{n}$$

其中 P_i 表示人脸、车牌、证件、声纹、位置、医疗信息、私人聊天或其他敏感信息命中。多模态隐私不能只看文本转录, 也要看原始媒体。

多模态样本门禁可以写成:

$$G_i = I(q_i \geq \tau_q) I(a_i \geq \tau_a) I_{\{\mathrm{license}\}, i} I_{\{\mathrm{privacy}\}, i} I_{\{\mathrm{time}\}, i} I_{\{\mathrm{contam}\}, i}$$

其中 I_{ground} 表示多模态问答或指令回答确实基于输入模态, 而不是凭语言先验猜测。

最终上线门禁可以写成:

$$G_{\{\mathrm{mm}\}} = I(C_{\{\mathrm{modality}\}} \geq c_{\{\min\}}) I(R_{\{\mathrm{privacy}\}} \leq \rho_{\{\max\}}) I(R_{\{\mathrm{contam}\}} \leq \kappa_{\{\max\}})$$

面试里要强调: 多模态数据工程的核心是对齐、证据和治理, 不是把图片、音频、视频文件和文本简单拼接。

4. 图文对数据

图文对是视觉语言模型最基础的数据形态。一个样本通常包含图片和对应文本。

文本来源可能是:

1. 人工 caption。
2. 网页 alt text。
3. 图片周围文本。
4. 标题。
5. 社交媒体正文。
6. 商品描述。
7. OCR 结果。
8. 模型生成 caption。

每种来源都有问题。

人工 caption 质量高但规模小。alt text 常常缺失或很短。周围文本可能和图片无关。商品描述可能带营销。社交媒体文本口语化且噪声大。OCR 有识别错误。模型生成 caption 可能同质化或遗漏细节。

因此，图文对数据的关键是 image-text alignment。

5. 图文对质量过滤

图文对质量可以从多个维度评估：

1. 图片是否可打开。
2. 图片分辨率是否足够。
3. 图片是否重复、模糊、水印过重或压缩严重。
4. 文本是否为空、乱码、广告或模板。
5. 文本是否真的描述图片。
6. 是否包含 OCR 文字且需要保留。
7. 是否涉及人脸、车牌、证件、医疗信息等隐私。
8. 是否存在版权或来源限制。
9. 是否包含不适合训练的风险内容。

图文对齐常用信号包括：

1. CLIP-like 相似度。
2. 图像 captioning 模型打分。
3. OCR 与文本重合度。
4. 图像分类标签与文本关键词是否一致。
5. 人工抽样审计。

但相似度分数不能替代人工判断。比如一张新闻图和新闻标题可能语义相关但不描述画面；一张商品图和商品详情相关但文本很长；一张图表的关键在 OCR 和坐标轴，而不是普通 caption。

6. Caption 数据

Caption 是图像或视频片段的自然语言描述。好的 caption 应该准确、具体、不过度臆测。

常见 caption 问题包括：

1. 太泛：例如“一张图片”。
2. 太短：只有对象名，没有关系和场景。
3. 幻觉：描述了图中不存在的内容。
4. 主观推断：对人物意图、身份、情绪过度猜测。
5. 缺少细节：不描述文字、数量、位置和动作。
6. 偏见表达：对人群属性做不必要判断。
7. 时间错位：视频 caption 描述了别的片段。

高质量 caption 数据需要平衡准确性和丰富度。过短训练不出复杂理解；过长且含臆测会引入幻觉。

7. OCR 和文档图像数据

很多多模态任务不是“看图识物”，而是“读图中文字”。例如截图、表格、发票、论文页面、合同扫描件、网页截图、PPT、海报、菜单、地图。

OCR 数据的关键不只是识别文字，还要保留结构：

1. 文字内容。
2. 位置坐标。
3. 行列关系。
4. 表格结构。
5. 阅读顺序。

6. 字体和强调。
7. 图文混排关系。

如果只把 OCR 结果拼成一串文本，模型可能丢失表格、版面和空间关系。

文档图像数据还涉及敏感信息。票据、合同、证件、病历、聊天截图、后台截图都可能包含个人信息或商业秘密。进入训练前必须做来源授权、脱敏、访问控制和审计。

8. 图像数据去重和污染

图像数据也需要去重。重复可能来自转载、缩略图、不同尺寸、裁剪、加水印、镜像和压缩版本。

常见方法包括：

1. exact hash：识别完全相同图片。
2. perceptual hash：识别近似相同图片。
3. embedding 相似度：识别语义或视觉相似图片。
4. OCR 文本匹配：识别同一截图或文档。
5. 来源 URL 和元数据去重。

图像污染也存在。如果评测集图片、题目或答案出现在训练集中，VLM 评估会虚高。对于图像问答、OCR benchmark、文档理解 benchmark，都需要 train-eval overlap 检测。

9. 语音数据

语音数据的核心样本通常是 audio-transcript pair。

语音数据来源包括：

1. 有声书。
2. 播客。
3. 会议录音。
4. 课程视频音轨。
5. 字幕视频。
6. 电话客服录音。
7. 公开演讲。
8. TTS 合成数据。
9. 多语言语音数据集。

语音数据比文本多很多维度：

1. 采样率。
2. 音质。
3. 噪声。
4. 口音。
5. 语速。
6. 说话人。
7. 语言切换。
8. 情绪。
9. 背景音乐。
10. 时间戳。

Whisper 的经验说明，大规模、多语言、多任务弱监督语音数据能带来鲁棒性。但弱监督也意味着转录质量不均、字幕错位和噪声很常见。

10. 语音转录和时间对齐

语音训练最关键的问题之一是时间对齐。

如果一段 30 秒音频对应一句完整字幕，模型很难知道哪个词在哪个时间出现。如果字幕延迟或提前，模型会学到错误对齐。

常见处理包括：

1. VAD：切分语音和非语音区域。
2. speaker diarization：区分说话人。
3. forced alignment：把 transcript 对齐到音频时间轴。
4. segment filtering：过滤太长、太短、噪声过大或转录不匹配片段。
5. language identification：识别音频语言。
6. transcript normalization：统一标点、大小写、数字和口语填充词。

转录质量评估可以看 WER、CER、语言识别准确率、人工抽检和模型训练后的验证表现。

11. 语音数据隐私和合规

语音数据天然敏感。声音可能包含身份线索、情绪、健康状态、地点、背景谈话和个人信息。

语音数据治理要关注：

1. 录音授权。
2. 说话人同意。
3. PII 转录脱敏。
4. 声纹隐私。
5. 儿童声音和敏感人群。
6. 电话、会议、客服等场景的合规要求。
7. 数据访问控制。
8. 保留期限和删除机制。

如果目标是训练 ASR、TTS 或语音助手，隐私治理不能只处理文本转录，还要处理原始音频。

12. 视频数据

视频是最复杂的模态，因为它同时包含图像序列、声音、字幕、动作、场景变化和时间关系。

视频数据样本可能包含：

1. 原始视频。
2. 关键帧。
3. 音频轨。
4. 字幕。
5. ASR 转录。
6. OCR。
7. 镜头切分。
8. 时间戳。
9. 事件标签。
10. 视频 caption 或 QA。

视频模型需要理解的不只是单帧内容，还包括动作和事件：谁在什么时候做了什么，前后发生了什么变化，声音和画面是否对应。

13. 视频切片和时间对齐

视频不能总是整段进入训练。长视频通常要切成片段。

切片策略包括：

1. 固定长度切片。
2. 按镜头切分。
3. 按字幕句子切分。
4. 按事件检测切分。
5. 按音频活动切分。

每种策略都有 trade-off。

固定长度简单但可能切断事件。镜头切分符合视觉变化但不一定对应语义。字幕切分适合语言对齐但字幕可能错位。事件切分语义好但需要额外模型。

视频时间对齐要处理：

1. 字幕和语音是否同步。
2. 描述是否对应当前画面。
3. 音频事件是否对应视觉事件。
4. OCR 是否来自当前帧。
5. QA 是否需要跨片段上下文。

视频数据工程的难点就在这里：时间是一个额外维度。

14. 多模态指令数据

多模态助手需要 instruction data。例如：

1. “这张图里有什么？”
2. “读一下截图里的表格。”
3. “这段视频发生了什么？”
4. “这段录音主要讲了什么？”
5. “根据图片和文字回答问题。”
6. “比较两张图的差异。”
7. “从视频中找出关键步骤。”

这类数据通常来自人工标注、模型生成、任务数据集改写和真实产品反馈。

关键质量问题是：回答必须 grounded in modality。也就是说，回答应该基于图像、音频或视频内容，而不是只凭语言先验猜测。

15. 多模态数据的安全问题

多模态安全不只是文本安全。

图像可能包含人脸、证件、地址、车牌、医疗信息、儿童信息、暴力或成人内容。音频可能包含声纹和背景隐私。视频可能包含可识别人物、住址、工作场所、敏感事件和第三方隐私。

安全处理包括：

1. 来源授权。
2. PII 检测和脱敏。
3. 人脸、车牌、证件等敏感区域处理。
4. 不适宜内容分类。
5. 儿童和敏感人群保护。
6. 地理位置和元数据清理。

7. 访问控制和审计。
8. 数据删除和追踪机制。

多模态数据还要避免训练模型做不合适的身份识别、跟踪或敏感属性推断。

16. 版权和许可

多模态数据版权风险比文本更高。图片、音频、视频通常由创作者、平台、机构或个人持有权利。公开可访问不等于可训练。

版权治理要记录：

1. 来源 URL 或数据集。
2. license。
3. 使用范围。
4. 是否允许商业使用。
5. 是否允许再分发。
6. 是否允许生成衍生数据。
7. 下架和删除机制。
8. 数据版本和 hash。

尤其是视频和语音，平台条款、音乐版权、人物肖像权和字幕版权可能交织在一起。数据工程不能只看文件是否能下载。

17. 多模态质量评估

多模态数据质量评估要分模态看，也要看跨模态对齐。

图像维度：分辨率、清晰度、重复、水印、压缩、内容安全、OCR 可读性。

文本维度：caption 质量、语言、长度、是否广告、是否描述准确。

音频维度：采样率、噪声、语音活动、转录准确率、说话人、语言。

视频维度：帧率、镜头稳定性、字幕同步、音画一致、切片合理性。

跨模态维度：图文相似度、音文对齐、视频字幕对齐、OCR 与回答一致性、问答是否 grounded。

评估方法包括自动评分、人工抽检、小模型 ablation、下游 benchmark 和产品样例回归。

18. 面向专家：多模态数据是对齐问题，不是拼接问题

从专家视角看，多模态数据工程的本质是 alignment。

图像和文本要语义对齐；语音和转录要时间对齐；视频和字幕要事件对齐；OCR 和版面要空间对齐；多模态问答要证据对齐。

如果对齐差，模型会学到 shortcut。例如看到“猫”的 caption 就学猫，但图片里可能是狗；字幕讲上一幕，画面是下一幕；问题问表格，OCR 顺序却错乱。模型表面训练 loss 下降，真实能力却不稳定。

多模态数据还会放大偏见。图文对来自 web，会继承视觉文化偏见、职业刻板印象、地域偏差和语言偏差。语音数据会受口音、性别、年龄、录音设备影响。视频数据会受平台内容分布影响。

因此，多模态数据工程要同时处理对齐、质量、多样性、公平性、版权和隐私。

19. 一个可落地的多模态数据方案

如果面试官问：“如何建设图文、语音和视频数据？”可以按下面回答。

第一步，定义模型目标。是图文理解、OCR、文档问答、ASR、语音助手、视频理解，还是统一多模态助手。

第二步，分模态建池。图文对、OCR、语音转录、视频字幕、多模态指令数据分别建池，记录来源、license、时间、语言、质量和安全标签。

第三步，做基础清洗。图片去坏图、低清、水印和重复；音频去噪、切分、语言识别；视频做镜头切分、字幕对齐和关键帧抽取；文本去广告、乱码和模板。

第四步，做跨模态对齐评估。图文用相似度和人工抽检；音文用 forced alignment 和转录质量；视频用字幕、镜头和事件对齐；OCR 保留版面结构。

第五步，处理隐私和版权。识别人脸、车牌、证件、PII、声纹风险和平台许可，建立删除和追溯机制。

第六步，构造训练样本。包括 image-caption、image-QA、OCR-QA、audio-transcript、video-caption、video-QA 和多模态 instruction。

第七步，做配比实验。不同模态、任务和质量层级按目标能力混合，观察 OCR、视觉问答、ASR、视频理解和通用语言能力 trade-off。

第八步，评估闭环。用 benchmark、人工样例、红队安全、多语言、多口音和真实产品案例持续修正数据。

第九步，版本化治理。记录数据来源、hash、过滤规则、对齐模型、OCR/ASR 版本、标注规范和评估结果。

19.1 最小可运行多模态数据审计 demo

下面这个 demo 不依赖外部库，也不读写文件。输入是一组 toy 图文、OCR、语音、视频和多模态指令样本；输出包括保留样本、拒绝原因、分模态保留率、最终 mixture、平均对齐分和门禁结果。

它演示的是元数据级治理机制，不是真实 CLIP scorer、OCR、ASR、视频事件检测器、版权系统或隐私识别器。真实系统需要接入媒体解析、perceptual hash、embedding 检索、OCR / ASR / VAD / forced alignment、隐私脱敏、许可证审查、人工抽样和训练 ablation。

```
from collections import Counter, defaultdict
```

```
samples = [
    {"id": "img_product_caption", "kind": "image_text", "tokens": 520, "media_quality": 0.88, "alignment": 0.91, "annotation": "product"},
    {"id": "img_decor_misaligned", "kind": "image_text", "tokens": 430, "media_quality": 0.84, "alignment": 0.31, "annotation": "decor"},
    {"id": "ocr_doc_redacted", "kind": "ocr_doc", "tokens": 760, "media_quality": 0.82, "alignment": 0.86, "annotation": "doc"},
    {"id": "ocr_private_id", "kind": "ocr_doc", "tokens": 690, "media_quality": 0.86, "alignment": 0.88, "annotation": "private_id"},
    {"id": "audio_multilingual_clean", "kind": "audio_transcript", "tokens": 620, "media_quality": 0.80, "alignment": 0.80, "annotation": "audio"},
    {"id": "audio_noisy_bad_asr", "kind": "audio_transcript", "tokens": 580, "media_quality": 0.42, "alignment": 0.55, "annotation": "audio"},
    {"id": "video_recipe_aligned", "kind": "video_caption", "tokens": 980, "media_quality": 0.83, "alignment": 0.84, "annotation": "video"},
    {"id": "video_subtitle_shift", "kind": "video_caption", "tokens": 850, "media_quality": 0.79, "alignment": 0.75, "annotation": "video"},
    {"id": "video_eval_leak", "kind": "video_caption", "tokens": 710, "media_quality": 0.81, "alignment": 0.82, "annotation": "video"},
    {"id": "mm_instruction_grounded", "kind": "mm_instruction", "tokens": 670, "media_quality": 0.86, "alignment": 0.86, "annotation": "mm"},
    {"id": "mm_hallucinated_answer", "kind": "mm_instruction", "tokens": 640, "media_quality": 0.84, "alignment": 0.4, "annotation": "mm"},
    {"id": "video_unlicensed_clip", "kind": "video_caption", "tokens": 900, "media_quality": 0.80, "alignment": 0.78, "annotation": "video"}
]
```

```
THRESHOLD = 0.74
```

```
MAX_TIME_ERROR = {"image_text": 0.0, "ocr_doc": 0.0, "audio_transcript": 1.0, "video_caption": 2.0, "mm_instruction": 0.0}
```

```
def score(item):
    return round(0.38 * item["media_quality"] + 0.42 * item["alignment"] + 0.20 * item["annotation"], 3)
```

```

def reject_reason(item, q):
    if not item["license_ok"]:
        return "license_or_source"
    if item["privacy"]:
        return "privacy_or_sensitive"
    if item["contam"]:
        return "eval_contamination"
    if item["time_error"] > MAX_TIME_ERROR[item["kind"]]:
        return "time_misalignment"
    if not item["grounded"]:
        return "ungrounded_answer"
    if item["media_quality"] < 0.60:
        return "low_media_quality"
    if item["alignment"] < 0.62:
        return "low_alignment"
    if q < THRESHOLD:
        return "low_quality"
    return None

kept, rejected, rows = [], {}, []
for item in samples:
    q = score(item)
    reason = reject_reason(item, q)
    rows.append({"id": item["id"], "kind": item["kind"], "score": q, "reason": reason or "kept"})
    if reason:
        rejected[item["id"]] = reason
    else:
        kept.append(item)

raw_tokens = sum(item["tokens"] for item in samples)
kept_tokens = sum(item["tokens"] for item in kept)
kind_raw, kind_kept = defaultdict(int), defaultdict(int)
align_sum = defaultdict(float)

for item in samples:
    kind_raw[item["kind"]] += item["tokens"]
for item in kept:
    kind_kept[item["kind"]] += item["tokens"]
    align_sum[item["kind"]] += item["alignment"]

reason_counts = dict(sorted(Counter(rejected.values()).items()))
report = {
    "kept_ids": [item["id"] for item in kept],
    "rejected": dict(sorted(rejected.items())),
    "reason_counts": reason_counts,
    "retention": round(kept_tokens / raw_tokens, 3),
    "kind_retention": {k: round(kind_kept[k] / kind_raw[k], 3) for k in sorted(kind_raw)},
    "mixture": {k: round(kind_kept[k] / kept_tokens, 3) for k in sorted(kind_kept)},
    "avg_alignment": {k: round(align_sum[k] / sum(1 for item in kept if item["kind"] == k), 3) for k in sorted(kind_kept)},
    "score_preview": {row["id"]: row["score"] for row in rows},
}

gates = {

```

```

    "license_blocked": reason_counts.get("license_or_source", 0) == 1,
    "privacy_blocked": reason_counts.get("privacy_or_sensitive", 0) == 1,
    "contamination_blocked": reason_counts.get("eval_contamination", 0) == 1,
    "time_alignment": reason_counts.get("time_misalignment", 0) == 1,
    "grounding": reason_counts.get("ungrounded_answer", 0) == 1,
    "modality_coverage": set(kind_kept) == {"audio_transcript", "image_text", "mm_instruction", "ocr_doc", "video_caption"},
    "alignment_floor": all(v >= 0.80 for v in report["avg_alignment"].values()),
}
report["gates"] = gates
report["gate_pass"] = all(gates.values())

for key, value in report.items():
    print(f"{key}=", value)

assert report["kept_ids"] == [
    "img_product_caption",
    "ocr_doc_redacted",
    "audio_multilingual_clean",
    "video_recipe_aligned",
    "mm_instruction_grounded",
]
assert report["reason_counts"] == {
    "eval_contamination": 1,
    "license_or_source": 1,
    "low_alignment": 1,
    "low_media_quality": 1,
    "privacy_or_sensitive": 1,
    "time_misalignment": 1,
    "ungrounded_answer": 1,
}
assert report["retention"] == 0.425
assert report["kind_retention"] == {"audio_transcript": 0.517, "image_text": 0.547, "mm_instruction": 0.511, "ocr_doc": 0.524, "video_caption": 0.511}
assert report["mixture"] == {"audio_transcript": 0.175, "image_text": 0.146, "mm_instruction": 0.189, "ocr_doc": 0.214, "video_caption": 0.214}
assert report["gate_pass"] is True

```

运行后会看到类似输出：

```

kept_ids= ['img_product_caption', 'ocr_doc_redacted', 'audio_multilingual_clean', 'video_recipe_aligned', 'mm_instruction_grounded']
rejected= {'audio_noisy_bad_asr': 'low_media_quality', 'img_decor_misaligned': 'low_alignment', 'mm_hallucinated_answer': 'ungrounded_answer'}
reason_counts= {'eval_contamination': 1, 'license_or_source': 1, 'low_alignment': 1, 'low_media_quality': 1, 'privacy_or_sensitive': 1, 'time_misalignment': 1, 'ungrounded_answer': 1}
retention= 0.425
kind_retention= {'audio_transcript': 0.517, 'image_text': 0.547, 'mm_instruction': 0.511, 'ocr_doc': 0.524, 'video_caption': 0.511}
mixture= {'audio_transcript': 0.175, 'image_text': 0.146, 'mm_instruction': 0.189, 'ocr_doc': 0.214, 'video_caption': 0.214}
avg_alignment= {'audio_transcript': 0.83, 'image_text': 0.91, 'mm_instruction': 0.88, 'ocr_doc': 0.86, 'video_caption': 0.86}
gates= {'license_blocked': True, 'privacy_blocked': True, 'contamination_blocked': True, 'time_alignment': True, 'grounding': True, 'modality_coverage': True}
gate_pass= True

```

这个 demo 的重点是：多模态样本必须同时通过媒体质量、跨模态对齐、时序同步、隐私版权、污染隔离和 grounded answer 检查。任何一个环节失败，都可能让模型学到错误的视觉、语音或视频对应关系。

20. 常见面试题

20.1 图文对数据最大的问题是什么？

最大问题是图文不对齐。网页文本、alt text、标题和周围文本不一定描述图片。需要用相似度、OCR、规则和人工抽样评估对齐质量。

20.2 caption 数据怎么评估质量?

看是否准确、具体、不过度臆测，是否描述关键对象、关系、动作、文字和场景；还要检查是否有偏见、幻觉、模板化和时间错位。

20.3 OCR 数据为什么不能只存纯文本?

因为文档理解需要版面结构、坐标、阅读顺序、表格关系和图文位置。只存纯文本会丢失空间信息。

20.4 语音数据清洗重点是什么?

重点是音质、噪声、VAD、语言识别、说话人区分、转录准确率、时间对齐、隐私和授权。

20.5 视频数据为什么比图像更难?

视频有时间维度，同时包含帧、音频、字幕、事件和动作。难点是切片、字幕同步、音画一致和跨时间理解。

20.6 多模态数据有哪些隐私风险?

图片和视频可能包含人脸、车牌、证件、地址、儿童和敏感场景；音频可能包含声纹和背景隐私；OCR 和字幕可能暴露个人信息。

20.7 如何避免多模态模型学到幻觉?

提高跨模态对齐质量，过滤不相关 caption，保留 OCR 和证据结构，训练 grounded QA，评估回答是否基于输入模态，而不是凭语言先验猜测。

21. 常见误区

误区一：多模态数据就是图片加文字。

真正难点是图文、音文、视频字幕、OCR 版面和问答证据的对齐。

误区二：caption 越长越好。

长 caption 如果包含臆测和幻觉，会伤害模型 grounding。

误区三：ASR 数据只看转录文本。

还要看音频质量、时间戳、噪声、口音、说话人和隐私。

误区四：视频可以当很多图片处理。

视频的核心是时间、动作和事件演化，不能只抽帧忽略时序。

误区五：公开图片视频都可以训练。

公开可见不等于授权训练，尤其涉及版权、肖像、隐私和平台条款。

误区六：自动相似度高就一定好数据。

相似度只是信号，复杂图表、新闻、商品和文档场景仍需抽样审计。

22. 本章小结

多模态、语音与视频数据工程的核心是对齐、质量和治理。

本章要记住几句话：

1. 多模态数据不是简单拼接，而是跨模态对齐问题。
2. 图文对要关注 caption、OCR、图文相似度、版权和隐私。
3. 语音数据要关注转录质量、时间对齐、口音、噪声和声纹隐私。
4. 视频数据要关注切片、字幕同步、音画一致、事件对齐和长时序。
5. 多模态数据版权和隐私风险更高，必须做来源、许可、脱敏和删除机制。
6. 成熟系统要把自动评分、人工抽检、下游评估和版本治理结合起来。

如果面试中被问到多模态数据，最好的回答是：先按图文、OCR、语音、视频和指令数据分池治理，再做基础清洗、跨模态对齐、隐私版权处理、样本构造、配比实验和评估闭环。

第十章：Data Attribution 与 Valuation

前面几章一直在回答“数据怎么采、怎么洗、怎么配”。本章讨论一个更难的问题：一条数据、一个数据源、一个数据池到底贡献了多少？

这就是 Data Attribution 与 Data Valuation。

Data Attribution 关注归因：模型某个行为、某个错误、某个能力提升，可能来自哪些训练数据。Data Valuation 关注估值：某条数据、某类数据、某个来源对目标指标有多少价值，值不值得保留、上采样、购买、标注或继续扩充。

在大模型场景下，这两个问题都非常难。因为训练数据巨大、训练成本高、模型非凸、数据之间存在强交互，几乎不可能精确回答“这条数据贡献了多少”。但工程上仍然需要近似方法，否则数据决策只能靠直觉。

本章重点：数据贡献估计、influence functions、数据选择、主动学习、数据价值评估。

合规边界：本章讨论数据价值评估、错误分析、训练数据治理和审计，不提供训练数据反推、隐私抽取或绕过数据保护的方法。

0. 本讲资料边界与第二轮精修口径

按照 WRITING_PLAN.md 的要求，本讲精修前核对了 influence functions、Data Shapley、高效 Shapley 近似、TracIn、Dataset Cartography、DSIR、LESS 和 DataInf 等公开论文资料。

本讲聚焦大模型数据工程中可落地的数据归因和估值：源级 / 簇级 / 样本级近似、目标依赖效用函数、influence 近似、Shapley 近似、小模型 proxy、消融实验、数据选择、主动标注优先级、负价值数据识别和版本化审计。

目标指标 -> 数据单元 -> 弱信号估值 -> 小规模验证 -> 风险成本修正 -> 数据选择 -> 版本审计

本讲不把任何估值方法写成“大模型训练数据价值的精确答案”。大模型数据价值通常只能通过多种弱证据近似：小模型实验、源级 ablation、梯度相似、相似检索、人工审计、下游评测和成本风险分析共同支撑。

1. 先建立直觉：为什么需要数据归因和估值？

假设你训练了一个模型，发现它在数学题上提升了 5 分，在法律问答上下降了 3 分，同时安全拒答变得过度保守。你会问：

1. 哪些数据让数学变好了？
2. 哪些数据导致法律能力下降？
3. 哪些安全数据让模型过度拒答？
4. 某个高价购买的数据源到底值不值？
5. 如果只能再标注 10 万条数据，应该标哪类？
6. 如果要删掉 20% 低价值数据，删哪里损失最小？

没有 attribution 和 valuation，团队只能靠经验拍脑袋。数据工程就会停留在“收更多、洗更干净、试试看”的阶段。

有了归因和估值，团队可以更科学地做数据选择、数据采购、质量修复、配比优化和问题定位。

2. 来龙去脉：从解释单点预测到评估数据资产

数据归因最早不是为大模型设计的。传统机器学习里，人们关心某个训练样本对某个预测有什么影响。Influence functions 是一个代表性方向。Koh 和 Liang 的工作把经典稳健统计中的 influence functions 用于解释黑盒模型预测，尝试追溯某个预测背后最有影响的训练点。

数据估值则和合作博弈里的 Shapley value 有关。Data Shapley 提出用 Shapley 思想衡量每个训练样本对模型性能的边际贡献。Jia 等工作进一步研究了基于 Shapley value 的高效数据估值近似。

这些方法在中小规模监督学习中更容易解释。但到了大模型时代，问题规模变了：

1. 数据从百万级变成万亿 token。
2. 训练一次模型成本极高。
3. 单条样本影响很小，数据源和数据分布影响更大。
4. 训练过程复杂，包含预训练、继续预训练、SFT、偏好训练和安全训练。
5. 模型能力来自数据交互，不是简单线性叠加。

因此，大模型数据估值更常用数据源级、数据簇级、任务级和配比级近似，而不是逐条精确估值。

3. Data Attribution 和 Data Valuation 的区别

Data Attribution 问的是“某个结果来自哪里”。

例如：

1. 模型为什么会输出某个错误事实？
2. 某个 benchmark 提升主要来自哪类训练数据？
3. 某个安全误拒是否来自某批安全数据？
4. 模型记住某段文本，训练集中哪些样本最相关？

Data Valuation 问的是“某个数据有多值钱”。

例如：

1. 这个数据源是否提升目标指标？
2. 这个领域数据是否值得继续采购？
3. 哪些样本最值得人工标注？
4. 哪些数据可以删掉或降权？
5. 哪些低质量数据对模型有负贡献？

简单说：attribution 更偏解释和诊断，valuation 更偏决策和资源分配。

4. 为什么大模型数据价值很难精确计算？

大模型数据价值难在几个方面。

第一，成本太高。最朴素的方法是删掉某个数据源重新训练，再看效果变化。但训练大模型成本巨大，不可能对每个数据源反复重训。

第二，数据有交互。代码数据可能增强结构化推理，数学数据可能增强验证，二者一起效果大于单独相加。单独估值会忽略交互。

第三，指标多目标。某个数据源可能提升代码能力，但降低多语言能力；提升安全性，但增加误拒。它到底“值钱”还是“不值钱”，取决于目标函数。

第四，价值随模型阶段变化。预训练阶段低价值的数据，在 SFT 阶段可能很有价值；小模型有效的数据，大模型未必同样有效。

第五，数据质量和配比耦合。一个数据源本身好，但比例过高会过拟合；一个数据源噪声大，但少量保留可增加多样性。

所以，大模型中的数据价值不是静态属性，而是相对于模型、阶段、目标、配比和评估集定义的。

4.1 关键公式与估值指标

把训练数据写成样本集合：

$$D = \{z_i\}_{i=1}^n$$

其中 z_i 可以是一条预训练文档、一条 SFT 样本、一对 chosen / rejected 偏好样本，也可以是一个数据源 C_k 中的样本。大模型数据工程里更常见的估值单元不是单条样本，而是数据源、数据簇、任务池或数据版本：

$$C_k = \{z_i : c_i = k\}$$

其中 c_i 表示样本所属来源、领域、任务或标注批次。

估值必须先定义目标效用函数。一个常见写法是：

$$U(S) = \sum_{m=1}^M w_m M_m(S) - \lambda_R R(S) - \lambda_C C(S)$$

其中 S 是被选中的数据集合， $M_m(S)$ 是第 m 个目标指标，例如数学、代码、安全、通用能力或人工质量， w_m 是业务权重， $R(S)$ 是隐私、版权、安全和污染风险， $C(S)$ 是采购、清洗、标注、训练和维护成本。

数据源级 ablation 的加入价值可以写成：

$$\Delta_k^{\text{add}} = U(S \cup C_k) - U(S)$$

删除价值可以写成：

$$\Delta_k^{\text{drop}} = U(S) - U(S \setminus C_k)$$

如果 Δ_{add} 为正，说明加入数据源有收益；如果 Δ_{drop} 为正，说明删掉它会损失收益。两者都要结合风险和成本解释。

Influence functions 关心训练点 z_i 对测试点 z^* 的 loss 影响。经典近似形式是：

$$I_{\text{up}}(z_i, z^*) = -\nabla_{\theta} \ell(z^*, \hat{\theta})^{\top} H_{\hat{\theta}}^{-1} \nabla_{\theta} \ell(z_i, \hat{\theta})$$

其中 ℓ 是 loss， $\hat{\theta}$ 是训练后的参数， $H_{\hat{\theta}}$ 是经验风险 Hessian。这个公式表达的是“如果稍微上调训练点 z_i 的权重，测试点 z^* 的 loss 如何变化”。在大模型中直接求 H^{-1} 通常不可行，因此常用梯度相似、TracIn 式训练轨迹或小模型 proxy 做近似。

梯度相似 proxy 可以写成：

$$A_i = \frac{g_i^{\top} g_T}{\|g_i\|_2 \|g_T\|_2}$$

其中 g_i 是训练样本或数据源的梯度特征， g_T 是目标任务、目标验证集或错误样本的梯度特征。 A_i 越高，说明方向越接近，但它仍然不是严格因果证明。

Data Shapley 把每条数据看成参与者，价值为平均边际贡献：

$$\phi_i = \sum_{S \subseteq D \setminus \{z_i\}} \frac{|S|!(n-|S|-1)!}{n!} [U(S \cup \{z_i\}) - U(S)]$$

这个公式理论性质好，但精确计算需要枚举大量子集。大模型里通常只在小数据池上做近似，或者把 Shapley 思想用于源级、簇级和任务级估值。

最终数据选择可以写成一个带约束的预算问题：

$$\begin{aligned} & \max_{s_i \in \{0,1\}} \sum_i s_i v_i \\ & \sum_i s_i b_i \leq B, \quad R(\{z_i : s_i = 1\}) \leq \tau_R \end{aligned}$$

其中 v_i 是估计数据价值, b_i 是 token、标注或训练成本, B 是预算, τ_R 是风险上限。工程上还会加语言、领域、任务、来源和多样性约束。

5. 最朴素的方法：数据消融

数据消融是最直接的数据价值评估方法。

做法是：

1. 设定 baseline 数据配比。
2. 删除或降低某个数据源。
3. 训练模型或小模型。
4. 比较 validation loss、benchmark、安全指标和人工评测。

例如，想知道代码数据是否有价值，可以训练两个小模型：一个包含代码，一个不包含代码。再比较代码 benchmark、通用问答、多语言和安全表现。

优点：

1. 直观。
2. 解释性强。
3. 和最终训练目标一致。

缺点：

1. 成本高。
2. 只能评估少量候选。
3. 小模型结论不一定外推到大模型。
4. 删除一个数据源会改变整体配比，混入其他变量。

数据消融是最可靠但最贵的方法，通常用于关键数据源和最终决策。

6. 数据源级估值

大模型中比逐条样本更常用的是数据源级估值。

数据源可以是：

1. 某个网站集合。
2. 某类书籍。
3. 某个代码数据池。
4. 某个数学题库。
5. 某个多语言语料。
6. 某个合成数据生成版本。
7. 某批偏好标注数据。

评估方法包括：

1. 源级 ablation。
2. 源级上采样/下采样实验。
3. 源级 validation loss。
4. 下游任务关联分析。
5. 质量分与收益的相关性分析。
6. 训练曲线和梯度信号观察。

数据源级估值适合回答：“这个数据池值不值得继续投入？”

7. 样本级估值

样本级估值关注单条数据的价值。它更细，但更难。

可能用途包括：

1. 找出有害样本。
2. 找出高价值标注样本。
3. 找出错误标签。
4. 选择主动学习样本。
5. 清理低质量合成数据。
6. 解释某个模型错误。

在大模型预训练中，逐条样本估值通常不可行，因为样本太多、影响太小、训练不可重复。但在 SFT、偏好训练、安全数据、领域数据等较小数据池中，样本级估值更有现实意义。

例如，DPO 数据里某些 chosen/rejected 标注可能反了；安全数据里某些拒答样本导致误拒；领域 QA 中某些专家答案过期。样本级估值可以帮助定位这些问题。

8. Influence functions：从预测追溯训练点

Influence functions 的直觉是：如果把某个训练样本权重稍微增加或删除，模型在某个测试点上的 loss 会怎样变化？

它可以用于回答：哪些训练样本最影响这个预测？

优点：

1. 可解释性强。
2. 适合错误诊断。
3. 可用于发现数据错误。
4. 不一定需要完整重训每个样本。

局限：

1. 依赖近似。
2. 对深度非凸模型不稳定。
3. 计算 Hessian-vector 相关量成本高。
4. 大模型规模下很难直接使用。
5. 训练过程中的优化路径影响难以完全捕捉。

所以在大型模型里，influence 思想更常被用于小模型、embedding 近邻、数据簇分析、SFT 数据诊断，而不是直接对万亿 token 预训练做精确 influence 计算。

9. Shapley value 和 Data Shapley

Shapley value 来自合作博弈，思想是公平分配多个参与者对整体收益的贡献。用于数据估值时，每条数据被看作参与者，模型性能是整体收益。

Data Shapley 的直觉是：一条数据的价值等于它在不同数据子集里加入后带来的平均边际提升。

优点：

1. 理论性质好。
2. 能考虑数据之间的交互。
3. 可用于发现高价值、低价值和有害数据。
4. 可指导数据采购和数据清洗。

缺点：

1. 精确计算极其昂贵。
2. 需要大量训练或近似训练。
3. 在大模型预训练规模下不可直接使用。
4. 结果依赖目标评估集。
5. 样本价值不是通用常数。

因此，Shapley 更适合作为估值思想和小规模数据池工具，而不是大模型全量 token 估值的直接方案。

10. 近似方法：从精确估值到可用信号

工程上常见近似包括：

1. 小模型 proxy：用小模型或短训练评估数据价值。
2. 数据源 ablation：删除或降权整个数据源。
3. 子集采样：随机采样数据子集训练多个模型。
4. validation loss 分桶：看不同数据源对验证集 loss 的影响。
5. embedding 相似度：找和目标任务最相近的数据。
6. gradient similarity：看训练样本梯度是否与目标任务梯度方向一致。
7. nearest neighbor attribution：用检索找最相似训练样本。
8. LLM judge 质量评分：对 SFT、偏好、合成数据做辅助估值。
9. 人工审计：抽样检查高低分数据。

这些方法没有一个完美。成熟系统通常把多个弱信号组合起来，而不是迷信某一个分数。

11. 数据选择

数据估值最终要服务数据选择。

数据选择的目标包括：

1. 在固定 token budget 下选最有价值数据。
2. 删除低质量或负贡献数据。
3. 上采样目标能力相关数据。
4. 降低重复和污染。
5. 控制安全和隐私风险。
6. 平衡通用能力和专项能力。

常见数据选择策略：

1. 质量分阈值。
2. 来源信誉。
3. 与目标任务 embedding 相似。
4. 多样性采样。
5. 难例优先。
6. 低 loss 或中等 loss 筛选。
7. 人工审核优先级。
8. 小模型收益验证。

注意，数据选择不是只选“最像评测集”的数据。那样容易过拟合 benchmark，损害泛化和多样性。

12. 主动学习和标注优先级

主动学习关注：在标注预算有限时，哪些样本最值得标注？

在大模型数据工程中，主动学习常用于：

1. SFT 数据标注。
2. 偏好数据标注。
3. 安全边界样本标注。
4. 专业领域专家标注。
5. 多模态 caption 或 QA 标注。

常见选择信号包括：

1. 模型不确定性高。
2. 多个模型分歧大。
3. 用户频率高。
4. 风险等级高。
5. 目标能力薄弱。
6. 代表性强。
7. 与已有数据差异大。

主动学习的核心不是找最难样本，而是在价值、覆盖、风险和标注成本之间做权衡。

13. 负价值数据

不是所有数据都有正贡献。有些数据会伤害模型。

负价值数据包括：

1. 错误标注。
2. 低质量合成数据。
3. 重复模板。
4. 过时专业知识。
5. 含隐私或敏感信息的数据。
6. benchmark 泄漏数据。
7. 不安全或不合规回答。
8. 与目标产品风格冲突的数据。
9. 引导模型过度拒答的数据。

数据估值的一个重要用途就是发现负价值数据。删掉坏数据有时比增加好数据更有效。

14. Attribution 在错误分析中的用途

当模型出现错误时，data attribution 可以帮助定位原因。

例如：

1. 模型输出错误医学知识，追溯到过时网页或低质量论坛。
2. 模型在某语言上表现差，发现低资源语言数据被质量过滤误删。
3. 模型安全误拒，发现某类安全数据过度保守。
4. 模型代码生成过时 API，发现训练数据中旧版本文档占比过高。
5. 模型在某 benchmark 异常高，发现训练集中存在题目泄漏。

这里的 attribution 不一定是精确数学归因，更多是证据链：相似训练样本、数据源统计、版本变更、消融实验和人工审计共同支持结论。

15. 数据估值和 Data Mixture 的关系

Data valuation 直接服务 data mixture。

如果某个数据源对目标任务收益高，可以上采样或扩充。如果某个数据源对安全有副作用，可以隔离或降权。如果某个合成数据版本提升 benchmark 但降低人工质量，需要重新生成或调低比例。

但估值不能只看单项指标。

例如：

1. 代码数据提升 HumanEval，但可能影响自然语言风格。
2. 安全数据降低漏拒，但可能增加误拒。
3. 多语言数据提升目标语言，但可能占用英语和代码 token budget。
4. 合成数据提升格式遵循，但可能造成模板化。

所以数据估值要输出多维价值向量，而不是单一分数。

16. 数据估值的工程指标

一个数据源的工程价值可以从多个维度衡量：

1. 目标能力收益。
2. 通用能力副作用。
3. 安全风险。
4. 隐私风险。
5. 版权和授权成本。
6. 清洗成本。
7. 标注成本。
8. 去重后有效 token 数。
9. 多样性贡献。
10. 时效性。
11. 可追溯性。
12. 对产品场景的覆盖度。

真实项目里，数据价值不是学术意义上的准确率提升，而是收益、成本、风险和可治理性的组合。

17. 大模型中的 practical attribution

在大模型里，一个更实用的 attribution pipeline 可能是：

1. 对问题样本做 embedding 检索，找相似训练样本和数据源。
2. 检查数据版本和来源统计。
3. 分析相关数据源在训练前后的配比变化。
4. 做小规模 ablation 或 downweight 实验。
5. 观察目标评估集和人工样例变化。
6. 对可疑数据做人审。
7. 将结论转成清洗规则、配比调整或标注任务。

这比追求单条样本的精确因果归因更现实。

18. 面向专家：数据价值是目标依赖的边际贡献

从专家视角看，数据价值不是数据本身的固定属性，而是目标依赖的边际贡献。

同一条数据在不同情况下价值不同：

1. 在缺少数学数据时，一条高质量数学题很有价值。
2. 在数学数据已经过量时，它的边际价值下降。

3. 对通用助手有价值的数据，对法律模型未必有价值。
4. 对小模型有价值的数据，对大模型可能只是重复。
5. 对训练有价值的数据，对评估可信度可能有污染风险。

因此，讨论数据价值必须同时说明：目标模型、训练阶段、评估指标、已有数据分布和风险约束。

19. 一个可落地的数据归因和估值方案

如果面试官问：“如何评估一个数据源对大模型是否有价值？”可以按下面回答。

第一步，明确目标。是提升代码、数学、多语言、安全、事实性，还是降低幻觉和误拒。

第二步，建立数据元信息。记录来源、语言、领域、质量分、去重簇、license、token 数、时间和版本。

第三步，做离线质量评估。看噪声、重复、污染、隐私、版权和人工抽样质量。

第四步，做小规模训练实验。比较加入、删除、上采样、下采样该数据源后的多维指标变化。

第五步，做目标任务相关性分析。用 embedding、关键词、领域分类、validation loss 和人工样例分析它覆盖哪些能力。

第六步，做副作用评估。看通用能力、安全、误拒、幻觉、多语言和风格是否变差。

第七步，估算成本和风险。包括采购、清洗、标注、授权、隐私和维护成本。

第八步，形成决策。保留、扩充、上采样、降权、隔离、重洗或删除。

第九步，版本化记录。保存实验配置、数据版本、评估结果和决策理由。

19.1 最小可运行数据归因与估值 demo

下面这个 demo 不依赖外部库，也不读写文件。输入是一组 toy 数据源，输出包括源级价值排名、目标任务 attribution proxy、token budget 下的数据选择、小规模 Shapley 估值、负价值 / 阻断数据和污染阻断清单。

它演示的是数据估值工程闭环，不是真实 influence function、生产级 Shapley、完整小模型训练或大规模数据选择系统。真实系统需要接入训练日志、数据版本、评估矩阵、embedding / gradient 特征、消融实验、人工审计和合规风险系统。

```
from itertools import permutations
from math import sqrt
```

```
weights = {"math": 0.35, "code": 0.25, "safety": 0.20, "general": 0.20}
target_grad = [0.70, 0.55, 0.20, 0.10]
```

```
sources = [
    {"id": "math_verified", "tokens": 900, "quality": 0.92, "coverage": 0.78, "risk": 0.03, "cost": 0.22, "license_ok": 1},
    {"id": "code_tests", "tokens": 760, "quality": 0.88, "coverage": 0.71, "risk": 0.04, "cost": 0.18, "license_ok": 1},
    {"id": "safety_boundary", "tokens": 640, "quality": 0.84, "coverage": 0.66, "risk": 0.05, "cost": 0.15, "license_ok": 1},
    {"id": "zh_domain", "tokens": 820, "quality": 0.80, "coverage": 0.73, "risk": 0.06, "cost": 0.12, "license_ok": 1},
    {"id": "synthetic_template", "tokens": 700, "quality": 0.50, "coverage": 0.42, "risk": 0.22, "cost": 0.05, "license_ok": 1},
    {"id": "old_legal_forum", "tokens": 680, "quality": 0.46, "coverage": 0.35, "risk": 0.36, "cost": 0.08, "license_ok": 1},
    {"id": "benchmark_leak", "tokens": 520, "quality": 0.90, "coverage": 0.40, "risk": 0.70, "cost": 0.04, "license_ok": 1}
]
```

```
def dot(a, b):
    return sum(x * y for x, y in zip(a, b))
```

```

def norm(a):
    return sqrt(dot(a, a))

def cosine(a, b):
    return dot(a, b) / (norm(a) * norm(b))

def weighted_delta(src):
    return sum(weights[k] * src["delta"].get(k, 0.0) for k in weights)

def source_value(src):
    if not src["license_ok"] or src["privacy"] or src["contam"]:
        return -1.0
    raw = (
        weighted_delta(src)
        + 0.08 * src["quality"]
        + 0.05 * src["coverage"]
        + 0.05 * cosine(src["grad"], target_grad)
        - 0.12 * src["risk"]
        - 0.04 * src["cost"]
    )
    if src["risk"] > 0.30 or src["quality"] < 0.48:
        return -abs(raw)
    return raw

rows = []
for src in sources:
    rows.append({
        "id": src["id"],
        "weighted_delta": round(weighted_delta(src), 4),
        "grad_sim": round(cosine(src["grad"], target_grad), 3),
        "value": round(source_value(src), 4),
    })

ranked = sorted(rows, key=lambda x: x["value"], reverse=True)
negative = [row["id"] for row in ranked if row["value"] < 0]

budget = 2600
selected, used_tokens = [], 0
for row in sorted(rows, key=lambda r: r["value"] / next(s["tokens"] for s in sources if s["id"] == r["id"]), reverse=True):
    src = next(s for s in sources if s["id"] == row["id"])
    if row["value"] <= 0 or used_tokens + src["tokens"] > budget:
        continue
    selected.append(row["id"])
    used_tokens += src["tokens"]

players = ["math_verified", "code_tests", "synthetic_template"]
base_gain = {"math_verified": 0.30, "code_tests": 0.22, "synthetic_template": -0.06}

def utility(subset):
    subset = set(subset)

```

```

score = sum(base_gain[p] for p in subset)
if {"math_verified", "code_tests"}.issubset(subset):
    score += 0.08
if "synthetic_template" in subset and "math_verified" not in subset:
    score -= 0.05
return score

shapley = {p: 0.0 for p in players}
orders = list(permutations(players))
for order in orders:
    prefix = []
    for p in order:
        shapley[p] += utility(prefix + [p]) - utility(prefix)
        prefix.append(p)
shapley = {k: round(v / len(orders), 4) for k, v in shapley.items()}

report = {
    "ranked_sources": [(row["id"], row["value"]) for row in ranked],
    "top_attribution": [(row["id"], row["grad_sim"]) for row in sorted(rows, key=lambda x: x["grad_sim"], reverse=True)],
    "selected_under_budget": selected,
    "used_tokens": used_tokens,
    "negative_or_blocked": negative,
    "shapley_demo": shapley,
    "total_selected_value": round(sum(row["value"] for row in rows if row["id"] in selected), 4),
    "blocked_contamination": [src["id"] for src in sources if src["contam"]],
}

for key, value in report.items():
    print(f"{key}=", value)

assert report["selected_under_budget"] == ["code_tests", "math_verified", "safety_boundary"]
assert report["used_tokens"] == 2300
assert report["negative_or_blocked"] == ["old_legal_forum", "benchmark_leak"]
assert report["shapley_demo"] == {"math_verified": 0.365, "code_tests": 0.26, "synthetic_template": -0.085}

```

运行后会看到类似输出：

```

ranked_sources= [('math_verified', 0.1808), ('code_tests', 0.1599), ('zh_domain', 0.1288), ('safety_boundary', 0.1202),
0.0184), ('benchmark_leak', -1.0)]
top_attribution= [('math_verified', 0.942), ('benchmark_leak', 0.93), ('synthetic_template', 0.922)]
selected_under_budget= ['code_tests', 'math_verified', 'safety_boundary']
used_tokens= 2300
negative_or_blocked= ['old_legal_forum', 'benchmark_leak']
shapley_demo= {'math_verified': 0.365, 'code_tests': 0.26, 'synthetic_template': -0.085}
total_selected_value= 0.4609
blocked_contamination= ['benchmark_leak']

```

这个 demo 的重点是：高 attribution 相似度不等于可训练价值。benchmark_leak 和目标梯度很相似，但因为评测污染必须被阻断；old_legal_forum 虽然有一点覆盖度，但质量低、风险高且带来负向指标，应进入重洗或降权候选。

20. 常见面试题

20.1 什么是 data attribution?

data attribution 是追溯模型行为、预测、错误或能力变化可能来自哪些训练数据、数据源或数据分布的过程。它主要用于解释、诊断和审计。

20.2 什么是 data valuation?

data valuation 是估计数据对目标模型性能、能力、风险或产品价值的贡献，用于数据选择、采购、标注优先级和配比决策。

20.3 Influence functions 的核心思想是什么?

它估计某个训练样本权重发生微小变化时，对某个测试点预测或 loss 的影响，从而追溯哪些训练点最影响该预测。大模型中直接使用成本高，通常作为思想或局部诊断工具。

20.4 Data Shapley 的核心思想是什么?

把每条数据看作参与者，用它在不同数据子集中的平均边际贡献衡量价值。它理论性质好，但精确计算昂贵，大模型中通常只能做近似或用于小数据池。

20.5 大模型中如何实际评估数据源价值?

更常用源级 ablation、小模型 proxy、上采样/下采样实验、validation loss、目标任务评估、人工审计和成本风险分析，而不是逐条精确估值。

20.6 数据价值是不是一个固定分数?

不是。数据价值依赖目标模型、训练阶段、已有数据分布、评估指标和风险约束。同一数据在不同任务和配比下价值不同。

20.7 如何发现负价值数据?

可以结合低质量分、异常 loss、人工审计、错误归因、数据消融、安全回归和污染检测，找出错误标注、过时知识、低质合成、隐私风险或导致误拒的数据。

21. 常见误区

误区一：数据价值可以精确算出来。

在大模型场景下，价值通常只能近似估计，并且依赖目标和评估集。

误区二：单条数据估值最重要。

预训练中源级、簇级和任务级估值往往更实用；样本级估值更适合 SFT、偏好和安全数据。

误区三：某数据源提升一个 benchmark 就一定有价值。

还要看污染、泛化、副作用、安全、成本和授权。

误区四：低质量分数据一定没用。

某些真实口语、低资源语言或边界样本可能质量分低但对目标能力重要。

误区五：数据归因能证明严格因果。

多数工程归因是近似证据链，不是严格因果证明。

误区六：数据选择只选高价值样本。

还要保留多样性、长尾覆盖和真实分布，否则模型容易过拟合窄目标。

22. 本章小结

Data Attribution 与 Valuation 是大模型数据工程从“经验驱动”走向“证据驱动”的关键。

本章要记住几句话：

1. attribution 关注模型行为来自哪些数据，valuation 关注数据有多少决策价值。
2. influence functions 和 Data Shapley 提供了重要思想，但大模型中通常需要近似。
3. 数据价值不是固定属性，而是目标依赖的边际贡献。
4. 大模型更常做数据源级、数据簇级和任务级估值。
5. 好的数据估值要同时看收益、成本、风险、副作用和可治理性。
6. 最实用的方案是质量评估、小模型实验、消融、相似检索、人工审计和版本记录组成的证据链。

如果面试中被问到数据价值评估，最好的回答不是直接说“用 Shapley 算”，而是说明：先定义目标和评估矩阵，再建立数据元信息，做源级和小规模实验，结合 attribution 诊断、副作用评估、成本风险分析，最后形成可版本化的数据决策。

第十一章：Dataset Versioning 与 Governance

前面章节讲了数据采集、清洗、去重、配比、专项数据、合成数据、偏好安全数据、多模态数据和数据价值评估。本章讨论把这些能力组织成长期可运行系统所必须的底座：dataset versioning 与 governance。

如果没有数据版本和治理，团队可能训练出一个模型，却说不清楚它到底用了哪些数据、哪些规则、哪些过滤器、哪些来源、哪些授权、哪些删除请求是否生效。模型出了问题，也无法追溯到数据版本；想复现实验，也找不到当时的数据快照。

大模型时代，数据治理不再是合规部门的附属工作，而是训练工程的一部分。没有治理的数据系统，规模越大，风险越大。

本章重点：数据版本、元数据、可追溯性、权限、审计、删除请求和数据治理。

合规边界：本章讨论数据治理、权限控制、审计、删除机制和合规文档，不提供绕过访问控制、规避审计、保留或删除数据或滥用个人信息的方法。

0. 本讲资料边界与第二轮精修口径

按照 WRITING_PLAN.md 的要求，本讲精修前核对了 Datasheets for Datasets、Model Cards、W3C PROV、DVC 数据版本控制文档、OpenLineage、Hugging Face Dataset Cards、NIST AI RMF / Generative AI Profile 和 GDPR Article 17 删除权等公开资料。

本讲聚焦训练数据版本和治理的工程闭环：不可变快照、manifest、checksum、lineage、schema 演进、权限分级、删除请求、datasheet、model card 输入、审计指标和训练可复现记录。

数据来源 -> 处理流水线 -> 不可变快照 -> manifest -> lineage -> 权限 / 删除 / 审计 -> 训练日志 -> 发布文档

本讲不提供规避权限、保留或删除数据、绕过审计或滥用个人信息的方法。涉及删除请求和隐私治理时，正文只讨论工程机制和审计口径；具体法律义务应由合规和法务团队结合适用地区判断。

1. 先建立直觉：为什么数据需要版本？

代码工程里，每次改代码都会有 git commit。你可以知道某个 bug 是哪个 commit 引入的，也可以回退到旧版本。数据工程也需要类似能力。

假设你训练了两个模型：Model A 和 Model B。B 在数学上更好，但安全误拒变多。你需要知道：

1. B 是否用了新的数学数据？
2. 安全数据比例是否变化？
3. 清洗规则是否改过？
4. 去重阈值是否不同？
5. 是否新增了某个低质量合成数据源？
6. 某些用户删除请求是否在 B 的训练集中生效？

如果数据没有版本，这些问题都无法回答。

所以 dataset versioning 的核心目标是：让每一次训练都能追溯到确定的数据快照和处理流程。

2. 来龙去脉：从数据集文档到基础模型治理

传统机器学习时代，数据集通常比较小，研究者可以在论文中简单描述数据来源、样本数、划分方式和标注方法。但随着模型进入高风险应用，大家逐渐意识到数据集本身需要标准化文档。

Datasheets for Datasets 提出每个数据集都应配套 datasheet，记录动机、组成、采集过程、推荐用途等信息，提升透明度和责任意识。

Model Cards 则强调模型发布时要说明模型用途、评估结果、适用边界和风险。虽然 model card 面向模型，但它离不开数据治理：模型的行为风险很多来自训练数据。

基础模型进一步放大了问题。Foundation model 训练在宽泛数据上，随后被大量下游应用复用。上游数据缺陷会被下游系统继承和放大。因此，数据版本、来源、权限、风险和文档不再是“最好有”，而是模型可控、可审计、可复现的基础。

3. Dataset versioning 到底版本化什么？

很多人以为版本化就是给数据文件起个版本号。实际远不止如此。

需要版本化的内容包括：

1. 原始数据快照。
2. 数据来源列表。
3. 采集时间和采集方式。
4. 解析器版本。
5. 清洗规则和阈值。
6. 质量评分模型版本。
7. PII 和安全过滤器版本。
8. 去重算法和参数。
9. benchmark contamination 检测版本。
10. 数据 schema。
11. 数据分桶和标签。
12. data mixture 配置。
13. 采样权重。
14. 删除请求和处理状态。
15. license 和授权状态。

一个训练数据版本不是一个文件，而是一组数据、代码、配置、元数据和审计记录的组合。

4. 数据 lineage：数据从哪里来，到哪里去

Lineage 指数据血缘，也就是数据从源头到最终训练样本的完整路径。

例如一条训练样本可能经历：

1. 从某网页采集。
2. HTML 解析。
3. 正文抽取。
4. 语言识别。
5. 质量评分。
6. PII 过滤。
7. 去重聚类。
8. 分配到中文技术文档数据池。
9. 按某个采样权重进入训练 shard。

Lineage 要回答：这条样本来自哪里，经过哪些处理，为什么被保留，最后用于哪个模型训练。

没有 lineage，数据问题无法追责；有 lineage，模型异常可以追溯到具体来源、规则或数据版本。

5. 元数据 schema

大模型数据治理依赖元数据。每条样本至少应包含一些基础字段。

常见字段包括：

1. sample_id。
2. source_id。
3. source_url 或来源描述。
4. collection_time。
5. license。
6. language。
7. domain。
8. modality。
9. token_count。
10. quality_score。
11. safety_label。
12. pii_label。
13. dedup_cluster_id。
14. contamination_flag。
15. processing_version。
16. dataset_version。
17. access_level。
18. deletion_status。

这些字段不是为了好看，而是服务训练、审计、删除、配比和复现。

例如，如果要删除某个来源的数据，需要 source_id；如果要分析低资源语言表现，需要 language；如果要隔离 benchmark 污染，需要 contamination_flag；如果要重现实验，需要 processing_version 和 dataset_version。

5.1 关键公式与治理指标

一个数据版本可以抽象为：

$$V_t = (D_t, M_t, P_t, S_t, A_t)$$

其中 D_t 是数据快照， M_t 是 manifest， P_t 是处理 pipeline 和配置版本， S_t 是 schema 版本， A_t 是审计记录。训练日志必须能引用这个整体，而不是只记录一个数据目录名。

Manifest 可以写成 shard 清单：

$$M_t = \{(h_j, n_j, b_j, c_j)\}_{j=1}^{K_t}$$

其中 h_j 是第 j 个 shard 的内容 hash 或 checksum, n_j 是样本数, b_j 是 token 数或字节数, c_j 是来源、语言、license、质量和风险统计。

样本级 lineage 可以写成有序处理路径:

$$L_i = (s_i, o_i, p_{i1}, p_{i2}, \dots, p_{im}, v_i)$$

其中 s_i 是 source id, o_i 是原始对象 hash, p_{ij} 是第 j 个处理步骤, 例如 parse、quality、PII scan、dedup、contamination scan, v_i 是样本进入的数据版本。lineage 的价值是同时支持“从样本追来源”和“从来源追模型版本”。

Lineage 覆盖率可以写成:

$$C_{\{\mathrm{lineage}\}} = \frac{|\{i: L_i \setminus \mathrm{complete}\}|}{N}$$

License 记录完整率可以写成:

$$C_{\{\mathrm{license}\}} = \frac{|\{i: \ell_i \neq \mathrm{varnothing}\}|}{N}$$

删除请求的执行时延可以写成:

$$T_{\{\mathrm{delete}, r\}} = t_{\{\mathrm{closed}, r\}} - t_{\{\mathrm{received}, r\}}$$

如果请求 r 对应的来源、hash、用户或 license 状态无法映射到版本和 shard, 就不能声称删除机制可靠。

权限违规率可以写成:

$$R_{\{\mathrm{access}\}} = \frac{N_{\{\mathrm{denied} \text{ or } \mathrm{invalid}\}}}{N_{\{\mathrm{access}\}}}$$

这个指标不是越低越好: 如果高风险数据被低权限角色访问成功, 说明权限系统失效。审计时要同时看违规访问尝试是否被阻断。

数据文档完整率可以写成:

$$C_{\{\mathrm{doc}\}} = \frac{N_{\{\mathrm{completed} \text{ sections}\}}}{N_{\{\mathrm{required} \text{ sections}\}}}$$

最终治理门禁可以写成:

$$G_{\{\mathrm{gov}\}} = G_{\{\mathrm{manifest}\}} \land G_{\{\mathrm{lineage}\}} \land G_{\{\mathrm{license}\}} \land G_{\{\mathrm{access}\}} \land \dots$$

其中任何一项失败, 都意味着该数据版本不应直接进入正式训练或对外发布说明。

6. 数据快照和不可变性

训练数据版本最好是不可变快照。也就是说, 一旦某个版本用于训练, 就不应该原地修改。

如果发现错误, 应创建新版本, 而不是偷偷改旧版本。否则后续无法解释为什么同一个版本训练结果不同。

不可变快照可以通过以下方式实现:

1. 内容 hash。
2. 文件 manifest。
3. shard checksum。
4. 对象存储版本。
5. 数据库快照。
6. 配置文件版本。

训练日志应记录使用的数据 manifest、shard 列表、采样配置和过滤器版本。

7. 数据 manifest

Manifest 是数据版本的目录清单。它描述一个数据集版本包含哪些文件、每个文件的 hash、大小、样本数、token 数和来源统计。

一个 manifest 可以包含：

1. dataset_name。
2. dataset_version。
3. created_at。
4. creator。
5. parent_versions。
6. processing_pipeline_version。
7. shard_list。
8. shard_hash。
9. sample_count。
10. token_count。
11. language_distribution。
12. domain_distribution。
13. license_distribution。
14. quality_summary。
15. known_risks。

Manifest 是训练可复现的关键。如果没有 manifest，只知道“用了某个数据目录”，复现几乎不可靠。

8. 权限控制

不是所有数据都应该对所有人开放。

权限控制要区分：

1. 原始数据。
2. 清洗后数据。
3. 脱敏数据。
4. 高风险数据。
5. 用户数据。
6. 企业内部数据。
7. 标注数据。
8. 评测集和 benchmark。

常见策略包括：

1. 最小权限原则。
2. 按数据敏感度分级。
3. 访问日志。
4. 审批流程。
5. 加密存储。
6. 临时访问令牌。
7. 数据导出限制。
8. 定期权限审计。

评测集尤其要保护。训练团队如果随意访问测试集，可能造成污染和评估失真。

9. 删除请求和数据撤回

大模型数据治理必须考虑删除请求。用户、数据提供方、版权方或合规团队可能要求删除某些数据。

删除机制要回答：

1. 如何定位相关数据？
2. 哪些版本包含这些数据？
3. 是否进入过训练？
4. 是否在后续数据版本中删除？
5. 是否需要重新训练、继续训练或模型层面处理？
6. 如何记录删除证明？

删除不是简单从当前目录删文件。因为旧版本、缓存、shard、索引、embedding、训练日志和派生数据都可能包含相关内容。

因此，删除请求需要 lineage、source_id、hash、索引和版本记录支持。

10. 数据审计

数据审计是定期检查数据系统是否符合质量、合规和安全要求。

审计内容包括：

1. 来源是否合法合规。
2. license 是否记录完整。
3. PII 处理是否生效。
4. 删除请求是否执行。
5. 访问权限是否合理。
6. 数据版本是否可复现。
7. 高风险数据是否隔离。
8. benchmark 是否被污染。
9. 标注规范是否执行。
10. 数据文档是否完整。

审计不是训练结束后的形式流程，而应该嵌入数据 pipeline。

11. Datasheet：给数据集写说明书

Datasheets for Datasets 的核心思想是：数据集像硬件组件一样，也需要说明书。

一个 datasheet 应该回答：

1. 为什么创建这个数据集？
2. 数据由什么组成？
3. 如何采集？
4. 如何清洗和标注？
5. 包含哪些人群、语言、领域和模态？
6. 有哪些偏差和风险？
7. 适合什么用途？
8. 不适合什么用途？
9. 是否包含敏感信息？
10. 如何维护、更新和删除？

对大模型训练数据，datasheet 可以是内部文档，也可以在开源或对外合作时提供简化版本。重点是透明和可沟通。

12. 数据治理和 model card 的关系

Model card 说明模型的用途、评估、限制和风险。数据治理为 model card 提供事实基础。

例如 model card 中常见内容：

1. 训练数据概述。
2. 适用语言和领域。
3. 不适用场景。
4. 已知偏差。
5. 安全评估。
6. 隐私风险。
7. 更新和删除策略。

这些都需要数据版本和治理支撑。如果训练数据来源、过滤策略和风险标签都不清楚，model card 就只能写空话。

13. 数据版本和训练可复现

训练可复现需要记录：

1. 模型代码版本。
2. tokenizer 版本。
3. 数据版本。
4. data mixture 配置。
5. 采样随机种子。
6. 训练超参。
7. 过滤规则版本。
8. 评测集版本。
9. checkpoint 版本。
10. 环境和依赖版本。

其中数据版本经常被低估。很多训练无法复现不是因为模型代码变了，而是数据目录被重写、过滤器变了、数据来源更新了、shard 顺序变了。

14. 数据 schema 演进

数据系统会不断新增字段。例如一开始只有 source 和 language，后来增加 license、quality_score、pii_label、safety_label、dedup_cluster_id。

Schema 演进要注意兼容性：

1. 新字段默认值。
2. 旧数据如何回填。
3. 字段含义是否变化。
4. 标签体系是否更新。
5. 训练 sampler 是否依赖该字段。
6. 下游评估是否使用该字段。

字段含义变化必须版本化。比如 quality_score v1 和 v2 使用不同模型，不能混在一起当同一个分数解释。

15. 数据治理中的角色分工

数据治理不是某一个人的事。

常见角色包括：

1. 数据工程：采集、清洗、存储、版本。
2. 训练工程：采样、配比、训练日志。
3. 研究团队：数据实验、估值、评估。
4. 安全团队：风险分类、安全过滤、红队数据。
5. 法务合规：license、隐私、删除请求。
6. 标注团队：标注规范、质量审计。
7. 产品团队：用户反馈、真实场景。
8. 平台团队：权限、审计、基础设施。

成熟组织会把这些责任写进流程，而不是靠口头沟通。

16. 数据治理指标

数据治理也需要指标。

常见指标包括：

1. 数据版本可复现率。
2. 样本 lineage 覆盖率。
3. license 记录完整率。
4. PII 检测覆盖率。
5. 删除请求处理时延。
6. 权限审计通过率。
7. 数据文档完整率。
8. benchmark 污染检测覆盖率。
9. 高风险数据隔离率。
10. 数据质量回归通过率。

这些指标让治理从抽象要求变成可执行工程。

17. 面向专家：治理是模型行为控制面的底座

从专家视角看，dataset governance 是模型行为控制面的底座。

模型行为问题经常被归因于算法，但很多问题来自数据：某批安全数据过度保守、某个代码源污染评测、某个合成数据版本质量下降、某些低资源语言被误删、某个旧法规文本未更新。

如果没有数据版本和 lineage，团队只能在模型层面盲目调参。如果有治理系统，就可以追溯、回滚、重洗、重采样、隔离和审计。

治理的价值不是让流程变慢，而是让大规模模型迭代可控。

18. 一个可落地的数据版本与治理方案

如果面试官问：“如何设计大模型训练数据的 versioning 和 governance？”可以按下面回答。

第一步，建立统一数据 schema。每条样本记录 source、license、language、domain、quality、safety、PII、dedup、version 和 deletion_status。

第二步，建立不可变数据快照。每个训练数据版本有 manifest、hash、shard 列表、统计摘要和父版本。

第三步，记录 pipeline 版本。采集、解析、清洗、过滤、去重、质量评分、污染检测都要有代码和配置版本。

第四步，建立 lineage。支持从训练样本追溯到原始来源，也支持从来源追踪到哪些模型版本用过。

第五步，做权限分级。区分公开数据、授权数据、用户数据、敏感数据、评测集和高风险数据。

第六步，建立删除机制。支持按来源、hash、用户请求、license 状态定位和删除，并记录执行证明。

第七步，配套文档。为关键数据集写 datasheet，为模型发布提供 model card 所需的数据说明。

第八步，审计和监控。定期检查 license、PII、权限、污染、删除请求和数据版本可复现性。

第九步，训练集成。训练日志必须记录数据版本、mixture、sampler、tokenizer 和评测集版本。

18.1 最小可运行数据版本治理 demo

下面这个 demo 不依赖外部库，也不读写文件。输入是一组 toy 样本、访问请求、删除请求和文档完成状态；输出包括 manifest、可训练样本、license 分布、lineage 覆盖率、访问阻断、删除请求命中、datasheet 完成度、model card 输入是否完整和治理门禁。

它演示的是治理机制，不是真实 DVC、对象存储、权限系统、合规系统或生产数据平台。真实系统需要接入对象存储版本、数据目录、权限服务、审计日志、删除工单、dataset card / datasheet 文档和训练日志。

```
import hashlib
from collections import Counter, defaultdict

samples = [
    {"id": "s1", "source": "web_blog", "shard": "shard-a", "tokens": 520, "license": "permissive", "pii": False, "contam": False},
    {"id": "s2", "source": "math_docs", "shard": "shard-a", "tokens": 610, "license": "permissive", "pii": False, "contam": False},
    {"id": "s3", "source": "user_forum", "shard": "shard-b", "tokens": 430, "license": "review", "pii": True, "contam": True},
    {"id": "s4", "source": "benchmark_site", "shard": "shard-b", "tokens": 390, "license": "blocked", "pii": False, "contam": False},
    {"id": "s5", "source": "code_repo", "shard": "shard-c", "tokens": 740, "license": "permissive", "pii": False, "contam": False},
    {"id": "s6", "source": "old_vendor", "shard": "shard-c", "tokens": 300, "license": "expired", "pii": False, "contam": True}
]

required_lineage = {"crawl", "parse", "quality"}
required_docs = {"purpose", "composition", "collection", "processing", "risks", "maintenance"}
datasheet = {"purpose": True, "composition": True, "collection": True, "processing": True, "risks": True, "maintenance": True}
model_card_inputs = {"training_data": True, "eval_data": True, "limitations": True, "risk_summary": True, "deletion_requests": True}

access_requests = [
    {"user": "trainer", "sample": "s1", "allowed_levels": {"public", "internal"}},
    {"user": "contractor", "sample": "s5", "allowed_levels": {"public"}},
    {"user": "auditor", "sample": "s3", "allowed_levels": {"public", "internal", "restricted"}},
    {"user": "researcher", "sample": "s4", "allowed_levels": {"public", "internal"}},
]

deletion_requests = [{"request_id": "del-001", "source": "old_vendor"}, {"request_id": "del-002", "source": "missing"}]

def digest(text):
    return hashlib.sha256(text.encode("utf-8")).hexdigest()[:12]

def sample_hash(item):
    payload = f"{item['id']}|{item['source']}|{item['tokens']}|{item['license']}|{item['access']}"
    return digest(payload)

shards = defaultdict(list)
for item in samples:
    shards[item["shard"]].append(item)
```

```

manifest = []
for shard, rows in sorted(shards.items()):
    joined = ";".join(sample_hash(row) for row in sorted(rows, key=lambda x: x["id"]))
    manifest.append({"shard": shard, "sample_count": len(rows), "tokens": sum(row["tokens"] for row in rows), "checksum": joined})

eligible = [item for item in samples if item["license"] == "permissive" and not item["pii"] and not item["contam"] and not item["deletion"]]
license_counts = Counter(item["license"] for item in samples)
lineage_ok = [item["id"] for item in samples if required_lineage.issubset(set(item["lineage"]))]
lineage_coverage = round(len(lineage_ok) / len(samples), 3)

blocked_access = []
for req in access_requests:
    item = next(row for row in samples if row["id"] == req["sample"])
    if item["access"] not in req["allowed_levels"]:
        blocked_access.append((req["user"], req["sample"], item["access"]))

deletion_hits = {}
for req in deletion_requests:
    deletion_hits[req["request_id"]] = [item["id"] for item in samples if item["source"] == req["source"]]

deletion_compliant = all(
    item["deleted"] for req in deletion_requests for item in samples if item["source"] == req["source"]
)

datasheet_completion = round(sum(datasheet.values()) / len(required_docs), 3)
model_card_ready = all(model_card_inputs.values())

report = {
    "dataset_version": "data-v2026-06-06.1",
    "manifest": manifest,
    "eligible_ids": [item["id"] for item in eligible],
    "license_counts": dict(sorted(license_counts.items())),
    "lineage_coverage": lineage_coverage,
    "lineage_missing": [item["id"] for item in samples if item["id"] not in lineage_ok],
    "blocked_access": blocked_access,
    "deletion_hits": deletion_hits,
    "deletion_compliant": deletion_compliant,
    "datasheet_completion": datasheet_completion,
    "model_card_ready": model_card_ready,
}

gates = {
    "manifest_checksums": all(row["checksum"] for row in manifest),
    "lineage_minimum": report["lineage_coverage"] >= 0.65,
    "blocked_bad_data": set(report["eligible_ids"]) == {"s1", "s2", "s5"},
    "access_controls": len(blocked_access) == 2,
    "deletion_traceable": deletion_hits["del-001"] == ["s6"],
    "deletion_compliant": deletion_compliant,
    "datasheet_ready": datasheet_completion >= 0.80,
    "model_card_ready": model_card_ready,
}
report["gates"] = gates
report["governance_ready"] = all(gates.values())

for key, value in report.items():

```

```

print(f"{key}=", value)

assert report["eligible_ids"] == ["s1", "s2", "s5"]
assert report["lineage_coverage"] == 0.667
assert report["lineage_missing"] == ["s5", "s6"]
assert report["blocked_access"] == [("contractor", "s5", "internal"), ("researcher", "s4", "quarantine")]
assert report["deletion_hits"] == {"del-001": ["s6"], "del-002": []}
assert report["datasheet_completion"] == 0.833
assert report["governance_ready"] is False

```

运行后会看到类似输出：

```

dataset_version= data-v2026-06-06.1
manifest= [{ 'shard': 'shard-a', 'sample_count': 2, 'tokens': 1130, 'checksum': '8ee90eff217c' }, { 'shard': 'shard-
b', 'sample_count': 2, 'tokens': 820, 'checksum': '83cc9bdbbafef' }, { 'shard': 'shard-c', 'sample_count': 2, 'tokens': 1040
eligible_ids= ['s1', 's2', 's5']
license_counts= { 'blocked': 1, 'expired': 1, 'permissive': 3, 'review': 1 }
lineage_coverage= 0.667
lineage_missing= ['s5', 's6']
blocked_access= [( 'contractor', 's5', 'internal' ), ( 'researcher', 's4', 'quarantine' )]
deletion_hits= { 'del-001': ['s6'], 'del-002': [] }
deletion_compliant= True
datasheet_completion= 0.833
model_card_ready= False
governance_ready= False

```

这个 demo 的重点是：eligible_ids 只能说明样本通过了训练准入门禁，不代表整个数据版本已经治理完备。这里 model_card_ready=False，说明发布文档所需的删除策略事实基础仍缺失，因此 governance_ready=False 是正确结果。

19. 常见面试题

19.1 为什么大模型数据需要 versioning?

因为训练结果依赖具体数据快照、清洗规则、采样权重和过滤版本。没有 versioning，模型不可复现，问题不可追溯，删除和审计也无法可靠执行。

19.2 什么是 data lineage?

data lineage 是数据从原始来源到最终训练样本的血缘记录，包括采集、解析、清洗、过滤、去重、标签、版本和使用去向。

19.3 manifest 有什么作用?

manifest 是数据版本清单，记录 shard、hash、样本数、token 数、统计分布、pipeline 版本和风险摘要，是训练可复现和审计的基础。

19.4 删除请求为什么难?

因为数据可能存在于原始数据、清洗数据、shard、缓存、索引、embedding、旧版本和已训练模型中。需要 lineage、hash、版本和处理记录支持。

19.5 Datasheet for Datasets 解决什么问题?

它为数据集提供标准化说明，记录动机、组成、采集、清洗、用途、限制和风险，提升透明度和责任意识。

19.6 数据治理和 model card 有什么关系？

model card 中关于训练数据、适用范围、风险、偏差和评估的信息，都需要数据治理提供事实基础。

19.7 如何设计数据权限？

按数据敏感度和用途分级，遵循最小权限原则，记录访问日志，审批高风险数据访问，定期审计权限，并保护评测集避免污染。

20. 常见误区

误区一：数据版本就是文件名加日期。

真正的数据版本包括数据快照、manifest、pipeline、配置、schema、过滤器、采样权重和审计记录。

误区二：训练完再补文档也可以。

很多信息训练后无法补齐，文档和 lineage 应嵌入 pipeline。

误区三：公开数据不需要治理。

公开可见不等于授权使用，也可能包含隐私、版权、偏见和删除请求风险。

误区四：删除就是删当前目录。

还要处理旧版本、缓存、索引、派生数据和训练记录。

误区五：治理会拖慢研发。

缺少治理才会在模型出问题拖慢研发。好的治理让迭代可追溯、可回滚、可审计。

误区六：model card 可以脱离数据系统编写。

没有数据版本和统计，model card 很容易变成泛泛而谈。

21. 本章小结

Dataset Versioning 与 Governance 是大模型数据工程从“能训练”走向“能复现、能审计、能治理”的关键。

本章要记住几句话：

1. 大模型训练必须绑定不可变数据快照和 manifest。
2. Lineage 让样本来源、处理过程和模型使用去向可追溯。
3. 元数据 schema 是清洗、配比、权限、删除和审计的基础。
4. Datasheet 和 model card 是透明治理的重要文档形式。
5. 删除请求、权限控制、license 和隐私处理必须进入工程流程。
6. 治理不是研发阻力，而是大规模模型迭代的控制面。

如果面试中被问到数据治理，最好的回答是：不仅讲存储和版本号，还要讲 schema、manifest、lineage、pipeline 版本、权限、删除请求、审计、datasheet、model card 和训练可复现闭环。

第十二章：数据面试题

本章是第九册《大模型数据工程与数据智能》的收束章。前面 11 章已经系统讲过 web-scale 采集、清洗过滤、去重污染、data mixture、代码数学领域数据、合成与蒸馏、偏好与安全、多模态数据、data attribution、dataset governance。本章把这些内容组织成面试题框架。

大模型数据面试和普通数据工程面试不一样。普通数据工程更关注 ETL、存储、调度、数据仓库和实时链路；大模型数据工程还要关注训练效果、模型行为、安全边界、评估污染、数据配比、合规治理和训练可复现。

面试官真正想听的不是“我会用某个工具清洗数据”，而是你是否理解：数据如何塑造模型能力，数据问题如何变成模型问题，数据工程如何和训练、评估、安全、产品形成闭环。

本章高频问题：如何构建预训练数据？如何做去重？如何检测数据污染？如何设计 reasoning 数据？

合规边界：本章只讨论合法合规的数据工程、风险治理和面试表达，不提供绕过访问控制、抓取未授权数据、复原训练数据、规避安全策略或生成有害内容的方法。

0. 本讲资料边界与第二轮精修口径

按照 WRITING_PLAN.md 的要求，本讲精修前核对了 C4 / T5、The Pile、RefinedWeb、FineWeb、Dolma、DCLM 等公开数据集和数据配方工作，也复核了 Datasheets for Datasets、Model Cards、W3C PROV、OpenLineage、Hugging Face Dataset Cards、NIST AI RMF / Generative AI Profile 和 OWASP LLM Top 10 中与数据污染、数据治理、文档透明度相关的公开资料。

本讲是第九册收束章，不再展开每个算法细节，而是把前 11 章压缩成面试可迁移框架：如何回答、如何量化、如何证明、如何说清边界，以及如何复盘自己的回答。

面试题 -> 目标澄清 -> 数据来源 -> 处理门禁 -> 配比实验 -> 评估证据 -> 治理边界 -> 复盘改进

本讲不把任何公开数据集的具体比例写成唯一标准。真实训练数据配方会随模型目标、语言覆盖、授权边界、数据质量、评估集和训练阶段变化；面试中更重要的是讲清可审计的数据工程方法，而不是背某个固定比例。

1. 面试总框架：大模型数据题怎么答？

遇到大模型数据题，建议用一个固定框架：目标、来源、处理、配比、评估、治理。

1. 目标：模型要解决什么问题？通用 base model、代码模型、数学模型、行业模型、多模态模型还是安全对齐模型？
2. 来源：数据从哪里来？网页、书籍、代码、数学、领域文档、合成数据、偏好数据、多模态数据。
3. 处理：如何清洗、过滤、去重、脱敏、对齐、质量评分和污染检测？
4. 配比：不同数据源如何混合？哪些上采样，哪些下采样？不同训练阶段如何变化？
5. 评估：如何判断数据策略有效？小模型 ablation、validation loss、benchmark、人工评测、安全评估、线上反馈。
6. 治理：数据版本、lineage、license、权限、删除请求、datasheet、model card。

这个框架适用于绝大多数数据面试题。

1.1 关键公式与指标速查

数据面试不要求你现场推复杂公式，但优秀回答应该能把“我会清洗、去重、配比、治理”落到可量化指标。下面这些公式覆盖本章大多数追问。

训练候选集可以抽象为：

$$D = \bigcup_{m=1}^M D_m, \quad x_i = (u_i, b_i, q_i, r_i, a_i, v_i)$$

其中 D_m 是第 m 个数据池， u_i 是来源或样本 id， b_i 是 token 数， q_i 是质量分， r_i 是风险分或风险标记， a_i 是授权和访问状态， v_i 是数据版本。

Token 保留率：

$$R_{\text{keep}} = \frac{\sum_i b_i g_i}{\sum_i b_i}$$

其中 $g_i=1$ 表示样本通过准入门禁， $g_i=0$ 表示被过滤或隔离。回答清洗题时，只说删掉多少不够，还要解释保留下来的 token 是否覆盖目标语言、领域和能力。

某类风险命中率：

$$R_{\{\mathrm{risk}\}}(k)=\frac{\sum_i b_i h_{\{ik\}}}{\sum_i b_i}$$

其中 $h_{ik}=1$ 表示样本命中第 k 类风险，例如 PII、secret、license、contamination 或 safety risk。

Near dedup 常用 Jaccard 直觉：

$$J(A,B)=\frac{|A\cap B|}{|A\cup B|}$$

其中 A 和 B 是两段文本的 shingle 集合。MinHash / LSH 的面试重点是用签名近似 Jaccard，并减少全量两两比较。

Benchmark contamination 命中率：

$$R_{\{\mathrm{contam}\}}=\frac{N_{\{\mathrm{train}\ \mathrm{eval}\ \mathrm{overlap}\}}}{N_{\{\mathrm{query}\}}}$$

这里的 query 不只包括题目原文，还应包括答案、选项、解析、代码签名、测试样例和近重复变体。

Data mixture 对应的训练目标可以写成：

$$L_{\{\mathrm{mix}\}}(\theta)=\sum_{m=1}^M\alpha_m\mathbb{E}_{\{x\sim D_m\}}[L_{\{\theta\}}(x)],\quad\sum_{m=1}^M\alpha_m=1$$

其中 α_m 是第 m 个数据池的采样权重。这个公式帮助你解释：data mixture 不是拼文件，而是在改变模型实际优化的训练分布。

一种可解释的采样权重草案：

$$p_m=\frac{b_m^{\tau}q_m(1-r_m)}{\sum_j b_j^{\tau}q_j(1-r_j)}$$

其中 b_m 是清洗后 token 数， q_m 是池级质量分， r_m 是风险惩罚， τ 控制大池和小池之间的平滑。真实系统还要叠加 license、语言覆盖、目标能力和人工审计结果。

Effective epoch：

$$e_m=\frac{T}{p_m}\{b_m\}$$

其中 T 是计划训练 token 总数。如果小数据池 e_m 很高，要警惕记忆、过拟合和污染放大。

偏好数据标注一致率和长度偏置：

$$A_{\{\mathrm{pair}\}}=\frac{N_{\{\mathrm{agree}\}}}{N_{\{\mathrm{pair}\}}},\quad B_{\{\mathrm{len}\}}=\frac{1}{N}\sum_i(\ell_{i+}-\ell_{i-})$$

其中 ℓ_{i+} 和 ℓ_{i-} 分别表示 chosen / rejected 的长度。长度偏置不能单独证明数据坏，但它是偏好数据审计必须检查的信号。

安全数据的漏拒率和误拒率：

$$R_{\{\mathrm{leak}\}}=\frac{N_{\{\mathrm{unsafe}\ \mathrm{accepted}\}}}{N_{\{\mathrm{high}\ \mathrm{risk}\}}},\quad R_{\{\mathrm{over}\}}=\frac{N_{\{\mathrm{safe}\ \mathrm{refused}\}}}{N_{\{\mathrm{safe}\ \mathrm{allowed}\}}}$$

高质量安全模型不是拒答率最高，而是在漏拒和误拒之间保持稳定边界。

多模态对齐分的基本形式：

$$s_{\{\mathrm{align}\}}=\frac{u_{\{\mathrm{image}\}}\cdot u_{\{\mathrm{text}\}}}{\lVert u_{\{\mathrm{image}\}}\rVert_2\lVert u_{\{\mathrm{text}\}}\rVert_2}$$

这个分数只能辅助判断图文相关性，不能替代 OCR、时序、grounded QA、隐私和版权审计。

数据源价值的面试级表达：

$$V_s=\Delta U_s-\lambda_r R_s-\lambda_c C_s$$

其中 ΔU_s 是目标效用提升， R_s 是风险， C_s 是清洗、标注、存储和治理成本。数据价值不是绝对属性，而是相对于目标、阶段和约束的边际贡献。

治理门禁：

$$G_{\{\mathrm{data}\}}=G_{\{\mathrm{license}\}}\land G_{\{\mathrm{pii}\}}\land G_{\{\mathrm{contam}\}}\land G_{\{\mathrm{lineage}\}}\land G_{\{\mathrm{other}\}}$$

如果没有 license、PII、污染、lineage、删除请求和文档门禁，就不能声称这个数据版本已经可用于正式训练或发布说明。

最后，面试回答本身也可以量化复盘：

$S_{\{\mathrm{ans}\}} = w_c C + w_e E + w_m M + w_g G + w_b B - P$

其中 C 是框架覆盖，E 是工程证据，M 是指标和公式，G 是治理边界，B 是 trade-off 深度，P 是错误表述和超时惩罚。准备面试时，不要只问“我记住了吗”，要问“我的回答有没有目标、证据、指标、边界和取舍”。

2. 高频题一：如何构建大模型预训练数据集？

标准回答框架

我会把预训练数据建设看成一个端到端数据系统，而不是简单拼接文本。

第一步，明确模型目标和覆盖范围。比如是通用中文/英文 base model，还是偏代码、数学、多语言或行业模型。目标不同，数据来源和配比不同。

第二步，设计数据源。通用文本包括网页、书籍、百科、新闻、论坛、问答和技术文档；专项数据包括代码、数学、论文、领域文档；后续还要考虑合成数据、偏好数据和安全数据，但预训练阶段要控制比例。

第三步，建立采集合规流程。记录来源、license、时间、robots/ToS、授权状态和风险等级。不采集未授权、访问受限或高隐私风险数据。

第四步，做解析和清洗。HTML、PDF、代码、文档、多模态数据分别使用不同 parser；去乱码、模板、广告、低质量文本、PII 和高风险内容。

第五步，做语言识别、领域分类和质量评分。不要对所有数据用同一阈值，代码、数学、多语言和专业领域要单独规则。

第六步，做去重和污染检测。包括 exact dedup、near dedup、段落级重复、代码 fork、benchmark contamination。

第七步，设计 data mixture。按目标能力设置采样权重，控制通用文本、代码、数学、多语言、领域和合成数据比例。

第八步，做评估闭环。用小模型训练、数据 ablation、validation loss、下游 benchmark、安全评估和人工样例验证数据策略。

第九步，做版本治理。保存 manifest、lineage、schema、过滤规则、数据版本、删除请求和审计记录。

面试加分点

不要只说“爬网页 + 清洗 + 去重”。要强调数据目标、合规、配比、污染检测和版本可复现。

3. 高频题二：如何做 web-scale 数据采集？

标准回答框架

Web-scale 采集要先确定合法来源和目标语料范围。采集不是越多越好，而是要保证来源合规、可追溯、可清洗和可评估。

我会先做 source planning，把来源分为通用网页、百科、新闻、技术文档、论坛、问答、论文、代码和多语言来源，并记录来源元数据。

然后建立采集 pipeline：source registry、采集任务、原始数据存储、解析、正文抽取、语言识别、质量过滤、去重、版本化。

合规上，要记录来源 URL、采集时间、license、使用限制和删除机制；对于用户数据、登录后内容、访问受限内容和隐私风险高的数据，不能随意进入训练集。

最后用统计和抽样审计监控来源质量，包括有效文本比例、语言分布、重复率、PII 命中率、低质量比例和下游训练收益。

常见追问

面试官可能问：“如果某个来源规模很大但质量一般，怎么办？”

回答：先单独建池，不直接混入主训练集；做质量评分、去重、抽样审计和小模型 ablation；如果有价值则低权重采样或只保留高质量子集。

4. 高频题三：如何做数据清洗和质量评分？

标准回答框架

清洗过滤分层做。

第一层是基础有效性过滤：空文本、乱码、异常字符、极端长度、重复行、编码错误。

第二层是正文抽取和模板去除：去导航栏、页脚、广告、cookie 提示、推荐链接和 boilerplate。

第三层是语言和领域识别：按语言、领域、来源类型分桶，不同桶使用不同规则。

第四层是规则质量过滤：长度、符号比例、链接比例、重复 n-gram、低信息密度等。

第五层是模型质量评分：训练质量分类器或用 LLM judge 辅助，对可读性、信息密度、结构、来源可信度和安全风险打分。

第六层是安全和隐私过滤：PII、毒性、高风险内容、版权和敏感数据处理。

最后要做人工抽样审计和小模型训练验证，评估误删、漏删和下游收益。

面试加分点

强调清洗不是越狠越好。过度过滤会损失口语、多语言、低资源语言、专业符号、代码和数学数据。

5. 高频题四：如何做去重？

标准回答框架

去重要按粒度分层。

第一，文档级 exact dedup。对规范化文本做 hash，相同 hash 只保留一份。

第二，段落级重复统计。识别网页模板、免责声明、页脚、广告段落和高频重复片段。

第三，near dedup。用 shingling、MinHash、SimHash、LSH 或 embedding 检索识别转载、改标题、加页脚、轻微改写的重复。

第四，代码专项去重。处理 fork、vendor、自动生成文件、函数级重复、题库答案和 benchmark 泄漏。

第五，去重后重新统计 data mixture，因为去重会改变来源、语言和领域分布。

常见追问

面试官问：“去重是不是越彻底越好？”

回答：不是。目标是控制不合理重复，而不是删除所有相似内容。常见概念、API 用法、数学标准题型和用户意图多样表达都需要一定 exposure。过度去重会损害泛化和常见模式学习。

6. 高频题五：如何检测 benchmark contamination?

标准回答框架

污染检测要区分训练集内部重复和 train-eval overlap。benchmark contamination 重点是训练集与评测集重叠。

我会把评测集题目、答案、选项、解析、代码签名、测试样例都作为 query，而不只查题目原文。

检测方法包括 exact match、n-gram overlap、MinHash/LSH、embedding 检索、代码函数签名匹配、答案泄漏检测和人工审计。

对于高相似候选，要判断污染程度：只有题目、题目加答案、完整解析、改写版本、同源题库变体，风险等级不同。

处理方式包括从训练集中移除、隔离、降低权重、重新划分评测集或在评测报告中标注风险。

面试加分点

强调污染检测是评估可信度问题，不只是数据清洗问题。公开 benchmark 越流行，污染风险越高。

7. 高频题六：如何设计 data mixture?

标准回答框架

Data mixture 是训练目标的一部分。模型最终优化的是 mixture 分布上的 loss，所以配比会影响能力、风格和安全表现。

我会先定义模型目标，然后把数据按来源、语言、领域、能力、质量和风险打标签。通用文本提供世界知识和语言能力；代码增强结构化和程序能力；数学增强推理；多语言增强语言覆盖；领域数据增强行业能力；合成数据补长尾和格式。

初始配比可以参考经验，但必须通过小规模 ablation 验证。实验时要看多维评估：通用能力、代码、数学、多语言、安全、事实性、风格和人工样例。

配比不是固定不变的。预训练、继续预训练、SFT、DPO、安全训练的 mixture 应该不同。

常见追问

面试官问：“代码数据比例越高越好吗？”

回答：不一定。代码比例高通常提升代码和结构化能力，但会占用 token budget，可能影响自然语言、多语言和对话风格，还会增加 license、secrets、重复和题库污染风险。

8. 高频题七：如何建设代码数据?

标准回答框架

代码数据不能只收 .py、.java 源文件，还应包括 README、文档、issue、PR、commit message、测试、API reference 和编程问答。

清洗上要做语言识别、语法解析、license 检查、secrets 过滤、fork 去重、vendor 处理、自动生成文件过滤、benchmark contamination 检测。

结构化上可以按仓库、文件、函数、类、AST、测试粒度组织样本。训练样本可以构造 docstring-code、test-code、bug-fix、code-explanation、API usage。

评估上不能只看文本相似度，要看功能正确性、编译运行、单测通过率、安全和依赖版本。

面试加分点

强调单测价值：测试提供可执行约束，能训练和评估模型是否真正写出能运行的代码。

9. 高频题八：如何设计数学和 reasoning 数据？

标准回答框架

数学数据要按题目、答案、步骤、难度、领域和验证方式组织。来源包括教材、题库、竞赛题、课程、论坛、合成题、人工 step-by-step 解答和 verifier 数据。

清洗重点是题解对齐、公式解析、答案校验、步骤质量、重复题检测和 benchmark 污染。

Reasoning 数据不能只要最终答案。高质量数据应包含过程、约束、检查和错误候选。对于代码和数学，可以用测试、计算器、CAS 或 verifier 辅助验证。

配比上要谨慎，过多低质量 reasoning 数据会让模型学会伪推理和模板化解释。

常见追问

面试官问：“CoT 数据越多越好吗？”

回答：不是。CoT 质量比数量重要。错误过程、跳步过程和模板化过程会伤害模型。要用答案校验、过程审计、难度分桶和下游评估控制质量。

10. 高频题九：如何建设 synthetic data？

标准回答框架

Synthetic data 要先定义目标能力，而不是为了生成而生成。常见目标包括指令跟随、数学、代码、工具调用、多语言、安全和领域问答。

Pipeline 包括：种子设计、生成策略、teacher 选择、多样性控制、格式检查、去重、质量评分、正确性验证、安全过滤、配比实验和版本记录。

不同任务验证方式不同：数学可校验答案，代码可跑测试，事实问答要检索证据，领域问答要专家审计，安全数据要检查边界。

合成数据风险包括 teacher 偏差、错误放大、同质化、数据退化、污染和过拟合。不能无追踪地大规模混入。

面试加分点

说清 synthetic data 和 distillation data 区别：synthetic 强调生成来源，distillation 强调 teacher-student 能力迁移。

11. 高频题十：如何建设 preference data?

标准回答框架

Preference data 常见形式是 prompt、chosen、rejected，也可以是多个回答排序和标注理由。

构造方式包括同模型多采样、不同模型对比、人工写 chosen、模型生成 rejected、LLM judge 辅助和专家标注。

标注规范要明确 helpful、honest、harmless 的优先级，以及事实性、完整性、简洁性、语气、专业边界和安全性。

数据质量要检查标注一致性、长度偏置、拒答偏置、迎合偏置、rejected 是否太弱、多语言一致性和专业标注质量。

Preference data 可用于 reward model、RLHF、DPO 等方法。算法不同，但都依赖偏好数据质量。

常见追问

面试官问：“DPO 数据随便构造 chosen/rejected 可以吗？”

回答：不可以。噪声大、rejected 太弱或标注标准不一致，会让模型学到错误偏好。DPO 简化优化，不降低对数据质量的要求。

12. 高频题十一：如何建设安全数据?

标准回答框架

安全数据不只是拒答数据。它包括风险分类、拒答、边界允许、误拒修复、漏拒修复、安全替代建议、红队回归、多语言安全和专业高风险场景。

我会按风险分层：明确安全、敏感但可答、高风险、边界模糊、紧急风险、专业高风险。

拒答数据要训练模型不提供危险细节，同时提供安全替代方向。边界允许数据要训练模型在教育、防御、科普、治理场景中安全帮助，避免看到关键词就拒绝。

评估上同时看误拒和漏拒。好的安全模型不是拒答最多，而是边界最稳定。

合规提醒

回答红队数据时，强调用于发现失败模式、防御训练和回归测试，不传播可操作风险细节。

13. 高频题十二：如何建设多模态数据?

标准回答框架

多模态数据工程的核心是对齐，而不是拼接。

图文数据要处理 image-caption、alt text、周围文本、OCR、图文相似度、图片质量、版权和隐私。

语音数据要处理音频质量、VAD、speaker diarization、ASR 转录、forced alignment、采样率、口音、噪声和声纹隐私。

视频数据要处理切片、关键帧、字幕、音频、OCR、镜头变化、事件对齐和时间戳。

多模态指令数据要保证回答 grounded in modality，不能只凭语言先验猜测。

常见追问

面试官问：“图文相似度高就说明数据好吗？”

回答：不一定。相似度只是信号。新闻图、商品图、图表、文档截图和 OCR 场景都需要结合结构和人工审计。

14. 高频题十三：如何评估数据价值？

标准回答框架

数据价值不是固定属性，而是相对于目标模型、训练阶段、已有数据分布和评估指标的边际贡献。

我会从源级、簇级和样本级做估值。预训练更适合源级和簇级；SFT、偏好、安全数据可以做样本级分析。

方法包括数据消融、上采样/下采样实验、小模型 proxy、validation loss、embedding 相似度、gradient similarity、influence 思路、Data Shapley 近似和人工审计。

最终要输出多维价值：目标能力收益、通用能力副作用、安全风险、隐私风险、版权成本、清洗成本和治理成本。

面试加分点

不要说“直接用 Shapley 算”。大模型全量 Shapley 不现实，应该讲近似和工程证据链。

15. 高频题十四：如何做 dataset versioning 和 governance？

标准回答框架

大模型训练数据必须有不可变快照、manifest、lineage、schema 和 pipeline version。

每条样本应记录 source、license、language、domain、quality_score、safety_label、pii_label、dedup_cluster、contamination_flag、dataset_version、deletion_status。

Manifest 记录 shard、hash、token 数、样本数、来源分布、语言分布、质量统计和风险摘要。

Governance 包括权限控制、访问日志、删除请求、license 审计、PII 处理、benchmark 保护、datasheet 和 model card 支撑。

训练日志要记录数据版本、mixture、sampler、tokenizer、评测集版本和 pipeline 版本，否则模型不可复现。

常见追问

面试官问：“删除请求为什么难？”

回答：因为数据可能存在于原始数据、清洗数据、shard、缓存、索引、embedding、旧版本和已训练模型中，需要 lineage、hash、source_id 和版本记录支持。

16. 系统设计题：给你 100TB 原始 web 数据，如何做成训练集？

推荐回答

我会按“数据湖 + 处理 pipeline + 质量评估 + 版本治理”的方式设计。

第一层是 raw data lake，保存原始数据和元信息，记录来源、时间、授权和 hash。

第二层是解析层，对 HTML、PDF、代码、多模态分别解析，抽取正文、结构、OCR、语言和元数据。

第三层是清洗过滤层，做基础有效性过滤、模板去除、质量评分、PII、安全过滤和领域分类。

第四层是去重污染层，做 exact dedup、near dedup、段落重复、代码 fork、benchmark contamination。
第五层是数据分桶层，按语言、领域、来源、质量、安全和模态分桶，形成可采样数据池。
第六层是 mixture sampler，根据训练目标设置采样权重。
第七层是评估闭环，小模型 ablation、validation loss、benchmark、安全评估和人工审计。
第八层是治理，manifest、lineage、schema、license、权限、删除请求和 datasheet。

深挖回答

如果训练结果异常，我会先看数据版本和 mixture 是否变化，再看对应评估集相关数据源、污染检测、质量分布和小规模 ablation，而不是直接改模型。

17. 系统设计题：如何构建一个代码大模型的数据体系？

推荐回答

代码模型数据要分成源码、文档、测试、issue/PR、问答、教程和合成任务几个池。
采集时记录 license、仓库元数据、语言、star、更新时间、依赖和路径结构。
清洗时过滤二进制、自动生成文件、vendor、fork、secrets、过时或低质量项目。
结构化时按 repo-file-function-class-test 组织，构造 docstring-code、test-code、bug-fix、explain-code、API usage 样本。
污染检测要重点查 HumanEval、MBPP、竞赛题、题解、测试样例和参考答案。
评估要看功能正确性、单测通过率、代码安全、API 时效、多语言编程语言覆盖和自然语言能力副作用。

18. 系统设计题：如何构建一个医疗领域模型的数据体系？

推荐回答

医疗领域数据要优先考虑安全、权威性、时效性和隐私。
数据来源包括医学教材、指南、药品说明、综述、公开医学知识库、经授权的专业问答和专家标注数据。真实病历必须严格授权、脱敏和审计。
数据要记录来源、发布日期、适用地区、证据等级、版本和过期时间。清洗时处理 PII、错误 OCR、过期指南、低质量论坛和不可靠建议。
训练策略上，继续预训练用于学习医学术语和表达；SFT 用于学习问答格式和安全边界；RAG 用于最新指南和可引用知识；安全数据用于急症提示、免责声明和避免越权诊断。
评估上要有专家审计、事实性、引用正确性、拒答边界、风险提示和多语言能力。

19. 行为面试题：数据策略和模型效果冲突怎么办？

推荐回答

我会先明确冲突指标。比如某个数据策略提升数学 benchmark，但降低通用问答或增加安全误拒。
然后检查是否存在污染、过拟合或评估偏差。接着做分桶分析，看影响来自哪个来源、语言、领域或质量区间。
再通过小模型 ablation、采样权重调整和人工样例评估确认因果方向。

最后给出取舍方案：调整 mixture、隔离数据池、提高质量阈值、补边界数据、改变训练阶段或把部分能力交给 RAG/工具处理。

核心是不要只追单一 benchmark，而要看多目标 trade-off。

20. 专家追问：为什么数据质量比数据规模更重要？

严格说不是“质量一定比规模重要”，而是二者共同决定有效训练信号。

规模提供覆盖和多样性，质量提供有效信息密度和可靠监督。低质量大规模数据会浪费 token budget、引入噪声、增加记忆和污染风险；高质量但过窄的数据会导致覆盖不足和风格单一。

所以更准确的回答是：在固定 compute budget 下，我们关心的是有效 token，而不是原始 token。有效 token 由质量、去重、多样性、配比和目标相关性共同决定。

21. 专家追问：为什么数据问题会表现成模型问题？

因为模型是在训练分布上学习统计规律。数据中的重复会表现为记忆，低质量网页会表现为幻觉和差风格，污染会表现为 benchmark 虚高，偏好数据噪声会表现为对齐不稳定，安全数据过度保守会表现为误拒，多语言过滤误伤会表现为语言能力差。

因此，排查模型问题时不能只看架构、优化器和 prompt，也要看数据来源、版本、清洗、配比和后训练数据。

22. 专家追问：小模型数据实验能外推到大模型吗？

不能保证，但有参考价值。

小模型实验适合快速筛选明显好坏的数据策略，比如去重、清洗阈值、数据源质量和合成数据版本。但大模型可能出现不同 scaling behavior，某些数据在小模型上无效，在大模型上有效，反之亦然。

所以成熟做法是多尺度验证：小模型快速探索，中模型确认趋势，大模型谨慎放大，并持续监控训练曲线和下游评估。

23. 专家追问：如何避免数据工程过度优化 benchmark？

第一，做污染检测，确保 benchmark 原题、答案和解析不进入训练。

第二，评估集多样化，不只看单个公开 benchmark。

第三，保留人工评测、产品样例、holdout set 和红队评估。

第四，数据选择不要只按与 benchmark 相似度，而要保留多样性和真实分布。

第五，评估报告中记录数据版本、污染检测方法和残余风险。

24. 专家追问：数据治理会不会拖慢训练迭代？

短期看，治理需要额外成本；长期看，它会加快迭代。

没有治理时，模型变好或变坏都无法解释，删除请求无法处理，实验无法复现，安全事件无法追溯。团队会在问题出现后花更多时间补救。

好的治理不是繁文缛节，而是把 manifest、lineage、schema、权限、审计和版本记录自动化，让训练迭代可追踪、可回滚、可比较。

25. 开放题：如果让你从零搭建数据团队，你怎么分工？

可以分为：

1. 数据采集与平台：source registry、数据湖、调度、存储、权限。
2. 数据清洗与质量：解析、过滤、质量评分、去重、污染检测。
3. 数据策略与实验：data mixture、ablation、数据估值、训练反馈。
4. 专项数据：代码、数学、领域、多模态、合成数据。
5. 后训练数据：SFT、preference、安全、红队回归。
6. 治理与合规：license、PII、删除请求、datasheet、model card。
7. 标注与评估：标注规范、专家审计、人工评测、线上反馈。

这个回答体现你理解数据工程不是单一 ETL，而是跨训练、评估、安全和治理的系统。

26. 面试中的常见错误表达

错误一：我们把网上数据爬下来清洗一下就能训。

问题：忽略合规、质量、去重、污染、配比和治理。

错误二：数据越多越好。

问题：忽略有效 token、重复、噪声、风险和 compute budget。

错误三：去重就是 hash 去重。

问题：忽略 near dedup、段落重复、代码 fork 和 benchmark contamination。

错误四：安全数据就是拒答数据。

问题：忽略边界允许、误拒修复、多语言和专业高风险场景。

错误五：合成数据可以无限扩。

问题：忽略 teacher 偏差、同质化、数据退化和验证。

错误六：数据治理是上线后的事。

问题：没有 lineage 和版本，训练阶段就已经不可复现。

27. 最后一套万能回答模板

如果你遇到没准备过的数据题，可以用这个模板：

第一，我会先明确模型目标和评估指标，因为数据质量和价值都是目标相关的。

第二，我会把数据按来源、语言、领域、模态、质量和风险分池，而不是混成一个大文件。

第三，我会建立清洗过滤流程，包括解析、正文抽取、质量评分、PII、安全过滤、去重和污染检测。

第四，我会设计 data mixture 和采样权重，通过小模型 ablation 和多维评估验证。

第五，我会关注专项数据，比如代码、数学、领域、多模态、合成、偏好和安全数据，它们要单独治理。

第六，我会建立版本和治理体系，包括 manifest、lineage、license、权限、删除请求和 datasheet。

第七，我会用训练结果、评估结果、人工审计和线上反馈反向迭代数据策略。

这套模板足以覆盖大多数大模型数据工程面试题。

27.1 最小可运行数据面试复盘 demo

下面这个 demo 不依赖外部库，也不读写文件。输入是一次 toy mock interview 的 4 道数据工程题回答记录；输出包括每题得分、平均分、指标覆盖率、缺失框架、缺失风险和下一步修正计划。

它演示的不是自动评分面试官，而是把复盘从“感觉没答好”变成可执行清单。真实面试准备时，可以把 covered、metrics、evidence、risks、tradeoffs 换成你自己的录音复盘结果。

```
from collections import Counter
```

```
FRAMEWORK = {"target", "source", "processing", "mixture", "evaluation", "governance"}
RISK_KEYS = {"license", "pii", "contamination", "safety", "deletion"}
WEIGHTS = {"framework": 0.30, "metrics": 0.18, "evidence": 0.18, "risk": 0.16, "depth": 0.18}

answers = [
    {
        "id": "q_pretrain_dataset",
        "topic": "pretraining data",
        "covered": {"target", "source", "processing", "mixture", "evaluation", "governance"},
        "metrics": {"token_retention", "risk_hit_rate", "contamination_rate", "lineage_coverage"},
        "evidence": {"small_model_ablation", "manual_audit", "version_manifest"},
        "risks": {"license", "pii", "contamination", "deletion"},
        "tradeoffs": {"quality_vs_coverage", "dedup_vs_diversity"},
        "red_flags": set(),
        "time_sec": 165,
    },
    {
        "id": "q_near_dedup",
        "topic": "dedup and contamination",
        "covered": {"processing", "evaluation"},
        "metrics": {"jaccard", "duplicate_rate"},
        "evidence": {"toy_minhash_demo"},
        "risks": {"contamination"},
        "tradeoffs": {"dedup_vs_diversity"},
        "red_flags": {"said_dedup_more_is_always_better"},
        "time_sec": 150,
    },
    {
        "id": "q_preference_safety",
        "topic": "preference and safety data",
        "covered": {"target", "source", "processing", "evaluation", "governance"},
        "metrics": {"annotator_agreement", "length_bias", "unsafe_leak_rate", "over_refusal_rate"},
        "evidence": {"rubric", "red_team_regression"},
        "risks": {"pii", "safety", "contamination"},
        "tradeoffs": {"helpfulness_vs_safety", "judge_cost_vs_human_review"},
        "red_flags": set(),
        "time_sec": 178,
    },
    {
        "id": "q_governance",
        "topic": "dataset versioning",
        "covered": {"source", "processing", "mixture", "governance"},
        "metrics": {"lineage_coverage", "license_completion", "datasheet_completion"},
        "evidence": {"manifest"},
    },
]
```

```

        "risks": {"license", "deletion"},
        "tradeoffs": {"iteration_speed_vs_traceability"},
        "red_flags": set(),
        "time_sec": 205,
    },
]

```

```

def score_answer(item):
    framework_score = len(item["covered"] & FRAMEWORK) / len(FRAMEWORK)
    metric_score = min(len(item["metrics"]) / 3, 1.0)
    evidence_score = min(len(item["evidence"]) / 2, 1.0)
    risk_score = len(item["risks"] & RISK_KEYS) / len(RISK_KEYS)
    depth_score = min(len(item["tradeoffs"]) / 2, 1.0)
    penalty = 0.10 * len(item["red_flags"])
    if item["time_sec"] > 180:
        penalty += 0.05
    total = (
        WEIGHTS["framework"] * framework_score
        + WEIGHTS["metrics"] * metric_score
        + WEIGHTS["evidence"] * evidence_score
        + WEIGHTS["risk"] * risk_score
        + WEIGHTS["depth"] * depth_score
        - penalty
    )
    return round(max(total, 0.0), 3)

```

```

scores = {item["id"]: score_answer(item) for item in answers}
missing_framework = Counter()
missing_risks = Counter()
revision_plan = []

```

```

for item in answers:
    missing = sorted(FRAMEWORK - item["covered"])
    for name in missing:
        missing_framework[name] += 1
    for risk in sorted(RISK_KEYS - item["risks"]):
        missing_risks[risk] += 1
    if scores[item["id"]] < 0.72 or item["red_flags"]:
        revision_plan.append(
            {
                "id": item["id"],
                "add_framework": missing[:3],
                "add_metrics": max(0, 3 - len(item["metrics"])),
                "fix_red_flags": sorted(item["red_flags"]),
            }
        )

```

```

average_score = round(sum(scores.values()) / len(scores), 3)
metric_coverage = round(sum(min(len(item["metrics"]), 3) for item in answers) / (3 * len(answers)), 3)
readiness_gate = average_score >= 0.78 and not revision_plan

```

```

report = {
    "scores": scores,

```

```

    "average_score": average_score,
    "metric_coverage": metric_coverage,
    "missing_framework": dict(sorted(missing_framework.items())),
    "missing_risks": dict(sorted(missing_risks.items())),
    "revision_plan": revision_plan,
    "readiness_gate": readiness_gate,
}

```

```

for key, value in report.items():
    print(f"{key}=", value)

```

```

assert scores == {
    "q_pretrain_dataset": 0.968,
    "q_near_dedup": 0.332,
    "q_preference_safety": 0.886,
    "q_governance": 0.574,
}

```

```

assert average_score == 0.69
assert metric_coverage == 0.917
assert readiness_gate is False

```

运行后会看到类似输出：

```

scores= {'q_pretrain_dataset': 0.968, 'q_near_dedup': 0.332, 'q_preference_safety': 0.886, 'q_governance': 0.574}
average_score= 0.69
metric_coverage= 0.917
missing_framework= {'evaluation': 1, 'governance': 1, 'mixture': 2, 'source': 1, 'target': 2}
missing_risks= {'contamination': 1, 'deletion': 2, 'license': 2, 'pii': 2, 'safety': 3}
revision_plan= [{'id': 'q_near_dedup', 'add_framework': ['governance', 'mixture', 'source'], 'add_metrics': 1, 'fix_red
readiness_gate= False

```

这里最重要的不是分数，而是 `revision_plan`。它指出 near dedup 题答得太局部，缺少 source、mixture、governance，还出现了“去重越多越好”的错误表述；governance 题虽然讲了 manifest 和 lineage，但缺少目标澄清、评估证据，并且超时。下一轮复盘应该优先修这两类问题。

28. 本章小结

大模型数据面试的核心不是背工具，而是建立系统视角。

你要让面试官相信你理解：

1. 数据决定模型能力和行为。
2. 数据工程必须和训练、评估、安全、产品闭环。
3. 预训练、SFT、DPO、安全、多模态和 RAG 需要不同数据策略。
4. 去重、污染、配比、合成、偏好和治理都是一体化问题。
5. 好的数据系统不仅能训练模型，还能解释、复现、审计和迭代模型。

如果只能记住一句话：大模型数据工程不是“收集更多数据”，而是在目标、质量、覆盖、风险、配比、评估和治理之间做系统性优化。